

SIEMENS

SINUMERIK

SINUMERIK ONE Technologies

Function Manual

Introduction	1
Fundamental safety instructions	2
Installation and activation of loadable compile cycles	3
Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)	4
Transformation package handling (Run MyCC / RCTRA)	5
Retrace support (Run MyCC /RESU)	6
High-speed laser switching signal (Run MyCC /HSLC)	7
Axis pair collision protection (Protect MyMachine /CC Axis Monitor)	8
Punching and nibbling	9
Appendix	A

Valid for

controller
SINUMERIK ONE

CNC software Version 6.23

01/2024

A5E48053723B AH

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury will result if proper precautions are not taken.
--

WARNING
--

indicates that death or severe personal injury may result if proper precautions are not taken.

CAUTION
--

indicates that minor personal injury can result if proper precautions are not taken.
--

NOTICE

indicates that property damage can result if proper precautions are not taken.
--

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

WARNING
--

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.
--

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Table of contents

1	Introduction	9
1.1	About SINUMERIK	9
1.2	About this documentation	9
1.3	Documentation on the internet	12
1.3.1	Documentation overview SINUMERIK ONE	12
1.3.2	Documentation overview SINUMERIK operator components	12
1.4	Feedback on the technical documentation	13
1.5	mySupport documentation	13
1.6	Service and Support.....	14
1.7	Using OpenSSL	15
1.8	Compliance with the General Data Protection Regulation	15
2	Fundamental safety instructions.....	17
2.1	General safety instructions.....	17
2.2	Warranty and liability for application examples	17
2.3	Cybersecurity information	17
3	Installation and activation of loadable compile cycles.....	19
3.1	Overview	19
3.2	Installation	21
3.2.1	Requirement: Compatibility of the interface versions	21
3.2.2	Installation of a compile cycle directly on the controller	22
3.2.3	Installing a compile cycle from an external computer.....	23
3.3	Activation in the channel	26
3.4	Creating alarm texts.....	27
3.5	Upgrading a compile cycle	28
3.6	Deleting a compile cycle.....	30
4	Distance control 1D/3D in position controller cycle (Run MyCC /CLCX).....	31
4.1	Brief description.....	31
4.1.1	Function description	32
4.2	Clearance control.....	34
4.2.1	POS mode.....	34
4.2.1.1	Overview	34
4.2.1.2	POS mode.....	35
4.2.2	GAIN mode	35
4.2.2.1	Control dynamics - GAIN mode.....	35
4.2.3	Dead times in the clearance control.....	37

4.2.4	Optimizing the control behavior	38
4.3	Technological features of clearance control	38
4.4	Sensor collision monitoring	39
4.5	Startup	40
4.5.1	Setup steps	41
4.5.2	Activating the technological function	42
4.5.3	Configuring the memory	43
4.5.4	Sensor input and sensor signals	43
4.5.4.1	Setting up the analog sensor input	43
4.5.4.2	Parameterizing the input signals	44
4.5.4.3	Simulation of the sensor signals	44
4.5.4.4	Linearizing the signal of the clearance sensor	46
4.5.4.5	Compensation of the time delay of the sensor	47
4.5.5	Parameter settings for clearance control	50
4.5.6	Configuration of the basic MD of CLCX	51
4.5.7	Definition of the real-time variables for synchronized actions	53
4.6	Programming	56
4.6.1	Activating and deactivating clearance control (CLC)	56
4.6.2	NC commands and Synact variables	58
4.6.3	Synact variables for diagnostics	60
4.6.4	Available technology commands	61
4.6.5	Diagnosis of the CLCX state	63
4.7	3D clearance control	64
4.7.1	3D clearance control - overview	64
4.7.2	3D control direction	65
4.7.3	Parameterization of the programmable control direction	67
4.7.4	3D clearance control and 5-axis transformation	69
4.8	Multiple CLCX instances in different channels	71
4.8.1	Multiple CLCX instances in one channel	71
4.8.2	Macros for operating and monitoring multiple instances in one channel	73
4.8.3	Programming several instances within one channel	75
4.8.4	Additional voltage/clearance characteristics	76
4.9	Function-specific display data	78
4.9.1	Channel-specific GUD variables	79
4.10	Function-specific alarm texts	80
4.11	Supplementary conditions	80
4.11.1	I/O modules	80
4.11.1.1	External smoothing filters	82
4.11.2	Function-specific supplementary conditions	82
5	Transformation package handling (Run MyCC /RCTRA)	87
5.1	Brief description	87
5.2	Kinematic transformation	88
5.3	Definition of terms	88
5.3.1	Units and directions	88
5.3.2	Definition of positions and orientations using frames	89
5.3.3	Definition of a joint	90

5.4	Configuration of a kinematic transformation.....	91
5.4.1	General machine data	92
5.4.2	Parameterization using geometry data	93
5.5	Descriptions of kinematics	106
5.5.1	3-axis kinematics	106
5.5.2	4-axis kinematics	114
5.5.3	5-axis kinematics	120
5.5.4	6-axis kinematics	125
5.5.5	Special kinematics	125
5.6	Tool orientation	131
5.6.1	Orientation programming for 4-axis kinematics	133
5.6.2	Orientation programming for 5-axis kinematics	135
5.7	Singular positions and how they are handled.....	136
5.8	Call and application of the transformation	137
5.9	Actual value display	138
5.10	Tool programming	139
5.11	Cartesian PTP travel with handling transformation package.....	140
5.12	Startup	140
5.12.1	General commissioning.....	140
5.12.2	Function-specific commissioning	140
5.13	Supplementary conditions	142
6	Retrace support (Run MyCC /RESU).....	143
6.1	Brief description.....	143
6.2	Function description	144
6.2.1	Function	144
6.2.2	Definition of terms	146
6.2.3	Functional sequence (principle)	147
6.2.4	Maximum retraceable contour area	150
6.3	Startup	151
6.3.1	Activation	151
6.3.2	Definition of the RESU working plane	151
6.3.3	Memory configuration: Block memory.....	152
6.3.4	Memory configuration: Heap memory	153
6.3.5	RESU main program memory area	154
6.3.6	Storage of the RESU subroutines	155
6.3.7	ASUB enable	155
6.3.8	PLC user program.....	156
6.4	Programming.....	157
6.4.1	RESU Start/Stop/Reset (CC_PREPRE).....	157
6.5	RESU-specific part programs	158
6.5.1	Overview	158
6.5.2	Main program (CC_RESU.MPF)	159
6.5.3	INI program (CC_RESU_INI.SPF)	160
6.5.4	END program (CC_RESU_END.SPF)	162

6.5.5	Retrace support ASUB (CC_RESU_BS_ASUP.SPF)	163
6.5.6	RESU ASUB (CC_RESU_ASUP.SPF).....	163
6.6	Retrace support	164
6.6.1	General information.....	164
6.6.2	Block search with calculation on contour	165
6.6.3	Reposition	165
6.6.4	Temporal conditions concerning NC start	166
6.6.5	Block search from last main block.....	167
6.7	Function-specific display data	168
6.7.1	Channel-specific GUD variables.....	168
6.8	Function-specific alarm texts	169
6.9	Boundary conditions	169
6.9.1	Function-specific supplementary conditions	169
6.9.1.1	Continue machining within subprograms	169
6.9.1.2	Continue machining within program loops	170
6.9.1.3	Machining continuation on full circles	170
6.9.1.4	Automatically generated contour elements	170
6.9.2	Supplementary conditions for standard functions.....	171
6.9.2.1	Axis replacement	171
6.9.2.2	Traversing movements of the channel axes.....	171
6.9.2.3	Block numbers	171
6.9.2.4	Block search.....	172
6.9.2.5	Transformations.....	172
6.9.2.6	Compensations.....	173
6.9.2.7	Frames	173
6.9.2.8	Tool offsets	173
7	High-speed laser switching signal (Run MyCC /HSLC)	175
7.1	Brief description.....	175
7.2	Functional description.....	176
7.2.1	General information.....	176
7.2.2	Calculating the switching positions	176
7.2.2.1	Block-related switching signal output	176
7.2.2.2	Path length-related switching signal output.....	178
7.2.3	Calculating the switching instants	179
7.2.4	Switching frequency and switching position distance	179
7.2.5	Approaching switching position	180
7.2.6	Programmed switching position offset.....	181
7.2.7	Response to part program interruption	181
7.3	Startup	182
7.3.1	Activation	182
7.3.2	Memory configuration	182
7.3.3	Parameterizing the digital on-board outputs	182
7.3.4	Parameterizing the switching signal	183
7.3.5	Parameterization of the geometry axes	183
7.4	Programming.....	184
7.4.1	Activating the block-related switching signal output (CC_FASTON).....	184
7.4.2	Activating the path length-related switching signal output (CC_FASTON_CONT).....	185
7.4.3	Deactivation (CC_FASTOFF).....	186

7.5	Function-specific alarm texts	187
7.6	Supplementary conditions	187
7.6.1	Block search.....	187
7.6.2	Transformations.....	188
7.6.3	Compensations.....	189
7.6.4	Tool radius compensation (TRC)	189
7.6.5	Continuous-path mode	189
7.6.6	Software cams	189
8	Axis pair collision protection (Protect MyMachine /CC Axis Monitor)	191
8.1	Brief description.....	191
8.2	Functional description.....	191
8.3	Commissioning	192
8.3.1	Enabling the technology function (option)	192
8.3.2	Activating the technology function.....	193
8.3.3	Activating the supplementary functions	193
8.3.4	Definition of an axis pair.....	193
8.3.5	Retraction direction.....	194
8.3.6	Offset for the machine coordinate systems	194
8.3.7	Protection window	195
8.3.8	Orientation	196
8.3.9	Protection window extension	196
8.3.10	Activating the protection function	197
8.3.11	Axis-specific acceleration.....	198
8.3.12	Monitoring status (GUD)	198
8.3.13	PLC interface: Axis-specific braking operations	199
8.3.14	PLC interface: Axis pair-specific activation of the protection function	200
8.4	Boundary conditions	201
8.4.1	Axes	201
8.4.2	Interpolatory couplings	201
8.4.3	Simultaneous programming of the protected axes	202
8.5	Examples	202
8.5.1	Collision protection	202
8.5.2	Collision protection and distance limiter	204
9	Punching and nibbling.....	207
9.1	Brief Description	207
9.2	Stroke control	207
9.2.1	General information.....	207
9.2.2	High-speed signals.....	208
9.2.3	Criteria for stroke initiation.....	209
9.2.4	Axis start after punching	211
9.2.5	PLC signals specific to punching and nibbling.....	212
9.2.6	Punching and nibbling-specific reactions to standard PLC signals.....	213
9.2.7	Signal monitoring	213
9.3	Activation and deactivation	214
9.3.1	Language commands.....	214
9.3.2	Functional expansions	218

9.3.3	Compatibility with earlier systems	222
9.4	Automatic path segmentation	224
9.4.1	General information.....	224
9.4.2	Operating characteristics with path axes	225
9.4.3	Response in connection with single axes	230
9.5	Rotatable tool	235
9.5.1	General information.....	235
9.5.2	Coupled motion of punch and die	236
9.5.3	Tangential control	237
9.6	Protection zones	240
9.7	Supplementary conditions	241
9.8	Examples	241
9.8.1	Examples of defined start of nibbling operation	241
A	Appendix.....	247
A.1	List of abbreviations.....	247
A.2	Available IPCs.....	252
	Index	253

Introduction

1.1 About SINUMERIK

From simple, standardized CNC machines to premium modular machine designs – the SINUMERIK CNCs offer the right solution for all machine concepts. Whether for individual parts or mass production, simple or complex workpieces – SINUMERIK is the highly dynamic automation solution, integrated for all areas of production. From prototype construction and tool design to mold making, all the way to large-scale series production.

Visit our website for more information SINUMERIK (<https://www.siemens.com/sinumerik>).

1.2 About this documentation

This documentation is part of the documentation category of SINUMERIK Function Manuals.

SINUMERIK Function Manuals

The SINUMERIK Function Manuals describe the NC functions of a SINUMERIK control.

The target group are configuring engineers, technologists, commissioning engineers and programmers.

Each Function Manual covers a specific topic area and contains all the function descriptions for this topic area.

The following table shows which Function Manuals are available for your SINUMERIK control and which topic areas a Function Manual covers:

Function Manual	Topic area
Basic functions	Basic functions of a CNC control
Axes and spindles	Axis and spindle functions, axis couplings
Transformations	Transformation functions
Monitoring and compensating	Axis monitoring and compensation functions, collision avoidance
Tools	Functions for selection, correction and monitoring of tools
Synchronized actions	Functionality of synchronized actions
PLC	Structure and functions of the PLC
Tool management	Function, commissioning and programming of tool management
Technologies	Extended technological functionalities

Overview of contents

You will find an overview of the main chapters of the descriptions of functions that are contained in a Function Manual on the title page.

Validity

The title page also contains all data about the validity of a document, i.e. for which SINUMERIK control and for which software version this edition of the Function Manual is valid.

System data

In the descriptions of functions, the system data relevant for a function (machine data, setting data, system variables, interface signals and alarms) are only described with the level of detail that is strictly necessary to understand the function. You will find detailed information on this data in the relevant List Manuals and for alarms in the Diagnostics Manual.

Basic Program Plus and Basic Program

In the TIA Portal, two different types of basic program are available for programming the PLC functionality:

- Basic Program Plus works only with symbolic addressing. The functionality is implemented via data types, PLC blocks, functions and instructions. Basic Program Plus is available only for SINUMERIK ONE.
- Basic Program works with both symbolic and absolute addressing. The programs are thus largely compatible with already existing programs from the SIMATIC Manager. Basic Program is available for ONE and MC controllers.

Basic Program Plus and Basic Program cannot be used together.

PLC blocks and data types

The descriptions of functions often refer to the PLC signals and functions. In the text, they are primarily the blocks and data types of Basic Program Plus, which is only available for SINUMERIK ONE.

You can find detailed information in the PLC Function Manual for the relevant controller.

Interface signals

The symbolic addressing of Basic Program Plus is the first choice for addressing interface signals. To derive signals for Basic Program, there are tables at the end of the relevant section with the PLC signals used for the different signal directions, as shown in the following example:

NC → PLC

Basic Program Plus	Basic Program	
<Axis>.basic.in.enc1Synchronized	LBP_Axis*.E_RefSyn1	DB31,DBX60.4
...		

PLC → NC

Basic Program Plus	Basic Program	
<Axis>.basic.out.refPointApproachCam	LBP_Axis*.A_DelayRef	DB31,DBX12.7
...		

Note**Quantity structure**

In Basic Program Plus, axes, channels etc. are represented via instances of the corresponding objects.

The absolute addressing in Basic Program is based on the absolute maximum number of the following components: Mode groups (DB11), channels (DB21, ...), axes/spindles (DB31, ...)

Standard scope

This documentation only describes the functionality of the standard version. This may differ from the scope of the functionality of the system that is actually supplied. Please refer to the ordering documentation only for the functionality of the supplied drive system.

It may be possible to execute other functions in the system which are not described in this documentation. This does not, however, represent an obligation to supply such functions with a new control or when servicing.

For reasons of clarity, this documentation cannot include all of the detailed information on all product types. Further, this documentation cannot take into consideration every conceivable type of installation, operation and service/maintenance.

The machine manufacturer must document any additions or modifications they make to the product themselves.

Websites of third-party companies

This document may contain hyperlinks to third-party websites. Siemens is not responsible for and shall not be liable for these websites and their content. Siemens has no control over the information which appears on these websites and is not responsible for the content and information provided there. The user bears the risk for their use.

1.3 Documentation on the internet

1.3.1 Documentation overview SINUMERIK ONE

Comprehensive documentation about the functions provided in SINUMERIK ONE Version 6.13 and higher is provided in the Documentation overview SINUMERIK ONE (<https://support.industry.siemens.com/cs/ww/en/view/109768483>).



You can display documents or download them in PDF and HTML5 format.

The documentation is divided into the following categories:

- User: Operating
- User: Programming
- Manufacturer/Service: Functions
- Manufacturer/Service: Hardware
- Manufacturer/Service: Configuration/Setup
- Manufacturer/Service: Safety Integrated
- Information and training
- Manufacturer/Service: SINAMICS

1.3.2 Documentation overview SINUMERIK operator components

Comprehensive documentation about the SINUMERIK operator components is provided in the Documentation overview SINUMERIK operator components (<https://support.industry.siemens.com/cs/document/109783841/technische-dokumentation-zu-sinumerik-bedienkomponenten?dti=0&lc=en-WW>).

You can display documents or download them in PDF and HTML5 format.

The documentation is divided into the following categories:

- Operator Panels
- Machine control panels
- Machine Pushbutton Panel
- Handheld Unit/Mini handheld devices
- Further operator components

An overview of the most important documents, entries and links to SINUMERIK is provided at SINUMERIK Overview - Topic

Page (<https://support.industry.siemens.com/cs/document/109766201/sinumerik-an-overview-of-the-most-important-documents-and-links?dti=0&lc=en-WW>).

1.4 Feedback on the technical documentation

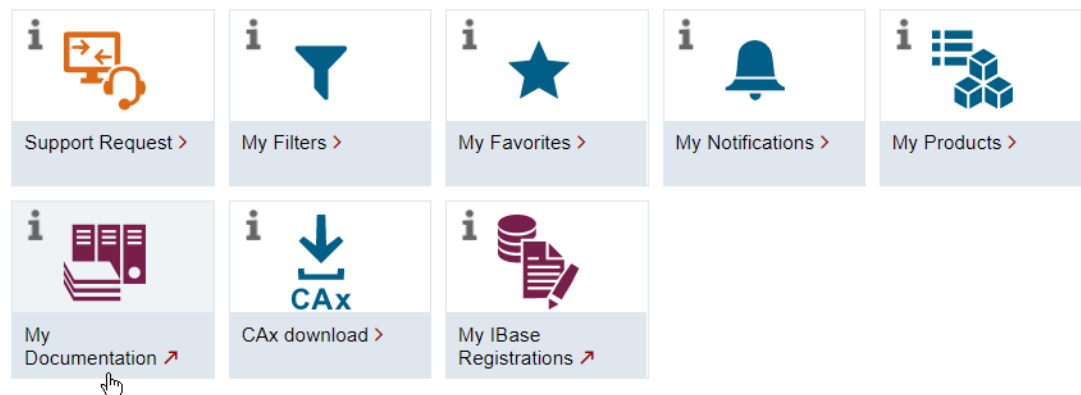
If you have any questions, suggestions or corrections regarding the technical documentation which is published in the Siemens Industry Online Support, use the link "Send feedback" link which appears at the end of the entry.

1.5 mySupport documentation

With the "mySupport documentation" web-based system you can compile your own individual documentation based on Siemens content, and adapt it for your own machine documentation.

To start the application, click on the "My Documentation" tile on the "mySupport links and tools" (<https://support.industry.siemens.com/cs/ww/en/my>) portal page:

mySupport Links and Tools



The configured manual can be exported in RTF, PDF or XML format.

Note

Siemens content that supports the mySupport documentation application can be identified by the presence of the "Configure" link.

1.6 Service and Support

Product support

You can find more information about products on the internet:

Product support (<https://support.industry.siemens.com/cs/ww/en/>)

The following is provided at this address:

- Up-to-date product information (product announcements)
- FAQs (frequently asked questions)
- Manuals
- Downloads
- Newsletters with the latest information about your products
- Global forum for information and best practice sharing between users and specialists
- Local contact persons via our Contacts at Siemens database (→ "Contact")
- Information about field services, repairs, spare parts, and much more (→ "Field Service")

Technical support

Country-specific telephone numbers for technical support are provided on the internet at address (<https://support.industry.siemens.com/cs/ww/en/sc/4868>) in the "Contact" area.

If you have any technical questions, please use the online form in the "Support Request" area.

Training

You can find information on SITRAIN at the following address (<https://www.siemens.com/sitrain>).

SITRAIN offers training courses for automation and drives products, systems and solutions from Siemens.

Siemens support on the go



With the award-winning "Industry Online Support" app, you can access more than 300,000 documents for Siemens Industry products – any time and from anywhere. The app can support you in areas including:

- Resolving problems when implementing a project
- Troubleshooting when faults develop
- Expanding a system or planning a new system

Furthermore, you have access to the Technical Forum and other articles from our experts:

- FAQs
- Application examples
- Manuals
- Certificates
- Product announcements and much more

The "Industry Online Support" app is available for Apple iOS and Android.

1.7 Using OpenSSL

This product can contain the following software:

- Software developed by the OpenSSL project for use in the OpenSSL toolkit
- Cryptographic software created by Eric Young.
- Software developed by Eric Young

You can find more information on the internet:

- OpenSSL (<https://www.openssl.org>)
- Cryptsoft (<https://www.cryptsoft.com>)

1.8 Compliance with the General Data Protection Regulation

Siemens observes standard data protection principles, in particular the data minimization rules (privacy by design).

1.8 Compliance with the General Data Protection Regulation

For this product, this means:

The product does not process or store any personal data, only technical function data (e.g. time stamps). If the user links this data with other data (e.g. shift plans) or if he/she stores person-related data on the same data medium (e.g. hard disk), thus personalizing this data, he/she must ensure compliance with the applicable data protection stipulations.

Fundamental safety instructions

2.1 General safety instructions

 WARNING
Danger to life if the safety instructions and residual risks are not observed If the safety instructions and residual risks in the associated hardware documentation are not observed, accidents involving severe injuries or death can occur. • Observe the safety instructions given in the hardware documentation. • Consider the residual risks for the risk evaluation.

 WARNING
Malfunctions of the machine as a result of incorrect or changed parameter settings As a result of incorrect or changed parameterization, machines can malfunction, which in turn can lead to injuries or death. • Protect the parameterization against unauthorized access. • Handle possible malfunctions by taking suitable measures, e.g. emergency stop or emergency off.

2.2 Warranty and liability for application examples

Application examples are not binding and do not claim to be complete regarding configuration, equipment or any eventuality which may arise. Application examples do not represent specific customer solutions, but are only intended to provide support for typical tasks.

As the user you yourself are responsible for ensuring that the products described are operated correctly. Application examples do not relieve you of your responsibility for safe handling when using, installing, operating and maintaining the equipment.

2.3 Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

2.3 Cybersecurity information

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit
<https://www.siemens.com/cybersecurity-industry>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under
<https://new.siemens.com/cert>.

Further information is provided on the Internet:

Industrial Security Configuration Manual (<https://support.industry.siemens.com/cs/ww/en/view/108862708>)



WARNING

Unsafe operating states resulting from software manipulation

Software manipulations, e.g. viruses, Trojans, or worms, can cause unsafe operating states in your system that may lead to death, serious injury, and property damage.

- Keep the software up to date.
- Incorporate the automation and drive components into a state-of-the-art, integrated industrial cybersecurity concept for the installation or machine.
- Make sure that you include all installed products in the integrated industrial cybersecurity concept.
- Protect files stored on exchangeable storage media from malicious software by with suitable protection measures, e.g. virus scanners.
- Carefully check all cybersecurity-related settings once commissioning has been completed.

Installation and activation of loadable compile cycles

3

3.1 Overview

Compile cycles are functional expansions of the NC system software that can be created by the machine manufacturer and/or by Siemens and then imported in the controller later.

As part of the open NC system architecture, compile cycles have comprehensive access to data and functions of the NC system level via defined software interfaces. In this way, compile cycles significantly extend the functionality of the controller.

Siemens compile cycles

Compile cycles created by Siemens are available as options requiring a license.

The customer receives the corresponding software license with the order. The software package for installing the compile cycle is provided by the regional Siemens sales partner.

The scope of delivery also includes a manual that describes the functionality of the compile cycle.

There is also a description in this Function Manual for the following Siemens compile cycles:

- Distance control 1D/3D in position controller cycle (Run MyCC /CLCX) (Page 31)
- Transformation package handling (Run MyCC /RCTRA) (Page 87)
- Retrace support (Run MyCC /RESU) (Page 143)
- High-speed laser switching signal (Run MyCC /HSLC) (Page 175)
- Axis pair collision protection (Protect MyMachine /CC Axis Monitor) (Page 191)

Commissioning

Installation files

In addition to the file for loading the compile cycle (ELF file), the software package for installing a Siemens compile cycle also contains other specific installation files such as alarm text and definition files.

Example:

Software package for the "Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)" compile cycle		
Directory	File	Description
 elf	clcx_*.elf	Loadable compile cycle file (ELF file) The extension ".ELF" stands for "executable and linking format".
 doc_alarms	clcx_alarms_deu.ts clcx_alarms_eng.ts	Language-specific alarm text files

3.1 Overview

Software package for the "Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)" compile cycle		
Directory	File	Description
 defVariables	clcx_gud.def	GUD definition file
	clcx_mmac.def	Macro definition file
	clcx_SynActVar.xml	Template file for the NC/PLC variable view

Installation

To integrate a compile cycle into the NC system software, the installation files must be copied into the NC file system and the data activated.

More information: → Chapter "Installation (Page 21)"

Activation in the channel

A specific global NC machine data is created for each Siemens Compile cycle during installation. This machine data must be used to activate the functionality of the compile cycle in the designated channel. The functionality of the compile cycle can only be used in the channel after this activation and a new controller start-up.

More information: → Chapter "Activation in the channel (Page 26)"

Additional commissioning steps

All other commissioning steps, such as setting the function parameters, are specific to the compile cycle. Their description can be found in the function description of the respective compile cycle.

Creating alarm texts

Specific alarm text files are part of a compile cycle software package from Siemens. After successful installation, they are stored in the system.

If a new alarm is required, you can add your own alarm text files.

More information: → Chapter "Creating alarm texts (Page 27)"

Upgrading a compile cycle

To upgrade a compile cycle, an NC archive must be created after the ELF file has been replaced and read back into the controller.

More information: → Chapter "Upgrading a compile cycle (Page 28)"

Deleting a compile cycle

For the most efficient use of NC memory resources, only the compile cycles actually required on the machine should be loaded. Compile cycles that are no longer used should be completely removed from the controller. To do this, the ELF file must be deleted and then an NC archive created and read back into the controller.

More information: → Chapter "Deleting a compile cycle (Page 30)"

3.2 Installation

3.2.1 Requirement: Compatibility of the interface versions

A SINUMERIK-specific interface is used for communication between the compile cycle and NC system software. This requires that the interface version of a compile cycle to be loaded is compatible with the interface version of the NC system software.

Note

If the interface versions of the compile cycle and NC system software are incompatible, it is not possible to load the compile cycle. Alarm 7200 is issued after the control has started up.

Interface version number

The compatibility of interface versions is indicated by their interface version numbers.

Number structure

Interface version numbers consist of a three-digit number (1st digits), a dot, followed by another three-digit number (2nd digits).

Example: 008.100

Compatibility criteria

The interface versions of the compile cycle and NC system software are compatible if the following criteria are met:

- The 1st digits of the interface version numbers of the compile cycle and NC system software must be **identical**.
- The 2nd digits of the interface version number of the compile cycle must be **less than or equal to** the 2nd digits of the interface version number of the NC system software.

Determining the interface version numbers

Interface version number of the NC system software

The interface version number of the NC system software is displayed in the user interface under:

Diagnostics > Version > CNC software > Details > Compile Cycles

Interface version number of a compile cycle

The interface version number of a compile cycle can be taken from the name of the ELF file. It results from the six-digit number between the letter sequence "if" and the file extension ".elf".

Example:

clcx_010601if008002.elf has the interface version number 008.002

3.2.2 Installation of a compile cycle directly on the controller

Requirements

To be able to install a compile cycle directly on the controller, the following requirements must be met:

- The interface version of the compile cycle must be compatible with the interface version of the NC system software.
More information: → "Requirement: Compatibility of the interface versions (Page 21)"
- The compile cycle software package must be available on a storage medium that can be connected directly to the controller, e.g. USB flash drive.
- Access authorization of protection level 1 "Machine manufacturer" (default password "SUNRISE") must be available.
- If there are no cursor keys on the operator panel, a mouse or keyboard must be connected to perform the operator actions.

Procedure

To copy the files of a compile cycle software package from a storage medium (here: USB flash drive), carry out the following steps:

1. Insert the USB flash drive in the USB interface on the front of the operator panel.
2. In the user interface, go to the "Commissioning" area > "System data".
3. Open the USB flash drive ("USB" directory).
4. In the "elf" directory, select the loadable compile cycle file <cc>_*.elf and copy the file.
5. Within the "System data" area, switch to the directory "NC data" > "Compile cycles" and insert the file there.
6. Switch to the area "Commissioning" > "Machine data" and restart the controller ("Reset (po)" softkey).
The compile cycle file is loaded during controller start-up.

Note

You can recognize the successful installation of the compile cycle file by the newly available machine data 6xxxx.

7. Switch back to the "Commissioning" > "System data" area.
8. Copy the remaining installation files of the compile cycle software package to the designated target directories in the NC file system and activate the data.
The following table shows the procedure using the example of the "Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)" compile cycle:

File	Copying to target directory	Activating the data
clcx_alarms_deu.ts clcx_alarms_eng.ts	/card/oem/sinumerik/hmi/Ing	Restart SINUMERIK Operate to store the alarm text files in the system.
clcx_gud.def	NC data/definitions	- Select the file and rename it. The following names are permitted: SGUD.DEF, MGUD.DEF, UGUD.DEF or GUD4.DEF ... GUD9.DEF – or – Insert the contents of the file into an existing GUD definition file. - Read in the GUD definition file again ("Activate" softkey).
clcx_mmac.def	NC data/definitions	- Select the file and rename it as follows: MMAC.DEF – or – Insert the contents of the file into an existing MMAC definition file. - Read in the MMAC definition file again ("Activate" softkey).
clcx_SynActVar.xml	HMI data/templates/variable masks	To use the template file, select the file in the operating area "Diagnostics" > "NC/PLC variables" and load the template ("Load mask" softkey).

3.2.3 Installing a compile cycle from an external computer

Requirements

To install a compile cycle on the controller from an external computer (PG/PC), the following requirements must be met:

- The interface version of the compile cycle must be compatible with the interface version of the NC system software.
More information: → Chapter "Requirement: Compatibility of the interface versions (Page 21)"
- The external computer on which the compile cycle software package is located must be connected to the controller via a network connection (TCP/IP).
- Access authorization of protection level 1 "Machine manufacturer" (default password "SUNRISE") must be available.

3.2 Installation

- All required connection and login data must be known.
- The "WinSCP" program must be installed on the external computer.

Procedure

To integrate the files of a compile cycle software package from an external computer into the NC system software, carry out the following steps:

1. Start the "WinSCP" program on the external computer (PG/PC)
2. Establish a connection to the controller by selecting an appropriate profile or by entering the host name or IP address, the user name, and the password.
3. Copy the loadable compile cycle file (<cc>_*.elf) from the external computer to the following directory in the NC file system:
/card/oem/sinumerik/oa/

4. Restart the SINUMERIK controller to load the compile cycle file.

Note

You can recognize the successful installation of the compile cycle file by the newly available machine data 6xxxx.

5. Copy the remaining installation files of the compile cycle software package to the designated target directories in the NC file system and activate the data.

The following table shows the procedure using the example of the "Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)" compile cycle:

File	Copying to target directory	Activating the data
<cc>_alarms_deu.ts <cc>_alarms_eng.ts	/card/oem/sinumerik/hmi/Ing	Restart the user interface to store the alarm text files in the system.
<cc>_gud.def	NC data/definitions	- In the operating area "Commissioning" > "System data", open the directory "NC data" > "Definitions". - Select the file and rename it. The following names are permitted: SGUD.DEF, MGUD.DEF, UGUD.DEF or GUD4.DEF ... GUD9.DEF – or – Insert the contents of the file into an existing GUD definition file. - Read in the GUD definition file again ("Activate" softkey).
<cc>_mmac.def	NC data/definitions	- In the operating area "Commissioning" > "System data", open the directory "NC data" > "Definitions". - Select the file and rename it as follows: MMAC.DEF – or – Insert the contents of the file into an existing MMAC definition file. - Read in the MMAC definition file again ("Activate" softkey).
<cc>_SynActVar.xml	HMI data/templates/variable masks	To use the template file, select the file in the operating area "Diagnostics" > "NC/PLC variables" and load the template ("Load mask" softkey).

3.3 Activation in the channel

Requirements

To be able activate a compile cycle in the channel, the following requirements must be met:

- The installation (Page 21) must be completed.
- The option data must be set.

Note

If the option date is not set, the compile cycle is not activated and the following alarm is displayed after each controller start-up:

Alarm 7202 "XXX_ELF_option_bit_missing: <Bit number>"

Function-specific machine data

Each technology function loaded per compile cycle creates a function-specific global NC machine data in the number range 60900 to 60999 according to the following scheme:

\$MN_CC_ACTIVE_IN_CHAN_<cc>[<n>]

wit <cc> = Identifier of the compile cycle

h Example: CLCX

CLCX is the identifier for the "Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)" compile cycle

<n> = Index

Index	Meaning	
= 0	Activate compile cycle	
	Bit 0	Activate compile cycle in the first channel
	Bits 1 ... 9	As a rule, bits 1 ... 9 are used to activate the compile cycle in channels 2 ... 10. However, there may be deviations from this rule depending on the compile cycle. ⇒ The meaning of the bits > 0 can be found in the function description of the respective compile cycle.
> 0	Specific to the compile cycles ⇒ The meaning of the indices > 0 can be found in the function description of the respective compile cycle.	

Activation in the channel

As mentioned in the table for \$MN_CC_ACTIVE_IN_CHAN_<cc>[<n>], a compile cycle is activated in a channel via \$MN_CC_ACTIVE_IN_CHAN_<cc>[<n>] by setting the corresponding bit of index 0.

Example:

\$MN_CC_ACTIVE_IN_CHAN_CLCX[0], ⇒ Activate the "Distance control ..." compile cycle in
Bit0 = 1 the first channel.

Activate settings

To make the activation and all other settings in the machine data

\$MN_CC_ACTIVE_IN_CHAN_<cc>[<n>] effective, the SINUMERIK controller must be restarted.

**CAUTION****Deleting user data**

When a bit is set for the first time in the machine data \$MN_CC_ACTIVE_IN_CHAN_<cc>[0], the following alarm is displayed:

Alarm 4400 "MD modification causes reorganization of the buffered memory (data loss)"

And you are warned that all user data (part programs, tool data, etc.) will be deleted on the next run-up.

It may therefore be necessary to create an archive **after** setting the data and **before** triggering a new controller start-up.

3.4 Creating alarm texts

If a new alarm is required, the existing alarm texts in the system can be supplemented accordingly. This is possible both directly on the controller and externally on a PC.

A detailed description of how to create alarm texts can be found in the → Commissioning Manual "Final steps for commissioning".

At this point, we will therefore only briefly describe the creation of alarm texts directly on the controller with SINUMERIK Operate as the user interface. For better understanding, the procedure is described using a specific example. There, the following alarm text should be supplemented:

075999 0 0 "Channel %1 block %2 call parameter is invalid"

Procedure

1. Please copy the "oem_alarms_deu.ts" file from the "/siemens/sinumerik/hmi/template/lng" directory to the "/oem/sinumerik/hmi/lng" directory.
2. Rename the file ("xxx_deu.ts").

3.5 Upgrading a compile cycle

3. Open the file in the editor and add the new alarm number and the new alarm text in German:

```
<message>
  <source>075999/NCK</source>
  <translation>
    Channel %1 block %2 call parameter is invalid
  </translation>
</message>
```

Note

Each alarm starts with the `<message>` tag and ends with the `</message>` tag. The `<source>` tag contains the alarm number and the source URL. The `<translation>` tag contains the alarm text.

4. To create an alarm text file in a foreign language, copy the just changed file and modify the language code in the filename (e.g., "xxx_eng.ts for English").
5. Open the foreign language alarm text file in the editor and record the translated alarm text in the `<translation>` tag.
6. Copy the example configuration file "oem_slaesvcadapconf.xml" from directory "/siemens/sinumerik/hmi/template/cfg" into directory "/oem/sinumerik/hmi/cfg".
7. Rename the file to "slaesvcadapconf.xml".
8. Open the "slaesvcadapconf.xml" file in the editor and enter the new base name (filename of the newly created alarm text file without language code and postfix), e.g.:

```
<BaseNames>
  <BaseName_02 type="QString" value="xxx"/>
</BaseNames>
```
9. Restart SINUMERIK Operate.
The new alarm text is integrated into the system.

3.5 Upgrading a compile cycle

To upgrade a compile cycle, the old version of the ELF file must be replaced by a new version and then an NC archive must be created and read back into the NC.

Note

When creating and importing an archive (setup or backup archive), **all** existing compile cycles in the controller are also transferred. It is not possible to select individual compile cycles.

Note

To upgrade a compile cycle loaded into the controller, it is **under no circumstances** sufficient to simply replace the corresponding ELF file. If only the ELF file is exchanged without subsequently creating an NC archive and reading it in again, the NC software may behave undefined due to inconsistent memory and data management data.

Note

When upgrading the ELF file, check whether the other installation files of the compile cycle software package have also changed and may need to be reinstalled.

Procedure

Perform the following actions to upgrade a compile cycle:

1. Load the new ELF file into the same directory as the old ELF file.
Proceed as described in Chapter "Installation (Page 21)".
2. Delete the old ELF file.
3. Create an NC archive.
4. Read the created NC archive back into the controller.

Note

If several compile cycles are loaded in the controller, the other compile cycles remain unchanged when a compile cycle is upgraded according to the procedure described.

Note

If the new compile cycle creates new data, which were still not available in the previous version, then after subsequently booting the control, these are predefined default values.

Version check

The software version of a loaded compile cycle is displayed in the user interface under:

Diagnostics > Version > CNC software > Details > Compile Cycles

Alternatively, the software version of a compile cycle can be derived from the name of the loadable file.

Example:

clcx_010601if008002.elf has software version 01.06.01

3.6 Deleting a compile cycle

To completely remove a compile cycle from the controller, an NC archive must be created after deleting the ELF file and read back into the controller.

Note

To delete a compile cycle loaded into the controller, it is **not** sufficient to delete only the corresponding ELF file. If only the ELF file is deleted, the following data is retained in the retentive memory of the controller:

- Activation data: \$MN_CC_ACTIVE_IN_CHAN_<cc>[<n>]
 - Function-specific machine data
-

Procedure

Perform the following actions to delete a compile cycle:

1. In the user interface, go to the area "Commissioning" > "System data" and delete the ELF file in the "NC data > Compile cycles" directory.
2. Switch to the "Setup archives" area and create an NC archive.
3. Read the created NC archive back into the controller.

Note

If several compile cycles are loaded in the controller, the other compile cycles remain unchanged when a compile cycle is deleted according to the procedure described.

Distance control 1D/3D in position controller cycle (Run MyCC /CLCX)

4

4.1 Brief description

Function

The "Clearance control" technology function is used to maintain a one-dimensional (1D) or three-dimensional (3D) clearance required for technological reasons with the aid of an external sensor. The distance to be maintained is, for example, the distance between a laser machining head and the workpiece surface to be machined. If a force transducer is used instead of a clearance sensor, a pressing force can alternatively be maintained (e.g. for friction welding).

Function code

The code for the "Clearance control" technology function for function-specific identifiers of program commands, machine data, etc. is:

CLC/CLCX (= clearance control)

Function extension CLCX

With the function extension CLCX, the technology function "CLC" has been fundamentally revised.

- CLCX permits you to linearize the signal of the clearance sensor.
- The velocity voltage tables are replaced by clearance voltage tables.
- Real-time variables (\$A_DBx) facilitate diagnostics and operation.
- Integration of jump-up movements between laser cuts without explicit deactivation of the CLC avoids idle times for the synchronization of the IPO position.
- The "POS mode" is available in addition to the "GAIN mode". Especially a reapproach after a lift-off movement is executed faster via the POS mode. When the clearance setpoint is reached and maintained, the control behavior of the POS mode is rougher than that of the GAIN mode.

It is therefore possible to switch automatically between POS and GAIN mode.

Versions

The clearance control is offered with the following control variants:

- POS mode (Page 34); the clearance is approached via the clearance signal of the sensor and the evaluation of the actual positions of the axis encoders.
- GAIN mode (Page 35); the clearance is controlled via the clearance signal from the sensor and the control loop gain.

4.1 Brief description

Multiple CLCX instances

Up to four instances of the technology function "Clearance control" can be assigned to the active channels of a controller. It is also possible to use several CLCX instances in one channel (Multiple CLCX instances in different channels (Page 71)).

Compile cycle

The "Clearance control" technology function is a compile cycle.

A description of the system-specific availability and handling of compile cycles can be found under Installation and activation of loadable compile cycles (Page 19).

4.1.1 Function description

The further description of the functionality of the technology function "clearance control" is exemplified by the technology "laser cutting".

Laser cutting

During laser cutting, an expanded, parallel laser beam is guided via several mirrors or an fiber-optic cable to a focusing lens mounted in the laser machining head. The focusing lens focuses the laser beam at its focal point. Typical focal lengths are from 5 to 20 cm.

The position of the focal point in the workpiece is an extremely critical process parameter in laser cutting, which must be kept constant within a tolerance of $\leq 100 \mu\text{m}$.

The distance between the focal point and the workpiece, which is also a key process variable, is usually measured by means of a high-speed capacitive clearance sensor. The analog output voltage of the clearance sensor contains the information about the current distance of the sensor from the workpiece surface.

The output voltage of the clearance sensor is transferred as a digital input value via an analog I/O module to the controller, where it is linearized and converted into a distance value if required. Deviations from the clearance setpoint ultimately generate superimposed position and velocity setpoints for the motion axes of the machining head.

System overview

The following figure provides an overview of the system components required for clearance control:

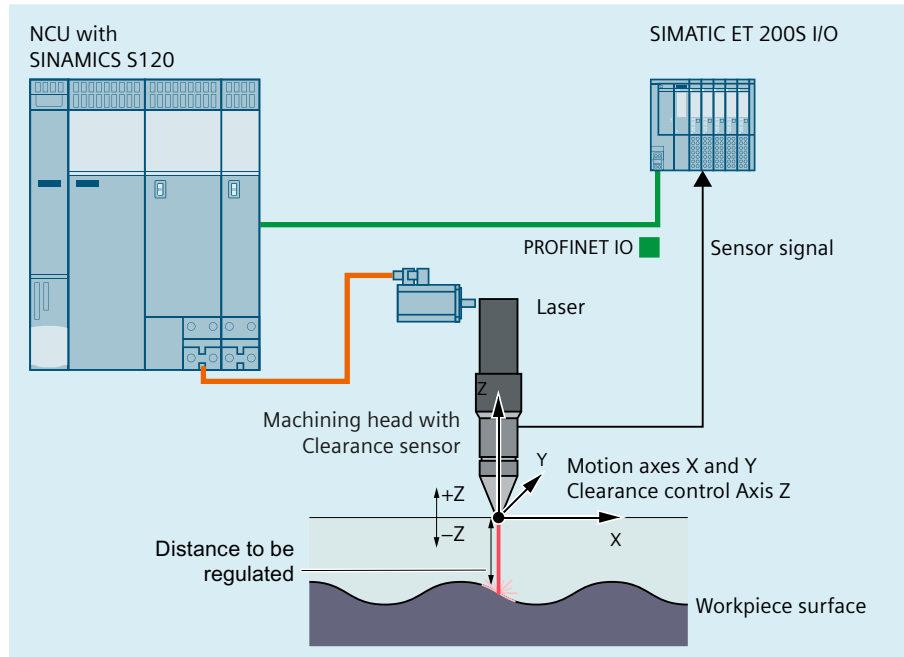


Figure 4-1 System components for clearance control with SINUMERIK NCU

1D machining

For 1D machining, only one axis is influenced by the clearance control. For example, axis Z, as shown in the machine configuration described in the system overview (see previous diagram). Clearance control acts only in the direction of the Z axis.

3D machining

Three linear axes are used to position the tool. A detailed description can be found under 3D clearance control - overview (Page 64).

4.2 Clearance control

4.2.1 POS mode

4.2.1.1 Overview

Properties of the POS mode

The system deviation of the clearance control is the difference between the programmed setpoint clearance and the current actual clearance reported by the clearance sensor.

In POS mode this system deviation is calculated together with the actual position of the axis to a target position for the clearance control axis. For this purpose, the delay time of the sensor in relation to the actual position of the axis encoders must be compensated for in the best possible way.

The POS mode does not cause overshooting, which depends on the size of the initial system deviation, and is therefore more suitable for approaching from a greater distance than the GAIN mode.

4.2.1.2 POS mode

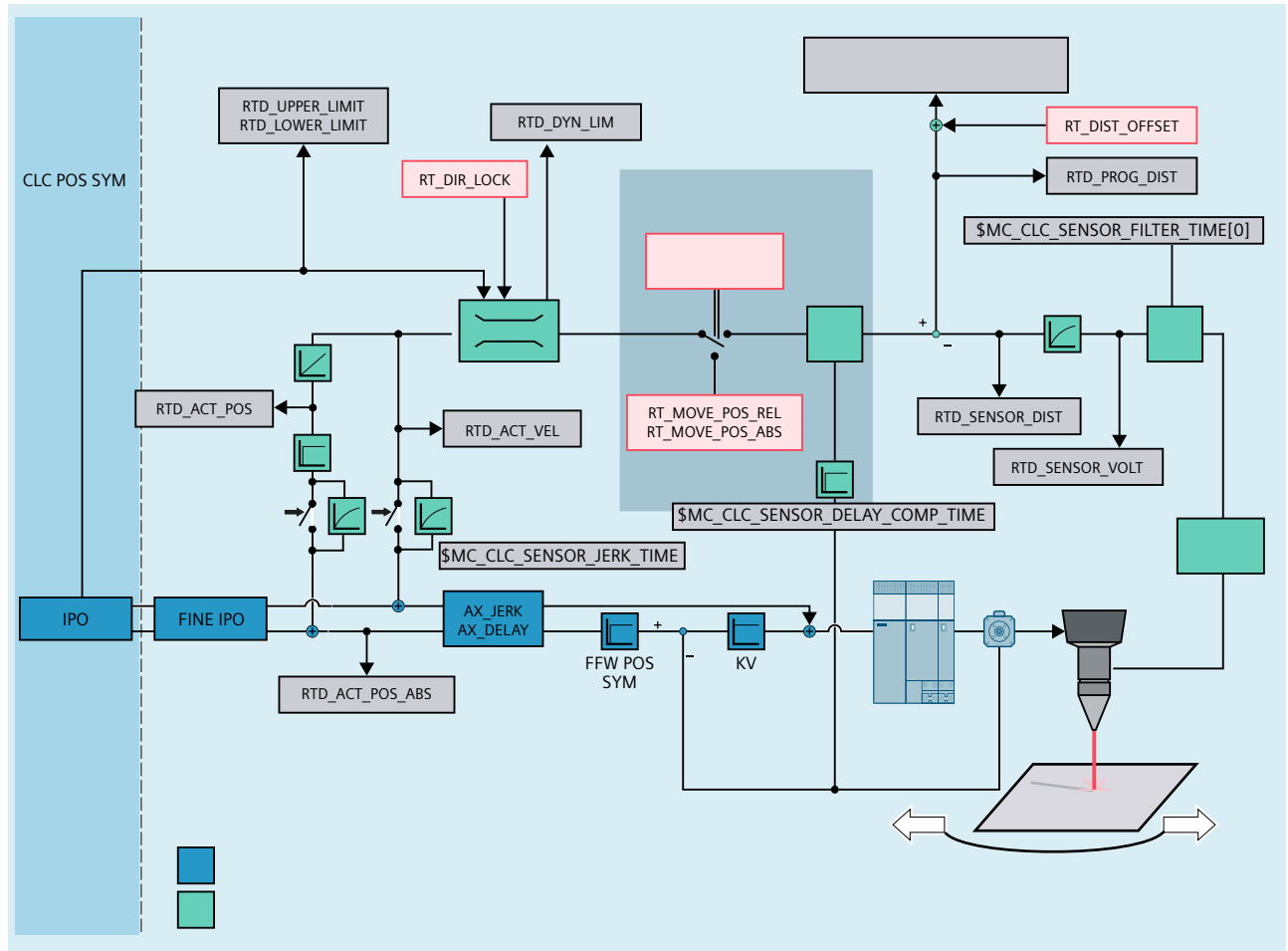


Figure 4-2 POS mode

4.2.2 GAIN mode

4.2.2.1 Control dynamics - GAIN mode

Control loop gain K_v

The system deviation of the clearance control is the difference between the programmed setpoint clearance and the current actual clearance reported by the clearance sensor. In GAIN mode, this system deviation is multiplied by the control loop gain K_v : The result is the control speed that moves the axis toward the clearance setpoint.

The control loop gain $K_{v_{CLC}}$ in the superimposed sensor control loop is defined as:

4.2 Clearance control

$$KV_{CLC} = \frac{\text{Speed [mm/min]}}{\text{System deviation [mm]}}$$

Especially when approaching from a great distance, i.e. with an initially large system deviation, the GAIN mode generates high control speeds which lead to overshooting of the axis position. The maximum adjustable control loop gain is limited by the dynamics of the closed control loop (sensor-control-axis) and the tolerable overshoot. Small values of the control loop gain cause time losses due to a slow approach.

Max. control loop gain

The maximum achievable control loop gain is determined by the following delay and reaction times of the overall system:

1. Reaction time of sensor
2. Delay time of the A/D converter
3. Delay/dead times of signal processing and transmission
4. Reaction time of position controller
5. Reaction times of speed and current controllers
6. Time constants of motor and mechanical system

Points 1, 3 and 4 are relevant in practice.

In total, the influencing variables result in an effective time constant. If the control loop gain is set too high in relation to this time constant, natural oscillations of the axis/axes to be controlled will occur in the range of a few hertz.

The aim of setting up the clearance control in GAIN mode is to adjust the decisive time constants in such a way that the highest possible control loop gain can be set without the occurrence of natural oscillations and with a tolerable overshoot amplitude when approaching.

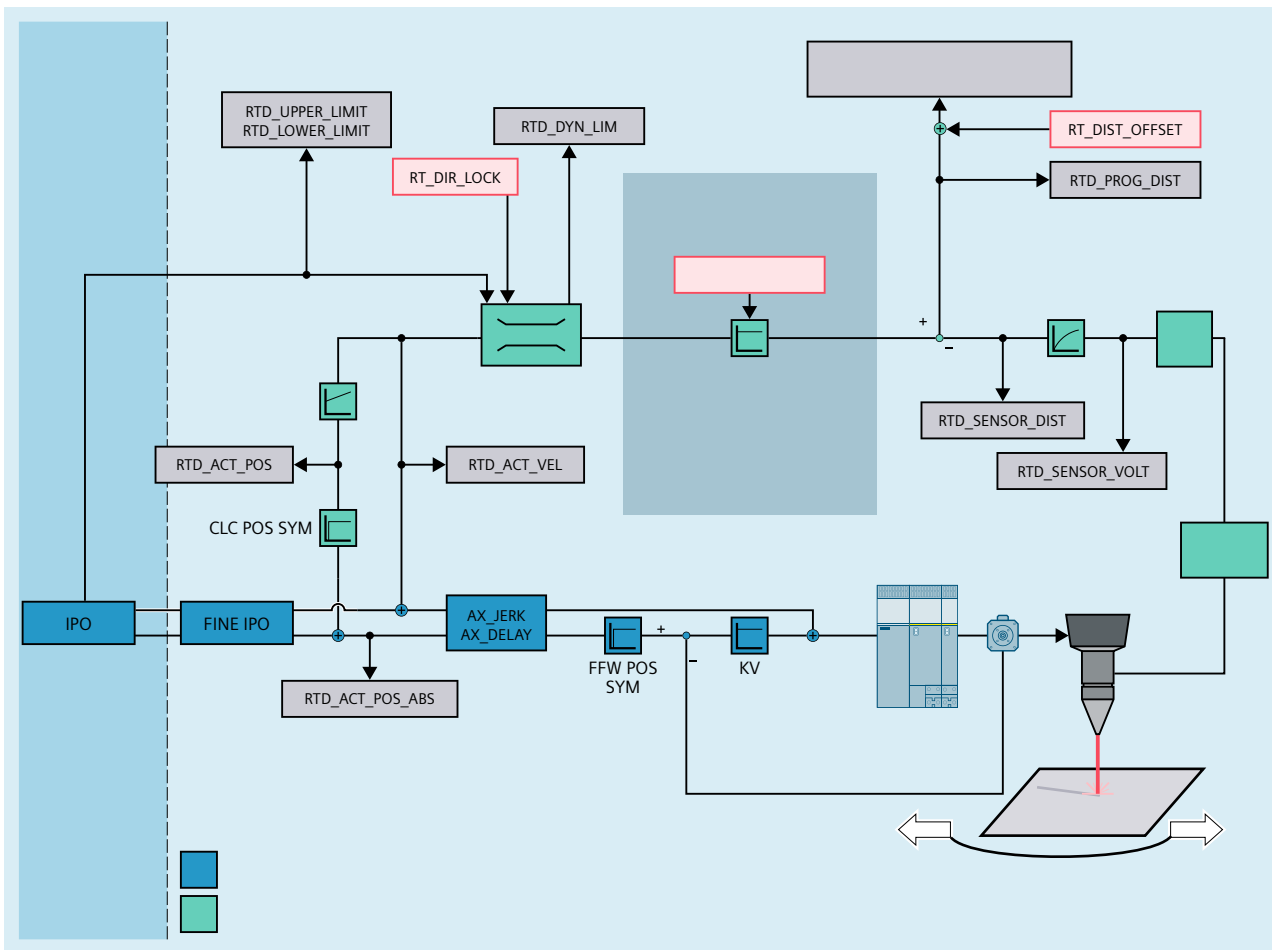


Figure 4-3 GAIN mode

4.2.3 Dead times in the clearance control

Dead time

In order to achieve a highly dynamic control behavior, the clearance control is implemented in the position controller cycle layer of the NC kernel.

For I/O modules that are connected via the isochronous fieldbus (PROFINET IRT) and when using SINAMICS drives there is a system-dependent dead time T_{dead} of:

$$T_{\text{dead}} = 2 * \text{position controller cycle} + 2 * \text{speed controller cycle} + \text{PROFINET } T_i$$

In order to achieve an optimum dynamic response of the clearance control, the reaction time (or dead time) of the sensor should not be significantly longer than the system-related dead time.

4.2.4 Optimizing the control behavior

Optimizing the control behavior

If the control behavior of the axis becomes too hard due to feedforward control, the control behavior can be optimized. Instructions for using the jerk filter can be found under Startup (Page 40).

The speed filters of the SINAMICS S120 drive provide additional damping capabilities:

- Parameter 1414 ff. (time constant, speed setpoint filter 1, 2)

Note

Each damping measure in the axis control loop results in an additional contribution to the increase of the total time constant of the superimposed sensor control loop!

Further information:

A complete description of the velocity feedforward control is provided in:
Function Manual Monitoring and compensating; Compensations, dynamic feedforward control (following error compensation)

4.3 Technological features of clearance control

Clearance control is characterized by the following technological properties:

- **Sensor characteristic**
The voltage/clearance characteristic of a sensor can be specified with up to 16 breakpoints.
- **Sensors**
Several sensors with different gain characteristics can be used. The active sensor characteristic can be switched over block-synchronously by means of a language command in the part program.
- **Control loop gain of the clearance control (GAIN mode)**
The control loop gain of the clearance control, parameterized by NC machine data, can be changed block-synchronously by a voice command in the part program. In addition, the real-time variable CLC_RT_GAIN_FACTOR can multiply the gain.
- **Motion limitation**
The lower and upper limits configured in the NC machine data for the axis movements induced by clearance control can be changed block-synchronously by means of a language command in the part program.
An alarm appears when a limit is reached. The alarm response can be parameterized (stop of the traversing motions or display only). The current position offset can be frozen via the real-time variable CLC_RT_TECH_CMD (or a PLC signal).
- **Switch-off behavior**
The switch-off behavior of the clearance control can be programmed either with synchronization to the current axis positions (no compensating motion) or with compensating motion of the axes back to the last programmed IPO positions.
- **Programmable clearance setpoint**
The clearance setpoint can be programmed block-synchronously. An additional offset can be added via the real-time variable CLC_RT_DIST_OFFSET.

- **State data of the clearance control**

The following real-time variables can be read at any time:

CLC_RTD_SENSOR_VOLT, the current sensor signal in Volt

CLC_RTD_SENSOR_DIST, the distance calculated over the selected characteristic

- **Sensor signal**

The sensor signal can be smoothed by a sliding average value filter with adjustable time constant:

62525 \$MC_CLC_SENSOR_FILTER_TIME[0]

- **Variables for synchronized actions**

Additional functions such as jump-up or blocking of sensor motion can be controlled via the real-time variable CLC_RT_TECH_CMD without having to switch the CLC function off and on again.

4.4 Sensor collision monitoring

Tip-touch signal from the sensor

If the clearance sensor used has an additional "tip-touch" signal for detecting a collision of the sensor with the workpiece to be machined, this signal can be made available to the clearance control via a digital NC peripheral input.

As a reaction to this signal, the clearance control performs a fast retraction movement in all clearance-controlled axes. The retraction movement is executed independently of the current value of the velocity override at traversing velocity in positive control direction by a specified distance. At the same time, the alert 75007 is issued and a real-time variable indicates the "tip-touch" state. Synchronized actions can be used to program the desired reaction to the path. After the cause of the collision has been eliminated, the retracted axis can be reset to control mode via the real-time variable CLC_RT_TECH_CMD.

Parameterization

The digital peripheral input, which the tip-touch signal is wired to, is assigned to the clearance control via the following machine data:

MD62504 \$MC_CLC_SENSOR_TOUCHED_INPUT (digital peripheral input for "sensor collision" signal)

The digital input is specified via the same input numbering that is used to address the corresponding system variable \$A_IN[input number].

If a negative input number is specified, the CLCX evaluates the tip-touch signal internally in inverted form (wire break protection). In this case a falling signal edge triggers the tip-touch reaction.

Internal tip-touch detection

If tip-touch detection is programmed, the clearance signal from the sensor can be monitored to determine whether it falls below a set distance threshold: Machine data 62526

\$MC_CLC_TIP_TOUCH_DISTANCE.

4.5 Startup

To avoid false tripping, the signal should be averaged over a suitable time (e.g. 30 ms) before comparison with the distance threshold using the mean value filter available for this purpose:

Machine data 62525 \$MC_CLC_SENSOR_FILTER_TIME[1].

The response of the CLCX to the undershooting of the tip-touch distance threshold is the same as when an external tip-touch signal is received.

4.5 Startup

Compile cycle

Before setting up the technology function, ensure that the corresponding compile cycle has been loaded and activated (see Installation and activation of loadable compile cycles (Page 19)).

General relationships

The clearance control works within the position controller task. The WCS and MCS positions on the standard HMI screen do **not** show the offset generated by the clearance control. Only if you switch off the CLC function with the CLC(0) command will the IPO positions displayed by default be resynchronized, and thus the axis positions actually reached will be displayed.

The clearance control generates the overlaid movement within the CLC coordinate system:

- The zero point is the position at which the CLC function was activated.
- The arguments of CLC_LIM (<lower limit>, <upper limit>) refer to this coordinate system. CLC_RTD_ACT_POS also refers to the CLC coordinate system.
Only the real-time variable CLC_RTD_ACT_POS_ABS shows the axis position in the regular MCS coordinates. In the 3D case, this variable shows the same position as CLC_RTD_ACT_POS.

Assign CLCX instance to a channel

A CLCX instance must be assigned to one channel each for activation. Several CLCX instances are described under Multiple CLCX instances in one channel (Page 71).

Activating an instance in a channel

MD60940 \$MN_CC_ACTIVE_IN_CHAN_CLCX[0] = 'H1'

- Bit n = 1
- n = channel number - 1
 - Bit0 = 1st channel
 - Bit1 = 2nd channel, etc.

Notes

If you want CLCX to act on a single axis only (and not in a 3D application), preferably do not configure the CLC axis as a geometry axis (\$MC_AXCONF_GEOAX_ASSIGN_TAB[2]=0). This has the advantage that the position synchronization required when CLCX is deactivated cannot affect XY path motion functions, but requires a change in programming. When the Z axis is no longer a geometry axis, the geometry axis name of the Z axis (typically "Z") can no longer be used to program rotating frames within the XY plane. Therefore, program rotations with CRPL(1, 0) instead of CROT(Z, 0).

Jerk filter

Do not apply the standard jerk filter (MD32400 \$MA_AX_JERK_ENABLE=0) to limit the axis jerk caused by the CLCX motion, as this filter will cause a delay within the sensor control loop. If you need to limit the axis jerk when moving the axis with the regular interpolator, program SOFT movement mode instead and limit the jerk with MD32431 \$MA_MAX_AX_JERK. CLCX provides a separate jerk filter (MD62518 \$MC_CLC_SENSOR_JERK_TIME) to limit the jerk during CLCX jerk movements. If a filter time constant greater than zero is set there, the CLCX automatically switches the filter on and off in a sliding manner according to the current situation:

- The jerk filter is switched on when approaching the plate and in case of predefined "jumps" of the axis; in normal sensor-controlled control operation it is switched off. Nevertheless, the specified filter time should also be kept as short as possible for this jerk filter (in the <20 ms range).

4.5.1 Setup steps

Setting up CLCX

To set up CLCX for optimum performance and productivity, it is recommended that the following steps be performed:

- Optimize the axis/axes by setting up Feed-Forward, DSC and other functions that provide the best possible precision and dynamics.
- Set up the analog input of the sensor for isochronous operation (PROFINET IRT), see Setting up the analog sensor input (Page 43).

4.5 Startup

- Linearize the clearance sensor input using the machine data tables 62510 __MC_CLC_SENSOR_VOLTAGE_TABLE_1[0... 15] and 62511 __MC_CLC_SENSOR_DIST_TABLE_1[0...15], see also Linearizing the signal of the clearance sensor (Page 46).
 - To obtain optimum performance of the CLCX function, the sensitive clearance range of the sensor should be as large as possible (preferably up to 20 mm) and linearized as well as possible.
 - The highest entry in the clearance table and the correct linearization of the distance within the entire table range are particularly important in order to obtain the optimum sensor-controlled approach movement from a far-away starting position to the workpiece surface.
 - The linearization of the sensor characteristic curve can be automated with an appropriate part program. CLCX simplifies a quick sensor calibration check, since both the analog input voltage information and the resulting clearance of the currently selected table available are available, even when CLCX is switched off.
- When using the POS mode and the enhanced approach function, the following is recommended:
 - The time delay between the relative position measurement of the sensor and the absolute axis position detection of the encoder must be compensated for in the best possible way, see Compensation of the time delay of the sensor (Page 47).
- The commissioning and testing of all functions that do not directly depend on the axis dynamics and the specific sensor characteristic can also be carried out via Create MyVirtual Machine with simulated axes and a simulated sensor.

4.5.2 Activating the technological function

The technology function is activated via the machine data:

MD60940 \$MN_CC_ACTIVE_IN_CHAN_CLCX[0], bit n = 1

n = channel number - 1; Bit0 = 1st channel, Bit1 = 2nd channel, etc.

Note

A maximum of four instances of the CLCX technology function can be activated on one NCU.

4.5.3 Configuring the memory

The CLCX technology function requires additional data in the NC-internal block memory. For each CLCX instance that is active in the channel, the values specified must be entered in the following memory-configuring channel-specific machine data. If the memory requirement for other compile cycle functions is already entered in this MC, the values must be added to the existing entries:

- MD28090 \$MC_MM_NUM_CC_BLOCK_ELEMENTS: + **4** (number of block elements for compile cycles)
- MD28100 \$MC_MM_NUM_CC_BLOCK_USER_MEM: + **20** (size of block memory for compile cycles (DRAM), in KB)

4.5.4 Sensor input and sensor signals

4.5.4.1 Setting up the analog sensor input

Recommended hardware

It is recommended to convert and read in the analog input signals of the capacitive clearance sensor with the following hardware:

Table 4-1 Recommended SIMATIC modules

Article number	Device	Description
6ES7 151-3BA60-0AB0	SIMATIC ET200S IM151-3PN HS Firmware V3.0	Profinet Interface Module
6ES7 138-4CA01-0AA0	SIMATIC ET200S PM-E DC24V	Power Module
6ES7 134-4FB52-0AB0	SIMATIC ET200S 2AI U HS	Module with 2 analog inputs

In order to achieve the best dynamic response of the clearance control, the analog input, which contains the clearance information, must be isochronously transferred to the NC.

Configure the PROFINET IRT operation in the "HWConfig" area of the Simatic Manager or in the corresponding function of the TIA Portal for:

- the bus segment
- the interface module (the head module)
- the analog input module to which the sensor is connected
- the digital input for tip-touch detection of the sensor - if available

4.5 Startup

4.5.4.2 Parameterizing the input signals

The following input signals must be parameterized in the machine data:

- Input voltage of the clearance sensor
 - 1 analog input
- "Sensor-tip-touch" input signal (optional)
 - 1 digital input

Analog input

The PROFINET address of the analog input module configured on the PLC side must be entered on the NC side in the following machine data:

- MD10362 \$MN_HW_ASSIGN_ANA_FASTIN (per analog input) (hardware assignment of the fast analog NC inputs)

Digital input

Unless one of the onboard inputs of the NCU is used for this purpose, the PROFINET address of the digital input module configured on the PLC side must be entered in the following machine data:

- MD10366 \$MN_HW_ASSIGN_DIG_FASTIN (per-bit input block) (hardware assignment of the external digital NC inputs)

A complete description of the analog and digital inputs can be found in:

Further information

Function Manual Basic Functions; digital and analog NC I/O

4.5.4.3 Simulation of the sensor signals

Preparing a simulation

The properties of the single-axis CLCX can be tested without a real clearance sensor and, if necessary, even without real axes. In this case, a sensor/plate simulation integrated in the CLCX software closes the control loop.

The axes are switched to simulation mode as usual with the following machine data:

- 30240__\$MA_ENC_TYPE[Z]=0
- 30130__\$MA_CTRLOUT_TYPE[Z]=0

The simulation can only simulate the fundamental function of the sensor, but not the real dynamic response of the clearance control.

To apply the simulation, the following definitions must be added to the definition file, which also contains the other GUD definitions of the CLCX:

- DEF CHAN REAL CLC_SIM_SHEET_POS[3]
- DEF CHAN REAL CLC_SIM_SENSOR_DELAY[2]

A restart of the NC kernel is required for the new GUD definitions to take effect in the CLCX. Then the simulation of sensor and sheet metal (workpiece) can be used.

Defining the sensor simulation

The simulation only works if a simulated sensor delay time not equal to zero is set in the following GUD variable:

Programming example

```
CLC_SIM_SENSOR_DELAY[0] = 0.001234
; activates the simulation and uses a simulated
; sensor delay of 1.234 ms
```

Note

The content of the GUD variable CLC_SIM_SENSOR_DELAY[0] is intentionally deleted after each NC restart to deactivate the sensor simulation by default after the restart. This is to avoid the accidental use of the simulated sensor when a real sensor is connected.

Defining the sheet metal

To simulate the sheet metal, simply enter the desired sheet metal position in MCS coordinates into this GUD variable:

Programming example

```
CLC_SIM_SHEET_POS[0] = -30 ; simulates a sheet metal in the MCS position -30 mm.
```

Make sure that the simulated sheet metal is positioned UNDER the axis position where you switch on the CLC.

The other GUD elements can be applied optionally:

Programming example

```
CLC_SIM_SHEET_POS[2] = 0.005 ;simulates white noise of the sheet metal position of 5 µm
CLC_SIM_SHEET_POS[1] = 0.1 ; simulates a linear up/down movement of the sheet metal of
0.1 mm.
CLC_SIM_SENSOR_DELAY[1] = 2.0 ;... with a duration of 2 seconds
```

If you change the content of a GUD variable within a part program, note that it may take a short time until the CLCX actually evaluates a changed GUD value.

In order to obtain stable test conditions, it is therefore recommended to program a dwell time of one second (G4 F1) after a change of the GUD values or before switching on or enabling the CLCX motion.

In order for the simulation of the closed CLC control loop to behave as realistically as possible dynamically, you should compensate the set delay of the simulated sensor in the machine data \$MC_CLC_SENSOR_DELAY_COMP_TIME in the same way as is required for a real sensor:

```
$MC_CLC_SENSOR_DELAY_COMP_TIME = CLC_SIM_SENSOR_DELAY[0]
```

4.5.4.4 Linearizing the signal of the clearance sensor

Normalizing the analog input value

The first step in providing a linearized sensor signal is to normalize the analog input value:

- Connect a known static analog voltage to the sensor input.
- Adjust the value of MD62503 \$MC_CLC_ANALOG_IN_HW_NORM_FACT until the variable CLC_RTD_SENSOR_VOLT correctly indicates the voltage in V applied at the input. This can also be done without the CLCX being switched on by program command, as the variable is updated as soon as CLCX is activated via the PowerOn machine data.
- The same applies to the variable CLC_RTD_SENSOR_DIST, which displays the linearized sensor distance resulting from the applied input voltage with the currently effective sensor characteristic curve:

```
MD62510 $MC_CLC_SENSOR_VOLTAGE_TABLE_1[0... 15] MD62511
```

```
$MC_CLC_SENSOR_DIST_TABLE_1[0... 15]
```

or

```
MD62512 $MC_CLC_SENSOR_VOLTAGE_TABLE_2[0... 15] MD62513
```

```
$MC_CLC_SENSOR_DIST_TABLE_2[0... 15]
```

Calibration of the sensor table via "CLC_RTD_SENSOR_VOLT" and "CLC_RTD_SENSOR_DIST"

Recalibrating the sensor means storing the currently effective characteristic of the sensor in a CLCX sensor characteristic curve. For this purpose, the distance breakpoints to be measured are first selected by entering them in ascending order starting with zero, e.g. in the table values of the machine data \$MC_CLC_SENSOR_DISTANCE_TABLE_1. In areas where the characteristic curve is strongly non-linear, a correspondingly higher number of breakpoints should be selected.

Starting with the zero distance, at which the sensor should almost touch the sheet metal, the axis is moved upwards step by step by the entered distances via a part program with the CLCX function switched off. For each distance the voltage output by the sensor is written to the corresponding element of the voltage table \$MC_CLC_SENSOR_VOLTAGE_TABLE_1.

- Start with the greatest possible approximation
Greatest possible approximation means almost contact of the sensor with the sheet metal (zero distance).
- Gradually increase the distance and transfer the measured voltage to the element of the voltage table corresponding to the distance.

Note

- The first entry within the clearance table must be zero. If it is not possible to measure the sensor voltage at this (contact) position, a corresponding estimated sensor voltage should be entered in MD62510 \$MC_CLC_SENSOR_VOLTAGE_TABLE_x[0].
 - All subsequent clearance and voltage entries must be monotonically increasing.
-

The table ends with the entry in index 15 or with a clearance entry of 0.0 under an index smaller than 15.

For sensor-controlled approximation to the sheet metal from a distance greater than the sensor's sensing range, CLCX uses the maximum valid clearance in the table to limit the approach velocity so that the axis can be braked to the programmed clearance setpoint.

If the evaluation electronics of the sensor already linearizes the clearance signal well, the characteristic can consist of only two entries: the voltage at zero distance and the voltage at the largest measurable distance. This saves computing time in the CLCX for the evaluation of unnecessary linear intermediate points.

Improving the signal

If the input signal displayed in "CLC_RTD_SENSOR_VOLT" contains a high noise level, first try to improve the electrical quality of the signal by better shielding and grounding of the electrical connection. To use a more reliable sensor signal during calibration, you can temporarily increase the time constant of the mean value filter during calibration:

```
62525 __MC_CLC_SENSOR_FILTER_TIME[0]
```

Before the clearance control is switched on, the filter time must be reset. Too large a time constant of the mean value filter causes a sensor delay (half of the mean value time constant), which impairs the dynamic response of the control loop.

If there is a need for permanent filtering of the input signal, no more than a few milliseconds should be entered.

If you check the variable "CLC_RTD_SENSOR_DIST" in the display, note that these are calculated with the sensor clearance table that was last activated.

You can switch to the second sensor table, for example, by starting the following part program:

Example code

```
CLC_RT_TECH_CMD=120      ; Block CLCX motion to CLC(1)
CLC(1)
CLC_SEL(2)               ; Activate table 2
CLC(0)                   ; Deactivate CLC. Table 2 remains active
M30                      ; RESET deletes CLC_RT_TECH_CMD (see Figure 2)
```

4.5.4.5 Compensation of the time delay of the sensor

Time delay

The clearance information that the CLCX receives from the clearance sensor includes a time delay in relation to the actual position of the axis read by the encoder. If this delay is constant, its effect can be compensated.

- When using the "POS mode" (CLC_RT_CTRL_MODE=0), this compensation will significantly improve the quality of the sensor-controlled motion - especially when the sensor approaches the sheet metal from a greater distance.
- It is also important for the "GAIN mode" (CLC_RT_CTRL_MODE=1) to compensate for the sensor delay: As long as the distance during the approach phase is greater than the range of the sensor, the correct determination of the approach velocity depends on the correct compensation of the delay.

4.5 Startup

CLCX provides a function to determine and compensate for this delay. To use this function, the following GUD variables must be defined in one of the definition files MGUD, UGUD, GUD4...GUD7:

```
DEF NCK REAL CLC_SIN_GEN_FREQ
DEF NCK REAL CLC_SIN_GEN_AMPL
DEF CHAN REAL CLC_SENSOR_MATCH[ 2 ]
```

Only after the next NCK restart does the CLCX gain access to these variables.

To measure the delay, place a piece of sheet metal under the sensor and position the CLCX axis at a suitable height (approx. 1 mm) above this sheet metal. Make sure that the sheet metal does not vibrate during the test. If necessary, you must stabilize the sheet metal with clamps or a weight.

Using a part program

The following part program starts the integrated function generator of the CLCX. It generates a sinusoidal movement of the CLCX axis.

- Adjust the amplitude (CLC_SIN_GEN_AMPL) so that the sinusoidal motion is completely within the sensitive range of the sensor.
- Carefully ensure that the SynAct variable CLC_RTD_DYN_LIM remains at ZERO throughout the evaluation of the phase. This ensures that no dynamic limitation of the axis movement occurs during the measurement of the sensor deceleration.
- If dynamic limiting occurs, reduce the frequency or amplitude of the stimulating sinusoidal movement. Dynamic constraints interfere with the correct determination of the delay.
- During the sine wave movement, the time delay of the sensor relative to the encoder is calculated after each period. The evaluation can be monitored online in the GUD CLC_SENSOR_MATCH[0]. The number of periods already evaluated is displayed in CLC_SENSOR_MATCH[1].

Let the measurement run until the monitored value in CLC_SENSOR_MATCH[0] is sufficiently stable.

Example program

```
CLC_SIN_GEN_AMPL=5.0 ; Generates a sinusoidal motion with +-5mm amplitude.
                        ; This must lie within the sensitive range of the sensor.
CLC_SIN_GEN_FREQ=0.5 ; The frequency used is 0.5 Hz
G4 F1                 ; Wait until the GUD values in the CLCX have become safely effective
CLC_LIM( -30, 10 )    ; Here it is assumed that a sheet of metal is located max. 50 mm
                        ; below the position where the CLCX was switched on.
CLC_DIST=1,0          ; The clearance setpoint for this sheet metal is 1 mm.
CLC(1)                ; Activate CLC
G4 F1                 ; Wait for the axis to approach the clearance setpoint
whenever TRUE CLC_RT_TECH_CMD=99 ; Starts the sine movement
M0                    ; Monitor calculated delay in GUD CLC_SENSOR_MATCH[0]
                        ; Check if the measured value is stable against small variations of
amplitude and frequency.
M30
```

- It is important that the stroke of the sinusoidal movement, which is twice as large as CLC_SIN_GEN_AMPL specifies, remains completely within the sensitive range of the sensor.
- Select CLC_SIN_GEN_AMPL so that it corresponds to about one third of the largest entry of the sensor characteristic curve.
Example:

$$\text{CLC_SIN_GEN_AMPL} = 0.354 * \$\text{MC_CLC_SENSOR_DIST_TABLE}[\text{<highestIndexUsed>}]$$
reaches 70% of the sensitive sensor range.
If CLC_RTD_PROG_DIST does not exceed the remaining 30%, the delay can be evaluated correctly.
- After a few periods of sine movement executed within the M0 block, the GUD variable CLC_SENSOR_MATCH[0] displays the evaluated delay in seconds.

GUD/CLC_DISTANCE[0]	actClcPos	D	-45.986
GUD/CLC_DISTANCE[1]	minClcPos	D	-49
GUD/CLC_DISTANCE[2]	maxClcPos	D	-39
GUD/CLC_SIN_GEN_AMPL	Sinus-Gen. Amplitude in mm	D	5
GUD/CLC_SIN_GEN_FREQ	Sinus-Gen. Freq in Hz	D	0.5
.../CLC_SENSOR_MATCH[0]	Delay of Sensor rel. to Enc	D	0.000123

Figure 4-4 Determination of the sensor delay

Determine and adjust measured data

Modern clearance sensors connected via an isochronous PROFINET IO A/D converter should show a delay in the range of a few milliseconds or even less, largely independent of frequency.

The example above shows a time delay of 0.123 ms.

If you get a stable and positive delay during the measurement, transfer the measured sensor delay in seconds as compensation value to the following machine data:

- MD62519 \$MC_CLC_SENSOR_DELAY_COMP_TIME = CLC_SENSOR_MATCH[0]
- Execute NEWCONFIG or RESET to make the compensation value effective.

CLCX compares the position information read by the sensor with the position information of the encoder.

This is required for the "POS mode" and usually for any sensor-controlled approach starting at a distance outside the sensitive range of the sensor.

If you repeat the measurement with an already correctly determined entry in \$MC_CLC_SENSOR_DELAY_COMP_TIME, CLC_SENSOR_MATCH[0] will still show the delay of the uncompensated sensor.

4.5.5 Parameter settings for clearance control

Part programs

For function-specific naming of the programmable NC address CLC_DIST, the standard name OMA1 must be reconfigured in the following machine data:

- MD10712 \$MN_NC_USER_CODE_CONF_NAME_TAB[0] = "OMA1" (list of re-configured NC codes)
- MD10712 \$MN_NC_USER_CODE_CONF_NAME_TAB[1] = "CLC_DIST"

1D/ 3D clearance control

The choice between 1D or 3D clearance control is based on the machine data:

- MD62500 \$MC_CLC_AXNO = <n> (Axis assignment of clearance control)
 - <n> > 0: 1D clearance control. <n> is the channel axis number of the clearance-controlled axis
 - <n> = -1: 1st 5-axis transformation configured in the channel
 - <n> = -2: 2nd 5-axis transformation configured in the channel
 - <n> = -3: 3rd 5-axis transformation configured in the channel
 - <n> = -4: 4th 5-axis transformation configured in the channel

Input signals

The input signals of the clearance sensor parameterized above are made known to the clearance control via the following machine data (see also Chapter "Parameterizing the input signals (Page 44)"):

- MD62502 \$MC_CLC_ANALOG_IN = <n> (Analog input for clearance control)
<n> = input number, corresponds to the addressing of the system variable \$A_INA[<n>]
- MD62504 \$MC_CLC_SENSOR_TOUCHED_INPUT = <m> (Input bit assignment for the "sensor collision" signal)
<m> = input number, corresponds to the addressing of the system variable \$A_IN[<m>]

Exact stop

In order to satisfy the programmed block change condition "Exact stop coarse/fine reached" (G601/G602), the traversing velocity generated by the clearance control in the clearance-controlled axes must be lower than the standstill velocity tolerance at least for the duration of the standstill delay time.

In order to optimize the block change time with active clearance control, the following machine data must be adjusted if necessary:

- MD36000 \$MA_STOP_LIMIT_COARSE[<x>] (Exact stop coarse)
- MD36010 \$MA_STOP_LIMIT_FINE[<x>] (Exact stop fine)
- MD36020 \$MA_POSITIONING_TIME[<x>] (Delay time exact stop fine)

- MD36040 \$MA_STANDSTILL_DELAY_TIME[<x>] (Delay time for standstill monitoring)
- MD36060 \$MA_STANDSTILL_VELO_TOL[<x>] (Threshold velocity/speed 'Axis/spindle in stop')
<x> = identifier of the clearance-controlled machine axis

4.5.6 Configuration of the basic MD of CLCX

Basic machine data

Table 4-2 Machine data for configuration of CLCX

Machine data	Description
MD62500 \$MC_CLC_AXNO	> 0: Activates the 1D clearance control for the channel axis with the specified axis number. This axis must not be a modulo rotary axis. n < 0: activates the 3D clearance control. A prerequisite for activating the 3D clearance control is that at least one of the first four 5-axis transformations possible in the channel is configured. -1: the first 5-axis transformation configured with \$MC_TRAFO_TYPE_n in the channel (16 ≤ transformer type ≤ 149) defines the three Cartesian axes on which clearance control acts. ... -4: the fourth 5-axis transformation configured in the channel is selected.
MD 62502 \$MC_CLC_ANALOG_INPUT	Number of the analog input to which the clearance sensor is connected.
MD62507 \$MC_CLC_A_DB_RT_VAR_START_INDEX	Start address of the real-time variables \$A_DBR / \$A_DBB (e.g. 1000) The CLCX function reads and writes to the \$A_DBx variables that follow this start address. Therefore 100 bytes from this start address should not be used for other functions.
MD \$MC_CLC_SENSOR_TOUCHED_INPUT	Number of the digital input to be used for collision monitoring (provided the sensor outputs a tip-touch signal).
MD62528 \$MC_CLC_PROG_ORI_AX_MASK	Axis mask for simulated axes when using the 3D-CLC function with freely programmed direction specification.
MD28090 \$MC_MM_NUM_CC_BLOCK_ELEMENTS	Configures the number of block elements that can be used for compile cycles. Each instance of the CLCX in the channel requires 4 block elements.
MD28100 \$MC_MM_NUM_CC_BLOCK_USER_MEM	Configures the size of the dynamic memory available for compile cycles in the internal block. Each instance of the CLCX in the channel requires 20 KB.
MD28150 \$MC_MM_NUM_VDIVAR_ELEMENTS	Number of block elements available for writing PLC variables. Recommended value for programming synchronized actions to use the CLCX function: 20

4.5 Startup

"New" machine data for CLCX

Table 4-3 Additional machine data for CLCX

Number	Identifier: \$MC_	Description
62503	CLC_ANALOG_IN_HW_NORM_FACT	Normalization of the internal voltage values to the input voltage in Volts
62507	CLC_A_DB_RT_VAR_START_INDEX	Start index of the CLCX real-time variable \$A_DBx
62509	CLC_DISTANCE_SETPOINT	Defines a default distance that is used after reset as long as no distance has been explicitly programmed.
62511	CLC_SENSOR_DIST_TABLE_1 [0... 15]	First clearance table for linearization of the sensor characteristic curve
62513	CLC_SENSOR_DIST_TABLE_2 [0... 15]	Second clearance table for linearization of the sensor characteristic curve
62514	CLC_DISTANCE_CTRL_GAIN	Defines the default value of the CLC circuit gain. This value is effective in "GAIN mode" after reset as long as no GAIN value has been explicitly programmed.
62515	CLC_PLUS_DIR_GAIN_FACTOR	Increase the gain factor if the current distance of the sensor becomes smaller than the programmed distance setpoint.
62518	CLC_SENSOR_JERK_TIME	Jerk filter for smoothing the non-sensor-controlled CLCX motion.
62519	CLC_SENSOR_DELAY_COMP_TIME	Value to compensate for the delay time of the sensor used. This adjustment is required for the sensor-guided approach from large distances and for the POS mode.
62525	CLC_SENSOR_FILTER_TIME	- Index0: Time constant of the mean value filtering of the sensor input signal. - Index1: Time constant of the mean value filtering for internal calculation of the tip-touch signal.
62526	CLC_TIP_TOUCH_DISTANCE	Threshold value for triggering internal tip-touch monitoring.
62527	CLC_MODE_SWITCH_DIST_ERROR	Threshold value for automatic switching from POS mode to GAIN mode and vice versa. An entry >0 activates the automatic switchover.

Table 4-4 List of redeclared machine data for CLCX

Number	Identifier:	Description
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[0] - = "OMA1"	
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[1] - = "CLC_DIST"	

"Old" machine data for CLC

This machine data is no longer provided or used by CLCX.

Table 4-5 CLC machine data that is no longer required

Number	Identifier: \$MC_	Description
62511	CLC_SENSOR_VELO_TABLE_1 [0 ... 15]	First velocity table for linearization of the sensor characteristic curve
62513	CLC_SENSOR_VELO_TABLE_2 [0 ... 15]	Second velocity table for linearization of the sensor characteristic curve
62522	CLC_OFFSET_ASSIGN_ANAOUT	Assignment of internal additional analog value to sensor signal
62523	CLC_LOCK_DIR_ASSIGN_DIGOUT	Assignment of digital output for interlock CLC

List of redeclared machine data for CLC.

Table 4-6 List of redeclared machine data for CLC

Number	Identifier:	Description
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[0] = "OMA1"	-
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[1] = "CLC_GAIN"	-
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[2] = "OMA2"	-
10712	\$MN_NC_USER_CODE_CONF_NAME_TAB[3] = "CLC_VOLT"	-

4.5.7 Definition of the real-time variables for synchronized actions

Real-time variables

CLCX uses the following two data types to display and operate its own real-time variables:

- "\$A_DBR; variables of data type "REAL".
- "\$A_DBB" variables of data type "BYTE".

The individual \$A_DBx variables are located at fixed byte offsets starting with the configured start address (via MD62507 \$MC_CLC_A_DB_RT_VAR_START_INDEX). For clear use of the variables in synchronized actions, it is recommended to assign functionally descriptive names to the \$A_DBx variables using MACRO definitions. In the "NC/PLC variables" operating menu of the SINUMERIK Operate, the same names can be entered manually in the "Comment" column of the variable list. This view allows the variables to be displayed and operated during commissioning.

The name selection of this documentation uses the prefix CLC_RT* for variables that require input and CLC_RTD* for variables that are only used to display states of the CLCX. RTD stands for "Real-Time Display". The SINUMERIK Operate allows you to save or load a commented variable view.

4.5 Startup

In the delivery package of the CLCX you will find the file "clcSynActVar.xml". If you adapt this file with regard to the variable start address you have selected and copy it to the "HMI Data/Template/User/VariableMasks" directory, you can directly load the variable view shown in the following figure.

NC/PLC variables clcxSynActVar*				Operand
Variable	Comment	F	Value	+
\$A_DBR[1828]	CLC_RTD_SENSOR_VOLT	D	8.25	Operand
\$A_DBR[1832]	CLC_RTD_SENSOR_DIST	D	8.5	-
\$A_DBR[1836]	CLC_RTD_PROG_DIST	D	8	Insert variable
\$A_DBR[1848]	CLC_RTD_LOWER_LIMIT	D	-42	Display comments
\$A_DBR[1844]	CLC_RTD_UPPER_LIMIT	D	1	Details
\$A_DBR[1848]	CLC_RTD_ACT_POS	D	-39.5	Change
\$A_DBR[1852]	CLC_RTD_ACT_POS_ABS	D	8.5	Delete all
\$A_DBR[1856]	CLC_RTD_ACT_VEL	D	8	
\$A_DBB[1868]	CLC_RTD_STATE	D	2	
\$A_DBB[1861]	CLC_RTD_TECH_MODE	D	3	
\$A_DBB[1862]	CLC_RTD_DYN_LIM	H	88888888	
\$A_DBR[1888]	CLC_RT_DIST_OFFSET	D	8	
\$A_DBR[1816]	CLC_RT_GAIN_FACTOR	D	1	
\$A_DBR[1884]	CLC_RT_MOVE_POS_REL	D	5	
\$A_DBR[1888]	CLC_RT_MOVE_POS_ABS	D	18	
\$A_DBB[1812]	CLC_RT_TECH_CMD	D	8	
\$A_DBB[1813]	CLC_RT_CTRL_MODE	H	88888888	

Alarm list
Mes-sages
Alarm log
V NC/PLC variab.
Remote diag.
Version

Figure 4-5 Real-time variables CLCX

Defining names of the real-time variables

To have symbolic names available for programming the real-time variables in synchronized actions, you can add the following global definitions to the definition file "MMAC.DEF" of the NC. With the same MACRO definitions, symbolic names which are only needed in a cycle program can also be defined locally in the header of this program.

Example program

```
; SPATH=/_N_DEF_DIR
; CLCX interface variables:
; ===== 76 bytes used =====
DEFINE CLC_RT_DIST_OFFSET AS $A_DBR[1000]
DEFINE CLC_RT_MOVE_POS_REL AS $A_DBR[1004]
DEFINE CLC_RT_MOVE_POS_ABS AS $A_DBR[1008]
DEFINE CLC_RT_TECH_CMD AS $A_DBB[1012]
DEFINE CLC_RT_CTRL_MODE AS $A_DBB[1013]
; Reserved byte index 1014
; Reserved byte index 1015
DEFINE CLC_RT_GAIN_FACTOR AS $A_DBR[1016]
DEFINE CLC_RT_TABLE3_PARAM AS $A_DBR[1020]
; Reserved dword index 1024
DEFINE CLC_RTD_SENSOR_VOLT AS $A_DBR[1028]
DEFINE CLC_RTD_SENSOR_DIST AS $A_DBR[1032]
DEFINE CLC_RTD_PROG_DIST AS $A_DBR[1036]
DEFINE CLC_RTD_LOWER_LIMIT AS $A_DBR[1040]
DEFINE CLC_RTD_UPPER_LIMIT AS $A_DBR[1044]
DEFINE CLC_RTD_ACT_POS AS $A_DBR[1048]
DEFINE CLC_RTD_ACT_POS_ABS AS $A_DBR[1052]
DEFINE CLC_RTD_ACT_VEL AS $A_DBR[1056]
DEFINE CLC_RTD_STATE AS $A_DBB[1060]
DEFINE CLC_RTD_TECH_MODE AS $A_DBB[1061]
DEFINE CLC_RTD_DYN_LIM AS $A_DBB[1062]
; Reserved byte index 1063
; Reserved Dword Index 1064
DEFINE CLC_RT_TRACE_FLAG AS $A_DBD[1068]
M17
```

For assigning values to \$A_DBx variables without using a synchronized action, it is recommended to set the following machine data:

```
MD28150 $MC_MM_NUM_VDIVAR_ELEMENTS = 20
```

This setting allows the following simple programming of the \$A_DBx real-time variables:

```
N100 CLC_RTD_ACT_POS=5.0 ; this assignment creates an executable NC
block with an IPO cycle processing time
```

Alternatively, a synchronized action with a one-time fulfilled condition would be used:

```
N99, when TRUE do CLC_RTD_ACT_POS=5.0 ; assignment is appended to
the next executable NC block
N100...; next executable NC block>
```

See also

NC commands and Synact variables (Page 58)

4.6 Programming

4.6.1 Activating and deactivating clearance control (CLC)

Syntax

CLC(*Mode*)

Mode

- Format: integer
- Value range: -1, 0, 1, 2, 3, 4

CLC(...) is an NC procedure call and must therefore be programmed in a separate part program block.

Functionality

The following options are available for switching clearance control on/off:

- CLC(1)
Switching on the clearance control
The evaluation of the sensor collision signal is deactivated.
- CLC(2)
Switching on the clearance control
The evaluation of the sensor collision signal is activated.
- CLC(3)
Switching on the 3D clearance control with freely programmable direction. The evaluation of the sensor collision signal is deactivated.
- CLC(4)
Switching on the 3D clearance control with freely programmable direction. The evaluation of the sensor collision signal is activated.
- CLC(0)
Switching off the clearance control without canceling the position offset of the axes
If the clearance-controlled axes are still moving at the switch-off time due to the sensor signal, they are stopped first. Then the positions in the workpiece coordinate system (WCS) are synchronized to the axis positions reached at standstill. The command causes an implicitly triggered preprocessing stop (STOPRE).
- CLC(-1)
Switching off the clearance control with canceling the position offset of the axes.
If the clearance-controlled axes are still moving at the switch-off time due to the sensor signal, they are stopped first. The position offset to the last programmed IPO position is then retracted by an axis motion.

RESET behavior

CLC(0) is executed implicitly on a reset (NC RESET or end of program). The IPO positions shown in the standard display (WCS or MCS) are synchronized to the axis positions currently reached in the servo without any traversing motion taking place. The next programmed traversing block starts at the position reached by sensor specification.

Constraints

Please note the following supplementary conditions:

- Continuous path mode
Switching the clearance control (CLC(Mode)) on/off during active continuous-path mode (G64/G64x) results in a brief stop of the path motion. In cases where this path stop should be avoided, the clearance control can already be activated before a path section with constant path velocity. The control can then be blocked by means of appropriate real-time influencing until a "on-the-fly release" occurs at the desired position.
- Block change with exact stop
If Exact stop at end of block (G60/G09 with G601/G602) is active, the block change may be delayed due to small axis movements which may be caused by a noisy sensor signal. If necessary, you may have to adapt the parameters of standstill monitoring. See above.
- Tool radius compensation
The 3D clearance control may only be deactivated if no tool radius compensation is active in the channel at the time of deactivation (G40). The following alarm is issued when tool radius compensation (G41/G42) is active:
 - Alarm "75015 Channel *number* Block *number* CLC(0) with active TRC."

4.6.2 NC commands and Synact variables

NC variables and synchronized actions variables for operation

Table 4-7 NC variables and synchronized actions variables

Name	Type	Description	Action on reset
CLC(<intArg>)	NC procedure	Modal NC command. Must be programmed in a separate block Switches CLC on and off <intArg>: 1 Switch on without tip-touch monitoring <intArg>: 2 Switch on with activated tip-touch monitoring <intArg>: 3 Switch on with "3d free direction programming" <intArg>: 4 Switch on with "3d free direction programming with tip-touch monitoring". <intArg>: 0 Switch off by synchronizing the servo position <intArg>: -1 Switch off by moving back to the switch-on position	CLCX is switched off with CLC(0)
CLC_SEL(<intArg>)	NC procedure	Modal NC command Selects the voltage clearance table to be used for linearizing the clearance sensor. A new setting is only effective if CLC is activated. 1: MD 62510 / MD 62511 2: MD 62512 / MD 62513 10...20: Voltage clearance table defined by GUD variable	The last selected table remains active when CLCX is switched off or a RESET is performed. Table 1 is selected by default when the program is started.
CLC_LIM(<realArg1>, <realArg2>)	NC procedure	Modal NC command Defines the limits of CLC motion relative to the switch-on position: <realArg1>: Lower limit <realArg2>: Upper limit	MD 62505 [0] and MD 62506 [0] initialize the limit values
CLCX_PREP_DATA(SENSOR_GUD_TAB)	NC function	Non-modal NC command Checks the up to 11 additional clearance voltage tables created in GUD and makes them available for application with the command CLC_SEL(10..20) – provided that the GUD definitions of all tables were free of errors.	Unchanged
CLC_DIST=<realValue>	NC address	Modal NC command <realValue> defines the clearance between the sensor tip and the sheet metal.	MD 62509 initializes the clearance
CLC_GAIN=<realValue>	NC address	This NC address of the old CLC is no longer supported in CLCX. The GAIN factor set with MD 62514 can now be changed by the synchronized action variable CLC_RT_GAIN_FACTOR.	MD 62514 initializes the GAIN

Name	Type	Description	Action on reset
CLC_RT_DIST_OFFSET	SynAct variable \$A_DBR[x+0]	Real-time offset in mm, which is added to the clearance setpoint programmed with CLC_DIST=<>	Initialized with zero
CLC_RT_GAIN_FACTOR	SynAct variable \$A_DBR[x+16]	Real-time factor by which the CLC loop gain set by the MD \$MC_CLC_DISTANCE_CTRL_GAIN is multiplied.	Is initialized with 1.0
CLC_RT_MOVE_POS_REL	SynAct variable \$A_DBR[x+4]	Relative distance in mm to which the CLC axis jumps when the command CLC_RT_TECH_CMD=10 is given or when "tip-touch monitoring" is triggered.	Unchanged
CLC_RT_MOVE_POS_ABS	SynAct variable \$A_DBR[x+8]	Absolute MCS position in mm to which the CLC axis jumps when the command CLC_RT_TECH_CMD=11 is given. In the 3D case: absolute jump position within the CLC coordinate system.	Unchanged
CLC_RT_TECH_CMD	SynAct variable \$A_DBB[x+12]	Real-time command that triggers the following actions: • Jump-up of the CLC motion • Freezing of the CLC motion • Switch to the mode for determining the sensor delay time For a complete list of commands, see Available technology commands (Page 61). With CLC_RT_TECH_MODE=0 the active technology command is terminated and the sensor-controlled mode is reactivated.	Initialized with zero
CLC_RT_CTRL_MODE	SynAct variable \$A_DBB[x+13]	Selects the control mode: • Bit 0 = 0: POS mode • Bit 0 = 1: GAIN mode • Bit 4 = 1: causes the CLC velocity limit to use the value set in MD \$MC_CLC_SENSOR_VELO_LIMIT[1].	Bit_0 is initialized with the value of Bit_12 of MD 62508. Bit_4 of \$A_DBB[x+13] remains unchanged after a reset
CLC_RT_TRACE_FLAG	SynAct variable \$A_DBB[x+68]	Variable to start and stop the integrated CLCX trace.	Unchanged

4.6 Programming

4.6.3 Synact variables for diagnostics

Available synchronized actions variables for diagnostics

Table 4-8 Diagnostics variables

Name of the SynAct variable	Type and byte offset	Description
CLC_RTD_SENSOR_VOLT	\$A_DBR[x+28]	Shows the currently applied input voltage in Volts. Allows you to set the correct normalization factor and update/check the voltage clearance tables. The sensor voltage is displayed even if CLC is not active.
CLC_RTD_SENSOR_DIST	\$A_DBR[x+32]	Shows the distance that the sensor is currently measuring. This value results from the actual sensor voltage and the currently selected voltage clearance table. The table is evaluated even if CLC is actually deactivated. The last selected table number remains active after RESET. The distance is always displayed in mm, regardless of any inch/metric settings.
CLC_RTD_PROG_DIST	\$A_DBR[x+36]	Displays the clearance setpoint resulting from the programmed modal NC command CLC_DIST=<value> and the real-time offset set with the SynAct variable CLC_RT_DIST_OFFSET. The clearance setpoint is always displayed in mm, regardless of all inch/metric settings.
CLC_RTD_LOWER_LIMIT	\$A_DBR[x+40]	Shows the effective lower limit of the CLC motion. The value refers to the position at which the CLC was switched on. With 3D-CLC, the limitation can be additionally restricted if the programmed TCP motion approaches the software limit of an axis. This value is always displayed in mm, regardless of any inch/metric settings.
CLC_RTD_UPPER_LIMIT	\$A_DBR[x+44]	Shows the effective upper limit of CLC motion. The value refers to the position at which the CLC was switched on. With 3D-CLC, the limitation can be additionally restricted if the programmed TCP motion approaches the software limit of an axis. This value is always displayed in mm, regardless of any inch/metric settings.
CLC_RTD_ACT_POS	\$A_DBR[x+48]	The current set position of the CLC motion. The zero point to which this coordinate refers is the position at which the CLC was switched on. The set position is always displayed in mm, regardless of any inch/metric settings.
CLC_RTD_ACT_POS_ABS	\$A_DBR[x+52]	The current set position of the CLC motion in MCS coordinates. This position is always displayed in mm, regardless of any inch/metric settings. In the 3D case, this variable shows the same position as CLC_RTD_ACT_POS.
CLC_RTD_ACT_VEL	\$A_DBR[x+56]	The current speed setpoint of the CLC motion. This speed is always displayed in mm/min, regardless of any inch/metric settings.
CLC_RTD_STATE	\$A_DBB[x+60]	Displays the current status of the CLC motion. See Diagnosis of the CLCX state (Page 63).
CLC_RTD_TECH_MODE	\$A_DBB[x+61]	Shows the state that resulted from the last given TECH command.

Name of the SynAct variable	Type and byte offset	Description
CLC_RTD_DYN_LIM	\$A_DBB[x+62]	The bits indicate the various dynamic limits currently acting on the input variable of the CLC interpolator and limiter. If all bits are zero, the sensor's motion specification is currently not limited.
CLC_RTD_FREE_DIR_ANGLE	\$A_DBB[x+72]	Angle (in degrees) between the orientation generated by "free direction programming" and the current tool orientation of the 5-axis transformation.

4.6.4 Available technology commands

Technology commands available for synchronized action

The following table lists the technology commands that can be programmed in synchronized actions while the CLC is active. The right column shows the corresponding mode that is output by the variable CLC_RTD_TECH_MODE.

Table 4-9 Technology commands

CLC_RT_TECH_CMD	Description	CLC_RTD_TECH_MODE
0	This command cancels any other currently active command and moves the CLC axis sensor-controlled up to the programmed distance CLC_RTD_PROG_DIST above the plate.	0 ... 5
10	jump_up_rel : Starting from the current position, the CLC axis moves with the maximum permissible dynamic response by the distance specified in the variable CLC_RT_MOVE_POS_REL.	10
11	jump_up_abs : The CLC axis moves with the maximum permissible dynamic response up to the MCS position specified in the variable CLC_RT_MOVE_POS_ABS. Note: In the 3D case, the target position of the jump is specified absolutely in relation to the CLC coordinate system. For the reference to the regular MCS, the specification of three coordinates would be necessary.	11
20	freeze_both_dirs : Locks the CLC axis at its current position	20
21	freeze_down_pos : Prevents the CLC axis from moving lower than its current position. The axis can still be moved upward (sensor-controlled) and can also be moved back again to the frozen position.	21
22	freeze_down_dir : Prevents the CLC axis from moving further down. The sensor can only move the axis upwards to higher positions.	22

4.6 Programming

CLC_RT_TECH_CMD	Description	CLC_RTD_TECH_MODE
30	This number reports a "tip-touch" event if tip-touch monitoring has been activated with the switch-on command CLC(2) or CLC(4): Either the external tip-touch signal or the internal tip-touch evaluation has triggered a motion of the CLC axis by the distance specified in the variable CLC_RT_MOVE_POS_REL. This state cannot be actively programmed. However, the tip-touch state can be canceled and the sensor-controlled motion can be continued by overwriting the pending command code 30 with 0 or another valid command code in CLC_RT_TECH_CMD.	30
99	Switches on the sine-wave generator to determine the sensor delay The GUD CLC_SIN_GEN_AMP determines the amplitude (in mm) of the generator and CLC_SIN_GEN_FREQ the frequency (in Hz).	99

Executing a technology command in the IPO task

In principle, all CLC_RT_TECH_CMD commands that are not equal to "0" must be repeated within each IPO cycle as long as the command is to be executed. If you do not repeat the command in an IPO cycle, CLCX receives the command "CLC_RT_TECH_CMD=0" and terminates the current technology actions (Auto-Cancelling).

Example:

For example, if you want to cross a hole already cut in the sheet metal with the cutting head raised by 5 mm, you can program the following instruction:

Programming example

```
CLC_RT_MOVE_REL_POS=5.0 ; prepares a relative jump-up of 5 mm
```

```
...
```

```
whenever TRUE do CLC_RT_TECH_CMD=10 ; keeps the jump_up_rel command active during the next executable block
```

The Auto-Cancelling property of CLC_RT_TECH_CMD makes the programming of synchronized actions more user-friendly in many cases because you do not need an additional command to terminate the action.

On the other hand, this setting prevents the corresponding commands from being entered into the \$A_DBB variable by the PLC or manually via the variable view of the SINUMERIK Operate, since with both methods the input cannot be repeated in every IPO cycle.

Executing a technology command with "AUTO-HOLD"

To nevertheless use the technology commands from the PLC or from the variable view of the SINUMERIK Operate, you can optionally select the Auto Hold mode which can be applied for each technology command not equal to zero. You must increase the respective command value by 100 for this purpose.

Example:

Set the technology command `CLC_RT_TECH_CMD = 10` (jump_up_rel) with auto hold mode in a synchronized action:

Programming example

```
When TRUE do CLC_RT_TECH_CMD=110 ; in the first cycle of the next executable block, the
jump_up_rel command becomes active in AUTO-HOLD
```

...

A technology command issued in Auto-Hold mode remains active until you explicitly cancel it by executing `CLC_RT_TECH_CMD=0` or another technology command.

As long as no technology command greater than zero is active, CLCX controls the axis in sensor-controlled mode.

CLC_RTD_TECH_MODE

As long as no technology command is issued, the CLC moves the axes in sensor-driven control mode. The variable `CLC_RTD_TECH_MODE` reports back details of the active control mode according to the following table:

CLC_RTD_TECH_MODE	Description
0	The CLC function is switched off
1	The CLC motion is driven by the sensor. It approaches the sheet metal, but the sensor is still outside its sensitive range.
2	The "POS mode" is active (<code>CLC_RT_CTRL_MODE=0</code>) The CLC motion is driven by the sensor, which is now in its sensitive range.
3	The "GAIN mode" is active (<code>CLC_RT_CTRL_MODE=1</code>) The CLC motion is driven by the sensor, which is now in its sensitive range.
4	Like mode 2, but the internal JERK filter is still activated. In sensor-driven mode, the jerk filter is switched off as quickly as possible.
5	Like mode 3, but the internal JERK filter is still activated. In sensor-driven mode, the jerk filter is switched off as quickly as possible.

4.6.5 Diagnosis of the CLCX state

Motion state of CLCX

The synchronized action variable `CLC_RTD_STATE` shows the detailed motion status of the CLCX axis:

Table 4-10 Synchronization variable `CLC_RTD_STATE`

CLC_RTD_STATE	Description
0	CLC is deactivated.
1	CLC axis is moving.
2	CLC motion is at a standstill according to the period set in MD 62520 and the position tolerance set in MD 62521 [0].

4.7 3D clearance control

CLC_RTD_STATE	Description
3	CLC stopped at lower motion limit.
4	CLC stopped at upper motion limit.
5	CLC motion is at standstill and at the same time the current sensor position is within the tolerance window to the programmed clearance setpoint specified in MD 62521 [1].
10	CLC stops: Identifies the IPO cycles required by the CLC to decelerate the axis motion to a standstill after an off command is received
11	CLC moves back: Appears while the axis is moving back to the last IPO position if the deactivation was programmed with CLC(-1).
12	CLC has reached the stop state: Marks the IPO cycle in which the CLC motion is stopped after state 10.

CLCX limiter/interpolator

The synchronized action variable CLC_DIAG_RTD_DYN_LIM shows the detailed state of the CLCX interpolator/dynamic limiter.

As long as this variable shows a value of zero, the motion specified by the sensor or internally can be executed without being limited by the dynamic response of the CLC axis.

Table 4-11 Display CLCX interpolator with CLC_DIAG_DYN_LIM

CLC_RTD_DYN_LIM	Description
bit_0	Brakes with predictable overshoot
bit_1	Braking to a fixed target position
bit_2	Current negative acceleration limit reached
bit_3	Current positive acceleration limit reached
bit_4	Current negative velocity limit reached
bit_5	Current positive velocity limit reached
bit_6	Lower position limit reached
bit_7	Upper position limit reached

4.7 3D clearance control**4.7.1 3D clearance control - overview****4 and 5-axis transformation**

A prerequisite for the application of the 3D clearance control is that a 4 or 5-axis transformation has been defined via TRAORI.

Three linear axes are used to position the tool. One or two rotary axes are used for the orientation of the tool vector (5-axis machining). The clearance control only affects the linear axes. Two options are available for the direction of the sensor-guided control motion in space ("the 3D control direction"):

- in the direction of the vector of the programmed tool orientation (standard case)
- in any direction, which must be programmed by three additional vector components within each traversing block

Note

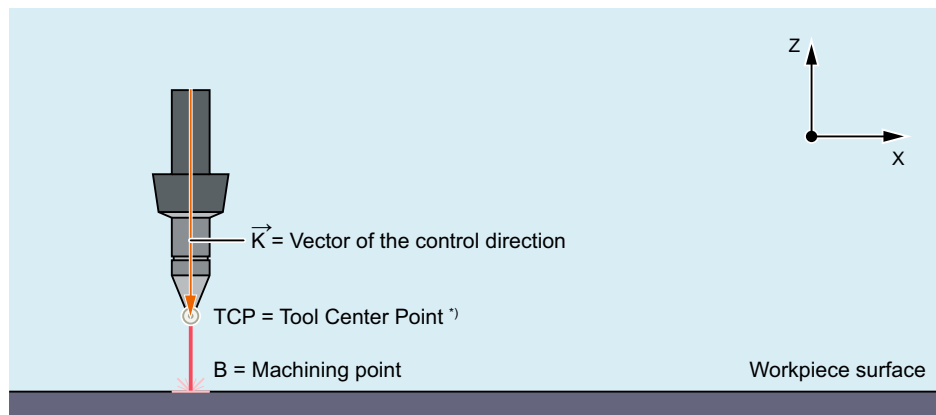
The 3D clearance control can only be used on Cartesian machines with generic 4 or 5-axis transformation.

The 3D CLC cannot be used on robots or other non-Cartesian kinematics. There, the TOFF function, which operates in the IPO cycle, can be used.

4.7.2 3D control direction

Standard 3D control direction

In the standard case, the control direction of the clearance control and the vector of the tool orientation are identical. Consequently, the compensation movement of the clearance control is normally always in the direction of the tool orientation.



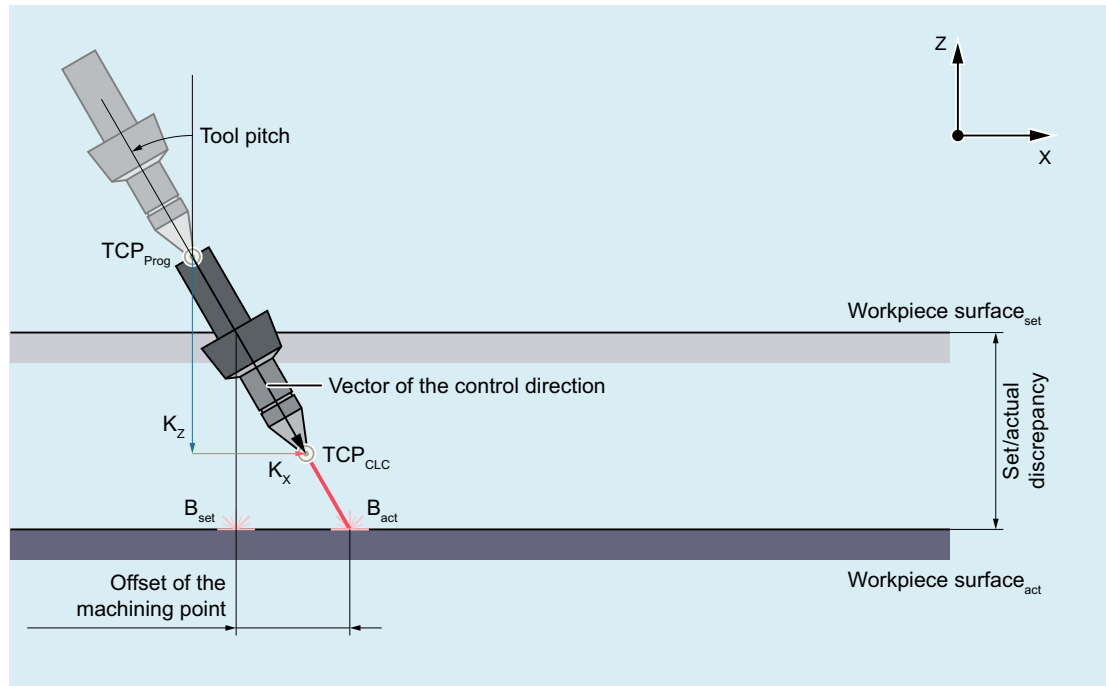
*) The programmed TCP corresponds to the ideal machining point.

Figure 4-6 Clearance control with standard control direction

Note

In all the figures in this section, the traversing movement of the machining head needed in order to machine the workpiece is in the direction of the Y coordinate, i.e. perpendicular to the drawing plane.

If, for technological reasons, a setting angle of the beam is required so that the tool orientation is no longer perpendicular to the workpiece surface, correction movements of the clearance control of the standard control direction result in a geometric offset of the machining point on the workpiece surface.



The reason for the offset of the machining point is the vector component (KX) of the control direction parallel to the workpiece surface. The TCP of the tool, and thus the machining point B, is shifted by this amount.

When using the software option "freely programmable control direction", the correction movements of the clearance control can be made in the direction of the programmed vector, i.e. it is no longer bound to the beam orientation (= tool orientation).

66

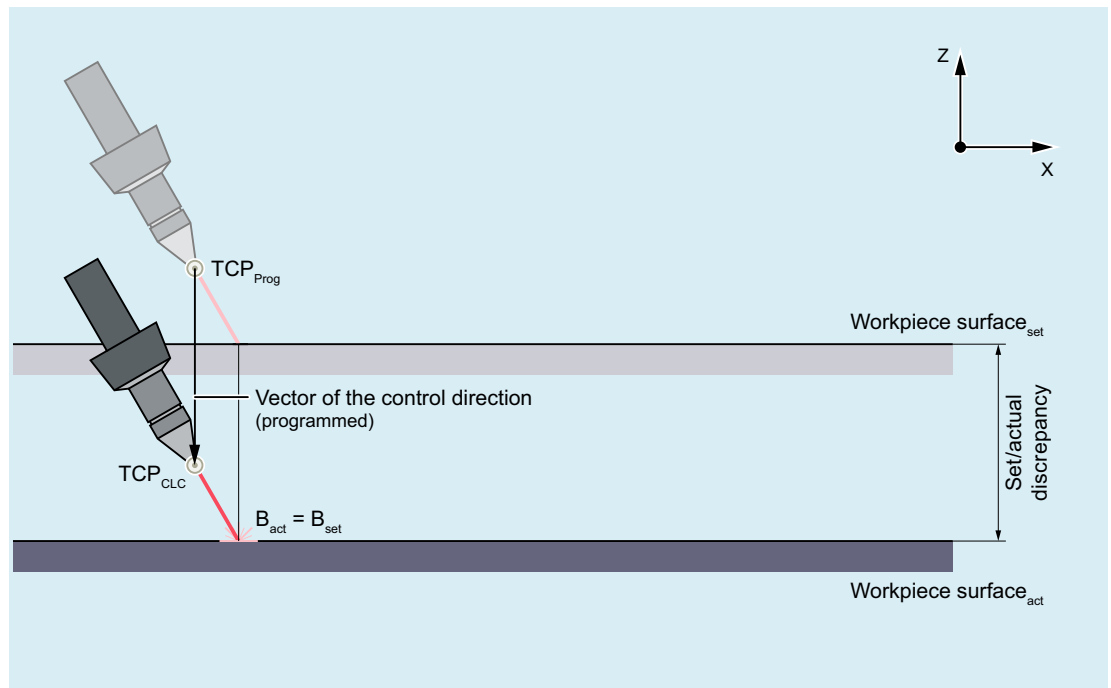


Figure 4-8 Programmable 3D control direction

4.7.3 Parameterization of the programmable control direction

Reference coordinate system

Three vector components to be programmed as additional axis positions are used to specify the direction in which the sensor-guided correction movement of the clearance control takes place.

The vector components refer to the basic coordinate system. Programmed frames that act on the contour and, if applicable, the tool orientation are not taken into account when calculating the control direction.

Axis positions for setting the vector of the control direction

The following conditions must be met for the specification of the vector components of the control direction via programmed axis positions:

1. The three axes used should be simulated NC axes:
 - MD30240 \$MA_ENC_TYPE[x]=0
 - MD30130 \$MA_CTRLOUT_TYPE[x]=0
2. The axes must be channel axes of the channel in which the clearance control is activated.
3. The axes must be linear axes. $\pm 1.0\text{mm}$ is recommended as maximum programmed position range of each vector component to avoid interpolation of long axis distances. The length of the programmed vector need not be standardized to 1.0.

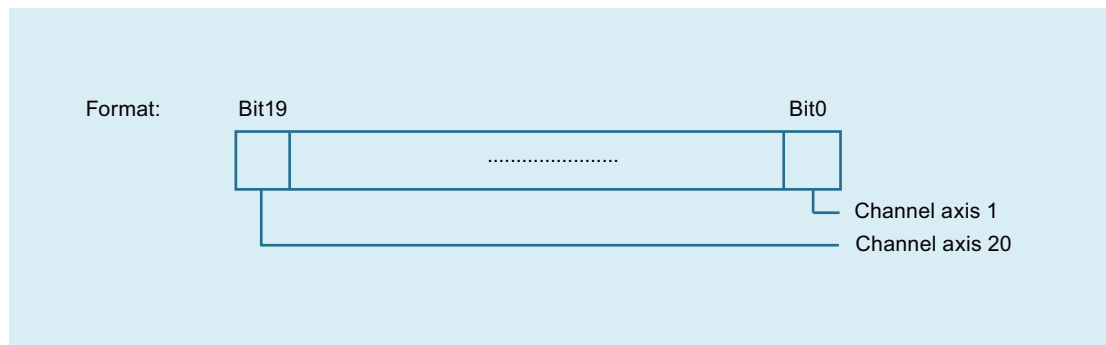
4.7 3D clearance control

4. The axes may not participate in an axis coupling, e.g. transformation, electronic gear, etc.
5. To ensure that the path dynamics are not limited due to insufficient axis dynamics, the following machine data of these axes must be set about a factor 100 higher than the corresponding values of the real geometry axes of the channel:
 - MD32000 \$MA_MAX_AX_VELO[x] (Maximum axis velocity)
 - MD32300 \$MA_MAX_AX_ACCEL[0,x]
 - MD32431 \$MA_MAX_AX_JERK[x] (Configuration of the position controller structure)
 x = name of the simulated axis

The machine data is used to determine which channel axes the vector components of the programmable control direction specify:

- MD62528 \$MC_CLC_PROG_ORI_AX_MASK (Progr. orientation vector: axis mask)

Each bit of this machine data corresponds to a channel axis according to the following scheme.



The selected channel axes are used in ascending numerical order in the channel for programming and interpolation of the three components x, y, z of the vector of the control direction.

Current difference angle

The current difference angle between the vector of the beam orientation and the vector of the control direction is displayed in the \$A_DBR variable CLC_RTD_FREE_DIR_ANGLE.

Monitoring of the maximum difference angle

The maximum allowed difference angle between the vector of the tool orientation and that of the control direction can be set via the following machine data:

MD65520 \$MC_CLC_PROG_ORI_MAX_ANGLE ()

If a value greater than zero is entered in this machine data and the current difference angle exceeds this value, alarm 75014 is output

4.7.4 3D clearance control and 5-axis transformation

Relationship between 3D clearance control and 5-axis transformation

If 3D clearance control is activated before the 5-axis transformation required for clearance control in the direction of the tool orientation has been activated, the clearance control function will be dependent on the selection of the active working plane (G17/G18/G19):

- G17: Direction of clearance control = Z
- G18: Direction of clearance control = Y
- G19: Direction of clearance control = X

Activation of 5-axis transformation

When 5-axis transformation is activated, the tool orientation specified by means of the rotary axis positions must tally with the control direction specified by the active working plane on activation of clearance control.

If the tool orientation of the 5-axis transformation and the control direction of the clearance control function do not tally, the following CLC alarm will appear:

- Alarm "75006 Channel *number* Block *number* CLCX: Abrupt change in orientation

Deactivation of 5-axis transformation

If the 5-axis transformation is switched off when clearance control is active, the CLC retains the last control direction before switching off the 5-axis transformation.

Vector of the control direction

Actual position of the direction axes

If the clearance control with freely programmable direction is activated with the value 0.0 in all three components of the direction vector, no control direction can be calculated from this. In this case, the following alarm is displayed:

- Alarm "75014 channel *number*, error ID: 1, angle 0.0"

Referencing the direction axes

Before activating clearance control with freely programmable direction CLC(3)/CLC(4), the simulated axes must be referenced for the components of the control direction.

Switching over clearance control

Direct switching over of the clearance control from CLC(1) or CLC(2) to CLC(3)/CLC(4) or vice versa is not possible. Such switchovers are ignored without a checkback signal. If such a switchover is necessary, the clearance control must first be switched off with CLC(0) or CLC(-1) and then switched on again in the desired mode.

Interpolation of the control direction

If the vector of the control direction with regard to its orientation is to follow a non-linear workpiece surface, e.g. an arc of a circle, this can be achieved by programming the direction axes accordingly.

Example

4.7 3D clearance control

Orientation of the control direction perpendicular to a semicircular workpiece surface. The programming of the traversing motion is not considered.

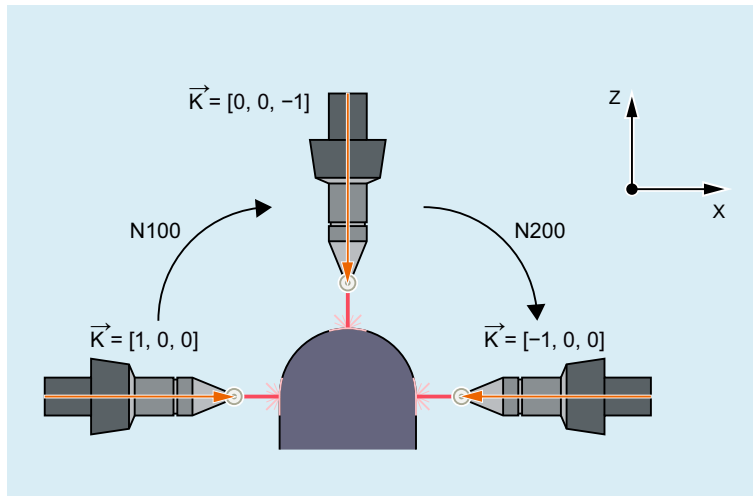


Figure 4-9 Interpolation of the vector of the control direction

Before the part program block N100, the control direction vector has been oriented to $[1, 0, 0]$ by programming the direction axes. In part program block N100, the end position of the control direction vector is oriented by programming the direction axes at $[0, 0, -1]$.

The intermediate values are generated by synchronous interpolation of all axes programmed in the part program block:

- Geometry and orientation axes for the movement of the machining head
- The three component axes for rotating the vector of the control direction.

It is necessary to break the movement down into part program blocks N100 and N200, because an antiparallel orientation of the control direction vector of $[1, 0, 0]$ at the start of the movement and $[-1, 0, 0]$ at the end of the movement (semi-circle) would otherwise result. In this case, the interpolator would interpolate only the X coordinate of the vector, and the orientation of the control direction vector would remain unchanged.

Antiparallel orientation of the vector of the control direction

If in a part program block an antiparallel orientation of the free control direction with respect to the previous direction is programmed, the following alarm will be displayed:

- Alarm "75018 Channel *number* Block *number* CLC in programmable direction, error ID: 1"

Note

Interpolation of the vector

As described above, the interpolation of the vector of the control direction is not a true vector interpolation, but is the result of the component-wise interpolation of the three simulation axes involved.

Rotation of the workpiece coordinate system

The programming of the three vector components of the control direction always refers to the basic coordinate system, as described above. If a rotating or mirroring frame is used for machining the workpiece, so that the coordinate axes of the programming coordinate

system (WCS) and those of the basic coordinate system are no longer parallel and of the same orientation, a corresponding conversion must also be made for the components of the control direction.



CAUTION

Not of the same orientation

If the workpiece coordinate system is rotated or mirrored in such a way that the coordinate axes of the basic and workpiece coordinate systems are no longer parallel and of the same orientation, the user is responsible for the corresponding transformation of the freely programmed control direction.

4.8 Multiple CLCX instances in different channels

4.8.1 Multiple CLCX instances in one channel

CLCX instance

Via a CLCX instance, the functionality of clearance control is applied to an axis or (for 3D CLC) to the Cartesian axes of a 4 or 5-axis transformation. Up to four instances can be configured within one NCU. All four instances can be used in one channel ("multi-head machine") or freely distributed to several channels of a control system. For example, two instances can run in the first channel and one instance each in the second channel and fourth channel.

Since a channel represents a machining context in which only exactly one 5-axis transformation can run, CLCX configurations with several instances per channel only make sense with the single-axis version.

Assigning CLCX instances to channels

As with other compile cycles, the following machine data determines the channels in which the CLCX function is to be used:

`$MN_CC_ACTIVE_IN_CHAN_CLCX[0] = 'Hxxx':`

- 'Hxxx' is a bit mask in hexadecimal format, which is used to select the channels in which at least one CLCX instance should be available.

Activating an instance in a channel

`$MN_CC_ACTIVE_IN_CHAN_CLCX[0] = 'H1'`

With this command you activate a CLCX instance in channel 1.

Activating one instance in several channels each

`$MN_CC_ACTIVE_IN_CHAN_CLCX[0] = 'H5'`

4.8 Multiple CLCX instances in different channels

With this command you activate one instance in channel 1 and one instance in channel 3 (H5 = binary 0101).

Activating multiple instances in channels

To activate more than one instance in a channel, the number of instances to be activated in the respective channel must be specified in \$MN_CC_ACTIVE_IN_CHAN_CLCX[0] in addition to the channel selection described. This happens in the machine data \$MN_CC_ACTIVE_IN_CHAN[1]. Each hexadecimal digit of this machine data determines the number of instances in the assigned channel:

```
$MN_CC_ACTIVE_IN_CHAN_CLCX[1] = 'H103'
```

With this entry you activate three instances in channel 1 and one instance in channel 3, provided that the channel assignment from the last command is effective simultaneously:

```
$MN_CC_ACTIVE_IN_CHAN_CLCX[0] = 'H5'
```

To activate the change from a configuration with one instance per channel to a configuration with multiple instances, you must first upload an archive with the new setting after you have set \$MN_CC_ACTIVE_IN_CHAN_CLCX[1] accordingly, and then download the archive back to the control system. A simple restart of the NC kernel is not enough for this.

Converting machine data to multiple instances

The archive download deletes the buffered memory of the NC kernel and thus also the CLCX machine data. This is necessary to convert the following eight CLCX channel machine data units from single data to array data. Then the first CLCX instance within the channel reads the value of the first array element, the second instance reads the value of the second array element, and so on.

The following list shows the eight machine data units that become arrays during the download process.

The four machine data units, which must always be given an individual setting for each CLCX instance in the channel, are marked with an exclamation mark in the list.

List of the machine data converted to an array

```
62500 $MC_CLC_AXNO[0]
62500 $MC_CLC_AXNO[1] ; !

62502 $MC_CLC_ANALOG_INPUT[0]
62502 $MC_CLC_ANALOG_INPUT[1] ; !

62503 $MC_CLC_ANALOG_IN_HW_NORM_FACT[0]
62503 $MC_CLC_ANALOG_IN_HW_NORM_FACT[1]

62504 $MC_CLC_SENSOR_TOUCHED_INPUT[0]
62504 $MC_CLC_SENSOR_TOUCHED_INPUT[1] ; !

62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[0]
62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[1] ; !!

62509 $MC_CLC_DISTANCE_SETPOINT[0]
62509 $MC_CLC_DISTANCE_SETPOINT[1]

62514 $MC_CLC_DISTANCE_CTRL_GAIN[0]
62514 $MC_CLC_DISTANCE_CTRL_GAIN[1]

62519 $MC_CLC_SENSOR_DELAY_COMP_TIME[0]
62519 $MC_CLC_SENSOR_DELAY_COMP_TIME[1]
```

All other channel-specific machine data is shared by the CLCX instances within the same channel. This data can therefore not be set individually for each CLCX instance.

It is recommended to increase the resources of the compile cycle block memory in the channels accordingly where more than one CLCX instance is being executed:

```
28090 $MC_MM_NUM_CC_BLOCK_ELEMENTS
28100 $MC_MM_NUM_CC_BLOCK_USER_MEM
```

Detailed descriptions for renaming the NC addresses can be found at Programming several instances within one channel (Page 75), 10712 \$MN_NC_USER_CODE_CONF_NAME_TAB.

4.8.2 Macros for operating and monitoring multiple instances in one channel

Adjusting real-time variables

According to the setting of the instance-specific start offset in \$MC_CLC_A_DB_RT_VAR_START_INDEX[n], the macros of the real-time variables must be adjusted for the operation of the CLX instances. Since the \$A_DBx variables are global, the names should be selected so that both the channel number and the number of the instance using the data are evident.

4.8 Multiple CLCX instances in different channels

Example macro

The following setting is provided:

```
channel_1:
62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[0]=1000
62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[1]=1100
channel_2:
62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[0]=2000
62507 $MC_CLC_A_DB_RT_VAR_START_INDEX[1]=2100
```

The following definitions can be used for this setting:

Example macro

```
MMAC. DEF:
; SPATH=/_N_DEF_DIR
; CLCX interface variables:
; === channel1_instance1 =====
DEFINE CLC_RT_DIST_OFFSET AS $A_DBR[1000]
; or "CLC11_RT_..." (to be really consistent)
DEFINE CLC_RT_MOVE_POS_REL AS $A_DBR[1004]
DEFINE CLC_RT_MOVE_POS_ABS AS $A_DBR[1008]
...
; === channel1_instance2 =====
DEFINE CLC12_RT_DIST_OFFSET AS $A_DBR[1100]
DEFINE CLC12_RT_MOVE_POS_REL AS $A_DBR[1104]
DEFINE CLC12_RT_MOVE_POS_ABS AS $A_DBR[1108]
...
; === channel2_instance1 =====
DEFINE CLC21_RT_DIST_OFFSET AS $A_DBR[2000]
DEFINE CLC21_RT_MOVE_POS_REL AS $A_DBR[2004]
DEFINE CLC21_RT_MOVE_POS_ABS AS $A_DBR[2008]
...
; === channel2_instance2 =====
DEFINE CLC22_RT_DIST_OFFSET AS $A_DBR[2100]
DEFINE CLC22_RT_MOVE_POS_REL AS $A_DBR[2104]
DEFINE CLC22_RT_MOVE_POS_ABS AS $A_DBR[2108]
...
```

Depending on which functions you actually want to use, you do not have to define all variables.

Note

If the real-time variables are to be used within a part program which can be called from different channels, the channel-dependent part of the start offset used in the macro must not be fixed, but must depend on the channel which is currently using the index (\$P_CHANNO).

Example

```
; === instance1 =====
DEFINE CLC_RT_DIST_OFFSET AS $A_DBR[$P_CHANNO*1000+0]
DEFINE CLC_RT_MOVE_POS_REL AS $A_DBR[$P_CHANNO*1000+4]
...
; === instance2 =====
DEFINE CLC2_RT_DIST_OFFSET AS $A_DBR[$P_CHANNO*1000+100]
DEFINE CLC_RT_MOVE_POS_REL AS $A_DBR[$P_CHANNO*1000+104]
...
```

4.8.3 Programming several instances within one channel

Additionally configured instances

If additional CLCX instances are configured in a channel, extended NC commands are available for switching on and off and for parameterizing each individual instance.

Regular commands:

```
CLC()
```

```
CLC_SEL() ; selects the characteristic to be used. Standard:
characteristic 1
```

```
CLC_LIM()
```

Extended commands, e.g. for the second instance:

```
CLC2()
```

```
CLC2_SEL() ; selects the characteristic to be used. Standard:
characteristic 1 !!
```

```
CLC2_LIM()
```

If the sensor used for the second instance has an individual characteristic curve, this must be determined and entered as Characteristic_2. At the beginning of each part program the Characteristic_2 must then be selected with the command CLC2_SEL(2).

Programmable clearance

The effective clearance for the second instance can be programmed modally with the following NC address:

```
CLC2_DIST=x.y
```

To do this, you must replace the default name of the OA NC address "OMA2" in the following machine data:

```
10712 $MN_NC_USER_CODE_CONF_NAME_TAB[0]="OMA1" ; default name of
the 1st OA NC address
```

```
10712 $MN_NC_USER_CODE_CONF_NAME_TAB[1]="CLC_DIST" ; Renaming
```

4.8 Multiple CLCX instances in different channels

10712 \$MN_NC_USER_CODE_CONF_NAME_TAB[2]="OMA2" ; default name of the 2nd OA NC address

10712 \$MN_NC_USER_CODE_CONF_NAME_TAB[3]="CLC2_DIST"; Renaming

As long as the setpoint is not programmed, the value entered in \$MC_CLC_DISTANCE_SETPOINT[1] acts as the default distance of the second CLCX instance in the channel.

4.8.4 Additional voltage/clearance characteristics

Defining additional characteristics

The channel-specific machine data of CLCX offers two sensor characteristics per channel for linearization and adaptation of different clearance sensors:

62510 __MC_CLC_SENSOR_VOLTAGE_TABLE_1[0..15]

62511 __MC_CLC_SENSOR_DIST_TABLE_1[0..15]

62512 __MC_CLC_SENSOR_VOLTAGE_TABLE_2[0..15]

62513 __MC_CLC_SENSOR_DIST_TABLE_2[0..15]

In cases where more than two characteristic curves are required in a channel, i.e. in particular in applications that configure several CLCX instances within a channel to operate up to 4 CLCX axes within this channel), up to eleven additional sensor characteristic curves can be defined with the following global GUD:

DEF NCK REAL CLCX_TAB_D[16] ; All GUD characteristics use the same clearances. This table applies to all voltage tables.

DEF NCK REAL CLCX_TAB_V[11, 16]; ; 11 voltage tables with 16 breakpoints each

DEF NCK INT CLCX_TAB_STATUS; ; the variable for error messages

If fewer than 11 characteristics are required or if the characteristics are defined with fewer than 16 clearance breakpoints, the dimension of the GUD arrays can be reduced. The minimum configuration that provides only one global additional sensor characteristic curve defined by only two voltage clearance points is as follows:

DEF NCK REAL CLCX_TAB_D[2]

DEF NCK REAL CLCX_TAB_V[2]

The following rules apply to the definition of the sensor characteristic curves defined by GUD as for the corresponding machine data:

- The first entry in the clearance table CLCX_TAB_D[0] must be zero.
- The entries in the clearance table must increase monotonically.
- The entries in the voltage tables must either increase or decrease monotonically.

A zero entry in the clearance table indicates the end of the table. This means that a table in particular may be shorter than specified in the GUD definition.

Typically, the sensor calibration is automated in a cycle program that moves the sensor head up step by step from a position where the sheet metal is almost touched to the clearances defined in CLCX_TAB_D[m] and writes the read-out voltage values to the corresponding element of CLCX_TAB_V[n,m].

After the characteristic curves have been measured and stored in GUD, the following NC function provided by the CLCX must be called to transfer all GUD values to the internal data representation which can be used by the CLCX function:

```
CLCX_TAB_STATUS = CLCX_PREP_DATA("SENSOR_GUD_TAB")
```

During the NC restart, the CLCX calls this function internally. In this way, the additional characteristics can be used without having to explicitly repeat the call of this NC function at each NCK restart.

In any case, the GUD variable CLCX_TAB_STATUS reflects the status of the last transmission and must be checked to detect errors within the current GUD definition. Such errors do not set the alarm 75001 with error ID: -1 or -2 as invalid characteristic curves entered via machine data do.

Display CLCX_TAB_STATUS

If CLCX_TAB_STATUS shows one of the following negative values, none of the additional characteristics defined via GUD can be used. You must first fix the specified problem.

Table 4-12 Status displays

-1	CLCX_PREP_DATA was called while CLC is active and one of the GUD tables is in use.
-2	CLCX_PREP_DATA was called with an argument unequal to "SENSOR_GUD_TAB".
-3	CLCX_PREP_DATA internal READ_ERROR when reading the argument.
-10	Expected GUD names CLCX_TAB_V, CLCX_TAB_D not found.
-11	The GUD data definition must be global: "DEF NCK REAL ..."
-12	The GUD data definition must be of data type REAL: "DEF NCK REAL ..."
-13	CLCX_TAB_D and CLCX_TAB_V: The size of the first array dimension must be at least 2.
-14	CLCX_TAB_V: The size of the second array dimension must be at least 2.
-16	Internal READ_ERROR when reading a GUD
-20	CLCX_TAB_V: The order of the entries does not increase or decrease monotonously.
-21	CLCX_TAB_D: Order of entries does not increase monotonously.
-25	CLCX_TAB_D: The first entry in the clearance table is not equal to zero.

If there was no problem in any of the table definitions during the transmission of the GUD values, the number of successfully transmitted voltage values is reported back as a positive number.

If, for example, 11 tables with 16 valid entries each have been defined, the following value is set in the status variable:

```
CLCX_TAB_STATUS = 176
```

4.9 Function-specific display data

If CLCX_TAB_STATUS shows that the additional internal sensor characteristics have been successfully updated from the GUD arrays, any instance of the CLCX function in any channel can use these characteristics by programming CLC_SEL(10) through CLC_SEL(20).

- CLC_SEL(10) uses the measured voltages CLCX_TAB_V[0, 0...15]
- CLC_SEL(11) uses the measured voltages from CLCX_TAB_V [1, 0...15]
- ...
- CLC_SEL(20) uses the measured voltages from CLCX_TAB_V[10, 0 ...15]

If a characteristic is programmed in CLC_SEL() that is currently not defined, CLCX sets the following alarm:

```
75002 Channel %1 NC block %2 CC_CLCX programming error, error code:
5
```

Error code 5: CLC_SEL(1x) programs without valid GUD characteristics set

4.9 Function-specific display data

The "Clearance control" technology function provides specific display data for supporting setup and for service purposes.

Possible applications

Possible applications of the display data include, for example:

- Determination of the form deviations and control errors occurring for a short time via the variables for the maximum and minimum position offset or the sensor input voltage.
- Determination of the voltage noise detected by the A/D converter using the variables for the maximum and minimum sensor input voltage. Prerequisites are a constant distance of the clearance sensor from the workpiece surface with blocked clearance control (e.g. with CLC_RT_GAIN_FACTOR = 0.0).

The minimum and maximum values are acquired in the position controller cycle.

Types of variables

The current display data is available both as channel-specific GUD (Global User Data) and as a synchronized action variable (Page 63). The min/max values are only available in the GUD.

4.9.1 Channel-specific GUD variables

The "Clearance control" technology function provides the following channel-specific GUD variables for SINUMERIK Operate:

Table 4-13 Channel-specific GUD variables

GUD variable	Designation	Unit	Access
CLC_DISTANCE[0]	Current position offset	mm	read only
CLC_DISTANCE[1]	Absolute minimum of position offset	mm	read / write
CLC_DISTANCE[2]	Absolute maximum of position offset	mm	read / write
CLC_VOLTAGE[0]	Current sensor input voltage	V	read only
CLC_VOLTAGE[1]	Absolute minimum of sensor input voltage	V	read / write
CLC_VOLTAGE[2]	Absolute maximum of sensor input voltage	V	read / write

Once the technology function has been set up, the GUD variables listed are not displayed automatically on the HMI interface.

SINUMERIK Operate

Proceed as follows to create and display the GUD variables for SINUMERIK Operate.

1. Set password
Enter the password for protection level 1: (machine manufacturer).
2. If no MGUD.DEF file exists yet:
Operating area switchover > Commissioning > System data > Open NC data directory:
Move the cursor to Definitions > New...
 - Name: MGUD
 - Type: DEF
Confirm with **OK**.
This opens the file in the editor.
1. Edit the GUD variable definitions
DEF CHAN REAL CLC_DISTANCE[3] ; Array of real, 3 elements
DEF CHAN REAL CLC_VOLTAGE[3] ; Array of real, 3 elements
M30
2. Save the file and close the editor
3. Activate the file MGUD.DEF

The GUD variables for clearance control are now displayed under:

Operating area switchover > Parameters > User variables > Channel GUD

4.11 Supplementary conditions

SINUMERIK NC

The newly created and already displayed GUD variables are only recognized and supplied with current values by the clearance control after an NC POWER ON-RESET.

Note

After creating the GUD variables, an NC POWER ON-RESET must be triggered so that the clearance control updates the GUD variables.

4.10 Function-specific alarm texts

For the procedure for creating function-specific alarm texts, see Creating alarm texts (Page 27).

The following table shows the alarm numbers of CLCX:

CLCX alarm number	CLC alarm number
75001	CLCX wrong MD configuration, error code: %2
75002	CLCX NC block %2 programming error, error code: %3
75003	CLCX NC block %2 invalid limits prog. in CLCX_LIM
75004	CLCX NC block %2 switched off with active tool radius compensation
75006	CLCX NC block %2 abrupt change of orientation
75007	CLCX NC block %2 sensor tip touched at pos. %3 mm
75008	CLCX NC block %2 position offset at lower limit %3
75009	CLCX NC block %2 position offset at upper limit %3
75011	CLCX NC block %2 prog. clearance exceeds %3 mm
75013	CLCX NC block %2 freely prog. direction error code %3
75014	CLCX NC block %2 freely prog. Direction real-time error code %3

4.11 Supplementary conditions

4.11.1 I/O modules

The analog output voltage of the clearance sensor for the A/D conversion must be connected to the NC via an I/O module with an analog input.

Connection options

With a SINUMERIK NCU, the SIMATIC ET 200S I/O is connected via PROFINET IO. The clearance sensor is connected via an analog S7 I/O module.

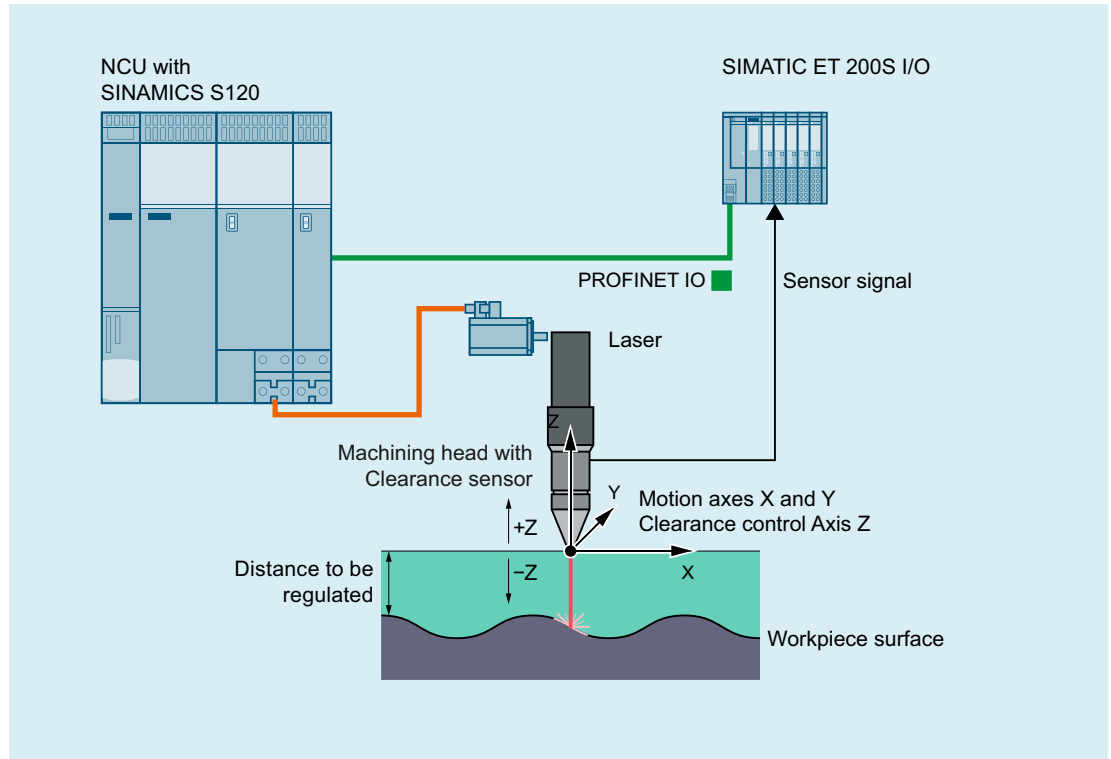


Figure 4-10 I/O modules connection for SINUMERIK NCU

Suitable I/O modules

As the A/D conversion time directly affects the dead time of the clearance control loop, only one I/O module may be used with low conversion time.

Suitable SIMATIC S7 I/O modules for the clearance control are, for example:

Analog I/O module 2 AI, U, High Speed for ET 200S

Analog I/O module 2 AO, U, High Speed for ET 200S

Suitable modules are described under Setting up the analog sensor input (Page 43).

4.11 Supplementary conditions

Connection of the I/O modules

The SIMATIC peripheral devices of the ET200 series, e.g. ET200M, are introduced and configured in the S7 project as usual by means of hardware configuration.

Note

To check whether a module selected from the hardware catalog complies with the module in the automation system, the following procedure is recommended:

1. Note the article numbers of all modules used in the automation system.
 2. Select the corresponding module in the hardware catalog and compare the article number of the module used in the automation system with the article number that is displayed in the hardware catalog. Both article numbers must match one another.
-

4.11.1.1 External smoothing filters

If an external filter is to be interconnected to smooth the output voltage of the clearance sensor before the A/D conversion of the output voltage by the I/O module, please ensure that the resulting time constant is small in relation to the NC position controller cycle.

Note

It is better for the control if electromagnetic shielding is used to ensure a large signal-noise ratio than if smoothing filters are used in the signal path.

4.11.2 Function-specific supplementary conditions

Complete NC stop

If in conjunction with an NC Stop, not only the programmed path motion but also the traversing movement of the clearance-controlled axes should be stopped, the following NC/PLC interface signals must be set:

- `<Chan>.basic.out.ncStop = 1` (NC stop)
- `<Chan>.basic.out.ncStopAxesAndSpindles = 1` (NC stop axes and spindles)

Tracking

If a clearance-controlled axis is to be switched as an alarm response or via the corresponding NC/PLC interface signal to "Track" mode, setpoint output will also cease for clearance control on this axis.

Traversing without software limit switch

If the clearance-controlled axes are to be moved without reference (traversing without software limit switch), values must nevertheless be entered in the axis-specific software limit switch machine data for the axes moved by the CLCX which do not restrict the intended traversing range:

- MD36100 \$MA_POS_LIMIT_MINUS (1st software limit switch minus)
- MD36110 \$MA_POS_LIMIT_PLUS (1st software limit switch plus)
- MD36120 \$MA_POS_LIMIT_MINUS2 (2nd software limit switch minus)
- MD36130 \$MA_POS_LIMIT_PLUS2 (2nd software limit switch plus)

The clearance control also considers these machine data in the non-referenced state of an axis.

No influence on the digital/analog inputs from the PLC

Both the analog input for the input voltage of the clearance sensor and the digital input used by the clearance control for external tip-touch detection cannot be influenced by the PLC as usual (lockable/overwritable):

- `<Nc>.digitalInputs[1].out.lock` (lock of digital NC inputs)
- `<Nc>.analogInputs[1].out.lock` (lock of analog NC inputs)

The CLCX reads the hardware status without PLC manipulation.

Gantry axes: Only leading axes

Only one of the clearance-controlled axes may be configured for the 3D-CLC as the leading axis of a gantry system:

MD37100 \$MA_GANTRY_AXIS_TYPE (Gantry axis definition)

The use of following axes in a gantry system is not permitted.

If one of the three geometry axes of the 3D CLC is a gantry leading axis, the CLC automatically outputs the offset value to its first gantry following axis as well. The axial CLC does not support gantry axes.

Displaying the axis position

The actual, current target axis position of a clearance-controlled axis as the sum of the programmed axis position and the current position offset of the clearance control is not displayed in the WCS/MCS positions of the machine root screen:

- SINUMERIK HMI Advanced:
The actual current axis position is only shown in the service screen: **Operating area switchover > Diagnostics > Service displays > Axis/spindle** displayed as the "Position setpoint" and as "Actual position value".
- SINUMERIK Operate:
The actual current axis position is only shown in the service screen: **Operating area switchover > Diagnostics > Axis dialog > Service axis** > displayed as position setpoints and as the "Actual position value measuring system 1 and 2".

4.11 Supplementary conditions

No virtual axes

Clearance-controlled axes must not be parameterized as virtual axes:

MD30132 \$MA_IS_VIRTUAL_AX[<axis>] (Axis is virtual axis)

Computing time requirement

On controllers where the computing time load of the NCU due to the cycle times of the interpolation and position control cycle is already extremely high compared to the recommended setting, the additional computing time requirement of the technology function "Clearance control" must be taken into account.

The additional required computing time results after the activation of the clearance control in the CLC (. . .) part program. If the interpolation or position controller cycle is exceeded, the following alarm appears:

- Alarm: "4240 Computing time overflow at IPO or position controller level, IP point in part program"

Execution of the part program is aborted.

1D clearance control function

Alarm "1016: System error, ID550010" can occur in the following situation:

- The clearance-controlled axis (e.g. Z axis) is parameterized as geometry axis
- Within a command sequence in which STOPRE is triggered implicitly or explicitly, the clearance control is deactivated with CLC(0)

We therefore recommend that the clearance-controlled axis of a 1D clearance control function (e.g. Z axis) is parameterized so that it is no longer the geometry axis of the channel.

Parameterization: Machine data

- MD20050 \$MC_AXCONF_GEOAX_ASSIGN_TAB[<Z axis>] = 0
- MD20060 \$MC_AXCONF_GEOAX_NAME_TAB[<Z axis>] = "NO_Z_AXIS"

Programming: Rotations around the Z axis

Since the Z axis is no longer a geometry axis in this configuration, its name can no longer be used for rotary frames. Therefore, for rotations around the Z axis, the predefined procedure CRPL () must be used instead of the predefined function CROT () :

CROT (Z, <angle>) → CRPL (1, <angle>)

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.out.ncStop	LBP_Chan*.A_NCStop	DB21,DBX7.3
<Chan>.basic.out.ncStopAxesAndSpindles	LBP_Chan*.A_NCStopASp	DB21,DBX7.4

Basic Program Plus	Basic Program	
<Nc>.digitalInputs[1].out.lock	LBP_NC.A_Disabl_Inp1 - 40	DB10.DBX0.0..7, .DBX122.0..7, DBX124.0..7, DBX126.0..7, DBX128.0..7
<Nc>.analogInputs[1].out.lock	LBP_NC.A_Disabl_analIn1 - 8	DB10.DBX146.0..7

4.11 Supplementary conditions

Transformation package handling (Run MyCC / RCTRA)

5

5.1 Brief description

The handling transformation package has been designed for use on **manipulators** and **robots**. The package is a type of modular system, which enables the customer to configure the transformation for his machine by setting machine data (provided that the relevant kinematics are included in the handling transformation package).

Section structure

The "Handling transformation package" section deals with the following topics:

- The "Kinematic transformation (Page 88)" section describes the kinematic transformation environment.
- The "Definition of terms (Page 88)" section explains some basic terms.
- The "Configuration of the kinematic transformation (Page 91)" section explains the machine data required for the configuration.
- The "Kinematic descriptions (Page 106)" section uses configuring examples to illustrate the most commonly used 2-axis to 5-axis kinematics that can be configured with the "Handling transformation package".
- The "Tool orientation (Page 131)" to "Tool programming (Page 139)" sections handle the programming, describing orientation programming, the entry of tool parameters and transformation calls.

Abbreviations

FL	Flange coordinate system
HP	Wrist point coordinate system
IRO	Internal robot coordinate system
p_3, q_3, r_3	Coordinates of the last basic axis
RO	Robot or base center point coordinate system (= basic coordinate system)
WS	Workpiece coordinate system
T	Tool coordinate system
x_3, y_3, z_3	Coordinates of the first wrist axis

5.2 Kinematic transformation

Task of a transformation

The purpose of a transformation is to transform movements in the tool tip, which are programmed in a Cartesian coordinate system, into machine axis positions.

Fields of application

The handling transformation package described here has been designed to cover the largest possible number of kinematic transformations implemented solely via parameter settings in machine data. Currently, kinematics with two to maximum six axes to be included in the transformation can be configured with this package, corresponding to up to five spatial degrees of freedom. In this case, a maximum of 3 degrees are available for translation and 3 degrees for orientation. This package thus allows a tool (milling cutter, laser beam) to be oriented by a 5-axis machine in any desired relation to the workpiece in every point of the machining space. The workpiece is always programmed in the rectangular workpiece coordinate system; any programmed or set frames rotate and shift this system in relation to the basic system. The kinematic transformation then converts this information into motion instructions for the real machine axes. The kinematic transformation requires information about the design (kinematics) of the machine, which are stored in machine data.

Kinematic categories

The handling transformation package is divided into two categories of kinematics, which can be selected via machine data:

MD62600 \$MC_TRAFO6_KINCLASS (kinematic category)

- STANDARD: This category includes the most commonly used kinematics.
- SPECIAL: Special kinematics

5.3 Definition of terms

5.3.1 Units and directions

Lengths and angles

In the transformation machine data, all lengths are specified in millimeters or inches and, unless otherwise stated, all angles in degrees at intervals of $[-180^\circ, 180^\circ]$.

Direction of rotation

In the case of angles, arrows in the drawings always indicate the mathematically positive direction of rotation.

5.3.2 Definition of positions and orientations using frames

In order to make a clear distinction from the term "frame" as it is used in the NC language, the following description explains the meaning of the term "frame" in relation to the handling transformation package.

Frame

A frame can be used to translate one coordinate system into another. In this respect, a distinction must be made between translation and rotation. Translation results in an offset; a rotation, a rotation of the coordinate system with respect to the initial system. Coordinates X, Y and Z are used as designator for a translation. They are defined so as to result in a legal coordinate system.

Translation

A translation is always specified in relation to the coordinate directions of the initial system. These directions are assigned to machine data as follows:

- X direction: ..._POS[0]
- Y direction: ..._POS[1]
- Z direction: ..._POS[2]

Rotation

A rotation is specified in RPY angles A, B and C (R=roll, P=pitch, Y=yaw). The positive direction of rotation is defined by the right hand rule, i.e. if the thumb on the right hand is pointing in the direction of the axis of rotation, then the fingers are pointing in the positive angular direction. Angles A and C are defined in the interval [-180; +180], and B in the interval [-90; +90].

The definitions of the RPY angles are as follows:

- Angle A: 1. rotation about the Z axis of the initial system
- B angle: 2. Rotation through the rotated Y axis
- C angle: 3. rotation about the twice rotated X axis

The RPY angles are assigned to machine data as follows:

- Angle A: ..._RPY[0]
- B angle: ..._RPY[1]
- C angle: ..._RPY[2]

The following diagram shows an example for a rotation through the RPY angle. The initial coordinate system X1, Y1, Z1 is transitioned into the coordinate system X4, Y4, Z4

1. Rotation through angle A around the Z1 axis
2. Rotation through angle B around the Y2 axis
3. Rotation through angle C around the X3 axis.

5.3 Definition of terms

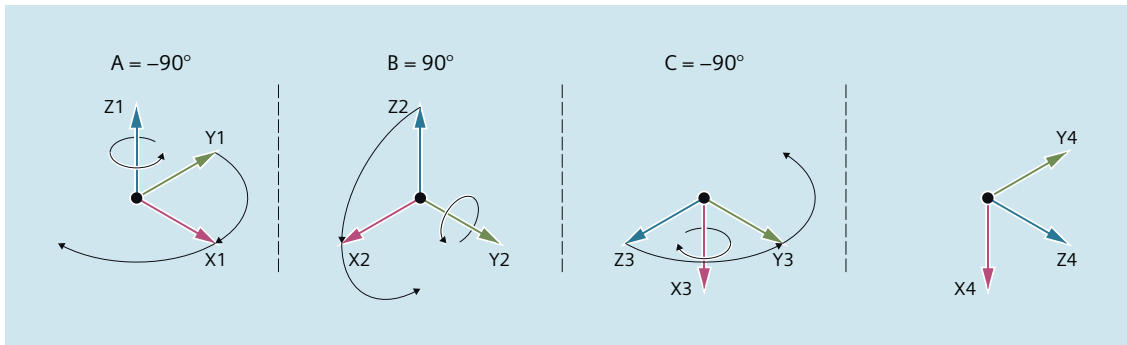


Figure 5-1 Example of rotation through RPY angles

5.3.3 Definition of a joint

Meaning

A sliding joint is implemented using a translatory axis, and a swivel joint, using a rotary axis.

The basic axis identifiers result from the arrangement and sequence of the individual joints. These are specified using letter codes (S, C, R, N).

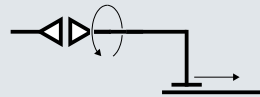
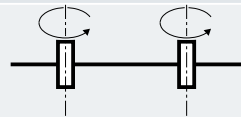
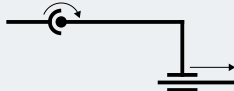
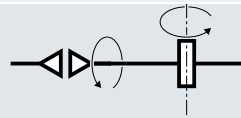
Identifier	Designation	Icons		
S	Sliding joint			
C	Sliding joint Swivel joint			
R	Swivel joint Swivel joint			
	Swivel joint Sliding joint			
N	Swivel joint Swivel joint			
FL	Flange for mounting of the tool		→ Positive axis direction → Positive axis direction ⊗ – Vertical into the plane of projection ⊙ – Vertical out of the plane of protection ↻ Positive direction of rotation Axis directions parallel to one another ⊥ Axis directions perpendicular to one another	
T	Tool			

Figure 5-2 Joint identifying letters

5.4 Configuration of a kinematic transformation

Meaning

In order to ensure that the kinematic transformation can convert the programmed values into axis motions, it must have access to some information about the mechanical construction of the machine. This information is stored in machine data:

- Axis assignments
- Geometry information

5.4 Configuration of a kinematic transformation

5.4.1 General machine data

MD24100 \$MC_TRAFO_TYPE_1 (definition of transformation 1 in the channel)

The value 4100 must be entered in this data for the handling transformation package.

MD24110 \$MC_TRAFO_AXES_IN_1 (axis assignment for transformation)

The axis assignment at the transformation input defines which transformation axis is mapped internally onto a channel axis. It is specified in machine data:

MD24110 \$MC_TRAFO_AXES_IN_1

There is a predetermined axis sequence for the handling transformation package, The basic axes are entered in the first three components (index 0...2). The upper three components (index 3...5) are assigned to the manual axes.

- MD24110 \$MC_TRAFO_AXES_IN_1[0] = 1 ; 1. Basic axis
- MD24110 \$MC_TRAFO_AXES_IN_1[1] = 2 ; 2. Basic axis
- MD24110 \$MC_TRAFO_AXES_IN_1[2] = 3 ; 3. Basic axis
- MD24110 \$MC_TRAFO_AXES_IN_1[3] = 4 ; 1. Manual axis
- MD24110 \$MC_TRAFO_AXES_IN_1[4] = 5 ; 2. Manual axis
- MD24110 \$MC_TRAFO_AXES_IN_1[5] = 6 ; 3. Manual axis

MD24120 \$MC_TRAFO_GEOAX_ASSIGN_TAB_1 (assignment between geometry axis and channel axis for transformation 1)

The translational degrees of freedom that are available for the transformation are entered via machine data:

MD24120 \$MC_TRAFO_GEOAX_ASSIGN_TAB_1

The 3 geometry axes normally correspond to Cartesian axis directions X, Y and Z.

In this case, the first three channel axis numbers must be transferred from MD24110 \$MC_TRAFO_AXES_IN_1 to MD24120 \$MC_TRAFO_GEO_AX_ASSIGN_TAB_1.

- MD24120 \$MC_TRAFO_GEO_AX_ASSIGN_TAB_1[0] = 1
- MD24120 \$MC_TRAFO_GEO_AX_ASSIGN_TAB_1[1] = 2
- MD24120 \$MC_TRAFO_GEO_AX_ASSIGN_TAB_1[2] = 3

5.4.2 Parameterization using geometry data

Modular principle

The machine geometry is parameterized according to a type of modular principle. With this method, the machine is successively configured in geometry parameters from its base center point to the tool tip, thereby producing a closed kinematic loop. In this case, frames are used to describe the geometry. While the control is powering up, the configuration machine data is checked and alarms displayed when relevant. All axes of the mode group are tracked as alarm response. The alarms can be deleted using POWER ON.

As shown in the following diagram "Closed kinematic loop illustrated by the example of a robot", the kinematic transformation converts the tool operating point (tool coordinate system: X_{TO} , Y_{TO} , Z_{TO}), specified relative to the base coordinate system (BCS = robot coordinate system: X_{RO} , Y_{RO} , Z_{RO}), in machine axis values (MCS positions: A1, A2, A3, ...). The operating point (XWZ, YWZ, ZWZ) is specified in the part program in relation to the workpiece to be machined (workpiece coordinate system WCS: X_{WS} , Y_{WS} , Z_{WS}). The programmable frames make it possible to create an offset between the workpiece coordinate system (WCS) and the base coordinate system (BCS).

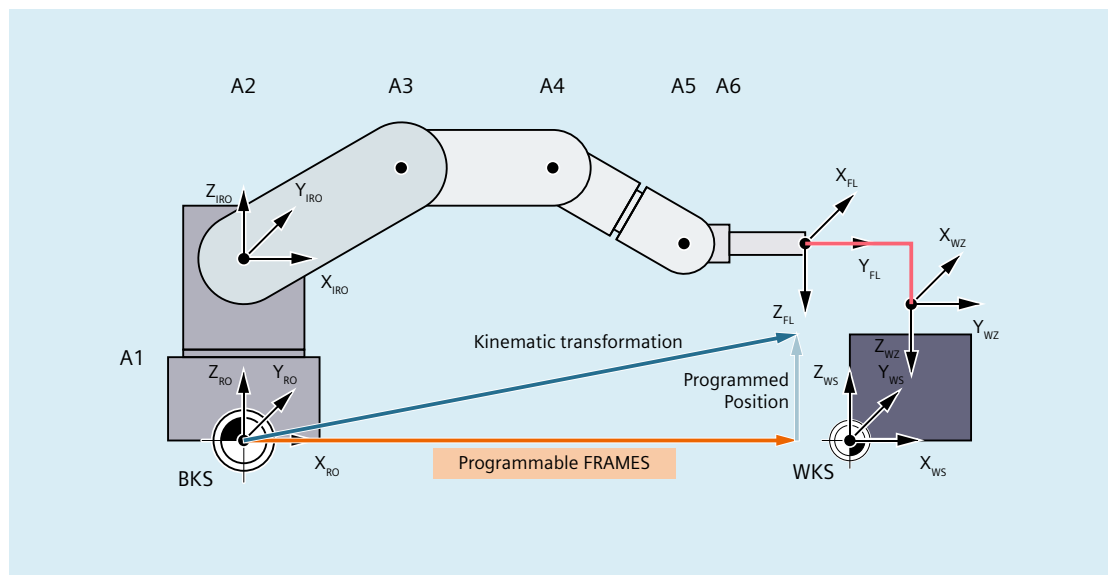


Figure 5-3 Closed kinematic loop illustrated by the example of a robot

Note

For more detailed information about coordinate systems, please see:

Further information:

Function Manual Basic Functions; Axes, Coordinate Systems, Frames" > "Coordinate Systems"

The following machine data is available for configuring kinematic transformations:

5.4 Configuration of a kinematic transformation

Frame between base center point and internal coordinate system

The frame T_IRO_RO links the base center point of the machine (BCS = RO) with the first internal coordinate system (IRO) determined by the transformation.

- MD62613 \$MC_TRAFO6_TIRORO_RPY (frame between base center point and internal coordinate system (rotation component), $n = 0...2$)
- MD62612 \$MC_TRAFO6_TIRORO_POS (frame between base center point and internal coordinate system (position component), $n = 0...2$)

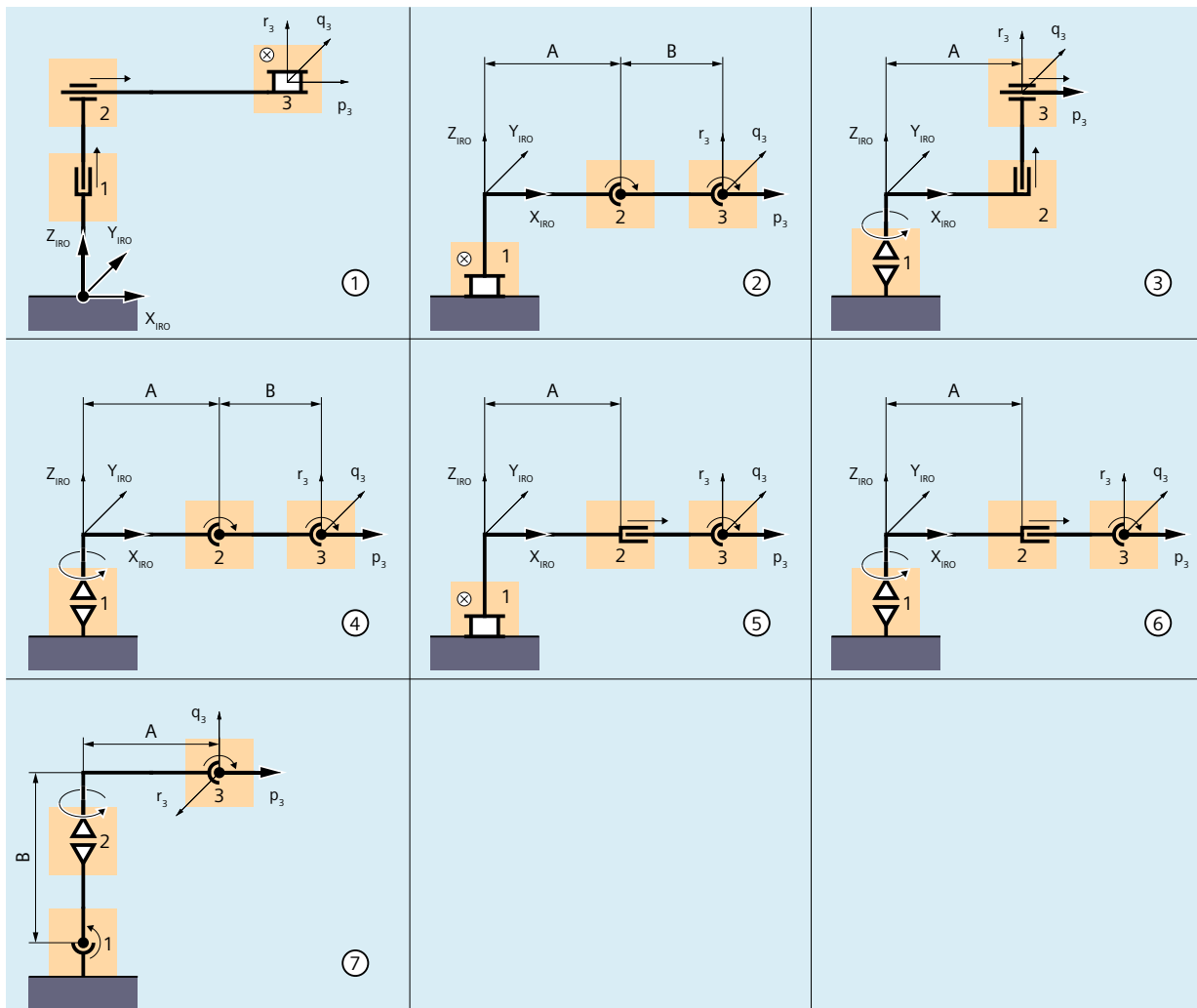
Basic axis arrangement

The type of basic axis arrangement is specified in machine data:

- MD62603 \$MC_TRAFO6_MAIN_AXES (basic axis identifier)

The first three axes included in the transformation are generally referred to as the "basic axes". They must always be mutually parallel or perpendicular.

The handling transformation package contains the basic axis kinematics shown in the following diagram. Every basic axis arrangement has its own special identifier (see Chapter "Definition of a joint (Page 90)").



- ① SS: MD62603 = 1, gantry (3 linear axes, rectangular)
- ② CC: MD62603 = 2, SCARA (1 linear axis, 2 rotary axes (in parallel))
- ③ CS: MD62603 = 6, SCARA (2 linear axes, 1 rotary axis (spin axis))
- ④ NR: MD62603 = 3, articulated arm (3 rotary axes (2 axes in parallel))
- ⑤ SC: MD62603 = 4, SCARA (2 linear axes, 1 rotary axis (swivel axis))
- ⑥ RR: MD62603 = 5, articulated arm (1 linear axis, 2 rotary axes (vertical))
- ⑦ NN: MD62603 = 7, articulated arm (3 rotary axes)

Figure 5-4 Overview of basic axis configuration

Basic axis lengths A and B

The basic axis lengths A and B are specified with machine data:

- MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB (basic axis lengths A and B, $n = 0 \dots 1$)

As Fig. "Overview of basic axis configurations" illustrates, these are specially defined for each type of basic axis.

5.4 Configuration of a kinematic transformation

Position of the 4th axis

Whether the 4th axis is mounted parallel, anti-parallel or perpendicular to the last rotary basic axis is specified in machine data:

- MD62606 \$MC_TRAFO6_A4PAR (axis 4 is parallel/anti-parallel to last basic axis)

Description of the wrist axis

Frame to attach the wrist axis

Frame T_X3_P3 links the last coordinate system of the basic axes with the first wrist axis coordinate system.

- MD62608 \$MC_TRAFO6_TX3P3_POS (attachment of wrist axis (position component), n = 0...2)
- MD62609 \$MC_TRAFO6_TX3P3_RPY (attachment of wrist axis (rotation component), n = 0...2)

Frame between wrist point and flange coordinate system

Frame T_FL_WP links the last hand wrist axis coordinate system with the flange coordinate system.

- MD62610 \$MC_TRAFO6_TFLWP_POS (frame between wrist point and flange coordinate system (position component), n = 0...2)
- MD62611 \$MC_TRAFO6_TFLWP_RPY (frame between wrist point and flange coordinate system (rotation component), n = 0...2)

This data is explained in more detail in the following sections.

Description of the wrist axis type

These parameters describe the wrist axis type.

- MD62604 \$MC_TRAFO6_WRIST_AXES (wrist axis identifier)
The term wrist axes generally refers to axes four to six.

Wrist axes included in every transformation

The fourth axis and all further axes are generally referred to as "wrist axes". The handling transformation package can only identify wrist axes with rotary axes. The wrist axis identifier for three-axis wrists is entered in machine data:

- MD62604 \$MC_TRAFO6_WRIST_AXES (wrist axis identifier)

When there are fewer than three wrist axes, the identifier for the inclined wrist axis or a central wrist axis is entered in the machine data:

- MD62604 \$MC_TRAFO6_WRIST_AXES (wrist axis identifier)

The current software only supports the central and inclined wrist axis types.

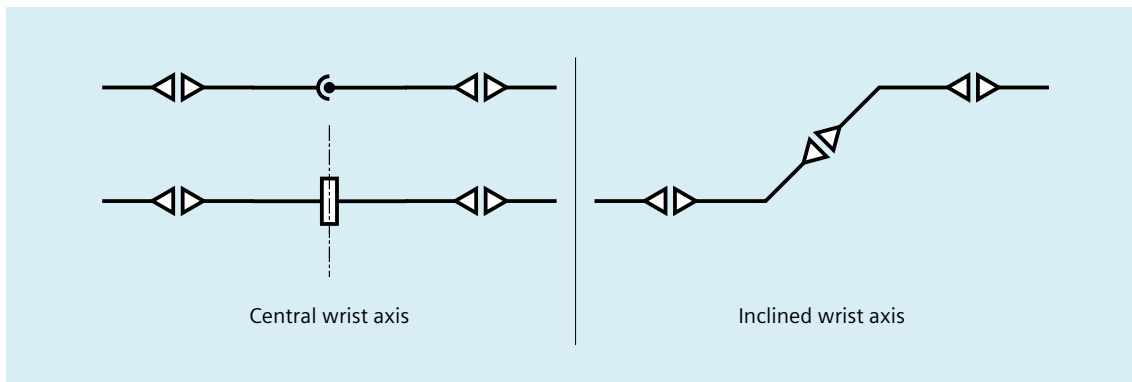


Figure 5-5 Overview of wrist axis configuration

Parameterization of wrist axes

With the following machine data, using a special frame type, the geometry of the wrist axis and/or the position of the coordinate system in the wrist axis with respect to one another is defined.

- MD62614 \$MC_TRAFO6_DHPAR4_5A (parameter A for configuring the wrist axis, $n = 0 \dots 1$)
- MD62615 \$MC_TRAFO6_DHPAR4_5D (parameter D for configuring the wrist axis, $n = 0 \dots 1$)
- MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA. (Parameter ALPHA for configuring the wrist axis, $n = 0 \dots 1$)

For this purpose, the machine data corresponds to certain frame components (see Section "Definition of positions and orientations using frames (Page 89)"):

- X component: MD62614 \$MC_TRAFO6_DHPAR4_5A $\triangleq$ _POS[0] (x component)
- Z component: MD62615 \$MC_TRAFO6_DHPAR4_5D $\triangleq$ _POS[2] (z component)
- C angle: MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA $\triangleq$ _RPY[2] (C angle)

The other components of the frame are zero.

Central wrist axis

On a central wrist axis, all wrist axes intersect at one point. All parameters must be set as shown in Table "Configuring data for a central wrist axis".

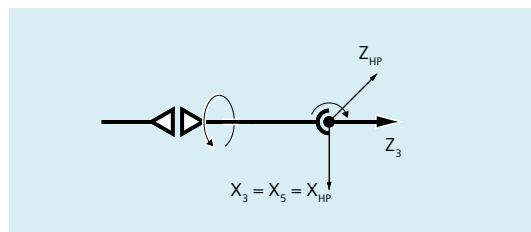


Figure 5-6 Central wrist axis, 5-axis machine

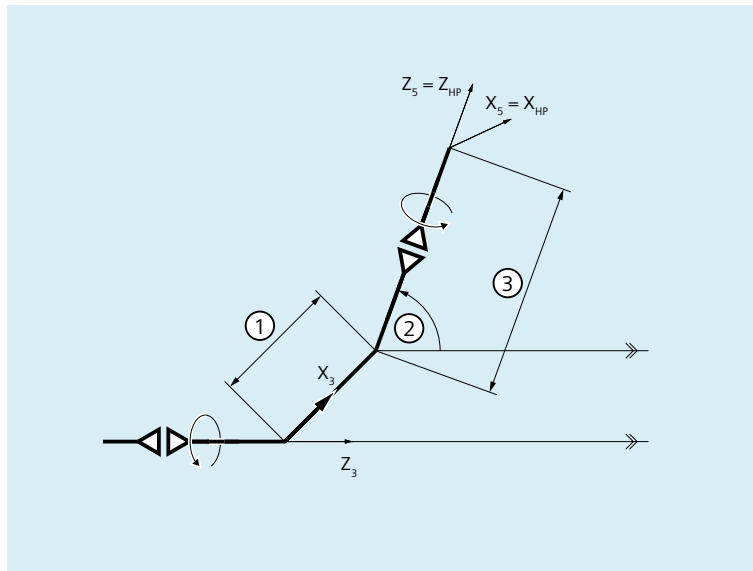
5.4 Configuration of a kinematic transformation

Table 5-1 Configuring data for a central wrist axis

Machine data	Value
MD62604 \$MC_TRAFO6_WRIST_AXES	2
MD62614 \$MC_TRAFO6_DHPAR4_5A	[0.0, 0.0]
MD62615 \$MC_TRAFO6_DHPAR4_5D	[0.0, 0.0]
MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA	[-90.0, 90.0]

Inclined wrist axis

The inclined wrist axis differs from the central wrist axis in two respects, i.e. the axes do not intersect nor are they mutually perpendicular. Parameters a_4 , d_5 , and α_4 are available for this type of hand, as shown in Table "Configuring data for a central wrist axis".



- ① MD62614 \$MC_TRAFO6_DHPAR4_5A[0]
- ② MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA[0]
- ③ MD62615 \$MC_TRAFO6_DHPAR4_5D[1]

Figure 5-7 Inclined wrist axis, 5-axis machine

Table 5-2 Configuring data for an inclined wrist axis (5-axis)

Machine data	Value
MD62604 \$MC_TRAFO6_WRIST_AXES	6
MD62614 \$MC_TRAFO6_DHPAR4_5A	[a4, 0.0]
MD62615 \$MC_TRAFO6_DHPAR4_5D	[0.0, d5]
MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA	[α4, 0.0]

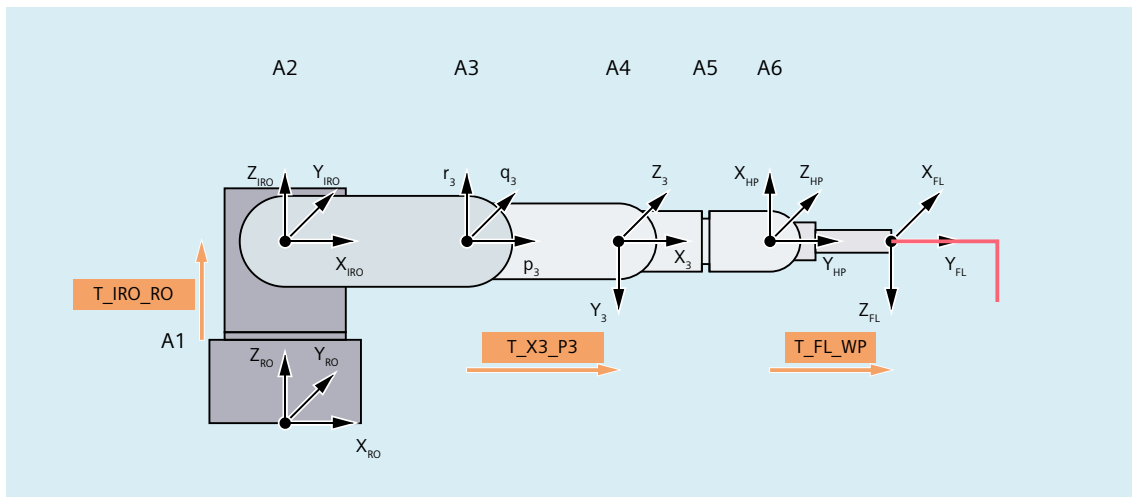


Figure 5-8 Link frames

Frame: T_IRO_RO

Frame T_{IRO_RO} provides the link between the base center point coordinate system (RO) defined by the user and the internal robot coordinate system (IRO). The internal robot coordinate system is predefined in the handling transformation package for each basic axis type and included in the kinematic diagrams for the basic axis arrangements. The base center point system is in the Cartesian zero point of the machine, corresponding to the basic coordinate system. If no **FRAMES** are programmed, the basic coordinate system equals the workpiece coordinate system.

Note

For more detailed information about **FRAMES**, please see:

Further information

Programming Manual Fundamentals

Frame T_{IRO_RO} is not subject to any restrictions for 5-axis kinematics.

The following restrictions apply in relation to 4-axis kinematics:

- The first rotary axis must always be parallel/anti-parallel to one of the coordinate axes of the base center point coordinate system (RO).
- No further restrictions apply to type SS basic axes.
- In the case of type CC, CS or SC basic axes, no further restrictions apply provided that the 4th axis is parallel to the last rotary basic axis.
- With respect to all other basic axes, and basic axes of type CC, CS or SC if the 4th axis is perpendicular to the last rotary basic axis, the Z axis of RO must be parallel to the Z axis of IRO.

5.4 Configuration of a kinematic transformation

Frame: T_X3_P3

Frame T_X3_P3 describes the method used to attach the wrist axis to the basic axes. Frame T_X3_P3 is used to link the coordinate system of the last basic axis (p3_q3_r3 coordinate system) with the coordinate system of the first wrist axis (x3_y3_z3 coordinate system). The p3_q3_r3 coordinate system is included in the kinematic diagrams for the basic axis arrangements.

The z3 axis is always on the 4th axis.

Depending on the number of axes to be included in the transformation, frame T_X3_P3 is subject to certain restrictions relating to the wrist and basic axes:

- For 5-axis kinematics, frame T_X3_P3 can be freely selected in the following cases:
 - If the basic axes are of the SS type.
 - If the basic axes are type CC, CS, or SC, the transformation must either include a central wrist axis or the 4th axis must be positioned parallel to the last rotary basic axis.
 - If the basic axes are type NR or RR, the transformation must either include a central wrist axis or the 4th axis must be positioned in parallel to the last basic rotary axis and an X flange must intersect the 5th axis.
 - If the basic axes are of the NN type, the transformation must include a central wrist axis.
- For 4-axes, ensure that the Z3 axis is always parallel/anti-parallel or perpendicular to the last basic axis.

Frame: T_FL_WP

Frame T_FL_WP links the flange with the last internal coordinate system (wrist point coordinate system) preset by the handling transformation package.

For kinematics with less than 6 axes, this frame is subject to certain restrictions. These restrictions are explained by the relevant kinematics.

Other configuring data

Number of transformed axes

The number of axes included in the transformation is defined in machine data:

- MD62605 \$MC_TRAFO6_NUM_AXES (number of transformed axes)

The number of transformed axes can currently lie between two and six axes.

Changing the axis sequence

Rearrangement of axes: MD62620

Note

With certain types of kinematics, it is possible to transpose axes without changing the behavior of the kinematic transformation. Machine data:

MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)

The axes on the machine are numbered consecutively from 1 to 6 and must be entered in the internal sequence in machine data:

MD62620 \$MC_TRAFO6_AXIS_SEQ[0] ...[4]

All other axis-specific machine data refer to the sequence of axes on the machine.

Table 5-3 Changing the axis sequence

Basic axis kinematics	Options for changing axis sequence
SS, CC	Any
SC	1 and 2
CS	2 and 3

Example 1

There are two kinematics, such as those shown in Fig. "Rearrangement of axes 1". Kinematic 1 is directly included in the handling transformation package. It corresponds to a CC kinematic with a wrist axis parallel to the last basic rotary axis.

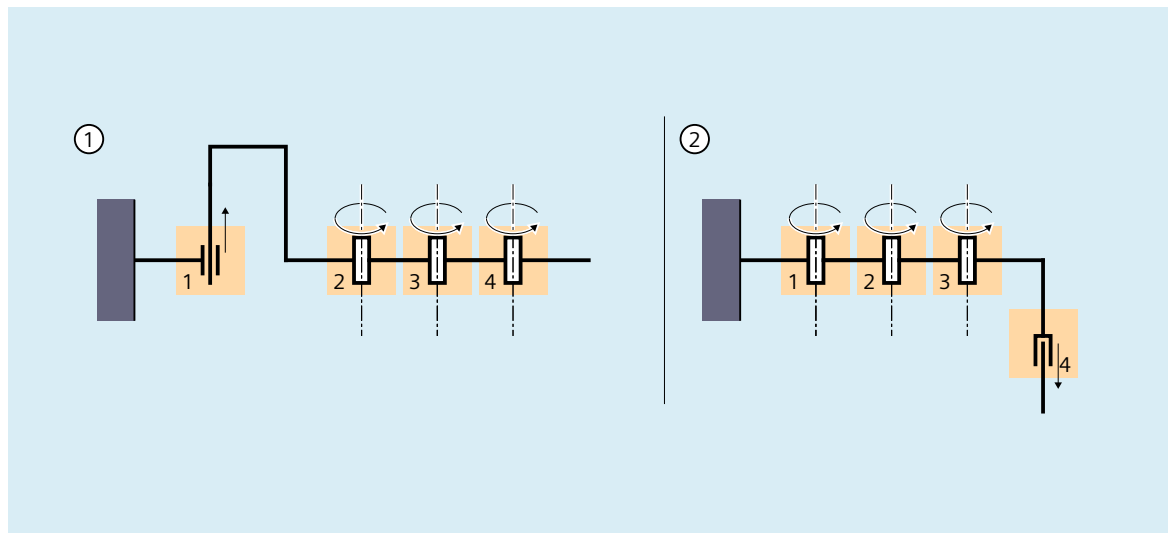
Kinematic 2 is the same as kinematic 1 inasmuch as it is irrelevant for the resulting robot movement whether the translational axis is axis 1 or axis 4. In this instance, the data for kinematic 2 must be entered as follows in machine data:

- MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)

The input is as follows:

- MD62620 \$MC_TRAFO6_AXIS_SEQ[0] = 4
- MD62620 \$MC_TRAFO6_AXIS_SEQ[1] = 1
- MD62620 \$MC_TRAFO6_AXIS_SEQ[2] = 2
- MD62620 \$MC_TRAFO6_AXIS_SEQ[3] = 3

5.4 Configuration of a kinematic transformation



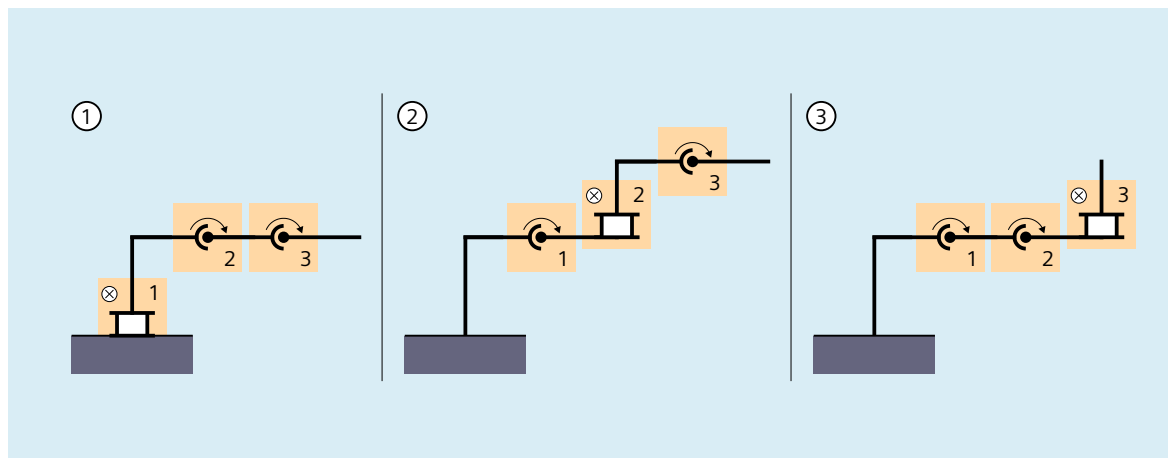
① Kinematics 1

② Kinematics 2

Figure 5-9 Rearrangement of axes 1

Example 2

This example involves a SCARA kinematic transformation as illustrated in Fig. "Rearrangement of axes 2", in which the axes can be freely transposed. Kinematic 1 is directly included in the handling transformation package. It corresponds to a CC kinematic. As regards the transposition of axes, it is irrelevant how many wrist axes are involved in the transformation.



① Kinematics 1: Axis sequence 1 2 3

② Kinematics 2: Axis sequence 2 1 3

③ Kinematics 3: Axis sequence 2 3 1

Figure 5-10 Rearrangement of axes 2

Changing the directions of axes

A rotational or offset direction is preset for each axis in the handling transformation package. This direction is not necessarily the same as the corresponding direction on the machine. The following machine data must be set to **-1** for the relevant axis if the direction is to be inverted, or otherwise to **+1**:

- MD62618 \$MC_TRAFO6_AXES_DIR[] (matching of physical and mathematical directions of rotation [axis no.]: 0...5)

Adapting the zero points of the axes

Offset between mathematical and mechanical zero points

The mathematical zero points of axes are preset in the handling transformation package. However, the mathematical zero point does not always correspond to the mechanical zero point (calibration point) of axes. In order for the zero point of each axis to fit one another, the difference between the mathematical zero point and the alignment point for each axis must be entered in the following machine data:

- MD62617 \$MC_TRAFO6_MAMES[] (offset between mathematical and mechanical zero points [axis no.]: 0...5)

The deviation to be entered corresponds to the difference between the mechanical zero point and the mathematically positive direction of rotation of the axis.

Example

Jointed arm kinematics are shown in the following diagram.

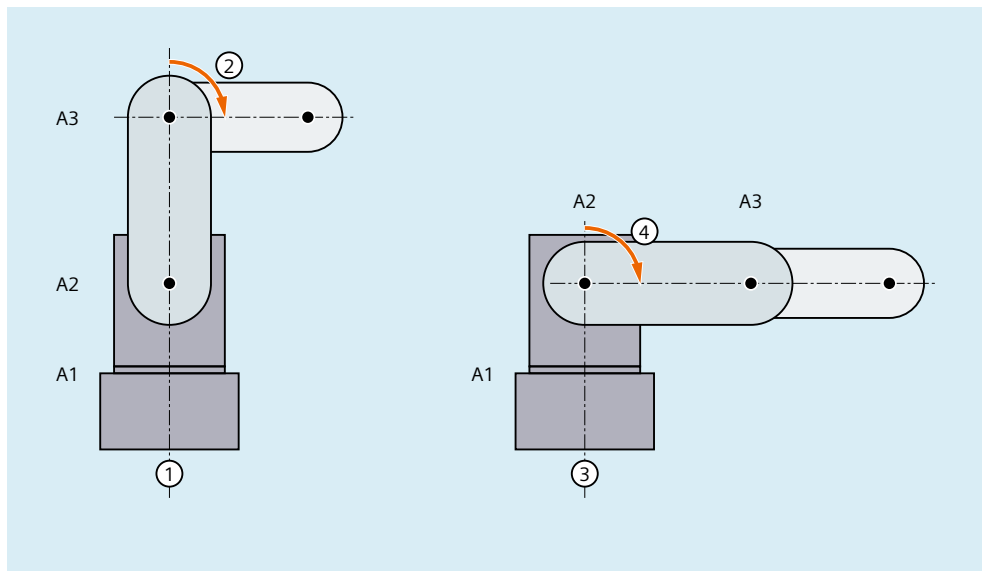
In the mathematical zero position ②, axis 2 (A2) has a value of 90°. This value should be entered into the machine data for axis 2 (index 1):

- MD62617 \$MC_TRAFO6_MAMES[1] = 90

Axis 3 (A3) is counted relative to the previous axis 2 (A2) in the kinematic chain, and in the mechanical zero position ① has the value: -90°:

- MD62617 \$MC_TRAFO6_MAMES[2] = -90

5.4 Configuration of a kinematic transformation



- ① mechanical zero position (adjusted position):
- ② MD62617 \$MC_TRAFO6_MAMES[2] = -90
- ③ mathematical zero position
- ④ MD62617 \$MC_TRAFO6_MAMES[1] = 90

Figure 5-11 Matching mathematical and mechanical zero points

Axis type for transformation

Which axis type is handled is specified in machine data:

- MD62601 \$MC_TRAFO6_AXES_TYPE (axis type for transformation [axis no.]: 0...5)

The transformation package distinguishes between the following axis types:

- Linear axis: MD62601 = 1
- Rotary axis: MD62601 = 3

Velocity and acceleration values for override control that limits the dynamic response

**CAUTION****Unnecessarily high limiting default values**

Since software release **RCTRA 07.05.00**, RCTRA transformation for limiting the Cartesian velocities and acceleration levels, takes into account machine data **MD62629** to **MD62632**.

In order to avoid unnecessarily limiting the velocities and acceleration levels, this machine data must be adapted to the machine capabilities. Frequently, unnecessarily high limiting default values are set here, as - up until now - this data has not been used.

For traversing axes with active RCTRA transformation and override controls that limit the dynamic performance, dedicated substitute limit values have been introduced for velocities and acceleration levels for Cartesian motion components.

The override controller within the transformation is only active if motion is interpolated, whose dynamic limits are not secured - or not completely secured - through motion planning in the Look Ahead function of the control.

The velocity controller is active in the following cases:

- When traversing in the JOG mode with transformation
- For overlaid motion in realtime, e.g. via system variable \$AA_OFF[X]
- For velocity limits as a result of Safety Integrated (SG level active)

Cartesian velocity

The velocities for individual translatable motion directions when traversing with override controller are specified using machine data:

- MD62629 \$MC_TRAFO6_VELCP[i] (Cartesian velocity [No.]: 0...2)
 - Index i = 0 : X component of basic system
 - Index i = 1 : Y component of basic system
 - Index i = 2 : Z component of basic system

Cartesian acceleration rates

The acceleration levels for individual translatable motion directions when traversing with override controller are specified using machine data:

- MD62630 \$MC_TRAFO6_ACCCP[i] (Cartesian acceleration levels [no.]: 0...2)
 - Index i = 0 : X component of basic system
 - Index i = 1 : Y component of basic system
 - Index i = 2 : Z component of basic system

Orientation angle velocities

The velocities for individual orientation directions when traversing with override controller are specified using machine data:

- MD62631 \$MC_TRAFO6_VELORI[i] (orientation angle velocities [no.]: 0...2)
 - Index i = 0 : A angle
 - Index i = 1 : B angle
 - Index i = 2 : C angle

5.5 Descriptions of kinematics

Orientation angle acceleration rates

The acceleration levels for individual orientation directions when traversing with override controller are specified using machine data:

- MD62632 \$MC_TRAFO6_ACCORI[i] (orientation angle acceleration levels [no.]: 0...2)
 - Index i = 0 : A angle
 - Index i = 1 : B angle
 - Index i = 2 : C angle

Reduction factor for the velocity controller

A reduction factor for the velocity controller in the JOG mode can be specified with the machine data:

- MD62633 \$MC_ROBX_DYN_LIM_REDUCE (reduction factor for the velocity controller)

Time constant for the velocity controller

A time constant for the PT1 filter of the velocity controller can be specified with the machine data:

- MD62634 \$MC_ROBX_VEL_FILTER_TIME (time constant for the velocity controller)

5.5 Descriptions of kinematics

The following descriptions of kinematics for transformations involving 2 to 5 axes explain the general configuring procedure first before describing how the machine data need to be configured, using a configuring example for each kinematic type. These examples do not include all possible lengths and offsets. The direction data refer to the positive directions of traversal and rotation for the transformation. The axis positions correspond to their zero position for the relevant transformation.

5.5.1 3-axis kinematics

3-axis kinematics normally possess three translational degrees of freedom, but do not have a rotational degree of freedom for orientation. As a consequence, 3-axis kinematics only have basic axes.

Configuration

The procedure for configuring a 3-axis kinematic is as follows:

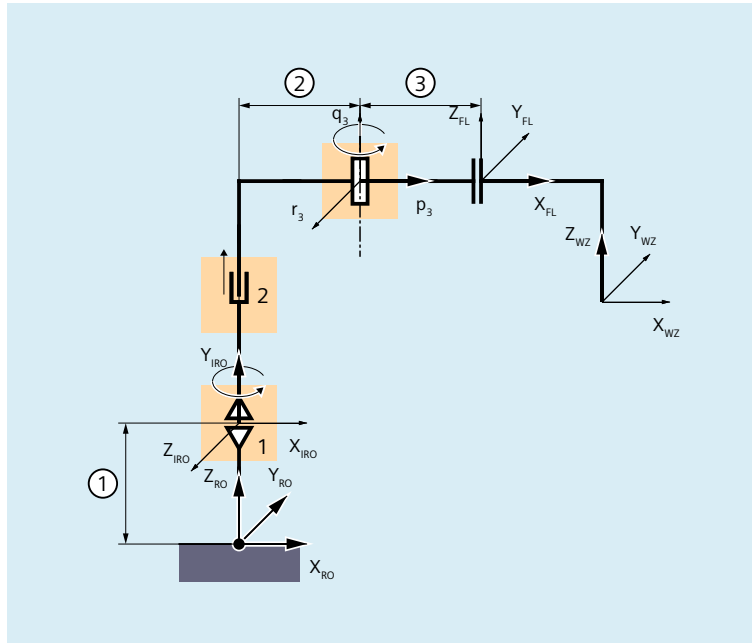
1. Enter "Standard" kinematic category in machine data:
MD62600 \$MC_TRAFO6_KINCLASS (kinematic category)
2. Set the number of axes for transformation in machine data:
MD62605 \$MC_TRAFO6_NUM_AXES = 3 (number of transformed axes)

3. Compare the basic axes with the basic axes contained in the handling transformation package. → Entries in the basic axis identifier in the machine data:
MD62603 \$MC_TRAFO6_MAIN_AXES (basic axis identifier)
4. If the axis sequence is not the same as the normal axis sequence, it must be corrected in machine data:
MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)
5. As identifier for the wrist axes, the following machine data must be set to 1 (no wrist axis):
MD62604 \$MC_TRAFO6_WRIST_AXES = 1 (wrist axis identifier)
6. Enter the axis types for the transformation in machine data:
MD62601 \$MC_TRAFO6_AXES_TYPE (axis type for transformation)
7. Compare the directions of rotation of axes with the directions defined in the handling transformation package and correct in machine data:
MD62618 \$MC_TRAFO6_AXES_DIR (matching physical and mathematical directions of rotation)
8. Enter the mechanical work offset in machine data:
MD62617 \$MC_TRAFO6_MAMES (offset between mathematical and mechanical zero points)
9. Enter the basic axis lengths in machine data:
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB (basic axis lengths A and B)
10. Define frame T_IRO_RO and enter the offset in machine data:
MD62612 \$MC_TRAFO6_TIRORO_POS (frame between base center point and internal system (position component))
Enter the rotation in machine data:
MD62613 \$MC_TRAFO6_TIRORO_RPY (frame between base center point and internal system (rotation component))
11. Determine the flange coordinate system. For this purpose, the p3_q3_r3 coordinate system must be regarded as the initial system. The offset is entered in machine data:
MD62610 \$MC_TRAFO6_TFLWP_POS (frame between wrist point and flange (position component))
The rotation is entered in machine data:
MD62611 \$MC_TRAFO6_TFLWP_RPY (frame between wrist point and flange (rotation component))

SCARA kinematics

SCARA kinematics are characterized by the fact that they possess both translational and rotary axes. The kinematics have different names depending on how the basic axes are arranged with respect to one another, e.g.: CC, SC, CS kinematics (see Chapter "Definition of a joint (Page 90)").

3-axis CC kinematics



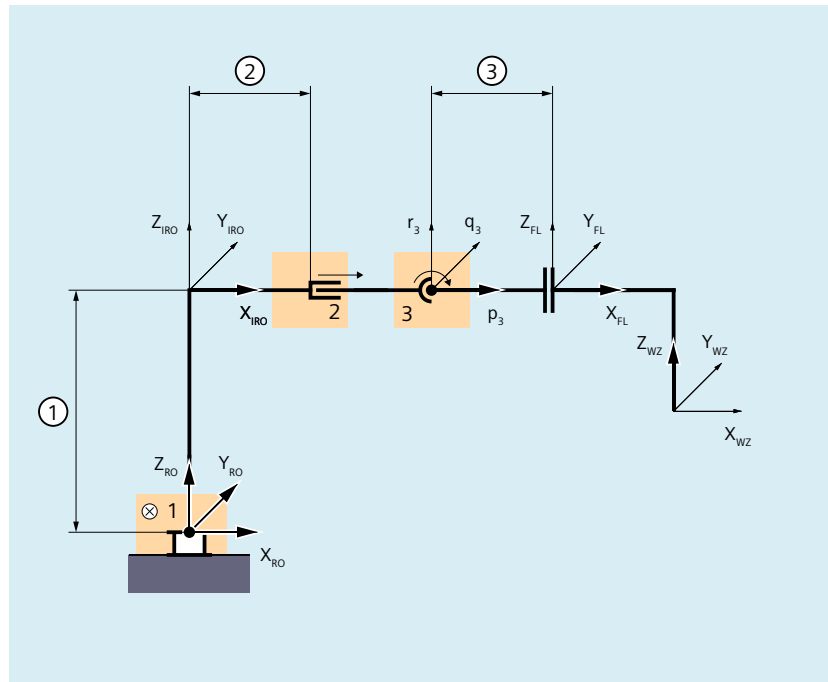
- ① MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[2]
 ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
 ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]

Figure 5-12 3-axis CC kinematics

Table 5-4 Configuration data for 3-axis CC kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[2, 1, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[0.0, 300.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 90.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[200.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, -90.0]

3-axis SC kinematics



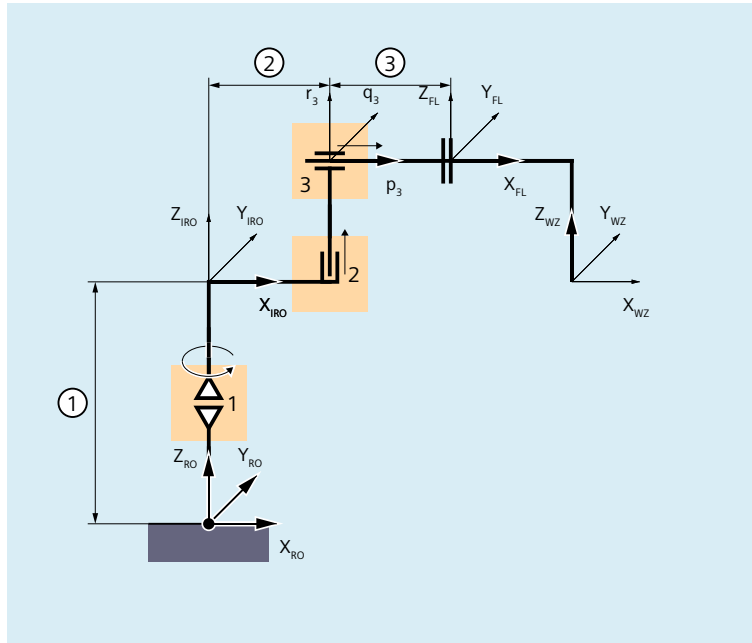
- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-13 3-axis SC kinematics

Table 5-5 Configuration data for 3-axis SC kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	4
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[1, 1, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[500.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[300.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

3-axis CS kinematic



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
 ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
 ③ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

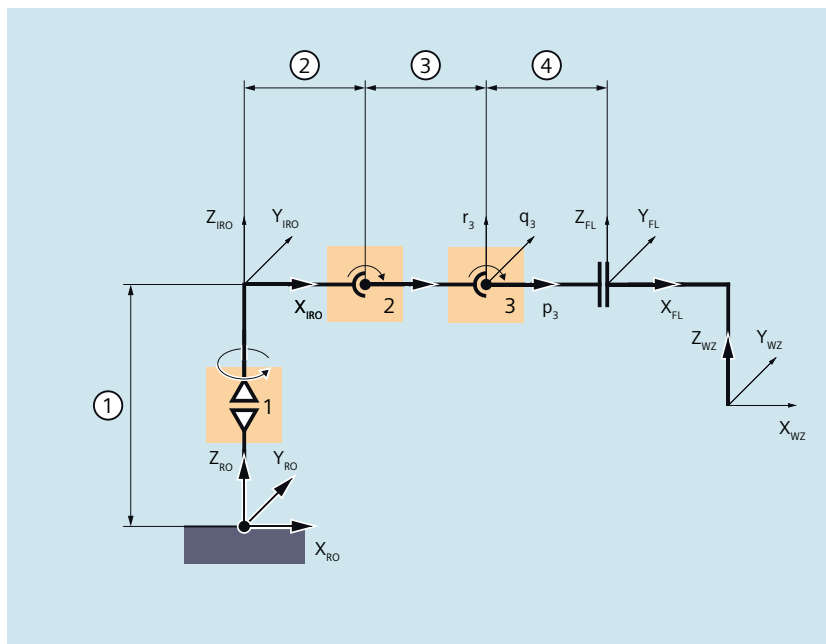
Figure 5-14 3-axis CS kinematic

Table 5-6 Configuration data for 3-axis CS kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	6
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 1, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[500.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[300.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

Articulated-arm kinematics

3-axis NR kinematics



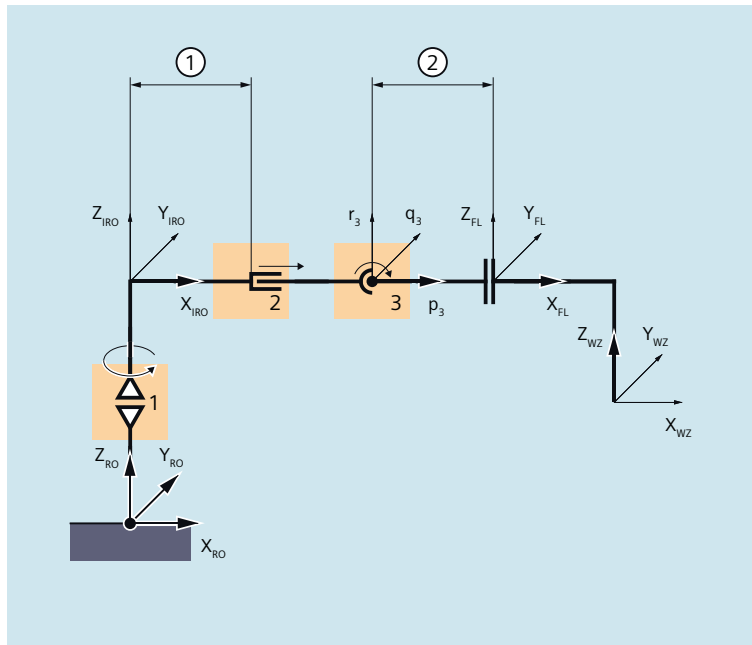
- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ④ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-15 3-axis NR kinematics

Table 5-7 Configuration data 3-axis NR kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	3
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[300.0, 500.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[300.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

3-axis RR kinematics



① MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]

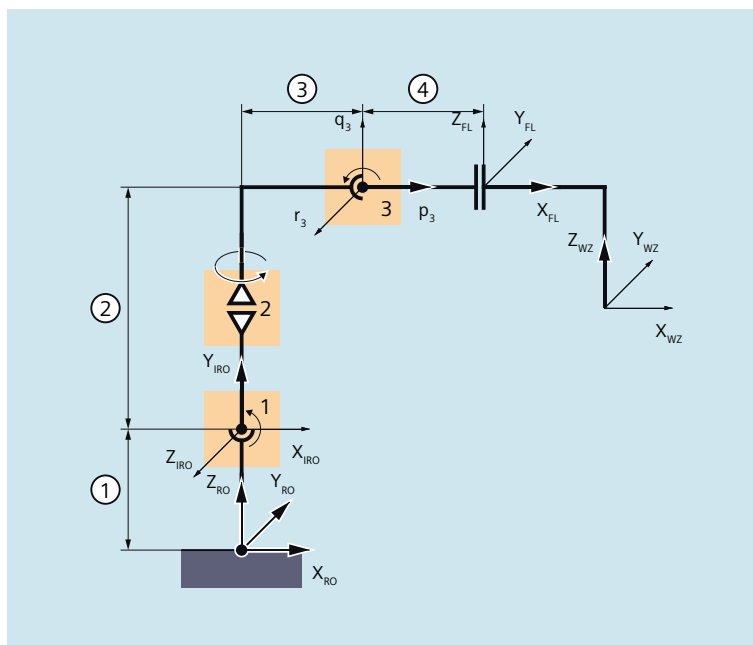
② MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-16 3-axis RR kinematics

Table 5-8 Configuration data for 3-axis RR kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	5
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_NAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[300.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 300.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[200.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

3-axis NN kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ④ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-17 3-axis NN kinematics

Table 5-9 Configuration data for 3-axis NN kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	7
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[300.0, 500.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 300.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 90.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPYFehler! Bookmark not defined.	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[400.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, -90.0]

5.5.2 4-axis kinematics

4-axis kinematics normally have 3 translational degrees of freedom and one rotational degree of freedom for orientation.

Restrictions

The following restrictions apply to 4-axis kinematics:

The frame T_FL_WP is subject to the following condition:

- MD62611 \$MC_TRAFO6_TFLWP_RPY = [0.0, 90.0, 0.0] (frame between wrist point and flange (rotation component))
- X flange and X tool must be parallel to the 4th axis.
- Two successive basic axes must be parallel or orthogonal.
- The 4th axis must only be mounted in a parallel or orthogonal way to the last basic axis.

Configuration

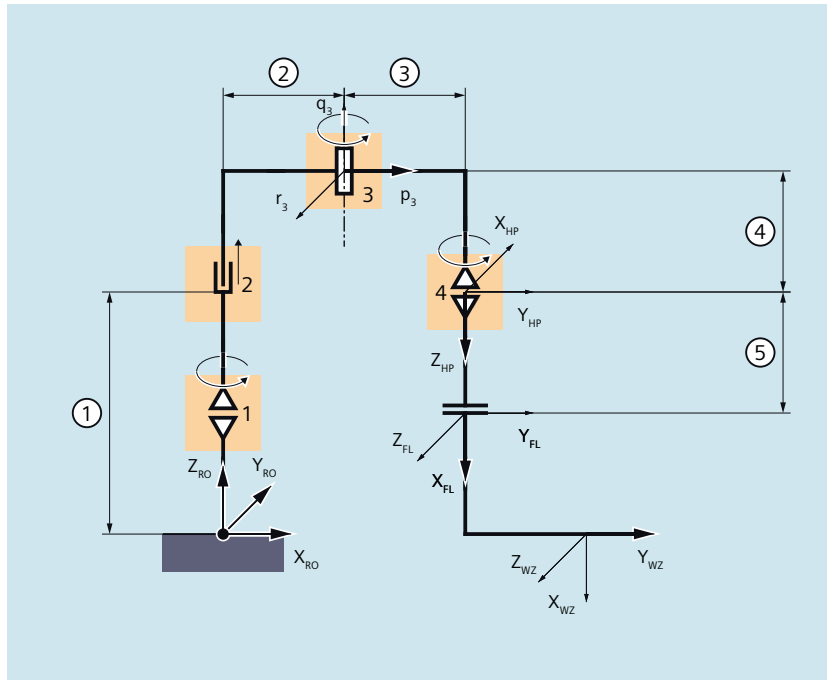
The procedure for configuring a 4-axis kinematic is as follows:

1. Enter "Standard" kinematic category in the machine data:
MD62600 \$MC_TRAFO6_KINCLASS (kinematic category)
2. Set the number of axes for transformation in the machine data:
MD62605 \$MC_TRAFO6_NUM_AXES=4 (number of transformed axes)
3. Compare the basic axes with the basic axes contained in the handling transformation package.
 - Enter the basic axis identifier in machine data:
MD62603 \$MC_TRAFO6_MAIN_AXES (basic axis identifier)
4. If the axis sequence is not the same as the normal axis sequence, it must be corrected in machine data:
MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)
5. As identifier for the wrist axes, the following machine data must be set (no wrist axis):
MD62604 \$MC_TRAFO6_WRIST_AXES = 1 (wrist axis identifier)
6. Whether axis 4 runs parallel/anti-parallel to the last rotary basic axis is entered into the machine data:
MD62606 \$MC_TRAFO6_A4PAR (axis 4 is parallel/anti-parallel to last basic axis)
7. Enter the axis types for the transformation in machine data:
MD62601 \$MC_TRAFO6_AXES_TYPE (axis type for transformation)
8. Compare the directions of rotation of axes with the directions defined in the handling transformation package and correct in the machine data:
MD62618 \$MC_TRAFO6_AXES_DIR (matching of physical and mathematical directions of rotation)
9. Enter the mechanical zero offset in the machine data:
MD62617 \$MC_TRAFO6_MAMES (offset between mathematical and mechanical zero points)

10. Enter the basic axis lengths in the machine data:
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB (basic axis lengths A and B)
11. Define frame T_IRO_RO and enter the offset in the machine data:
MD62612 \$MC_TRAFO6_TIRORO_POS (frame between base center point and internal system (position component))
Enter the rotation in the machine data:
MD62613 \$MC_TRAFO6_TIRORO_RPY (frame between base center point and internal system (rotation component))
12. Specification of frame T_X3_P3 to attach wrist axis. For this purpose, the p3_q3_r3 coordinate system must be regarded as the initial system. The offset is entered in the machine data:
MD62608 \$MC_TRAFO6_TX3P3_POS (attachment of wrist axis (position component))
The rotation is entered in the machine data:
MD62609 \$MC_TRAFO6_TX3P3_RPY (attachment of wrist axis (rotation component))
is entered.
13. Determine the flange coordinate system. For this purpose, the wrist point coordinate system must be regarded as the initial system. The offset is entered in the machine data:
MD62610 \$MC_TRAFO6_TFLWP_POS (frame between wrist point and flange (position component))
The rotation is entered in the machine data:
MD62611 \$MC_TRAFO6_TFLWP_RPY (frame between wrist point and flange (rotation component))

SCARA kinematics

4-axis CC kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ③ MD62608 \$MC_TRAFO6_TX3P3_POS[0]
- ④ MD62608 \$MC_TRAFO6_TX3P3_POS[1]
- ⑤ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

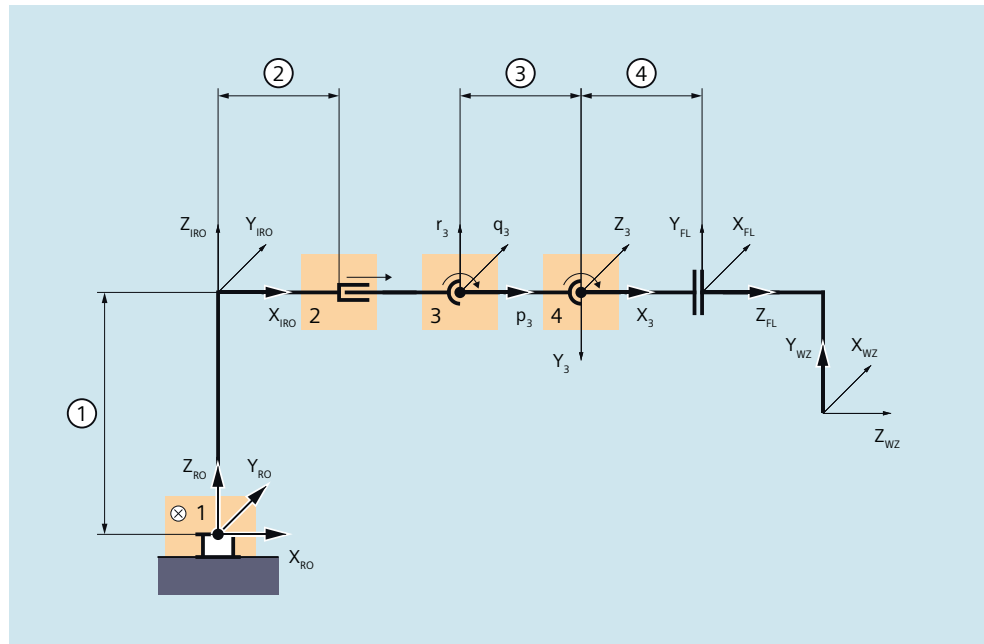
Figure 5-18 4-axis CC kinematics

Table 5-10 Configuration data for 4-axis CC kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	4
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62606 \$MC_TRAFO6_A4PAR	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[2, 1, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR)	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[0.0, 300.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 90.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[300.0, 0.0, -200.0]

Machine data	Value
MD62609 \$MC_TRAFO6_TX3P3_RPY	[-90.0, 90.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, 0.0, 200.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, -90.0, 0.0]

4-axis SC kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62608 \$MC_TRAFO6_TX3P3_POS[0]
- ④ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-19 4-axis SC kinematics

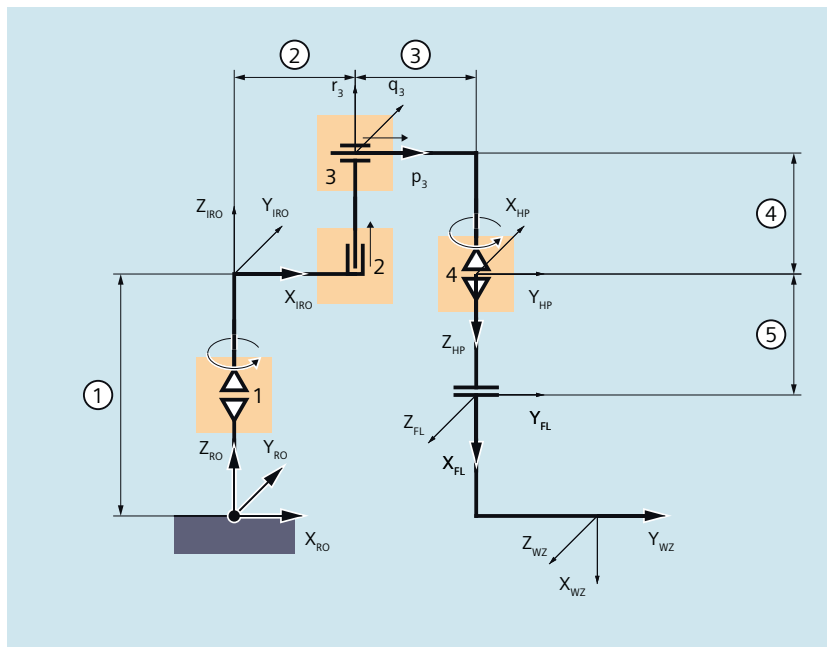
Table 5-11 Configuration data for 4-axis SC kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	4
MD62603 \$MC_TRAFO6_MAIN_AXES	4
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62606 \$MC_TRAFO6_A4PAR	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[1, 1, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[300.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 300.0]

5.5 Descriptions of kinematics

Machine data	Value
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[200.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, -90.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[200.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, -90.0, 180.0]

4-axis CS kinematic



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62608 \$MC_TRAFO6_TX3P3_POS[0]
- ④ MD62608 \$MC_TRAFO6_TX3P3_POS[2]
- ⑤ MD62610 \$MC_TRAFO6_TFLWP_POS[2]

Figure 5-20 4-axis CS kinematic

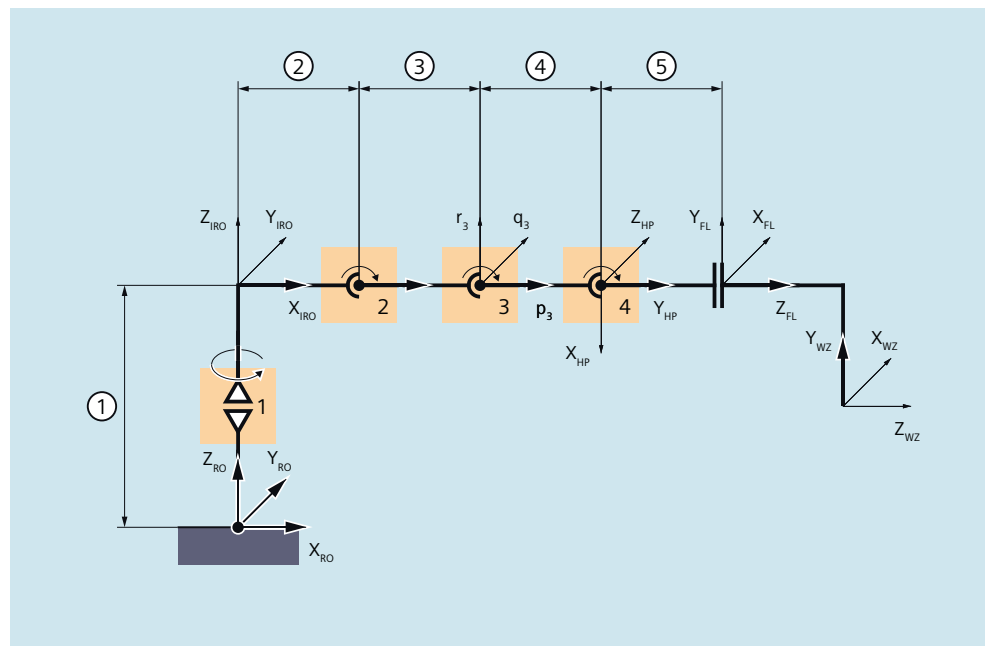
Table 5-12 Configuration data for 4-axis CS kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	4
MD62603 \$MC_TRAFO6_MAIN_AXES	6
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62606 \$MC_TRAFO6_A4PAR	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 1, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]

Machine data	Value
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[400.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 400.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[500.0, 0.0, -200.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[90.0, 0.0, 180.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, 0.0, 200.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, -90.0, 0.0]

Articulated-arm kinematics

4-axis NR kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ④ MD62608 \$MC_TRAFO6_TX3P3_POS[0]
- ⑤ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-21 4-axis NR kinematics

Table 5-13 Configuration data 4-axis NR kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	4
MD62603 \$MC_TRAFO6_MAIN_AXES	3

5.5 Descriptions of kinematics

Machine data	Value
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62606 \$MC_TRAFO6_A4PAR	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[300.0, 300.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[300.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, -90.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[200.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, -90.0, 180.0]

See also

Definition of a joint (Page 90)

5.5.3 5-axis kinematics

5-axis kinematics usually have 3 translatory degrees of freedom and 2 additional ones for orientation.

Restrictions

The following restrictions apply to 5-axis kinematics:

1. There are restrictions for the flange coordinate system because the X flange axis must intersect the 5th axis; nevertheless, it must not be parallel to it.
2. For 5-axis articulated arm kinematics, the following condition applies to frame T_FL_WP:
 - MD62610 \$MC_TRAFO6_TFLWP_POS = [0.0, 0.0, Z] (frame between wrist point and flange (position component))
 - MD62611 \$MC_TRAFO6_TFLWP_RPY = [A, 0.0, 0.0] (frame between wrist point and flange (rotation component))
3. Restrictions apply to the tool for 5-axis articulated arm kinematics:
 - 4th axis parallel to the 3rd Axis: 2-dimensional tool is possible [X, 0.0, Z]
 - 4th axis perpendicular to the 3rd Axis: only a 1-dimensional tool is possible [X, 0.0, 0.0]
4. Restrictions apply to the tool for 5-axis Scara kinematics:
 - 4th axis perpendicular to the 3rd Axis: 1-dimensional tool is possible [X, 0.0, 0.0]
5. Two successive basic axes must be parallel or orthogonal.
6. The 4th axis must only be mounted in a parallel or orthogonal way to the last basic axis.

Configuration

The following procedure is required to configure 5-axis kinematics:

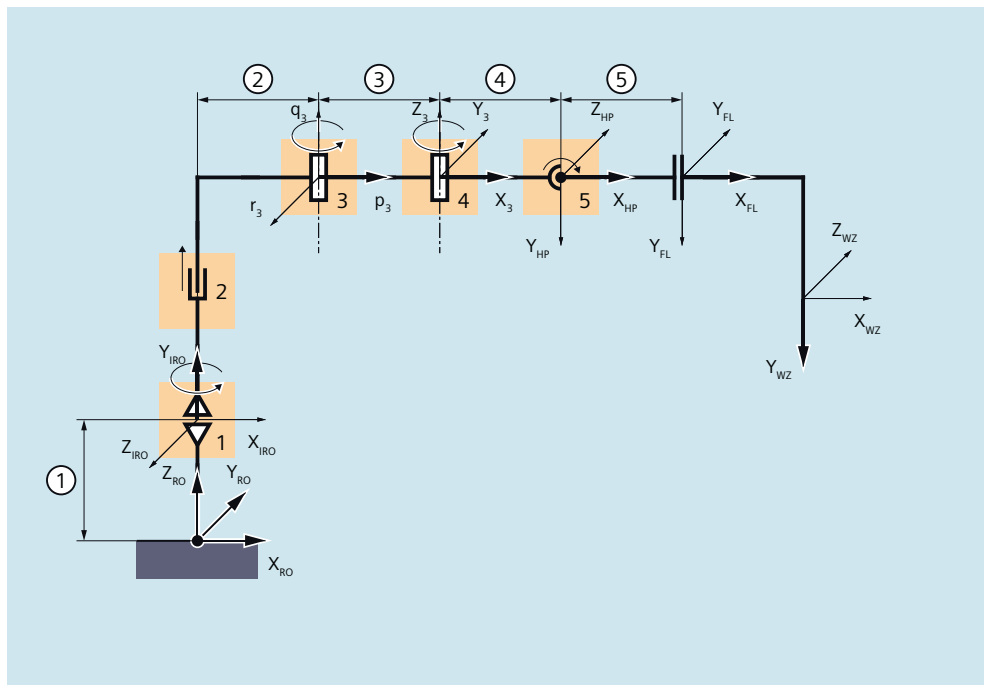
1. Enter "Standard" kinematic category in machine data:
MD62600 \$MC_TRAFO6_KINCLASS (kinematic category)
2. Set the number of axes for transformation in the machine data:
MD62605 \$MC_TRAFO6_NUM_AXES = 5 (number of transformed axes)
3. Compare the basic axes with the basic axes contained in the handling transformation package.
 - Enter the basic axis identifier in machine data:
MD62603 \$MC_TRAFO6_MAIN_AXES (basic axis identifier)
4. If the axis sequence is not the same as the normal axis sequence, it must be corrected in machine data:
MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)
5. Determination of the identifier for the wrist axes. A central wrist axis exists if axes 4 and 5 intersect. In all other cases, the identifier for inclined wrist axis must be entered in the machine data:
MD62604 \$MC_TRAFO6_WRIST_AXES (wrist axis identifier)
6. Whether axis 4 runs parallel/anti-parallel to the last rotary basic axis must be entered into the machine data:
MD62606 \$MC_TRAFO6_A4PAR (axis 4 is parallel/anti-parallel to last basic axis)
7. Enter the axis types for the transformation in machine data:
MD62601 \$MC_TRAFO6_AXES_TYPE (axis type for transformation)
8. Compare the directions of rotation of axes with the directions defined in the handling transformation package and correct in machine data:
MD62618 \$MC_TRAFO6_AXES_DIR (matching physical and mathematical directions of rotation)
9. Enter the mechanical work offset in the machine data:
MD62617 \$MC_TRAFO6_MAMES (offset between mathematical and mechanical zero points)
10. Enter the basic axis lengths in machine data:
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB (basic axis lengths A and B)
11. Define frame T_IRO_RO and enter the offset in machine data:
MD62612 \$MC_TRAFO6_TIRORO_POS (frame between base center point and internal system (position component))
Define frame T_IRO_RO and enter the rotation in machine data:
MD62613 \$MC_TRAFO6_TIRORO_RPY (frame between base center point and internal system (rotation component))
12. Specification of frame T_X3_P3 to attach wrist axis. The offset is entered in machine data:
MD62608 \$MC_TRAFO6_TX3P3_POS (attachment of wrist axis (position component))
The rotation is entered in the machine data:
MD62609 \$MC_TRAFO6_TX3P3_RPY (attachment of wrist axis (rotation component))

5.5 Descriptions of kinematics

13. Determination of the parameters for the wrist axes. For this purpose, only the parameters for axis 4 must be entered in the machine data:
MD62614 \$MC_TRAFO6_DHPAR4_5A[0] (parameter A for configuring the wrist axis)
MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA[0] (parameter ALPHA for configuring the wrist axis)
All other parameters should be set to 0.0
14. Determine the flange coordinate system. The wrist point coordinate system must be regarded as the initial system in this case. The offset is entered in the machine data:
MD62610 \$MC_TRAFO6_TFLWP_POS (frame between wrist point and flange (position component))
The rotation is entered in the machine data:
MD62611 \$MC_TRAFO6_TFLWP_RPY (frame between wrist point and flange (rotation component))

SCARA kinematics

5-axis CC kinematics



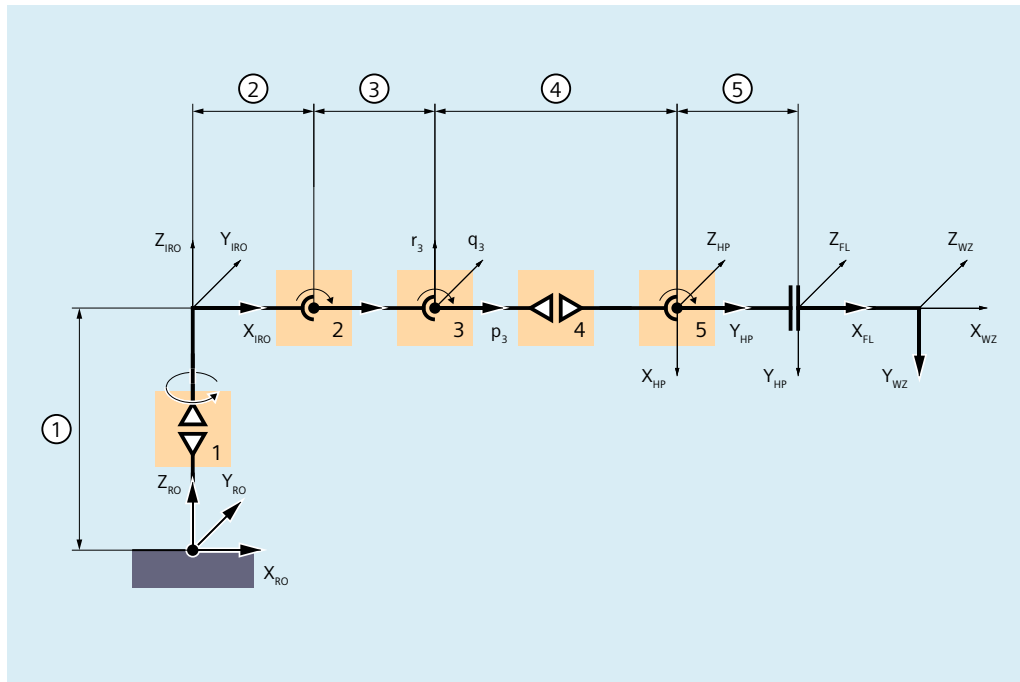
- ① MD62612 \$MC_TRAFO06_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62608 \$MC_TRAFO6_TXP3_POS[0]
- ④ MD62614 \$MC_TRAFO6_DHPAR4_5A[0]
- ⑤ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-22 5-axis CC kinematics

Table 5-14 Configuration data for 5-axis CC kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	5
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	5
MD62606 \$MC_TRAFO6_A4PAR	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 1, 3, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[2, 1, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[0.0, 500.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 90.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[300.0, 0.0, -200.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, -90.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[200.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]
MD62614 \$MC_TRAFO6_DHPAR4_5A	[200.0, 0.0]
MD62615 \$MC_TRAFO6_DHPAR4_5D	[0.0, 0.0]
MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA	[-90.0, 0.0]

5-axis NR kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[2]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ④ MD62608 \$MC_TRAFO6_TXP3_POS[0]
- ⑤ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-23 5-axis NR kinematics

Table 5-15 Configuration data 5-axis NR kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	1
MD62605 \$MC_TRAFO6_NUM_AXES	5
MD62603 \$MC_TRAFO6_MAIN_AXES	3
MD62604 \$MC_TRAFO6_WRIST_AXES	2
MD62606 \$MC_TRAFO6_A4PAR	0
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, 3, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[30.0, 300.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 500.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[500.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 90.0, 0.0]

Machine data	Value
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, -300.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[-90.0, 0.0, 0.0]
MD62614 \$MC_TRAFO6_DHPAR4_5A	[0.0, 0.0]
MD62615 \$MC_TRAFO6_DHPAR4_5D	[0.0, 0.0]
MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA	[-90.0, 0.0]

See also

Definition of a joint (Page 90)

5.5.4 6-axis kinematics

6-axis kinematics usually imply 3 degrees of freedom for translation and 3 more for orientation. This allows for the tool direction to be manipulated freely in space. Also, the tool can be rotated along its own axis to the machining surface or inclined with a tilting angle.

This kinematic type is supported by the software, but only for very specific solutions as the auxiliary method for this handling transformation package. A general, comprehensive and universally applicable software version is still not available for 6-axis kinematics.

5.5.5 Special kinematics**MD62602 \$MC_TRAFO6_SPECIAL_KIN (special kinematic type)**

Special kinematics are kinematics that are not directly included in the modular system of the Handling transformation package. They are frequently missing a degree of freedom or are characterized by mechanical links between the axes or with the tool. Set the following machine data for these kinematics:

MD62600 \$MC_TRAFO6_KINCLASS = 2 (kinematic category)

Which special kinematic is handled is specified in machine data:

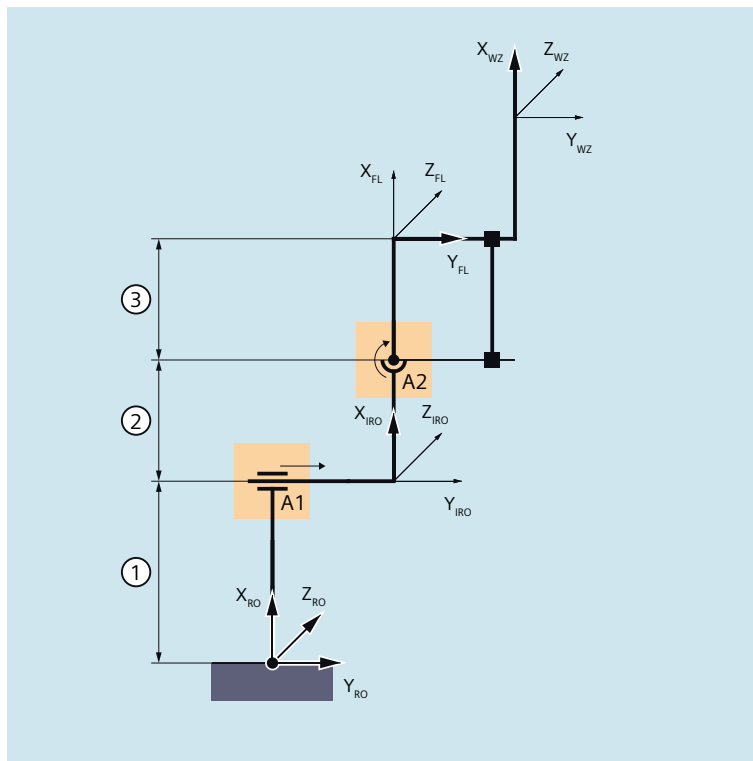
MD62602 \$MC_TRAFO6_SPECIAL_KIN (special kinematic type)

Special 2-axis SC kinematic

This special kinematic is characterized by the fact that the tool is always maintained in the same orientation via a mechanical linkage. It implies two Cartesian degrees of protection. The identifier for this kinematic is machine data:

MD62602 \$MC_TARFO6_SPECIAL_KIN = 3 (special kinematic type)

5.5 Descriptions of kinematics



- ① MD62612 \$MC_TRAFO6_TIRORO_POS[0]
- ② MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]

Figure 5-24 Special 2-axis SC kinematic

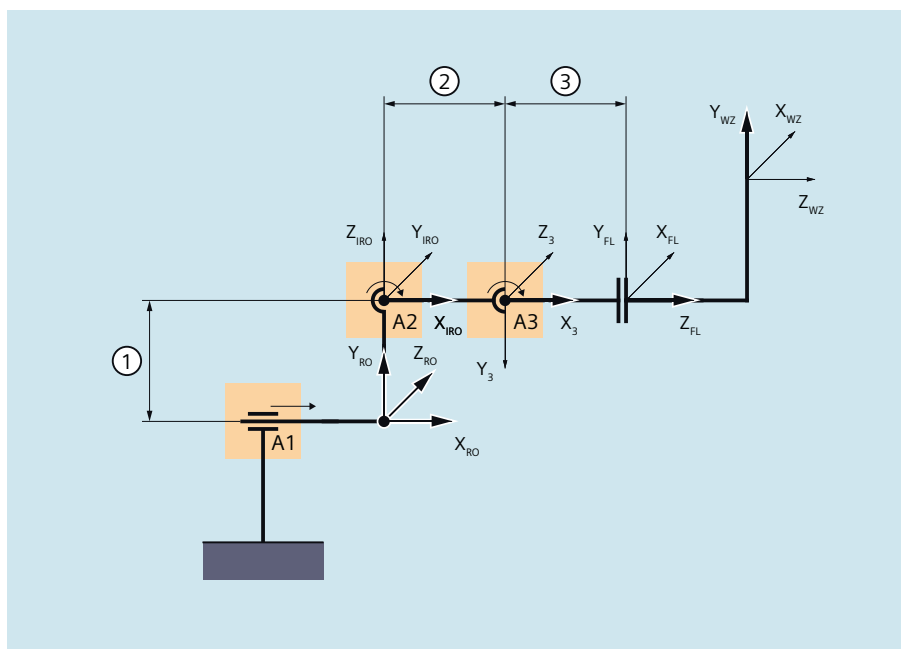
Table 5-16 Configuring data for a special 2-axis SC kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	2
MD62602 \$MC_TRAFO6_SPECIAL_KIN	3
MD62605 \$MC_TRAFO6_NUM_AXES	2
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[1, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[400.0, 500.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 300.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[0.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

Special 3-axis SC kinematic

The special kinematic has 2 Cartesian degrees of freedom and one degree of freedom for orientation. The identifier for this kinematic is:

MD62602 \$MC_TRAFO6_SPECIAL_KIN = 4 (special kinematic type)



① MD62612 \$MC_TRAFO6_TIRORO_POS[2]

② MD62608 \$MC_TRAFO6_TX3P3_POS[0]

③ MD62610 \$MC_TRAFO6_TFLWP_POS[0]

Figure 5-25 Special 3-axis SC kinematic

Table 5-17 Configuring data for a special 3-axis SC kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	2
MD62602 \$MC_TRAFO6_SPECIAL_KIN	4
MD62605 \$MC_TRAFO6_NUM_AXES	3
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[1, 3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[0.0, 0.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[0.0, 0.0, 400.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[400.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, -90.0]

Table 5-18 Configuring data for a special 4-axis SC kinematic

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	2
MD62602 \$MC_TRAFO6_SPECIAL_KIN	7
MD62605 \$MC_TRAFO6_NUM_AXES	4
MD62603 \$MC_TRAFO6_MAIN_AXES	2
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, 1, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[100.0, 400.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[100.0, 0.0, 1000.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, 0.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[300.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, 0.0, -600.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 90.0, 0.0]

Special 2-axis NR kinematics

This special kinematic is characterized by the fact that axis 1 and axis 2 are mechanically coupled. Another special feature is the tool. With this kinematic, it maintains its orientation in space irrespective of the positions of the other axes.

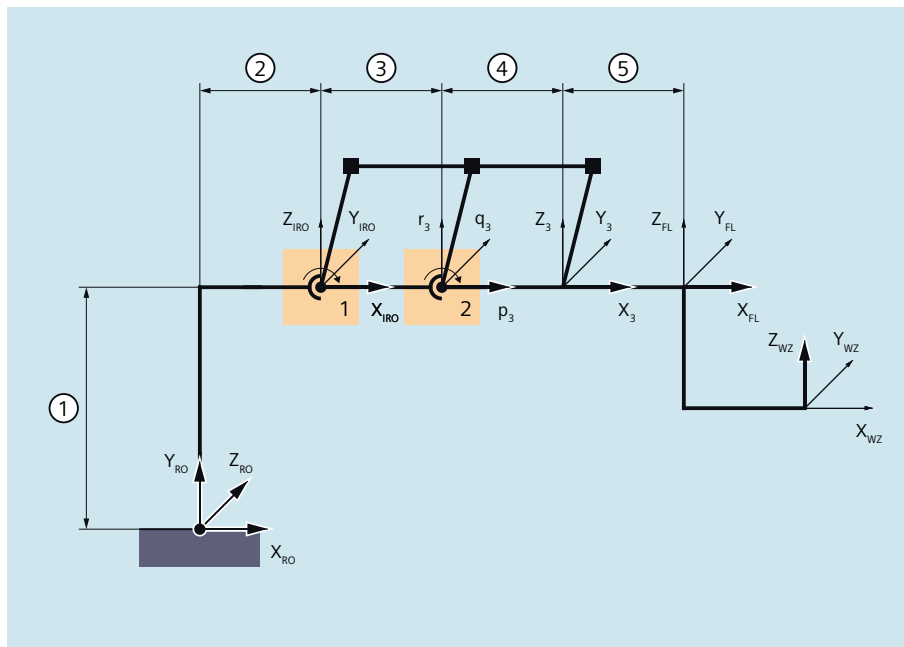
The identifier for this kinematic is:

MD62602 \$MC_TRAFO6_SPECIAL_KIN = 5 (special kinematic type)

Without a mechanical coupling between axis 1 and 2, it has the following identifier:

MD62602 \$MC_TRAFO6_SPECIAL_KIN = 8 (special kinematic type)

5.5 Descriptions of kinematics



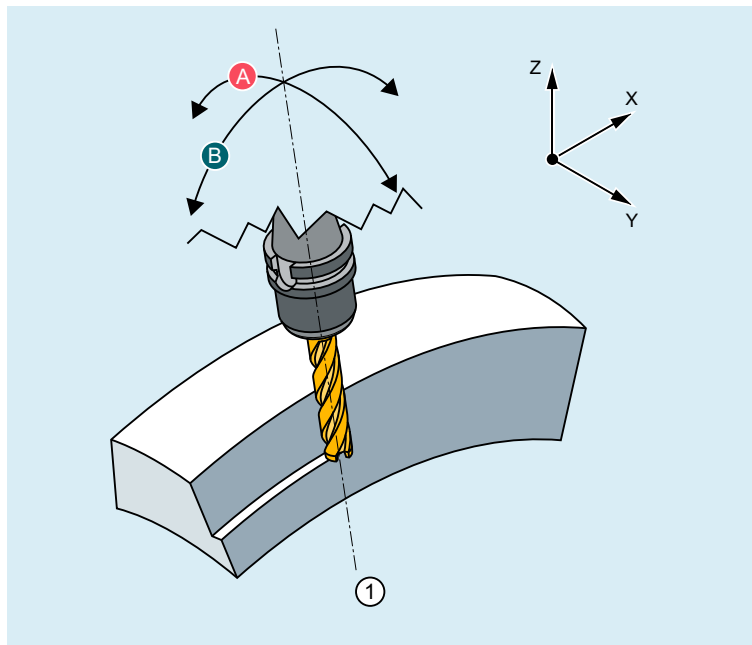
- ① MD62612 \$MC_TRAFO6_TIRORO_POS[1]
- ② MD62612 \$MC_TRAFO6_TIRORO_POS[0]
- ③ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[1]
- ④ MD62608 \$MC_TRAFO6_TX3P3_POS[0]
- ⑤ MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB[0]

Figure 5-27 Special 2-axis NR kinematics

Table 5-19 Configuration data for special 2-axis NR kinematics

Machine data	Value
MD62600 \$MC_TRAFO6_KINCLASS	2
MD62602 \$MC_TRAFO6_SPECIAL_KIN	5 (8)
MD62605 \$MC_TRAFO6_NUM_AXES	2
MD62603 \$MC_TRAFO6_MAIN_AXES	3
MD62604 \$MC_TRAFO6_WRIST_AXES	1
MD62601 \$MC_TRAFO6_AXES_TYPE	[3, 3, ...]
MD62620 \$MC_TRAFO6_AXIS_SEQ	[1, 2, 3, 4, 5, 6]
MD62618 \$MC_TRAFO6_AXES_DIR	[1, 1, 1, 1, 1, 1]
MD62617 \$MC_TRAFO6_MAMES	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB	[100.0, 400.0]
MD62612 \$MC_TRAFO6_TIRORO_POS	[100.0, 500.0, 0.0]
MD62613 \$MC_TRAFO6_TIRORO_RPY	[0.0, 0.0, -90.0]
MD62608 \$MC_TRAFO6_TX3P3_POS	[400.0, 0.0, 0.0]
MD62609 \$MC_TRAFO6_TX3P3_RPY	[0.0, 0.0, 0.0]
MD62610 \$MC_TRAFO6_TFLWP_POS	[0.0, 0.0, 0.0]
MD62611 \$MC_TRAFO6_TFLWP_RPY	[0.0, 0.0, 0.0]

5.6 Tool orientation



① Tool axis

Figure 5-28 Peripheral milling with 5-axis transformation

Machine data

Identifiers of the Euler angles

The identifiers with which the Euler angles are programmed in the NC program can be set using:

MD10620 \$MN_EULER_ANGLE_NAME_TAB (name of Euler angles)

Standard identifier: "A2", "B2", "C2"

Identifier of the direction vector components

The identifier with which the components of the direction vector are programmed in the NC program can be set using:

MD10640 \$MN_DIR_VECTOR_NAME_TAB (name of normal vectors)

The tool orientation can be located in any block. Above all, it can be programmed alone in a block, resulting in a change of orientation in relation to the tool tip which is fixed in its relationship to the workpiece.

Channel-specific input: Euler or RPY angle

Whether the tool orientation programming is realized in the channel with Euler angle or RPY angle can be set using:

MD21100 \$MC_ORIENTATION_IS_EULER (angle definition for orientation programming)

5.6 Tool orientation

G command for orientation interpolation

The type of interpolation for the orientation is specified using machine data:

MD21104 \$MC_ORI_IPO_WITH_G_CODE = <value>

<value>	Meaning
FALSE	G commands for orientation interpolation: ORIWKS and ORIMKS
TRUE	G commands of the 51st G group for orientation interpolation: ORIXES, ORIVECT, ORIPLANE, ...

Channel-specific initial setting for the tool orientation

G commands of the 25th G group: "Tool orientation reference" which takes effect after a channel or end of program reset, can be set via:

MD20150 \$MC_GCODE_RESET_VALUES[24] (25. G group: tool orientation reference)

Programming**Function**

The tool orientation can be programmed via:

- Directly programming the rotary axes
- Programming the Euler or RPY angle
- Programming the direction vector

The coordinate system in which motion is executed is defined using the ORIWKS and ORIMKS commands.

Syntax

ORIWKS
ORIMKS

Meaning

ORIWKS:	Tool orientation in the workpiece coordinate system (WCS) In the case of a change in orientation with the tool tip at a fixed point in space, the tool moves along a large arc on the plane extending from the start vector to the end vector. Transitioning a tool orientation at the start of a block to an orientation at the end of the block is only possible in the WCS.	
	Effective:	Modal
ORIMKS:	Tool orientation in the machine coordinate system (MCS) In the case of a change in orientation of a tool tip at a fixed point in space, linear interpolation takes place between the rotary axis positions.	
	Effective:	Modal

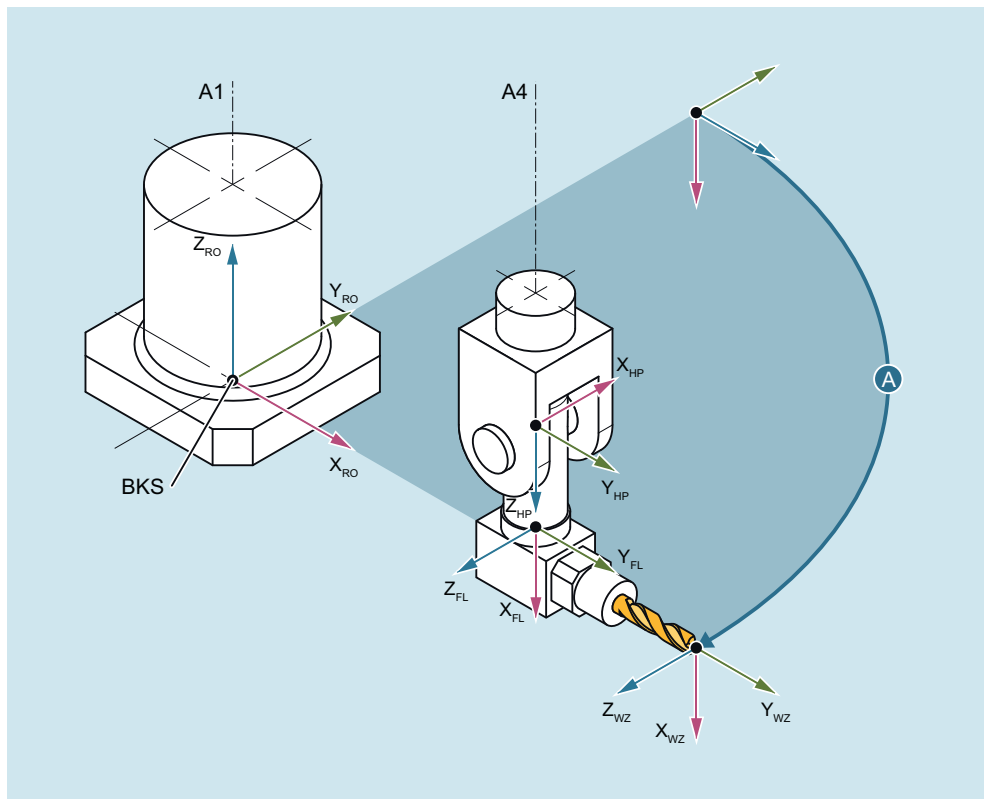
Note

- `ORIWKS` is the preferred approach for a program independent of a particular machine.
 - It is not possible to program the tool orientation using Euler angles, RPY angles or direction vectors for kinematics involving fewer than 5 axes. For 4-axis kinematics with only one rotary axis, there is only one degree of freedom for orientation. This orientation angle can only be programmed with orientation axis angle "A".
 - Transferring an orientation using `ORIMKS` is not allowed in the handling transformation package. With an active transformation, the machine axis angles are not programmed and traversed, but rather the "orientation angles" (RPY angles according to robotics definition, see Section "Definition of positions and orientations using frames (Page 89)").
-

5.6.1 Orientation programming for 4-axis kinematics

4-axis kinematics possess only one degree of freedom for orientation. When programming orientation using RPY angle, Euler angle or direction vector, it is not always possible to approach the specified orientation. If at all, this type of orientation programming is only practical for certain kinematics, where there is an invariance of the orientation angle with respect to the basis axes. This is the case for the Scara kinematics, for example.

For this reason, orientation programming is only permitted via "orientation angle" A for 4-axis kinematics. This angle corresponds to the RPY angle C of the robot definition. This means one rotation around the Z_{RO} axis, as illustrated in Fig. "Orientation angle for 4-axis kinematics".



A Virtual orientation angle

Figure 5-29 Orientation angle for 4-axis kinematics

Absolute calculation of the tool orientation for 4-axis kinematics

For 4-axis kinematics, the "virtual orientation angle" A is normally only defined within the range $-180^\circ < A \leq +180^\circ$. This means that, in the transition from 180° to 181° , angle A jumps to -179° . The following machine data must be set to avoid this step, and to ensure that angle A is continually controlled:

MD62643 \$MC_TRAFO6_SPECIAL_FEATURE_MASK, bit 0 = 1

As a consequence, for the following 4-axis kinematics, angle A is controlled in absolute terms:

- CC kinematics
- SC kinematics
- CS kinematics

This is on condition that the 4th axis is installed parallel to the last rotary basic axis (MD62606 \$MC_TRAFO6_A4PAR == 1).

Angle A is calculated from the sum of the rotary axes. Machine data MD62617 \$MC_TRAFO6_MAMES and MD62618 \$MC_TRAFO6_AXES_DIR are taken into account.

If, as a result of the rotations via MD62612 \$MC_TRAFO6_TIRORO_RPY and MD62611 \$MC_TRAFO6_TFLWP_RPY, additional rotations are obtained, then these must be compensated using MD62642 \$MC_TRAFO6_C_ANGLE_OFFSET. In machine data MD62642,

the angle, which is displayed as "virtual orientation angle" A at the mathematical zero position of the axes, must be entered with **inverted** sign.

5.6.2 Orientation programming for 5-axis kinematics

For 5-axis kinematics, when programming via orientation vector, it is assumed that the orientation vector corresponds to the x component of the tool.

When programming via orientation angle (RPY angle according to robotics definition), the x component of the tool is considered the initial point for rotations.

For this purpose, the vector in the x tool direction, as shown in Fig. "Orientation angle for 5-axis kinematic", is first rotated around the Z axis by the angle A and then around the rotated Y axis by the angle B. The rotation by the angle C is not possible for 5-axis kinematics because of the restricted degrees of freedom for the orientation.

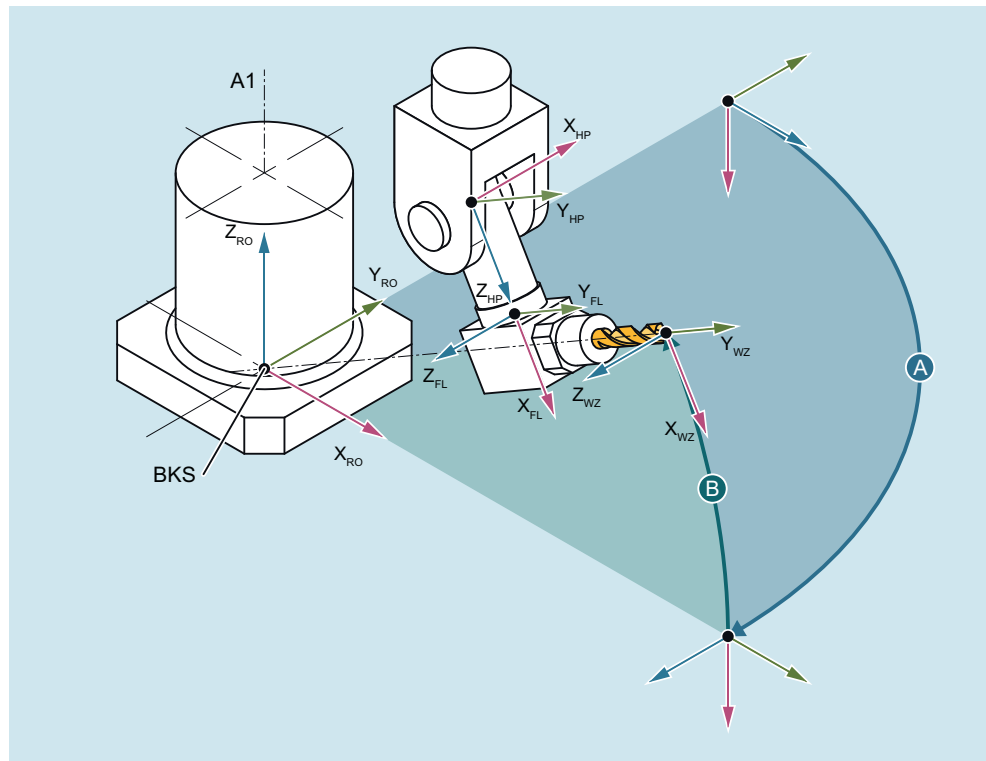


Figure 5-30 Orientation angle for 5-axis kinematic

Adaptation of the flange coordinate system

This machine data is used to set the flange coordinate system on the user side so that Z can be set as the tool direction for 5-axis kinematics:

MD62636 \$MC_TRAFO6_TFL_EXT_RPY

Definition of the tool direction

In this machine data, set whether the tool direction will be calculated according to robotics conventions or NC conventions:

5.7 Singular positions and how they are handled

MD62637 \$MC_TRAFO6_TOOL_DIR

Standard definition of the tool direction

The tool calculation normally used with machine tools can be set with machine data:

MD62637 \$MC_TRAFO6_TOOL_DIR = 1

This machine data also has an effect on the rotation sequence of the virtual orientation axes. For this purpose, the vector in the Z tool direction is first turned through angle A at the X axis and then through angle B at the rotated Y axis. The rotation by the angle C is not possible for 5-axis kinematics because of the restricted degrees of freedom for the orientation.

Note

More information can be found in:

- Function Manual, Special Functions, Section "Orientation axes"
 - Programming Manual, Job Planning; Section "Orientation axes".
-

5.7 Singular positions and how they are handled

The calculation of the machine axes to a preset position, i.e. position with orientation, is not always clear. Depending on the machine kinematic, there may be positions with an infinite number of solutions. These positions are called "singular".

Singular positions

- A singular position is, for example, characterized by the fact that the fifth axis is positioned at 0°. In this case, the singular position does not depend on a specified orientation. The fourth axis is not specified in this position, i.e., the fourth axis does not have any influence on the position or the orientation.
- A singular position also applies for articulated arm and SCARA kinematics if the third axis is at 0° or 180°. These positions are called leveling/diffraction singularity.
- Another singular position exists for articulated arm kinematics if the hand point is above the rotary axis of axis 1. This position is called overhead singularity.

Extreme velocity increase

If the path runs in close vicinity to a pole (singularity), one or several axes may traverse at a very high velocity. Alarm 10910 "Irregular velocity run in a path axis" is then triggered.

Behavior at pole

The unwanted behavior of fast compensating movements can be improved by reducing the velocity in the proximity of a pole. Traveling through the pole with active transformation is usually not possible.

5.8 Call and application of the transformation

Activation

The transformation is activated by the TRAORI (1) command.

When the transformation is activated, the following NC/PLC interface signals are output:

<Chan>.basic.in.transformationActive = 1 (transformation active)

<Chan>.basic.in.transformationActiveNumber = ... (Index number of the active transformation in the channel)

Note

If the machine data have not been defined for an activated transformation grouping, the NC program stops and the control displays the alarm 14100 "Orientation transformation does not exist".

Deactivation

The currently active transformation is deactivated by means of TRAFOOF or TRAFOOF().

Note

When the "Handling transformation package" transformation is deactivated, no preprocessing stop and no synchronization of the preprocessing with the main run is performed.

Reset/end of program

The behavior of the controller with regard to the transformations after start-up, end of program or reset is determined by the following machine data:

5.9 Actual value display

MD20110 \$MC_RESET_MODE_MASK, bit 7

Bit 7	Reset behavior, "active kinematics transformation"	
	= 0	The initial setting for the active transformation after end of part program or reset is defined by the following machine data: MD20140 \$MC_TRAFO_RESET_VALUE (transformation data record startup (reset/part program end)) = <Value>
		<value> Meaning
		0 No transformation is active after reset.
		1 to 8 The set transformation is active according to machine data: MD24100 \$MC_TRAFO_TYPE_1 (Definition of transformation 1 in channel) ... MD24460 \$MC_TRAFO_TYPE_8 (Transformation 8 in channel)
	= 1	The actual setting for the active transformation is retained after Reset or end of part program.

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.in.transformationActive	LBP_Ch* .E_Transform	DB21,DBX33.6
<Chan>.basic.in.transformationActiveNumber	LBP_Ch* .E_TransformNo	DB21,DBB388

5.9 Actual value display

MCS machine coordinate system

The machine axes are displayed in mm/inch and/or degrees in MCS display mode.

WCS workpiece coordinate system

If the transformation is active, the tool tip (TCP) is specified in mm/inch and the orientation by the RPY angles A, B and C in WCS display mode. The tool orientation results from turning a vector in the Z direction firstly with A around the Z axis, then with B around the new Y axis and lastly with C around the new X axis.

If the transformation is deactivated, the axes will be displayed with the channel axis names. Otherwise the geometry axis names will be displayed.

5.10 Tool programming

Meaning

The tool lengths are specified in relation to the flange coordinate system. Only 3-dimensional tool compensation is possible. Depending on which kinematic is involved, additional restrictions for the tool exist for 5- and 4-axis kinematics. For a kinematic as illustrated in Fig. "5-axis NR kinematic", only a 1-dimensional tool with length in the x direction is permitted.

The tool direction is dependent on the machine's initial setting, as specified with G commands G17, G18 and G19. The tool lengths refer to the zero position specified by G17. These should not be changed in the program.

Example

An example of a 2-dimensional tool mounted on a 5-axis Scara is described below (see Fig. "5-axis CC kinematic"). Type 100 (cutting tool) should be specified as identification for the tool. The tool lengths result from the specifications shown in Fig. "Tool length programming". X-TOOL must be entered as tool length x and Y-TOOL must be entered as tool length y in the tool parameters.

\$TC_DP1[1,1] = 100 ;	Cutting tool type
\$TC_DP3[1,1] = 0.0 ;	(z) length compensation vector
\$TC_DP4[1,1] = Y-TOOL ;	(y) length compensation vector
\$TC_DP5[1,1] = X-TOOL ;	(x) length compensation vector

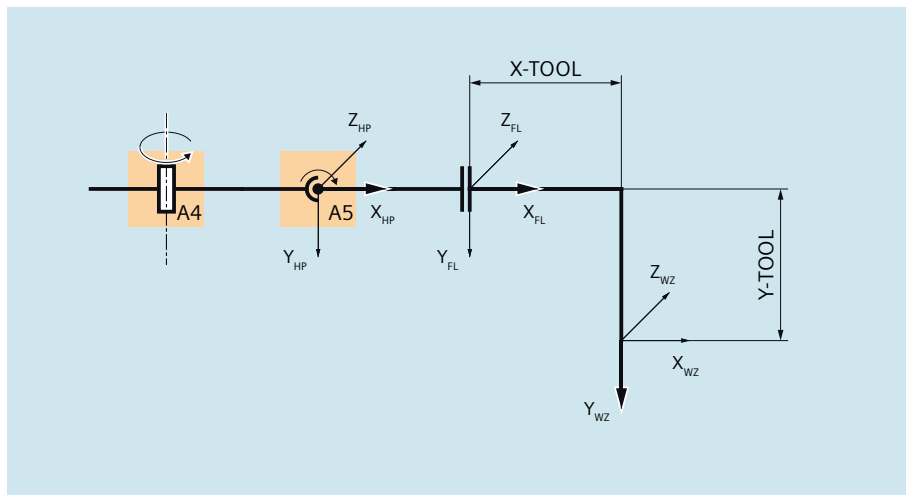


Figure 5-31 Tool length programming

5.11 Cartesian PTP travel with handling transformation package

It is possible to use the "Cartesian PTP travel" function with the handling transformation package (see Function Manual "Transformations", Chapter "Cartesian PTP travel"). For this, the following machine data must be set:

MD24100 \$MC_TRAFO_TYPE_1 = 4100 (definition of transformation 1 in the channel)

5.12 Startup

5.12.1 General commissioning

The handling transformation package is a compile cycle.

The general commissioning of a compile cycle is described in Chapter "Installation and activation of loadable compile cycles (Page 19)".

5.12.2 Function-specific commissioning

Option

The "Handling transformation package" function is an option for which a license must be purchased:

- Transformation Handling RCTRA, Order No.: 6FC5800-0AM31-0YB0

A detailed description of licensing, for instance the assignment between a license and hardware, is provided in:

Further information

Commissioning Manual, CNC Commissioning: NC, PLC, drive; licensing

User-specific alarm texts

To create user-specific alarm texts in conjunction with compile cycles, see Chapter "Creating alarm texts (Page 27)"

Parameterizing transformation

1. Enter the transformation type into the machine data:
 - MD24100 \$MC_TRAFO_TYPE_1 = **4100** (definition of transformation 1 in the channel)
2. Enter the assignment of the channel axes involved in the transformation in the machine data (axis numbers, starting at 1):
 - MD24110 \$MC_TRAFO_AXES_IN_1[0 ... 5] (axis assignment for transformation)

3. Enter the geometry axes corresponding to the Cartesian degrees of freedom of the machine in the machine data:
 - MD24120 \$MC_TRAFO_GEOAX_ASSIGN_TAB_1[0 ... 2] (assignment between geometry axis and channel axis for transformation 1)
4. Enter the kinematic identifier in the machine data:
 - MD62600 \$MC_TRAFO6_KINCLASS (kinematic category)
5. Enter the identifier for special kinematics, if there are any, in machine data:
 - MD62602 \$MC_TRAFO6_SPECIAL_KIN (special kinematic type)
6. Enter the number of axes in the machine data:
 - MD62605 \$MC_TRAFO6_NUM_AXES (number of transformed axes)
7. If the travel direction of the involved axes is opposed to the transformation definition, then change the factory setting in the machine data:
 - MD62618 \$MC_TRAFO6_AXES_DIR[] (matching of physical and mathematical directions of rotation)
8. Enter the data which define the basic axes:
 - MD62603 \$MC_TRAFO6_MAIN_AXES (basic axis identifier)
 - MD62607 \$MC_TRAFO6_MAIN_LENGTH_AB (basic axis lengths A and B)
9. Enter any changes to the axis sequence in the machine data:
 - MD62620 \$MC_TRAFO6_AXIS_SEQ (rearrangement of axes)
10. Enter the data which define the wrist axis:
 - MD62604 \$MC_TRAFO6_WRIST_AXES (wrist axis identifier)
 - MD62614 \$MC_TRAFO6_DHPAR4_5A (parameter A for configuring the wrist axis)
 - MD62615 \$MC_TRAFO6_DHPAR4_5D (parameter D for configuring the wrist axis)
 - MD62616 \$MC_TRAFO6_DHPAR4_5ALPHA (parameter ALPHA for configuring the wrist axis)
 - MD62606 \$MC_TRAFO6_A4PAR (axis 4 is parallel/anti-parallel to last basic axis)
11. Enter the geometry parameters:
 - Frame T_IRO_RO
 - Frame T_X3_P3
 - Frame T_FL_WP
12. Enter the position in relation to the calibration point in the machine data:
 - MD62617 \$MC_TRAFO6_MAMES (offset between mathematical and mechanical zero points)
13. Enter the Cartesian velocities and acceleration rates.

5.13 Supplementary conditions

Clearance control

The handling transformation package **cannot** be used together with the technology function: "**clearance control**".

Moving to fixed end stop

The handling transformation package **cannot** be used in conjunction with the "**Travel to fixed stop**".

Several transformations

The handling transformation package can be activated just once per channel in all channels.

Tool programming

Tools can only be parameterized by specifying tool lengths. It is not possible to program an orientation for the tool.

Programming of orientation

The programming possibilities of the orientation depend on the number of axes available on the machine:

- Number < 5:
 - Orientation axis angle
- Number = 5:
 - Orientation axis angle
 - Orientation vector

Singularities

A pole cannot be directly passed through when a transformation is active.

NOTICE
Traversing close to the pole
When traversing close to the pole, the axes involved can be overloaded. This is because the control does not automatically reduce or limit the feedrate, acceleration or jerk. As a consequence, the user is responsible in limiting the feedrate of the axes involved and/or the clearance to the pole.

Retrace support (Run MyCC /RESU)

6.1 Brief description

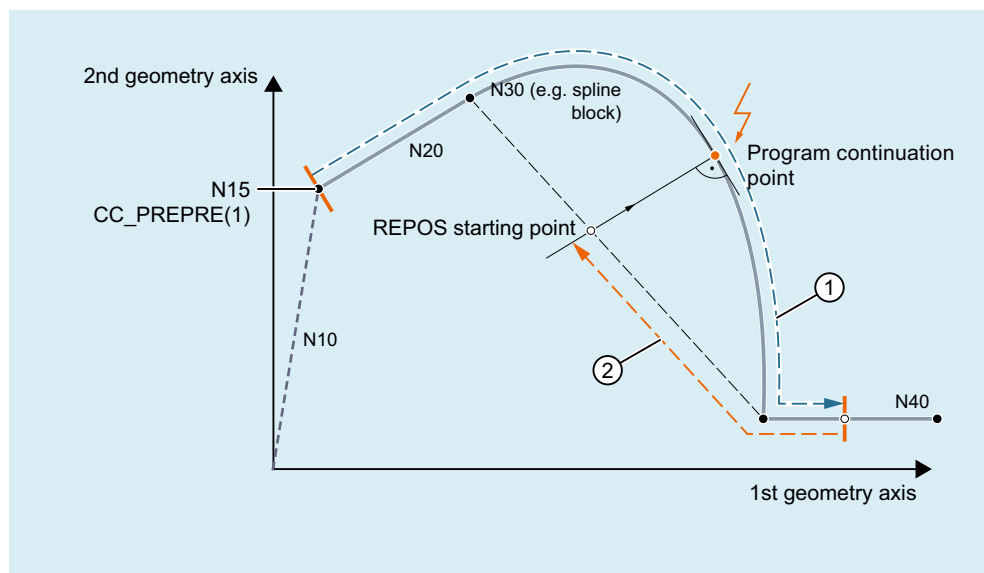
Function

The "Continue machining - Retrace support (RESU)" technological function supports the retracing of uncompleted 2-dimensional machining processes such as laser cutting, water jet cutting, etc.

In the event of a fault during the machining process, e.g. loss of the laser, RESU can be used even by machine operators who do not have specific knowledge of the active part program to interrupt machining and travel back along the contour from the interruption point to a program continuation point necessary for machining purposes.

After reaching the "continue machining" point, the machine operator triggers the re-machining. As part of the retrace process, an implicit block search takes place along the contour with calculation followed by repositioning on the contour and automatic retracing of the part program machining process.

The possibility of retrace support within a machining program is realized by programming RESU start and end points using the CC_PREPRE(...) procedure. The program continuation point can be arbitrarily selected within a contour range defined in this way.



- ① RESU start point or start of the RESU-capable contour range
- ② RESU end point or end of the RESU-capable contour range
- ③ Example of a program continuation point

Figure 6-1 Programmed contour with program continuation and interruption points

6.2 Function description

Precise retracing of contours is possible along all programmed contours comprising straight and circular elements. During retracing, other contour elements such as splines or automatically inserted non-linear contour elements (circle, parable, etc. e.g. through tool radius compensation) are mapped as straight lines between the start and end points of the corresponding contour element, thereby preventing precise retracing of contours.

Function code

The code for the "Continue Machining - Retrace support" technological function for function-specific identifiers of program commands, machine data, etc. is:

RESU (= REtrace SUpport)

Restrictions

The use of the "Continue Machining Retrace support" technological function is subject to the following restrictions:

- **The technology function is available only in the 1st channel of NC.**
- Program continuation or reverse travel is only possible for part program blocks that contain traversing blocks in the configured RESU working plane (e.g. 1st and 2nd geometry axes of the channel, see Section "Definition of the RESU working plane (Page 151)").

Further information

The "Continue Machining - Retrace support" technological function is a compile cycle. For the handling of compile cycles (see Function Manual *Technologies*, Section *Installation and activation of loadable compile cycles*).

See also

Installation and activation of loadable compile cycles (Page 19)

6.2 Function description

6.2.1 Function

Block search with calculation on contour

To be able to resume interrupted machining at a specific point in a part program, a block search can be carried out using the "Block search with calculation on contour" standard function. However, detailed knowledge of the part program is required to be able to enter the block number of the part program block required for the block search (i.e. the number of the block the search needs to locate).

Continue machining - retrace support

The "Continue machining - Retrace support" technological function supports the continuation of the machining operation by means of an implicit block search with calculation on the contour without the machine operator needing to know the part program block required.

Continue machining might be required for example in a laser cutting application if the laser is lost during the machining operation and machining needs to resume at the point at which it was interrupted.

RESU supports continue machining (retrace support) using the following automatically running sub-functions:

- Function-specific reverse travel along the contour to the required program continuation point
- Automatic identification of the part program block associated with the program continuation point
- Block search with calculation on the contour for the part program block identified
- Repositioning on the contour at the program continuation point
- Continuation of part program machining

To approach the required program continuation point exactly, it is possible to switch several times between reverse and forward travel along the contour during the continue machining process.

RESU-capable contour ranges

RESU is activated by programming the function-specific part program command `CC_PREPRE (1)`. Only the contour range lying between the RESU start (`CC_PREPRE (1)`) and the interruption point (NC stop) is retraceable in the sense of RESU.

Once RESU has been launched, all part program blocks in which traversing movements are programmed are logged by RESU for possible subsequent reverse travel. Contour ranges for which continuing machining is irrelevant can be excluded from the log using RESU stop `CC_PREPRE (0)`.

Contour ranges, which are not logged are bridged by straight lines between the start and end points during reverse/forward travel.

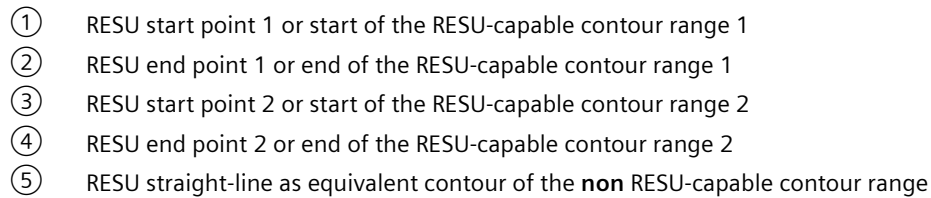


Figure 6-2 RESU-capable contour ranges

6.2.2 Definition of terms

The interruption point is the point of the contour at which the traversing movement comes to a standstill following an NC stop and reverse travel is activated.

The program continuation point is the point of the contour at which reverse travel terminates and program continuation is activated.

Retraceable contour ranges consist of traversing blocks in the configured RESU working plane (e.g. 1st and 2nd geometry axis of the channel), that are programmed in the part program between the RESU start command `CC_PREPRE(1)` and the RESU stop command `CC_PREPRE(0)` (see "Figure 6-2 RESU-capable contour ranges (Page 146)").

6.2.3 Functional sequence (principle)

The principle sequence of the RESU function between the interruption point, program continuation point and continuation of part program processing is described below.

Requirements

A part program with traversing blocks in the configured RESU working plane (Page 151) as well as the command for the RESU start has been started in the 1st channel.

Functional sequence

1. Interrupt part program processing:
The part program processing / traversing movement may be interrupted any number of traversing blocks after RESU start by NC stop.
2. Select reverse travel:
The selection of reverse travel is realized using the NC/PLC interface signal:
`<Chan>.basic.out.retraceSupportTraversBackwards = 1`
3. Reverse travel:
The contour is traversed in the RESU working plane (Page 151) in the reverse direction with NC start. Instead of the current machining program, RESU selects the automatically generated RESU main program (Page 159).
4. End reverse travel:
Once the required program continuation point on the contour has been reached, reverse travel is ended using NC stop.
5. Select forward travel (optional):
For forward travel, reverse travel must be deselected via NC/PLC interface signal:
`<Chan>.basic.out.retraceSupportTraversBackwards = 0`
6. Forward travel (optional):
The contour is traversed in the RESU working plane (Page 151) in the forward direction with NC start.
7. End forward travel (optional):
Once the required program continuation point on the contour has been reached, forward travel is ended using NC stop.

6.2 Function description

8. Retrace support:

Continue machining is initiated via PLC interface signal:

`<Chan>.basic.out.retraceSupportStart = 1` (start retrace support)

For retrace support, RESU automatically selects the original machining program and launches a block search with calculation as far as the program continuation point.

9. Continuation of part program machining:

Part program processing continues at the program continuation point in accordance with the "Block search with calculation" standard function when two NC start commands occur one after the other.

The first NC start command processes the action blocks. Retrace support

ASUP CC_RESU_BS_ASUP.SPF (Page 163) is initiated when the last action block is reached:

`<Chan>.basic.in.blockSearchLastActionActive == 1` (last action block active)

The second NC start command processes the approach block before part program processing is resumed.

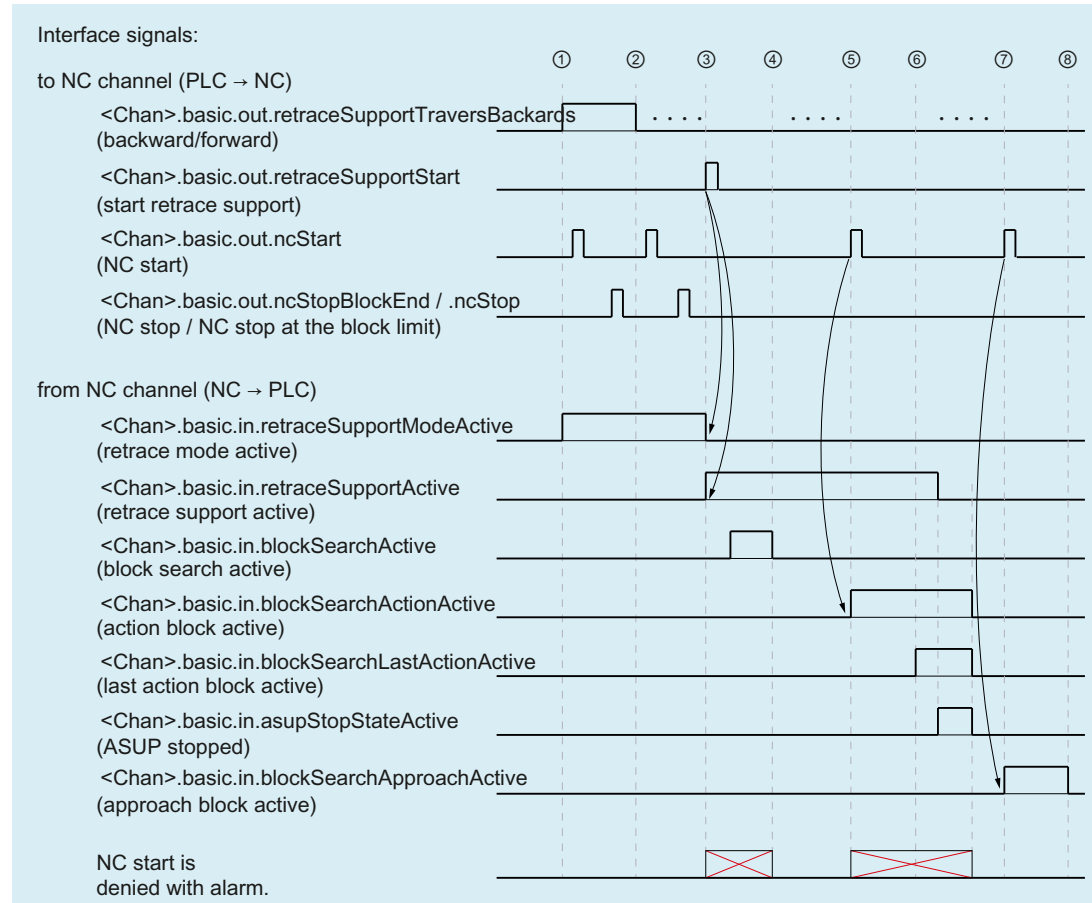
Note

Points 3 to 8 can be repeated as often as required.

Following retrace support, a new reverse travel is possible up to a maximum of the last program continuation point.

Signal characteristic of the NC/PLC interface signals

The principle sequence of the RESU function is illustrated in the following figure as a signal characteristic of the NC/PLC interface signals involved:



- ① Reverse travel is started.
- ② Forward travel is started (optional).
- ③ Continue machining is started (block search).
- ④ Search destination (target block) was found.
- ⑤ 1st NC START → Action blocks are output
- ⑥ Last action block is activated.
The RESU ASUP CC_RESU_BS_ASUP.SPF (Page 163) is triggered when the last action block is activated.
- ⑦ 2nd NC start → the approach block to the program continuation point is executed.
- ⑧ Part program processing (target block) resumed

Figure 6-3 Signal chart

6.2 Function description

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.out.retraceSupportTraversBackwards	LBP_Chan*.f0_1	DB21,DBX0.1
<Chan>.basic.out.retraceSupportStart	LBP_Chan*.f0_2	DB21,DBX0.2
<Chan>.basic.in.blockSearchLastActionActive	LBP_Chan*.E_LastActBlock	DB21,DBX32.6
<Chan>.basic.out.ncStart	LBP_Chan*.A_NCStart	DB21,DBX7.1
<Chan>.basic.out.ncStopBlockEnd	LBP_Chan*.A_NCStopBlock	DB21,DBX7.2
<Chan>.basic.out.ncStop	LBP_Chan*.A_NCStop	DB21,DBX7.3
<Chan>.basic.in.retraceSupportModeActive	LBP_Chan*.f32_1	DB21,DBX32.1
<Chan>.basic.in.retraceSupportActive	LBP_Chan*.f32_2	DB21,DBX32.2
<Chan>.basic.in.blockSearchActive	LBP_Chan*.E_BlockSearch	DB21,DBX33.4
<Chan>.basic.in.blockSearchActionActive	LBP_Chan*.E_ActBlock	DB21,DBX32.3
<Chan>.basic.in.blockSearchLastActionActive	LBP_Chan*.E_LastActBlock	DB21,DBX32.6
<Chan>.basic.in.asupStopStateActive	LBP_Chan*.E_ASUP_Stop	DB21,DBX318.0
<Chan>.basic.in.blockSearchApproachActive	LBP_Chan*.E_BegBlock	DB21,DBX32.4

6.2.4 Maximum retraceable contour area

In multiple machining continuation within a contour area, the reverse travel on the contour is always possible only up to the last machining continuation point (**W**). In first-time reverse travel after RESU start, reverse travel up to the start of the contour range is possible.

This response must be illustrated via the following graph. For the sake of simplicity, the interruption point (**U**) is always the same:

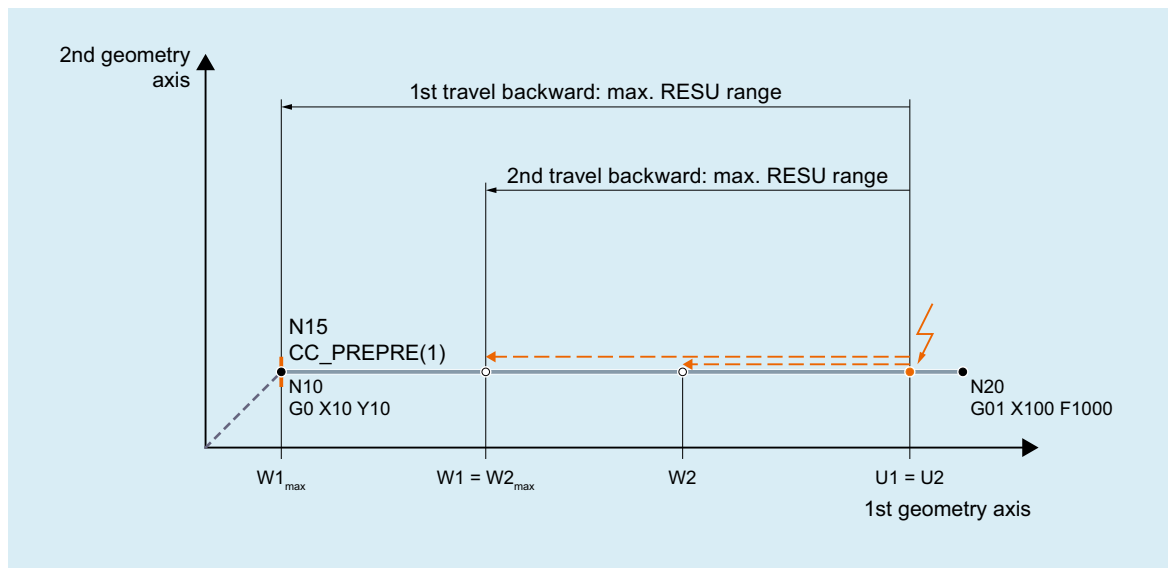


Figure 6-4 Maximum Retractable contour range

1st reverse travel

In first-time reverse travel, reverse travel is possible maximum up to the start of the first contour element (N20) after RESU start (N15) ($W1_{max}$).

If reverse travel takes place up to the machining continuation point W1, W1 defines the maximum RESU range for a possible further reverse travel after machining continuation and forward travel.

2nd reverse travel

A renewed reverse travel can now be undertaken maximum only up to the last machining continuation point $W2_{max} = W1$.

If reverse travel goes as far back as program continuation point W2, the maximum RESU range is restricted further.

6.3 Startup

6.3.1 Activation

It should be ensured prior to starting up the technology function the corresponding compile cycle is loaded and activated (see also Function Manual *Technologies*, Section *Installation and activation of loadable compile cycles*).

Activation

The "Continue machining - Retrace support" technology function is activated via the following machine data:

MD60900+i \$MN_CC_ACTIVE_IN_CHAN_RESU[0], bit 0 = 1

Note

The "Continue machining - Retrace support" technological function is available only in the **1st channel** of NC.

See also

Installation and activation of loadable compile cycles (Page 19)

6.3.2 Definition of the RESU working plane

Machining continuation/reverse travel is possible only for part program blocks that contain traversing blocks in the configured RESU working plane.

The RESU working plane is defined with the following machine data:

MD62580 \$MC_RESU_WORKING_PLANE

Value	Meaning
1	The RESU working plane is formed by the 1st and 2nd Geometry axes of the 1st channel (for G17).
2	The RESU working plane is formed by the 1st and 3rd Geometry axes of the 1st channel (for G18).
3	The RESU working plane is formed by the 2nd and 3rd Geometry axes of the 1st channel (for G19).

6.3.3 Memory configuration: Block memory

The technological function requires **additional** data in the NC-internal block memory. The values must be increased for the following memory configuring channel-specific machine data:

- MD28090 \$MC_MM_NUM_CC_BLOCK_ELEMENTS += **4** (number of block elements for compile cycles)
- MD28100 \$MN_MM_NUM_CC_BLOCK_USER_MEM += **20** (size of block memory for compile cycles (DRAM) in KB)

6.3.4 Memory configuration: Heap memory

Memory requirements

RESU requires compile cycles heap memory for the following function-specific buffers:

- **Block buffer**
The larger the block buffer (see "Figure 6-6 RESU-specific part programs (Page 159)"), the larger the number of part program blocks that can be traversed backwards.
32 bytes are required per part program block.
The block buffer can be parameterized directly.
- **Block search buffer**
The block search buffer contains the information required for processing subprogram searches in the context of RESU.
180 bytes are required for each subprogram. The block search buffer requires at least 2880 bytes (16 subprogram calls with 180 bytes each).
The block search buffer cannot be parameterized directly.
A function-specific GUD variable displays the size of the block search buffer (for creation of GUD variables, see Section "Channel-specific GUD variables (Page 168)").

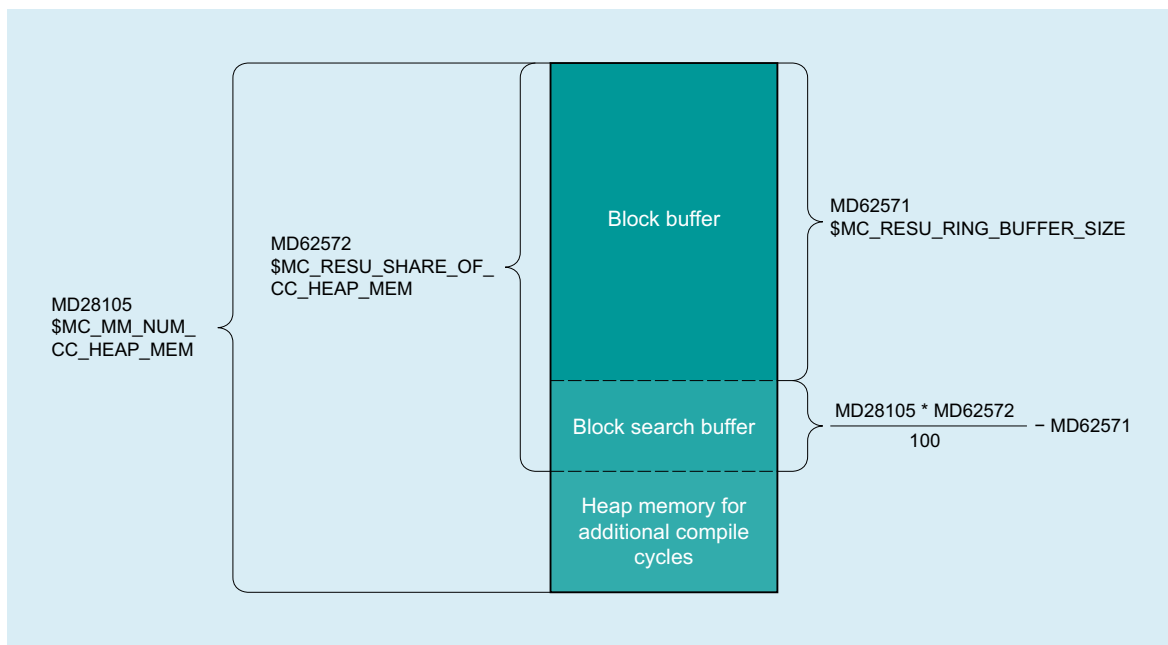


Figure 6-5 Division of the heap memory for compile cycles

Memory configuration

Size of the compile cycle heap memory

The size of the heap memory in KB that can be used by the user for compile cycles is defined via the memory configuring channel-specific machine data:

$MD28105 \$MC_MM_NUM_CC_HEAP_MEM$

For RESU, the already existing machine data value (x) is adjusted as follows:

6.3 Startup

MD28105 \$MC_MM_NUM_CC_HEAP_MEM = x + 50

Size of the block buffer

The size of the block buffer is adjusted via the machine data:

MD62571 \$MC_RESU_RING_BUFFER_SIZE

Default setting:

MD62571 \$MC_RESU_RING_BUFFER_SIZE = 1000

RESU portion of the total heap memory

The RESU portion of the total heap memory that can be used for compile cycles is set via the machine data:

MD62572 \$MC_RESU_SHARE_OF_CC_HEAP_MEM

Default setting:

MD62572 \$MC_RESU_SHARE_OF_CC_HEAP_MEM = 100

Error messages

The block search buffer requires at least 2880 bytes (corresponding to 16 subprogram calls with 180 bytes each). Otherwise, the following alarm will be generated during NC power-up:

Alarm 75600 "Channel 1 Retrace Support: Incorrect MD configuration, error no. 5"

If the block search buffer is not big enough during operation, the following alarm appears:

Alarm 75606 "Channel 1 retraceable contour shortened"

6.3.5 RESU main program memory area

Memory configuration

The storage location of the RESU main program CC_RESU.MPF can be set with the following machine data (see Section "Main program (CC_RESU.MPF) (Page 159)"):

MD62574 \$MC_RESU_SPECIAL_FEATURE_MASK (additional RESU features)

Bit	Value	Meaning
1	0	The RESU main program is stored in the dynamic NC memory (default).
	1	The RESU main program is stored in the static NC memory (default).

Storage in the dynamic NC memory (default)

If the RESU main program is created in the dynamic NC memory, the memory area available to the user for file storage must be increased as follows:

MD18351 \$MN_MM_DRAM_FILE_MEM_SIZE = x¹⁾ + 100

¹⁾ already available machine data value

Storage in the static NC memory

If the RESU main program is created in the static memory area of the NC, it is retained even after a POWER OFF. However, since RESU regenerates the RESU main program every time the retrace support function is used, this parameter setting is not recommended.

6.3.6 Storage of the RESU subroutines

Storage as user or manufacturer cycles

The following RESU-specific subroutines can be stored as user cycles or manufacturer cycles:

- INI program: CC_RESU_INI.SPF
- END program CC_RESU_END.SPF
- Retrace support ASUB CC_RESU_BS_ASUP.SPF
- RESU ASUB CC_RESU_BS_ASUP.SPF

The setting is done using machine data:

MD62574 \$MC_RESU_SPECIAL_FEATURE_MASK (additional RESU features)

Bit	Value	Meaning
2	0	The RESU-specific subroutines are stored as user cycles (default).
	1	The RESU-specific subroutines are stored as manufacturer cycles .

Series commissioning

Due to the default setting of MD62574 Bit 2, the RESU-specific subroutines along with their contents are stored as user cycles the first time the NC is powered up after the activation of the technological function.

If the setting is then changed to specify that the RESU-specific subroutines are to be stored as manufacturer cycles, the RESU-specific subroutines already created as user cycles are retained even after a new power-up and must be deleted.

As support for series commissioning, the RESU-specific subroutines present as user cycles can be deleted without prompting when the NC is powered up:

MD62574 \$MC_RESU_SPECIAL_FEATURE_MASK, Bit 3 = 1

6.3.7 ASUB enable

Note

A requirement for using ASUBs is that the "Cross-mode actions" option must be available.

6.3 Startup

The following machine data must be set for the start enable for the RESU-specific ASUB CC_RESU_ASUP.SPF while the channel is in the NC STOP state:

MD11602 \$MN_ASUP_START_MASK, bit 0 = 1 (ignore stop reason for ASUB)

MD11604 \$MN_ASUP_START_PRIO_LEVEL = 1 (priorities from which MD11602 is effective)

6.3.8 PLC user program

Requirements

The following functionality is necessary for the sequential coordination of the RESU function in the PLC user program:

```

IF      <Chan>.basic.in.retraceSupportActive      "Retrace support active" == 1
THEN    <Chan>.basic.out.retraceSupportTraversB "Forward/Reverse" = 0
        ackwards                                "Start retrace support" = 0
        <Chan>.basic.out.retraceSupportStart

IF      <ModeGroup>.basic.out.reset                "Mode group reset" == 1
OR      <Chan>.basic.out.reset                    "Reset" == 1
THEN    <Chan>.basic.out.retraceSupportTraversB "Forward/Reverse" = 0
        ackwards                                "Start retrace support" = 0
        <Chan>.basic.out.retraceSupportStart

```

The following signals should be reset for safety reasons:

```

IF      <Chan>.basic.out.retraceSupportStart      "Start retrace support" == 1
THEN    <Chan>.basic.out.retraceSupportTraversB "Forward/Reverse" = 0
        ackwards

IF      <Chan>.basic.out.retraceSupportTraversB "Forward/Reverse" == 1
THEN    ackwards                                "Start retrace support" = 0
        <Chan>.basic.out.retraceSupportStart

```

Program example

The following program extract implements the changes described above:

```

U      <Chan>.basic.in.retraceSupportActive      // IF      "Retrace support active" == 1
R      <Chan>.basic.out.retraceSupportTraversBack // THEN    "Forward/Reverse" = 0
wards
R      <Chan>.basic.out.retraceSupportStart      //          "Start retrace support" = 0
O      <ModeGroup>.basic.out.reset              // IF      "Mode group reset" == 1
O      <Chan>.basic.out.reset                   // OR      "Reset" == 1

```

```

R    <Chan>.basic.out.retraceSupportTraversBack // THEN  "Forward/Reverse" = 0
wards
R    <Chan>.basic.out.retraceSupportStart          //      "Start retrace support" = 0
U    <Chan>.basic.out.retraceSupportStart          // IF    "Start retrace support" == 1
R    <Chan>.basic.out.retraceSupportTraversBack // THEN  "Forward/Reverse" = 0
wards
U    <Chan>.basic.out.retraceSupportTraversBack // IF    "Forward/Reverse" == 1
wards
R    <Chan>.basic.out.retraceSupportStart          // THEN  "Start retrace support" = 0

```

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.out.retraceSupportTraversBackwards	LBP_Chan*.f0_1	DB21,DBX0.1
<Chan>.basic.out.retraceSupportStart	LBP_Chan*.f0_2	DB21,DBX0.2
<Chan>.basic.in.blockSearchLastActionActive	LBP_Chan*.E_LastActBlock	DB21,DBX32.6
<Chan>.basic.out.ncStart	LBP_Chan*.A_NCStart	DB21,DBX7.1
<Chan>.basic.out.ncStopBlockEnd	LBP_Chan*.A_NCStopBlock	DB21,DBX7.2
<Chan>.basic.out.ncStop	LBP_Chan*.A_NCStop	DB21,DBX7.3
<Chan>.basic.out.reset	LBP_Chan*.A_Reset	DB21,DBX7.7
<Chan>.basic.in.retraceSupportModeActive	LBP_Chan*.f32_1	DB21,DBX32.1
<Chan>.basic.in.retraceSupportActive	LBP_Chan*.f32_2	DB21,DBX32.2
<Chan>.basic.in.blockSearchActive	LBP_Chan*.E_BlockSearch	DB21,DBX33.4
<Chan>.basic.in.blockSearchActionActive	LBP_Chan*.E_ActBlock	DB21,DBX32.3
<Chan>.basic.in.blockSearchLastActionActive	LBP_Chan*.E_LastActBlock	DB21,DBX32.6
<Chan>.basic.in.asupStopStateActive	LBP_Chan*.E_ASUP_Stop	DB21,DBX318.0
<Chan>.basic.in.blockSearchApproachActive	LBP_Chan*.E_BegBlock	DB21,DBX32.4
<ModeGroup>.basic.out.reset	LBP_ModeGroup.A_MGReset	DB11,DBX0.7

6.4 Programming

6.4.1 RESU Start/Stop/Reset (CC_PREPRE)

Using procedure `CC_PREPRE` (**Prepare Retrace**) the "Start", "Stop" and "Reset" modes can be activated for retracing:

Syntax

```
CC_PREPRE (<mode>)
```

Meaning

CC_PREPRE:	Retrace mode	
	Alone in the block:	Yes
<mode>:	Mode	
	Data type:	INT
	Value range:	-1, 0, 1
	Value	Meaning
	-1	Starts logging the traversing blocks. The information required for traversing backward is logged on a block-specific basis in a RESU-internal block buffer. The traversing information is related to the two geometry axes of the RESU working plane e.g. the 1st and 2nd geometry axes of the channel: MD20050 \$MC_AXCONF_GEOAX_ASSIGN_TAB[x]; with x = 0 and 1 Or, if transformation is active: MD24120 \$MC_TRAFO_GEOAX_ASSIGN_TAB[x]; with x = 0 and 1
	0	Stops logging the traversing blocks. Thus irrelevant contour areas can be excluded from the log. These excluded contour areas are bridged by a straight line between the starting point and end point when traversing backward.
	1	Deactivates logging of the traversing blocks and deletes the function-internal block buffer. Contour areas located before the instant of deactivation of the part program are therefore no longer available to RESU.

RESET response

For the following reset events, CC_PREPRE (-1) is implicitly executed:

- Power on (warm restart)
- Channel reset
- End of program reset (M30)

6.5 RESU-specific part programs

6.5.1 Overview

The following programs are automatically generated within the scope of the retrace support functionality (continue machining). User-specific content can be incorporated in the programs listed.

Function	Program name	Can be changed
Main program	CC_RESU.MPF	No
INI program	CC_RESU_INI.SPF	Yes

Function	Program name	Can be changed
END program	CC_RESU_END.SPF	Yes
Continue machining ASUP	CC_RESU_BS_ASUP.SPF	Yes
RESU ASUP	CC_RESU_ASUP.SPF	No

Internal structure

The following diagram provides an overview of the internal structure of the technological function and the relationship between the various part programs.

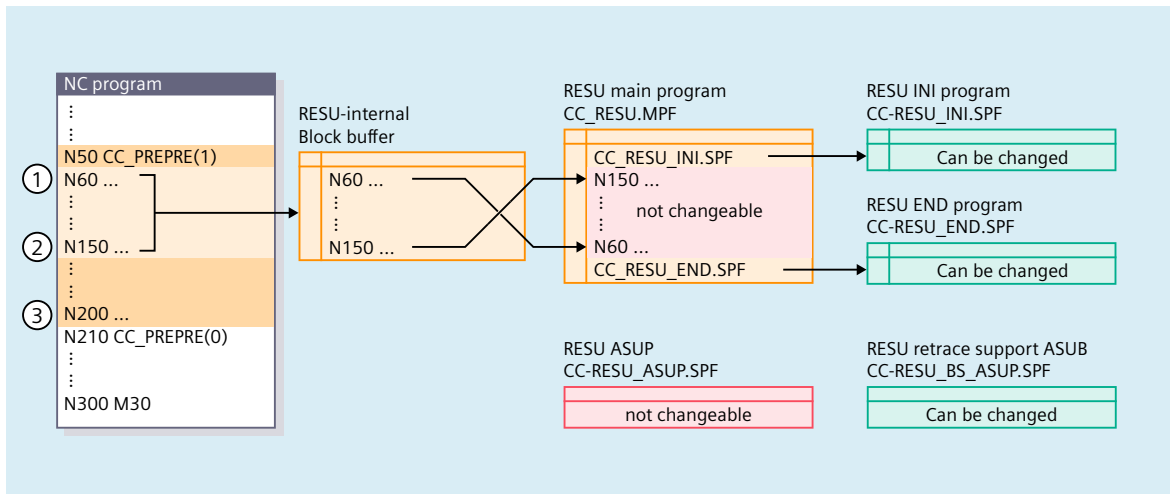


Figure 6-6 RESU-specific part programs

6.5.2 Main program (CC_RESU.MPF)

Function

In addition to the calls for the RESU-specific subprograms, the RESU main program "CC_RESU.MPF" contains the traversing blocks generated from the traversing blocks logged in the block buffer for reverse/forward travel along the contour. It is automatically regenerated by RESU whenever the status of the following NC/PLC interface signal changes after interruption of the part program:

<Chan>.basic.out.retraceSupportTraversBackwards (backward / forward)

Note

CC_RESU.MPF must not be changed. User-specific adjustments are to be made in the corresponding RESU-specific subprograms.

Error messages

By default, RESU generates traversing blocks for the entire retraceable contour range logged in the block buffer. If there is not enough memory for all traversing blocks to be generated in the parameterized memory area of the RESU main program (see Section "RESU main program memory area (Page 154)"), RESU reduces the number of generated traversing blocks.

The lacking memory and/or reduction in the number of the generated traversing blocks is indicated by an alarm:

RESU alarm 75608 "Channel *number* NC memory limit reached, RAM type *type*"

If the RESU main program is created in the static user memory, the following system alarm appears at the same time as the RESU alarm:

Alarm 6500 "NC memory full"

Note

If the number of traversing blocks generated is reduced due to insufficient memory, the entire retraceable contour can still be retraced for retrace support. To do this, proceed as follows:

- Reverse travel up to the end of the RESU main program
- Two-time change of the NC/PLC interface signal:
`<Chan>.basic.out.retraceSupportTraversBackwards` (backward/forward)

Using the current position as a new interruption point enables RESU to generate a new RESU main program.

Subsequently, travel is possible as far as the end of the retraceable contour range or, if the limits have changed, as far as the starting point of the last traversing block that can be generated.

The procedure described can be repeated as many times as required both for reverse and forward travel.

PLC signals

Basic Program Plus	Basic Program	
<code><Chan>.basic.out.retraceSupportTraversBackwards</code>	<code>LBP_Chan*.f0_1</code>	<code>DB21,DBX0.1</code>

6.5.3 INI program (CC_RESU_INI.SPF)

Function

The RESU-specific subprogram "CC_RESU_INI.SPF" contains the defaults required for the reverse travel:

- Metric input system: `G71`
- Absolute dimensions: `G90`
- To switch off the adjustable zero-point offsets / frames `G500`
(see Chapter "Frames (Page 173)");

- To switch off the active tool offsets (see Chapter "Tool T0 offsets (Page 173)":
- Switching off the tool radius offset: G40
- Traversing velocity: F200

Program structure

CC_RESU_INI.SPF has the following content by default:

```
PROC CC_RESU_INI
  G71 G90 G500 T0 G40 F200
  ;system frames that are present are deactivated
  ;actual value and scratching
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H01'
    $P_SETFRAME = ctrans()
  endif
  ;external zero point offset
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H02'
    $P_EXTFRAME = ctrans()
  endif
  ;tool carrier
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H04'
    PAROTOF
  endif
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H08'
    TOROTOF
  endif
  ;workpiece reference points
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H10'
    $P_WPFRAME = ctrans()
  endif
  ;cycles
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H20'
    $P_CYCFRAME = ctrans()
  endif
  ;transformations
  if $MC_MM_SYSTEM_FRAME_MASK B_AND 'H40'
    $P_TRAFRAME = ctrans()
  endif
  ; bit mask for global basic frames
  $P_NCBFRMASK = 0
  ;bit mask for channel-specific basic frames
  $P_CHBFRMASK = 0
  ;programmable frame
  $P_PFRAME = ctrans()
```

M17



CAUTION

Program changes

By changing the content of the RESU-specific subprogram "CC_RESU_INI.SPF", the user (machine manufacturer) takes over the responsibility for the correct sequence of the technological function.

Note

CC_RESU_INI.SPF may be changed.

CC_RESU_INI.SPF must not contain any RESU part program commands. CC__PREPRE (x) .

6.5.4 END program (CC_RESU_END.SPF)

Function

The task of the RESU-specific subroutine "CC_RESU_END.SPF" is to stop reverse travel once the end of the retraceable contour is reached. If the RESU function is parameterized appropriately, this scenario will not arise under normal circumstances.

Program structure

CC_RESU_END.SPF has the following content by default:

```
PROC CC_RESU_END
    M0
M17
```



CAUTION

Program changes

By changing the content of the RESU-specific subroutine "CC_RESU_END.SPF", the user (machine manufacturer) takes over the responsibility for the correct sequence of the technological function.

Note

CC_RESU_END.SPF may be changed.

CC_RESU_END.SPF must not contain any RESU part program commands. CC__PREPRE (x) .

6.5.5 Retrace support ASUB (CC_RESU_BS_ASUP.SPF)

Function

The NC is forced to approach the current path point during machining continuation with the help of the RESU-specific ASUB "CC_RESU_BS_ASUP.SPF":

- Reapproach next point on path: RMN
- Approach along line on all axes: REPOSA

Program structure

CC_RESU_BS_ASUP.SPF has the following content by default:

```
PROC CC_RESU_BS_ASUP SAVE
    RMN
    REPOSA
M17
```

 **CAUTION**

Program changes
By changing the content of the RESU-specific subroutine "CC_RESU_BS_ASUP.SPF", the user (machine manufacturer) takes over the responsibility for the correct sequence of the technological function.

Note

CC_RESU_BS_ASUP.SPF may be changed.

User-specific modifications must be inserted before the part program block RMN.

6.5.6 RESU ASUB (CC_RESU_ASUP.SPF)

Function

The RESU-specific ASUB "CC_RESU_ASUP.SPF" is required internally for the function. The ASUB is initiated if the following NC/PLC interface signal is switched over in the NC stop state:

<Chan>.basic.out.retraceSupportTraversBackwar (forward / backward)

6.6 Retrace support

Program structure

RESU-specific ASUB has the following content:

```
PROC CC_RESU_ASUP
    ; siemens system asup - do not change
    G4 F0.001
    M0
    REPOSA
M17
```

Note

CC_RESU_ASUP.SPF must not be changed.

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.out.retraceSupportTraversBackwards	LBP_Chan*.f0_1	DB21,DBX0.1

6.6 Retrace support

6.6.1 General information

Retrace support refers to the entire operation from initiation of retracing through the NC/PLC interface signal <Chan>.basic.out.retraceSupportStart (start retrace support) up to the continuation of part program processing on the programmed contour.

Requirement

In order for retrace support to function, the retrace mode, launched by means of the request for reverse travel, must be active in the channel:

<Chan>.basic.in.retraceSupportModeActive = 1 (retrace mode active)

(See Section "Functional sequence (principle) (Page 147)")

Subfunctions

The two essential subfunctions of retrace support are the standard NC functions:

- Block search with calculation on contour
- Repositioning on the contour via shortest route (REPOS RMN)

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.out.retraceSupportStart	LBP_Chan*.f0_2	DB21,DBX0.2
<Chan>.basic.in.retraceSupportModeActive	LBP_Chan*.f32_1	DB21,DBX32.1

6.6.2 Block search with calculation on contour

Function

The block search with calculation on the contour initiated implicitly by RESU within the framework of the retrace support has the following tasks:

- Set the program pointer on the part program block on which repositioning was done with the help of Reverse / Forward travel
- Calculates the axis positions on the basis of the programmed traversing blocks from the start of the part program to the target block
- Collates the instructions programmed from the start of the part program to the target block, which are executed in the action block. These include:
 - Auxiliary functions
 - Tool change
 - Spindle functions
 - Feedrate programming

All part program instructions which are not executed in the action block but are required for retrace support in the part program must be entered manually in the RESU-specific retrace support ASUB CC_RESU_BS_ASUP.SPF, e.g.:

- Synchronized actions
- M functions

Further information see Function Manual Basic Functions.

6.6.3 Reposition

Function

Following the end of the last action block (last traversing block before repositioning), NC Start outputs the approach block for repositioning all channel axes programmed in the part program as far as the target block.

Geometry axes

In the approach block, the geometry axes of the RESU working plane (e.g. 1st and 2nd geometry axes of the channel) traverse the shortest route along the contour to the program continuation point.

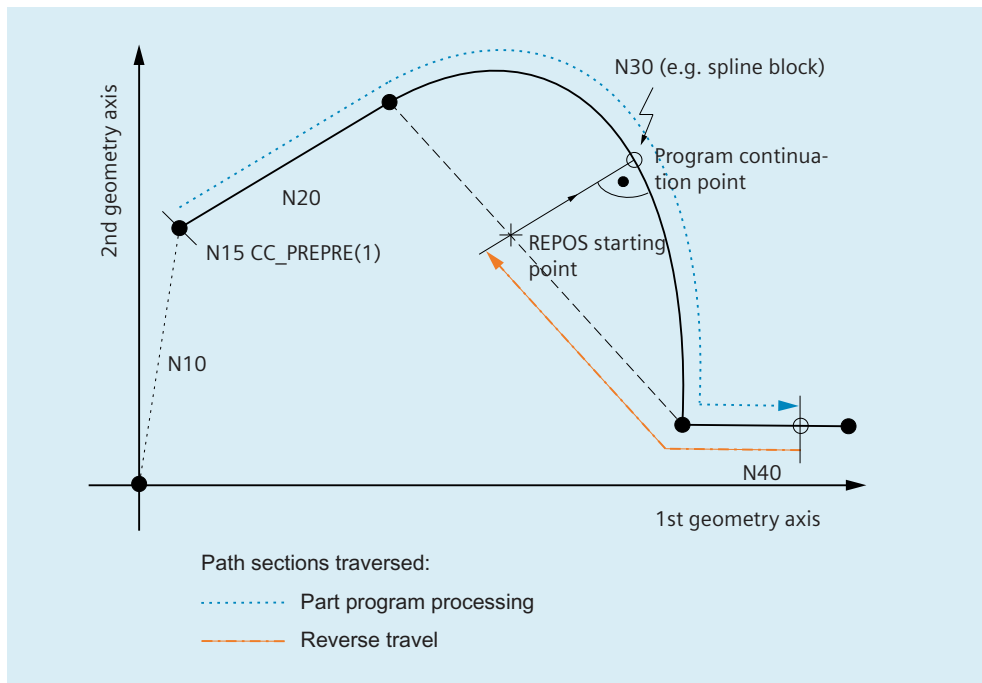


Figure 6-7 Retraceable contour ranges and REPOS

Channel axes

All other channel axes programmed in the part program travel to the relevant position calculated in the block search.

6.6.4 Temporal conditions concerning NC start

NC start is to be initiated twice by the machine operator within the framework of retrace support (see Section "Functional sequence (principle) (Page 147)").

The following conditions must be met:

- NC start for outputting the action blocks:
 - The block search must be completed:
`<Chan>.basic.in.blockSearchActive == 0` (block search active)
- NC start for outputting the approach block:
 - The RESU ASUB "CC_RESU_BS_ASUP" must be completed:
`<Chan>.basic.in.asupStopStateActive == 1` (ASUB stopped)

PLC signals

Basic Program Plus	Basic Program	
<Chan>.basic.in.blockSearchActive	LBP_Chan*.E_BlockSearch	DB21,DBX33.4
<Chan>.basic.in.asupStopStateActive	LBP_Chan*.E_ASUP_Stop	DB21,DBX318.0

6.6.5 Block search from last main block

The block search with calculation on the contour executed within the framework of the machining continuation via the use of the most powerful NCU in very large part programs can itself lead to computation times of several minutes up to the reaching of the target block.

A significant reduction of this waiting period is possible through the use of the "Block search from the last main block".

Functionality

For retrace support with block search from the last main block, the search for the target block takes place in 2 stages:

1. Block search without calculation from start of machining program to last main block before target block. Subprograms are ignored during this search, i.e. it takes place exclusively in the main program.
2. Block search with calculation on contour from main block to target block. This block search does not ignore subprograms.

Requirement

To enable a search from last main block for retrace support, at least one main block must be programmed after the RESU start `CC_PREPRE (1)`.

Main block

All instructions required for processing the subsequent section of the part program must be programmed in one main block.

Main blocks must be identified by a main block number consisting of the character ":" and a positive integer (block number).

Further information:

Programming Manual Fundamentals; Fundamentals of NC programming

Activation

Activation of the block search from the last main block is performed using the RESU-specific machine data:

MD62575 \$MC_RESU_SPECIAL_FEATURE_MASK_2, bit 0 (additional RESU features)

Bit	Value	Meaning
0	0	Retrace support is performed using block search with calculation on contour.
	1	Retrace support is performed using block search from the last main block.

Boundary conditions

In order that a new retrace support operation can take place following a retrace support operation with block search from last main block, the RESU start `CC_PREPRE (1)` must be programmed in the retrace support ASUB "CC_RESU_BS_ASUP.SPF".

Programming example:

```
PROC CC_RESU_BS_ASUP SAVE
    ; (synchronized actions, M functions, etc. required for retrace support)
    )
    CC_PREPRE (1)
    RMN
    REPOSA
M17
```

6.7 Function-specific display data

6.7.1 Channel-specific GUD variables

As display data for the size of the block search buffer, RESU provides the following channel-specific GUD variables:

GUD variable	Meaning	Value	Access
CLC_RESU_LENGTH_BS_BUFFER	Size of block search buffer	Byte	read only

After the startup of the technological function, the GUD variable is not displayed automatically on the operator panel.

Creating and displaying the GUD variable

Perform the following steps to create and display the GUD variables in the operator panel:

- Set password:
The password of protection level 1 (machine manufacturer) is to be input.
- Activate the "Definitions" display
- The file is recreated if an "SGUD.DEF" file is still not available:
Name: SGUD
Type: Global data /system
Confirm entry with **OK**.
This opens the file in the editor.

4. Edit the GUD variable definitions:

```
DEF CHAN REAL CLC_RESU_LENGTH_BS_BUFFER
M30
```

5. Save the file and close the editor.

6. Activate the "SGUD.DEF" file.

The GUD variable is not displayed on the operator panel.

Note

The new GUD variable, which is already being displayed, will be detected by the RESU function and supplied with an up-to-date value only following an NC POWER ON Reset. As a consequence, an NC POWER ON must be initiated after the creation.

Further information

The exact procedure to be following while creating and displaying GUD variables depends on the software version of the existing operator panel and is described in the:

- Universal/Turning/Milling/Grinding Operating Manual

6.8 Function-specific alarm texts

For the procedure for creating function-specific alarm texts, see Creating alarm texts (Page 27).

6.9 Boundary conditions

6.9.1 Function-specific supplementary conditions

6.9.1.1 Continue machining within subprograms

Subprogram call outside or inside a program loop

Clear retrace support within subprograms depends on whether the subprogram call is made outside or inside a program loop:

- Outside
Clear retrace support is possible if a subprogram is called outside a program loop.
- Inside
Clear retrace support may not be possible if a subprogram is called inside a program loop (see Section "Continue machining within program loops (Page 170)").

Number of passes P

Subprogram repetitions using number of passes P are taken into account for retrace support. This means that retrace support is performed in the part program with the correct reference to the part program block and number of passes P to the program continuation point of the contour.

6.9.1.2 Continue machining within program loops

In NC high-level language, program loops can be programmed using:

- LOOP ENDLLOOP
- FOR ENDFOR
- WHILE ENDWHILE
- REPEAT UNTIL
- CASE/IF-ELSE-ENDIF in conjunction with GOTOB



WARNING

Risk of collision

Significant contour deviations can occur during the machining if the programmed continuation point at the programmed contour is the result of a loop run that is not equal to the first loop run.

If retrace support is performed within program loops, the retrace support is always effective in the first loop run.

6.9.1.3 Machining continuation on full circles

In full circles, the block starting and end points coincide at one contour point. As no clear differentiation is possible in this case, one always starts from the block start point during machining continuation on this kind of a contour point. The first part program block following retrace support is then the circular block.

A contour point just before the block end point of the circular block must be selected to avoid traversing the circular block after the machining continuation.

6.9.1.4 Automatically generated contour elements

The automatic generation of non-linear / non-circular contour elements by the NC takes place, for example, when programming the following NC functions in the part program:

- RND
- G641/G642
- Tool radius compensation

For reverse / forward travel as part of RESU these contour elements can be replaced by straight lines between the start and end of the block.

6.9.2 Supplementary conditions for standard functions

6.9.2.1 Axis replacement

As long as RESU is active, the two geometry axes of the RESU working plane (e.g. the 1st and 2nd geometry axes of the channel) must not be transferred to another channel via axis replacement (`RELEASE (x) /GET (x)`).

The RESU activity:

- Starts:
 - with the part program command `CC_PREPRE (1)`
- Ends with:
 - the program end
 - or
 - with the part program command `CC_PREPRE (-1)`

6.9.2.2 Traversing movements of the channel axes

Other channel axes, except the two geometry axes of the RESU working plane, are not considered by RESU.

If traversing movements in other channel axes are required for machining continuation or reverse travel, these can either be undertaken by the machined operator manually, or programmed as travel block in the RESU-specific subroutine "CC_RESU_INI.SPF".



WARNING

Risk of collision when traversing

The machine operator must ensure that collision of the associated traversing movements does not take place during the entire machining continuation operation within the framework of the technological function of RESU.

6.9.2.3 Block numbers

The following RESU-specific subroutines and their subroutines must not contain block numbers:

- `CC_RESU_INI.SPF`
- `CC_RESU_END.SPF`

The following alarm appears in the event of an error:

Alarm 75604 "Reverse travel not possible, error no. *number*"

6.9 Boundary conditions

6.9.2.4 Block search

Block search with calculation

RESU is subject to the following constraints in the context of the "block search with calculation (on contour/at end of block)" standard function:

- The last block of the RESU part program command `CC_PREPRE (x)` run during the block search is effective in the target block.
- The retraceable contour range starts with the `REPOS` approach block.

Block search without calculation

RESU part program commands `CC_PREPRE (x)` have **no** effect in the "block searches without calculation" function.

6.9.2.5 Transformations

RESU can also be used for active kinematic transformation (e.g. 5-axis transformation) subject to restrictions, as the traversing movements of the two geometry axes of the RESU working plane are recorded in the basic coordinate system (BCS) and therefore before the transformation (see also Function Manual *Technologies*, Section *Transformation package - handling*).

Transformation changeover

While RESU is active, no transformation changes are permitted to take place and transformation must not be activated/deactivated.

The RESU activity:

- Starts:
 - with the part program command `CC_PREPRE (1)`
- Ends with:
 - the program end
or
 - with the part program command `CC_PREPRE (-1)`

Further information

Function Manual Transformations, Kinematics transformation

See also

Transformation package handling (Run MyCC /RCTRA) (Page 87)

6.9.2.6 Compensations

RESU can be used in interaction with compensations, because the traversing movements of the two geometry axes of the RESU working plane is recorded in the basic coordinate system (BCS) and therefore before the compensation.

A complete description of compensations can be obtained in:

Further information:

Function Manual Monitoring and Compensating; Compensations

6.9.2.7 Frames

RESU can be used in conjunction with frames.

However, as the traversing movements of the two geometry axes of the RESU working plane are recorded in the basic coordinate system (BCS) and therefore after the frames have been taken into account, the frame offsets must be deactivated during retrace support (reverse / forward travel).

The frame offsets are deactivated during retrace support via the standard default settings of the RESU-specific subprogram "CC_RESU_INI.SPF" (see Section: "INI program (CC_RESU_INI.SPF) (Page 160)").

Further information

Function Manual Basic Functions; Axes, coordinate systems, frames

6.9.2.8 Tool offsets

RESU can be used in conjunction with tool offsets.

However, as the traversing movements of the two geometry axes of the RESU working plane are recorded in the basic coordinate system (BCS) and therefore after the tool offsets have been taken into account, the tool offsets must be deactivated during retrace support (reverse / forward travel).

The tool offsets are deactivated during retrace support via the standard default settings of the RESU-specific subprogram "CC_RESU_INI.SPF" (see Section "INI program (CC_RESU_INI.SPF) (Page 160)").

Specific instances of tool radius compensation, e.g. compensation on outside corners `G450 DISC=x` may generate contour deviations between the contour traversed during retrace support and the contour programmed in the machining program.

Contour deviations will always occur wherever the tool radius offset generates contour elements which are non-linear or circular. For example, `G450DISC=x` with $x > 0$ generates parabolic or hyperbolic contour elements.

For further information regarding the subject of tool offsets, see Function Manual Tools.

High-speed laser switching signal (Run MyCC / HSLC)

7

7.1 Brief description

Function

The "Cycle-independent path synchronized switching signal output" technological function serves the purpose of switching time-critical, position-based machining processes on and off quickly, e.g. high speed laser cutting (HSLC; High Speed Laser Cutting).

The switching signal output can be block-related or path length-related:

- Block-related switching signal output
The switching signal output and therefore, the activation / deactivation of the machining is undertaken independently of the status changes:
 - Rapid travel mode G00 active / inactive
 - Programmed feed threshold fell short / exceeded
- Path length-related switching signal output
The switching signal output and therefore, the activation / deactivation of the machining takes place in a continuous change and is monitored along the traversed path.
This enables a regular machining without the individual switching positions having to be programmed explicitly.

The activation or selection of the mentioned options leads to the monitoring of the output of the digital signal which is programmed with a part program command.

I/Os

Only the on-board I/Os of the NC module can be used as the digital I/O via which the switching signal is output: The switching signal can only be output via one of the 4 on-board digital outputs on the NCU module.

Restrictions

The use of the "Cycle-independent path synchronized switching signal output" technological function is subject to the following restriction:

- **The technological function is available only in one channel of NC.**

Further information

The "cycle-independent path-synchronous switching signal output" technological function is a compile cycle. For the handling of compile cycles (see Section "Installation and activation of loadable compile cycles (Page 19)").

7.2 Functional description

7.2.1 General information

Note

The functionality is described with examples, with the help of the "High speed laser cutting technology (HSLC, High Speed Laser Cutting).

7.2.2 Calculating the switching positions

7.2.2.1 Block-related switching signal output

Switching criteria

During high-speed laser cutting, e.g. as used to manufacture perforated sheets, it is absolutely essential to switch the laser beam on/off exactly at the programmed setpoint positions during the machining process.

In order to minimize programming overheads, the switching positions of the technology function are calculated using the velocity of the geometry axes programmed in the part program block.

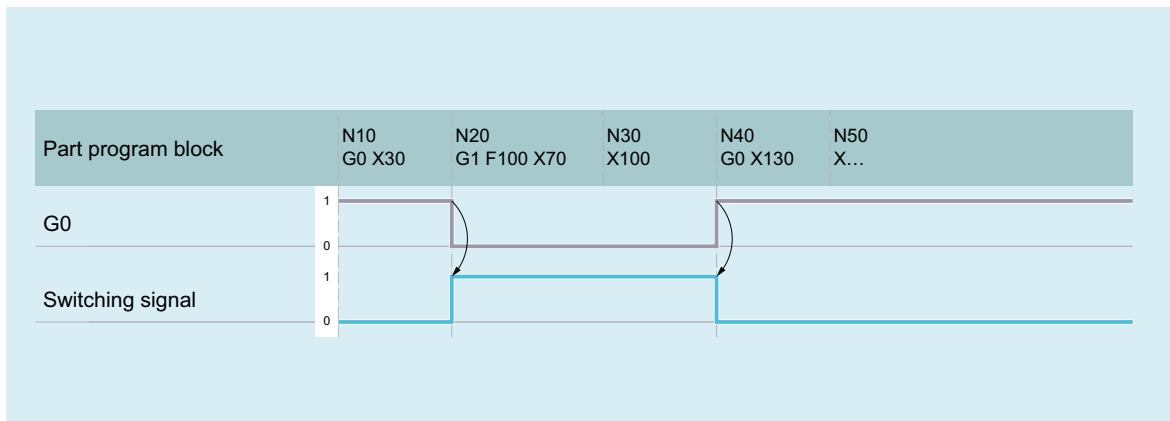
The following criteria define the setpoint position programmed in the part program block (end of block position) as a switching position:

1. G0 edge change
2. Overshooting/undershooting a freely programmable velocity threshold

G0 edge change as switching criterion

If G0 (rapid traverse) is **active** in a part program block (programmed or modal), the switching signal is switched **off**. Conversely, the following applies: If G0 (rapid traverse) is **not active** in a part program block, the switching signal is switched **on**. The G0 edge change marks the programmed end of block position of the previous block as the switching position.

Example:



The following block positions function as switching positions:

- Position X30 for G0 edge change from N10 to N20
- Position X100 for G0 edge change from N30 to N40

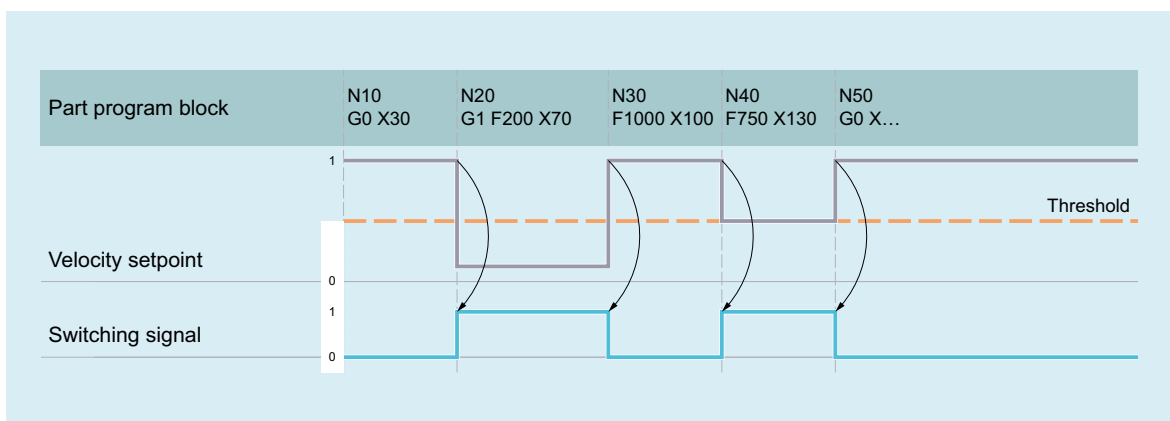
Freely programmable velocity threshold value as switching criterion

A freely programmable velocity threshold value is used to define the setpoint velocity programmed in the part program block at and above which the switching signal is activated/deactivated.

- If the setpoint velocity programmed in the part program block is **above** the programmed threshold, the switching signal is switched **off**.
- If the setpoint velocity is **at or below** the threshold value, the switching signal is switched **on**.

The edge change marks the programmed end of block position of the previous block as the switching position.

Example:



7.2 Functional description

The following block positions function as switching positions:

- Position X30 for edge change from N10 to N20
- Position X70 for edge change from N20 to N30

Note

G0 always deactivates the switching signal, regardless of the threshold value.

7.2.2.2 Path length-related switching signal output

Programmable paths as the switching criterion

In path-related switching signal output, the switching positions are defined with the help of the two freely programmable paths s_1 and s_2 .

Functional sequence

The path length-related switching signal output starts with a switch on signal at the beginning of the first traversing block after activation with CC_FASTON_CONT (refer to ① in the screen).

The processing is active till a deactivation signal is set after the traversing of a programmable path s_1 (refer to ② in screen). This way the processing is interrupted till a switch on signal is set again after the traversing of a programmable path s_2 (refer to ③ in screen).

The changeover between activation and deactivation signal, and therefore between processing and interruption phase is path length dependent at the end of either of the stretch sections s_1 and s_2 . This enables a continuous and regular machining without the individual switching positions having to be programmed explicitly.

The path length-related switching signal output ends with the start of the first traversing block (or of another executable block) after deactivation with CC_FASTOFF (refer to ④ in screen).

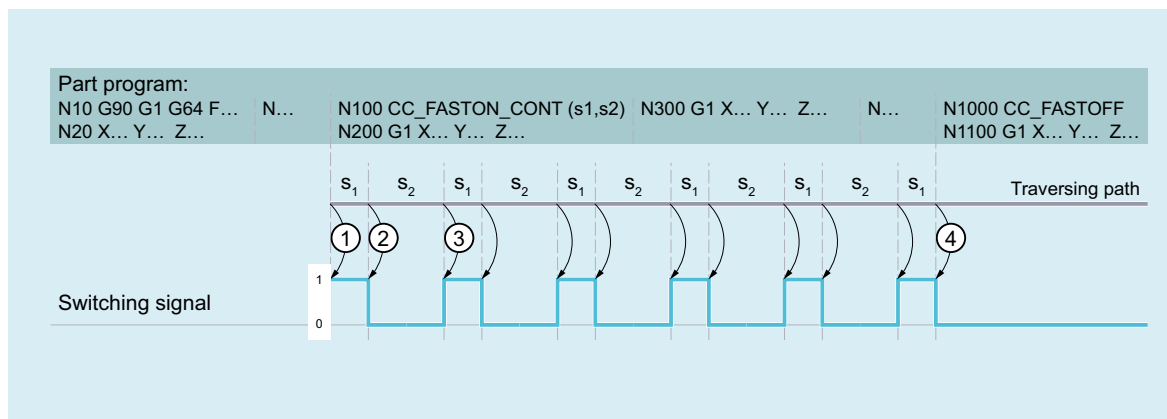


Figure 7-1 Path length-related switching signal output

7.2.3 Calculating the switching instants

In order for the switching to be as precise as possible at the switching positions calculated, the control calculates the positional difference between the actual position of the geometry axes involved and the switching difference in every position controller cycle.

If the positional difference is less than 1.5 position controller cycles, the control converts it into a temporal difference taking into account the current path velocity and acceleration rate of the geometry axes.

With the temporal difference specified, a hardware timer is started, which triggers the switching signal at exactly the instant calculated in advance regardless of the position controller cycle.

7.2.4 Switching frequency and switching position distance

Maximum switching frequency

The maximum switching frequency is: 1 signal edge change per IPO cycle

Note

Special case: IPO cycle time = position controller cycle time

In this case, the maximum switching frequency is:

1 signal edge change per **2** IPO cycles

Minimum switching position clearance

The minimum possible distance between the switching positions placed one above the other depends on:

- The duration of an IPO cycle
- The feed rate

The theoretically possible minimum distance can be determined from these dimensions as follows:

$$\text{Minimum switching position clearance} = \text{Programmed feed rate} * \text{IPO cycle time}$$

Example:

For IPO cycle times of 2 ms and position controller cycle times of 1 ms as well as a feed rate of 20000 mm/min, the theoretically possible minimum distance between switching positions one above the other is limited to:

$$20000 \text{ mm/min} * 2 \text{ ms} = 0.667 \text{ mm}$$

Value below minimum switching position distance

For path length-related switching signal output, the value may fall below the minimum switching position distance, e.g. due to:

- Increase in feed rate
- Decrease in programmable switching position distance s_1 and s_2

The following reactions take place if the value falls below the minimum:

- Alarm 75501 "Channel %1 HSLC: CC_FASTON_CONT speed too high" is displayed.
- The switching signal at the current switching position is omitted. To maintain the machining rhythm, the function also suppresses the switching signal at the following switching position.

The position of all the following switching positions is not affected by this. In the further course of the path, one can witness alternating output and suppression of two switching signals following each other.

7.2.5 Approaching switching position

If in block-related switching signal output a switching position is not reached exactly, e.g. in continuous-path mode and travel in more than one geometry axis, then switching takes place at the instant at which the positional difference between the actual position of the geometry axes involved and the programmed switching position increases again.

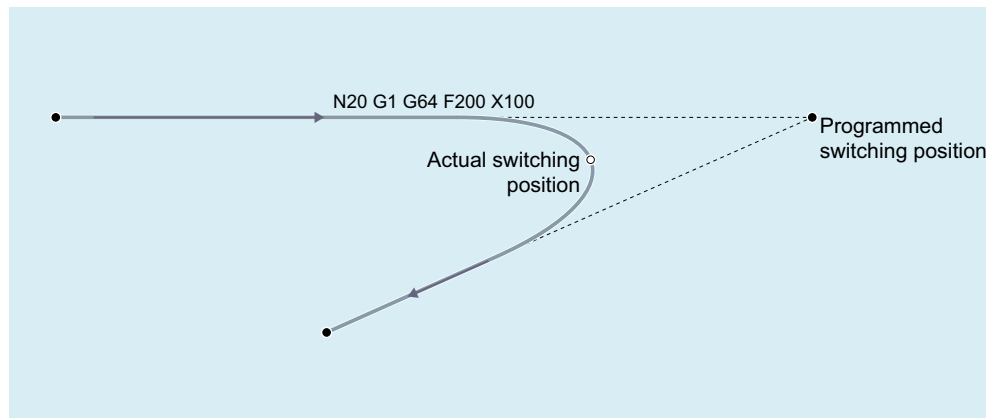


Figure 7-2 Switching position offset in path control mode

7.2.6 Programmed switching position offset

Programmed switching position offset

For block-related switching signal output, a positional offset of the switching position can be programmed :

- Offset distance **negative** = lead
With a negative offset distance, the switching position is offset **before** the set point position programmed in the part program block.
If an excessively large negative offset distance is programmed, i.e. the setpoint has already been exceeded by the time the edge is detected, the signal is switched immediately.
- Offset distance **positive** = follow-up
With a positive offset distance, the switching position is offset **behind** the set point position programmed in the part program block.

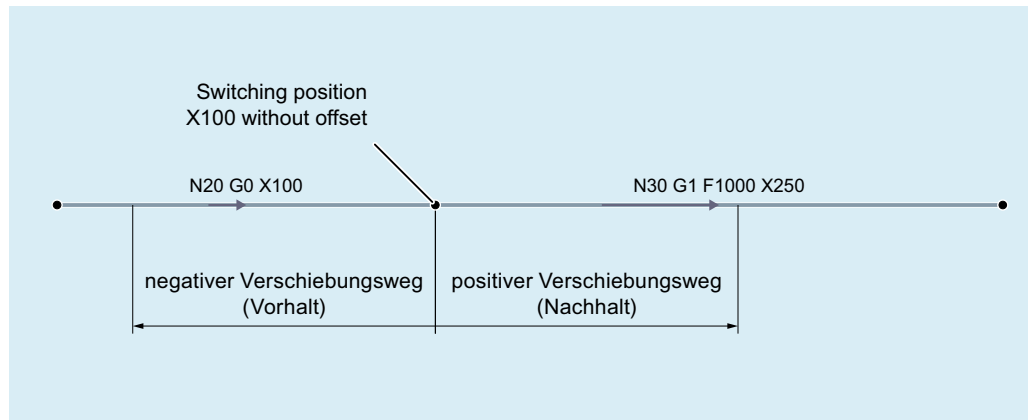


Figure 7-3 Programmed switching position offset

Path reference

The offset distance is a positional specification that refers to the programmed path. This way one can start from a simplified linear motion. Path curves are not taken into account.

Response to single block and G60

Due to the internal motion logic, negative offset distances (lead) have no effect when used with the following standard functions:

- Single Block
- Exact stop at block end (G60)

7.2.7 Response to part program interruption

Following an interruption in the part program (NC-STOP) and subsequent change to JOG mode, the technological function is deactivated or switching signals cease to be output.

7.3 Startup

The technology function is reactivated or switching signals are output again only after switching to the AUTOMATIC mode and continuing the part program (NC-START).

7.3 Startup

7.3.1 Activation

Before commissioning the technological function, ensure that the corresponding compile cycle has been loaded and activated (see also Section "Installation and activation of loadable compile cycles (Page 19)").

Activation

The "Cycle-independent path synchronized switching signal output" technology function is activated via the following machine data:

MD60948 \$MN_CC_ACTIVE_IN_CHAN_HSLC[0], Bit 0 = 1

Note

The "Cycle-independent path-synchronized switching signal output" is available only in **one** channel of NC.

7.3.2 Memory configuration

The technological function requires **additional** data in the NC-internal block memory. The values must be increased for the following memory configuring channel-specific machine data:

- MD28090 \$MC_MM_NUM_CC_BLOCK_ELEMENTS += **1** (number of block elements for compile cycles)
- MD28100 \$MN_MM_NUM_CC_BLOCK_USER_MEM += **10** (size of block memory for compile cycles (DRAM) in KB)

7.3.3 Parameterizing the digital on-board outputs

Parameter assignment

A digital output from the local I/O is required for the switching signal.

For this, at least 1 digital output byte must be defined through the following machine data:

MD10360 \$MN_FASTIO_DIG_NUM_OUTPUTS ≥ 1 (number of active digital output bytes)

Further information

A full description of the parameterization of a digital output can be found in:

- Function Manual Basic Functions; Digital and Analog NC I/O

7.3.4 Parameterizing the switching signal

Output number of the switching signal

Once the compile cycle has started up, the following function-specific machine data appears in the channel-specific machine data:

MD62560 \$MC_FASTON_NUM_DIG_OUTPUT (number of the digital output of the switching signal)

The number n of the on-board digital output through which the switching signal is to be output, must be entered in it:

n = 1, 2, 3 or 4

Disable

Entering the number of the digital output n = 0 deactivates the function. **No** message or alarm is output.

Effect on other output signals

The hardware-timer-controlled output of the switching signal at the parameterized output delays the signal output for the other digital on-board outputs, e.g. due to synchronized actions, by 2 IPO cycles.

7.3.5 Parameterization of the geometry axes

Default setting

Machines for high-speed laser cutting normally have two geometry axes that are configured in the following two machine data:

MD20050_\$MC_AXCONF_GEOAX_ASSIGN_TAB[0]

MD20050_\$MC_AXCONF_GEOAX_ASSIGN_TAB[1]

7.4 Programming

The calculation of the switching instants is derived from these two geometry axes.

Note

The configured axis selection for calculating the switching instants can be changed by redefining the first and second geometry axes in the part program with the help of program instructions `GEOAX (1, <axis name>)` and `GEOAX (2, <axis name>)`.

Important:

For the function change to be considered correctly, it should occur before the interpretation of the `CC_FASTON` command.

Changing the default setting

For a deviating machine configuration (e.g. definition of a third geometry axis), the default setting can be adjusted via the following machine data:

MD60948 \$MN_CC_ACTIVE_IN_CHAN_HSCL[1]

Value	Meaning
\$MN_CC_ACTIVE_IN_CHAN_HSCL[1]='H3'	Default setting. The calculation of the switching instants is derived from the first and second geometry axis.
\$MN_CC_ACTIVE_IN_CHAN_HSCL[1]='H7'	The calculation of the switching instants is derived from the first, second and third geometry axis.

Any change is effective only after the NC has powered up the next time.

7.4 Programming

7.4.1 Activating the block-related switching signal output (CC_FASTON)

Syntax

`CC_FASTON (DIFFON, DIFFOFF [, FEEDTOSWITCH])`

`CC_FASTON ()` is a procedure call and must therefore be programmed in a dedicated part program block.

Parameter

The parameters for the `CC_FASTON ()` procedure have the following meaning:

Parameter	Meaning
<DIFFON>	Length* of the offset distance for setting the switching signal.
<DIFFOFF>	Length* of the offset distance for deactivating the switching signal.

Parameter	Meaning
<FEEDTOSWITCH>	This parameter is optional. If the parameter is not specified in the procedure call, the G0 edge change is used as the switching criterion. If the parameter is specified in the procedure call, it contains as a switching criterion the velocity threshold value, which, when undershot or exceeded, activates or deactivates the switching signal accordingly.
* The basic unit (inch or mm) depends on the current dimensions programming (G70 / G71 / G700 / G710).	

Programming example

Programming	Comment
DEF REAL DIFFON= -0.08	; Length* of the offset distance for activating the switching signal = -0.08
DEF REAL DIFFOFF= 0.08	; Length* of the offset distance for deactivating the switching signal = 0.08
DEF REAL FEEDTOSWITCH= 20000	; Speed threshold value = 20000
CC_FASTON (DIFFON, DIFFOFF, FEEDTOSWITCH)	; Activating block-related switching signal output (with speed threshold value as switching criterion)

Changing parameters

The parameters for the CC_FASTON() procedure can be modified at any time during the execution of the part program. To do this, enter the procedure call again with the new parameter values. The switching criterion (G0 edge change / velocity threshold value) may also be changed.

Reset response

A reset (NC RESET or end of program) deactivates the function.

7.4.2 Activating the path length-related switching signal output (CC_FASTON_CONT)

Syntax

CC_FASTON_CONT (PATH_DISTANCE_ON, PATH_DISTANCE_OFF)

CC_FASTON_CONT() is a procedure call and must therefore be programmed in a dedicated part program block.

7.4 Programming

Parameter

The parameters for the `CC_FASTON_CONT()` procedure have the following meaning:

Parameter	Meaning
< PATH_DISTANCE_ON >	Length* of the stretch section with machining (s_1).
< PATH_DISTANCE_OFF >	Length* of the stretch section without machining (s_2).
* The basic unit (inch or mm) depends on the current dimensions programming (G70 / G71 / G700 / G710).	

Programming example

Programming	Comment
DEF REAL PATH_DISTANCE_ON = 0.5	; Length* of the stretch section with machining = 0.5
DEF REAL PATH_DISTANCE_OFF = 1.0	; Length* of the stretch section without machining = 1.0
CC_FASTON_CONT (PATH_DISTANCE_ON, PATH_DISTANCE_OFF)	; Activating path length-related switching signal output.

Changing parameters

The parameters for the `CC_FASTON_CONT()` procedure can be modified at any time during the execution of the part program. For this, the procedure call must be specified again with the new parameter values.

Reset response

A reset (NC RESET or end of program) deactivates the function.

7.4.3 Deactivation (CC_FASTOFF)

Syntax

`CC_FASTOFF`

`CC_FASTOFF` is a procedure call and must therefore be programmed in a dedicated part program block.

Functionality

The `CC_FASTOFF` procedure call deactivates the "Cycle independent, path synchronized switching signal output" function.

7.5 Function-specific alarm texts

For the procedure for creating function-specific alarm texts, see Creating alarm texts (Page 27).

7.6 Supplementary conditions

7.6.1 Block search

Switching signal output for block search

If a block search is on a part program block which lies after a `CC_FASTON()` procedure call for activating the technology function, then the switching signal is activated with the next traversing motion. One of the specific consequences of this is to initiate travel along the contour from the start position of the geometry axes back to the program continuation point with an activated switching signal.

Example

Normal process:

In the normal process of the part program machining, the switching signal is activated for the first time at the beginning of part program block N60.

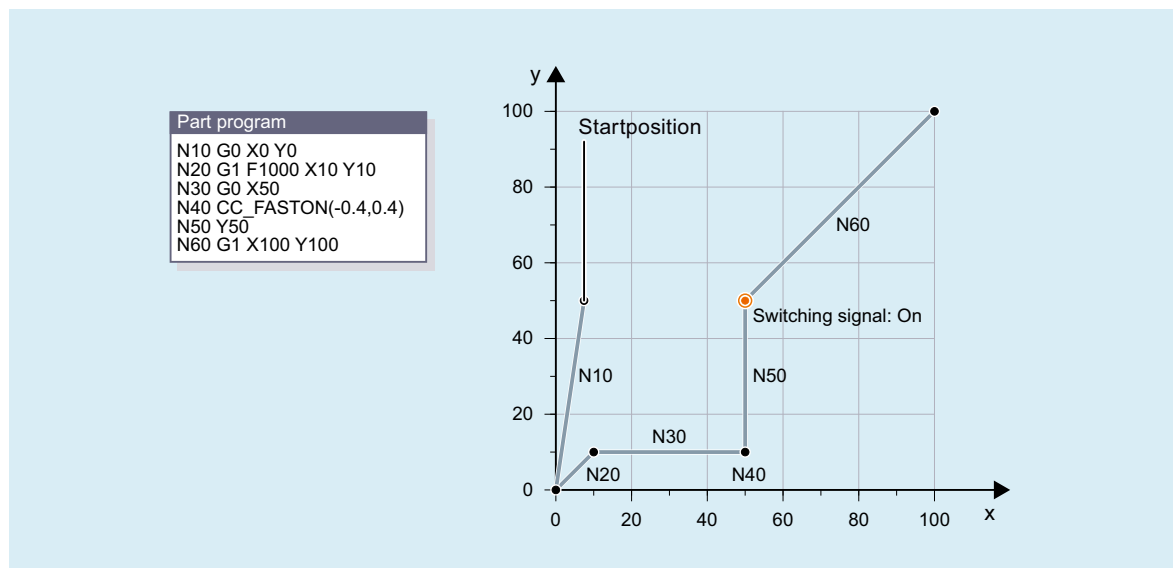


Figure 7-4 Switching signal for part program machining operation

Process sequence after block search:

If a block search is executed for the block end point of part program block N60 the switching signal is activated on reaching the start position of the geometry axes.

7.6 Supplementary conditions

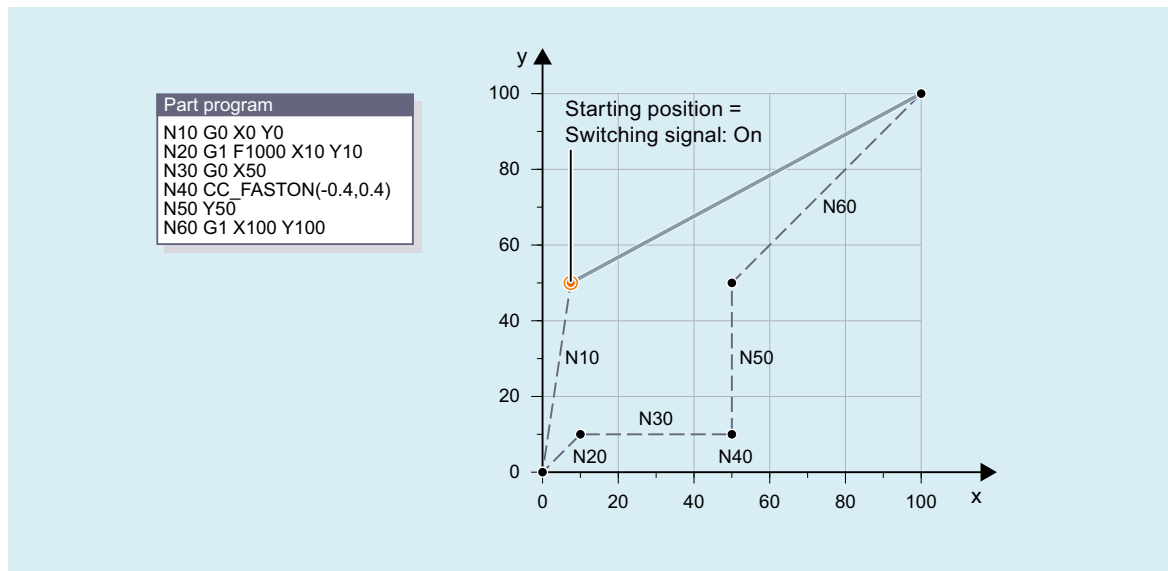


Figure 7-5 Switching signal after block search

Suppressing the switching signal output

The user (machine manufacturer) must take appropriate measures, e.g. disable the switching signal, in order to suppress the activation of the switching signal in the REPOS block in the constellation described above.

Note

It is the sole responsibility of the user (machine manufacturer) to suppress the output of the switching signal during repositioning, e.g. after a block search.

Further information

You will find a description of the block search in:

Function Manual Basic Functions; Mode Group, Channel, Program Operation, Reset Response

7.6.2 Transformations

The function will only run correctly with deactivated transformation. There is no monitoring function.

For a description of the transformations (see Section "Transformation package handling (Run MyCC /RCTRA) (Page 87)").

Further information

Function Manual Transformations; Kinematic Transformation

7.6.3 Compensations

The following compensations are considered while calculating the switching positions:

- Temperature compensation
- Sag compensation

A description of the compensations can be found in:

For further information

Function Manual Monitoring and Compensating; Compensations (K3)

7.6.4 Tool radius compensation (TRC)

As part of tool radius compensation, control-internal part program blocks (compensation blocks) are inserted into the part program. With reference to the switching signal output, a compensation block is always added to the next programmed part program block.

A description of the tool radius compensation can be found in:

Further information:

Function Manual Tools; Tool Offset, Tool Radius Compensation

7.6.5 Continuous-path mode

Continuous-path mode

Although the `CC_FASTON()`, `CC_FASTON_CONT()` and `CC_FASTOFF` procedure calls must be programmed in dedicated part program blocks, this will not lead to a drop in velocity while continuous-path mode is active (G64, G641, ...).

Continuous-path mode (ADIS)

If in continuous-path mode with programmable rounding response (G641 ADIS) a part program block is inserted in the part program, then the originally programmed switching position is not reached and the switching signal output takes place nevertheless in the center of the rounding block.

Further information

You will find a description of the continuous-path mode in:

Function Manual Basic Functions; Continuous-Path Mode, Exact Stop and LookAhead

7.6.6 Software cams

Because the hardware timer is also used for the "software cam" function, it is not possible to use the "clock-independent switching signal output" function with software cams at the same time.

7.6 Supplementary conditions

The following alarm appears in the event of an error:

Alarm 75500 "Channel *Channel No.*, wrong configuration of function: clock-independent switching signal output"

A description of the software cams can be found in:

Further information:

Function Manual Axes and Spindles; Position Switching Signals

Axis pair collision protection (Protect MyMachine /CC Axis Monitor)

8

8.1 Brief description

Note

Compile cycle

It should be ensured prior to starting up the function that the corresponding compile cycle is loaded and activated (see Function Manual "Technologies", Section "Installation and activation of loadable compile cycles").

Function

The "axis pair collision protection" function enables machine axes, which are arranged on the same guide element of a machine, to be monitored in pairs to ensure that no collisions occur and that the maximum distance between the two axes is not exceeded.

Function code

The code for function-specific identifiers of machine data, system variables etc., is:
PROTECT (axial collision PROTECTion)

Maximum number of axis pairs

A maximum of **20** axis pairs can be parameterized.

8.2 Functional description

The "axis pair collision protection" function is a protection function for machine axes which are arranged in a machine tool in such a way (on the same guide rail, for example) that incorrect operation or programming could cause them to collide with one another.

The machine axes are always monitored in pairs, i.e. parameters always need to be assigned for two machine axes in each case, which are then monitored in relation to one another. The machine axes being monitored may be located in different machine coordinate systems.

Collision protection

The function uses the current actual positions and actual velocities, as well as the offset for the machine coordinate systems and the axis-specific brake acceleration values, to calculate the distance between the standstill positions of the machine axes, on a cyclical basis. If the resulting distance is shorter than the protection window that has been set, the machine axes are decelerated to a standstill. This ensures that the minimum distance defined using the protection window is not undershot.

Distance monitoring

If the offset vector is selected accordingly, the function can also be used to monitor the maximum distance between the two machine axes (maximum distance vector).

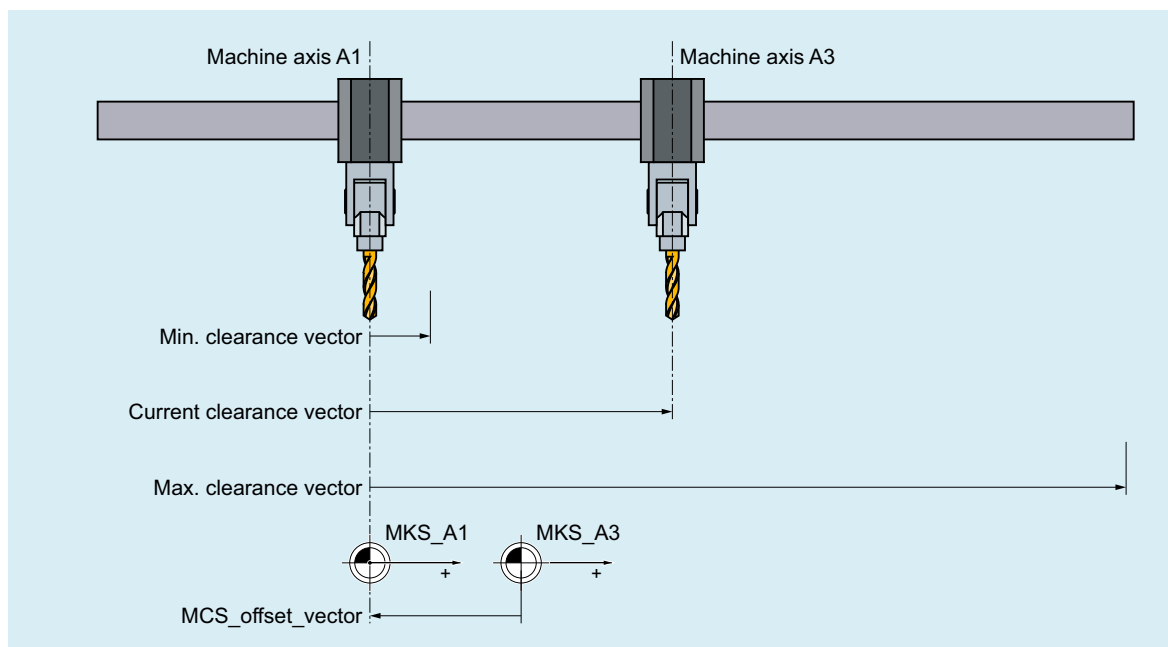


Figure 8-1 Basic design

Monitoring status

The actual status of an axis pair can be read out of `_PROTECT_STATUS` (Page 198), optionally defined global user variables in the NC program (GUD).

8.3 Commissioning

8.3.1 Enabling the technology function (option)

The function is an option, which must be assigned to the hardware via the license management.

6FC5800-0AN06-0YB0, "RMCC/PROT axis collision avoidance"

For test purposes, the function can be enabled by setting the option data:

MD19610 \$ON_TECHNO_EXTENSION_MASK[2], BIT4 = 1

8.3.2 Activating the technology function

Activation rule

The function must be activated on a channel-for-channel basis for the following NC channels:

- Independent of which channels are assigned to the machine axes to be monitored, always in the 1st channel of the NC
- In all channels that are assigned to the machine axes to be monitored (by the function) by parameterizing the machine data
- In all channels that are assigned at a later point in time to the machine axes to be monitored (by the function), e.g. as a result of axis replacement

Activation

The function is activated on a channel-for-channel basis using machine data:

MD60972 \$MN_CC_ACTIVE_IN_CHAN_PROT[0], bit n = 1

with n = 0, 1, 2, ..., corresponding to the (n+1)th channel of the NC

8.3.3 Activating the supplementary functions

The supplementary functions are activated for specific axis pairs using option data:

MD61535 \$MN_CC_PROTECT_OPTIONS[<a>]

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

Bit	Value	Meaning
0	1	Axis pair-specific activation/deactivation of the function "Axis pair collision protection" via the NC/PLC interface signal (Page 200)" Note After activating this supplementary function, the protection function is in the monitoring status (Page 198) == 1 (selected, but still not active)

8.3.4 Definition of an axis pair

A machine axis pair to be monitored is defined on an axis pair-specific basis in machine data:

MD61516 \$MN_CC_PROTECT_PAIRS[<a>] = <yyxx>

8.3 Commissioning

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

<yyxx>	Meaning
xx	1st and 2nd decimal place ⇒ axis number of the 1st machine axis
yy	3rd and 4th decimal place ⇒ axis number of the 2nd machine axis

Example

Definition of the 1st axis pair:

- 1st axis: 4th machine axis
- 2nd axis: 12th machine axis

MD61516 \$MN_CC_PROTECT_PAIRS[0] = 1204

8.3.5 Retraction direction

The direction of travel for retracting the corresponding machine axis is set using machine data:

MD61517 \$MN_CC_PROTECT_SAFE_DIR[<a>] = <yyxx>

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

<yyxx>	Meaning
xx	Retraction direction for the 1st axis of the axis pair
yy	Retraction direction for the 2nd axis of the axis pair
Retraction in the positive direction of travel of the machine axis: xx or yy > 0	
Retraction in the negative direction of travel of the machine axis: xx or yy = 0	

Note**Changing the retraction direction**

Changing the retraction direction in machine data MD61517

\$MN_CC_PROTECT_SAFE_DIR[<axis pair>] may only be carried out if the protection function for the axis pair is not active (MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] == 0).

8.3.6 Offset for the machine coordinate systems

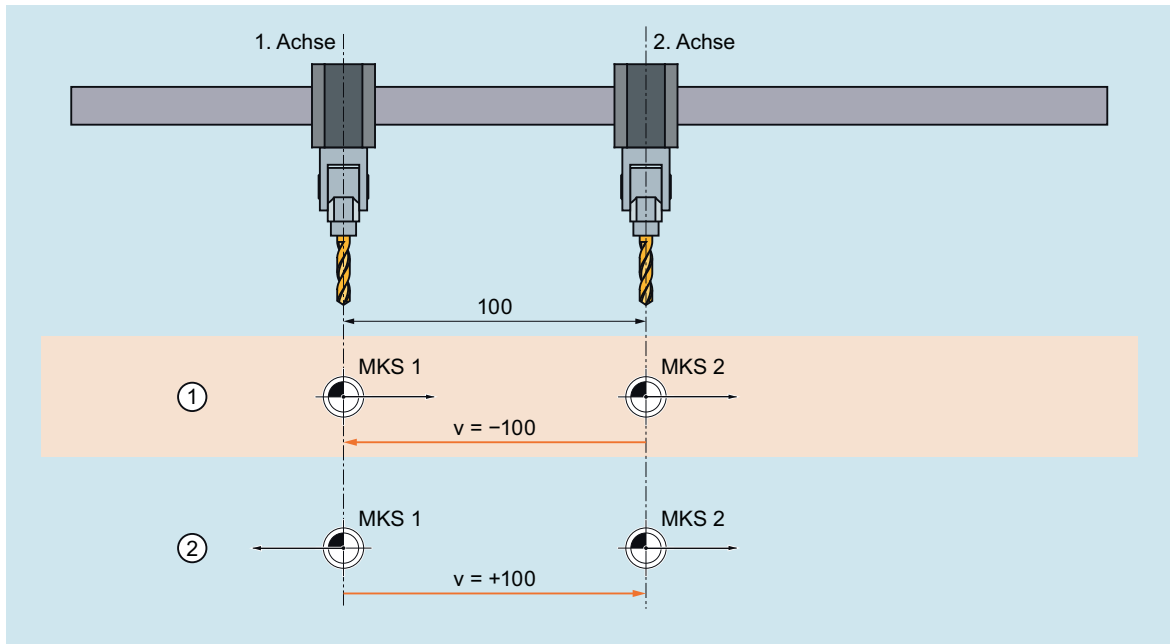
If the machine axes of the axis pair are located in different machine coordinate systems, then the appropriate offset vector must be specified in the following machine data:

MD61518 \$MN_CC_PROTECT_OFFSET[<a>] = <offset vector>

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

The offset vector should be specified as vector from the origin of the machine coordinate system of the 2nd axis of the axis pair to the origin of the machine coordinate system of the 1st axis, referred to the machine coordinate system of the 1st axis.

If both machine axes are located in the same machine coordinate system, an offset vector of 0 must be specified.



- ① Case 1: Equidirectional MCS orientation
- ② Case 2: Counterdirectional MCS orientation
- V Offset vector

Note

Changing the offset vector

Changing the offset vector in machine data MD61518 \$MN_CC_PROTECT_OFFSET[<axis pair>] may only be carried out if the protection function for the axis pair is not active (MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] == 0).

8.3.7 Protection window

The protection window and/or the minimum clearance is defined which the axes of the axis pair may not fall below using the machine data:

MD61519 \$MN_CC_PROTECT_WINDOW[<a>] = <minimum clearance>

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

When the clearance approaches the minimum clearance, the axes are braked with the function-specific acceleration (Page 198).

8.3 Commissioning

The protection window can be dynamically extended, e.g. in an NC P program using the protection window extension (Page 196).

Note**Changing the protection window**

Changing the protection window in machine data MD61519

\$MN_CC_PROTECT_WINDOW[<axis pair>] may also be carried out if the protection function for the axis pair is **active** (MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] ≠ 0).

8.3.8

Orientation

The orientation of the axes of the axis pair to one another is specified in the following machine data:

MD61532 \$MN_CC_PROTECT_DIR_IS_REVERSE[<a>] = <value>

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

<value>	Meaning
0	Equidirectional orientation
1	Counterdirectional orientation

Note**Changing the orientation**

Changing the orientation in machine data MD61532

\$MN_CC_PROTECT_DIR_IS_REVERSE[<axis pair>] may only be carried out if the protection function for the axis pair is not active (MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] == 0).

8.3.9

Protection window extension

The protection window (Page 195) can be dynamically **extended** using machine data:

MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[<a>] = <extension>

with a = 0, 1, 2, ... (maximum number of axis pairs - 1) corresponding axis pair 1, 2, 3, ...

The resulting effective protection window of an axis pair is as follows:

Effective protection window[<axis pair>] =

MD61519 \$MN_CC_PROTECT_WINDOW[<axis pair>] +

MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[<axis pair>]

It is **not** possible to reduce the protection window by entering a negative value.

Note

Changing the protection window extension

Changing the protection window extension in machine data MD61533

\$MN_CC_PROTECT_WINDOW_EXTENSION[<axis pair>] may also be carried out when the protection function is active, e.g. from the NC program - and can be activated by initiating "Activate machine data".

8.3.10 Activating the protection function

Static activation using machine data

The protection function for an axis pair is statically activated as soon as the following preconditions are fulfilled:

- Both machine axes of the axis pair have been referenced
- Valid machine axes have been parameterized for the axis pair:
MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] = <valid machine axis pair>
- The supplementary function "Axis pair-specific activation/deactivation via axis-specific NC/PLC interface signals" is not active:
MD61535 \$MN_CC_PROTECT_OPTIONS[<axis pair>], bit 0 = 0

Dynamic activation via NC/PLC interface signal

The protection function for an axis pair is dynamically activated as soon as the following preconditions are fulfilled:

- Both machine axes of the axis pair have been referenced
- Valid machine axes have been parameterized for the axis pair:
MD61516 \$MN_CC_PROTECT_PAIRS[<axis pair>] = <valid machine axis pair>
- The supplementary function "Axis pair-specific activation/deactivation via axis-specific NC/PLC interface signals" is active:
MD61535 \$MN_CC_PROTECT_OPTIONS[<axis pair>], bit 0 = 1
- The axis-specific NC/PLC interface signal to activate the protection function is set for one of the two machine axes of the axis pair:
`<Axis>.coupling.out.mcsCollisionProtection == 1`

Clearance less than the protection window

If, at the time when the protection function is activated, the distance between the two machine axes is less than the minimum distance of the protection window which has been parameterized, then the machine operator must retract the machine axes. In this particular state, the control only allows traversing motion in the parameterized retraction direction (Page 194) of the machine axes.

8.3 Commissioning

Monitoring status

The actual monitoring status of an axis pair can be read using global user variable `_PROTECT_STATUS` (Page 198).

PLC signals

Basic Program Plus	Basic Program	
<Axis>.coupling.out.mcsCollisionProtection	---	DB31, ... DBX24.3

8.3.11 Axis-specific acceleration

The acceleration with which the two machine axes of the axis pair are braked by the protection function when a critical distance is approached is set using:

MD63514 \$MA_CC_PROTECT_ACCEL[<axis>] = <acceleration>

with <axis>: Machine axis name, e.g. AX1, AX2, ...

Note**Without jerk limitation**

The braking acceleration set using MD63514 \$MA_CC_PROTECT_ACCEL acts without jerk limiting.

Priority of the function-specific acceleration

The protection function only uses the function-specific acceleration of the machine axes from MD63514 \$MA_CC_PROTECT_ACCEL to calculate the time at which the axis should be braked. The protection function does not take the actual acceleration of the machine axis in the channel into account.

Note**Path reference**

If machine axes being monitored by the protection function are traversed by a channel with a path reference to other axes, this path reference is lost as soon as the protection function causes the axis grouping to decelerate. The machine axes being monitored by the protection function are decelerated using their function-specific acceleration values from machine data item MD63514 \$MA_CC_PROTECT_ACCEL. The remaining axes in the axis grouping are decelerated using the current path acceleration value of the channel.

8.3.12 Monitoring status (GUD)

The actual status of one axis pair is displayed using the global user variable `_PROTECT_STATUS`.

The variable is not available in the default setting. When required, it must be defined in the GUD.DEF definition window.

Definition

DEF NCK INT _PROTECT_STATUS[<number of parameterized axis pairs>]

with <number of parameterized axis pairs> = 1, 2, 3, ... (maximum number of axis pairs)

Range of values

Value	Meaning: The monitoring of the axis pair is ...
0	Not active
1	selected, but not yet active
2	active, the axes are currently not being braked
3	active, the axes are currently being braked
4	deselected, however still active

8.3.13 PLC interface: Axis-specific braking operations

Within the general system variable field \$A_DBD, a double word (four bytes) can be used to define an axes-specific braking interface. When the axes of the axis pair critically approach one another, the actual braking of the machine axes is displayed via the braking interface.

MD61534 \$MN_CC_PROTECT_A_DBD_INDEX = <value>

<value>	Meaning
-1	Deactivation of the output.
≥ 0	Braking interface index within the system variable field: \$A_DBD[<index>] with index = 0, 4, 8, 12, ...

Note**Double word index**

The starting index can be specified in the machine data, byte-by-byte (0, 1, 2, ...). System variable \$A_DBD is accessed from the PLC using a double word. This means that a starting index, which is not located at a double word boundary (0, 4, 8, ...), is **rounded off** to the next double word boundary (0, 4, 8, 12, ...): Index = (index DIV 4) * 4

Braking interface

Each bit of the braking interface is assigned a machine axis:

Bit n → (n+1)th machine axis, with n = 0, 1, 2, ...

Bit	31	30	29	28	...	3	2	1	0
MA ¹⁾	---	31	30	29	...	4	3	2	1
1) Machine axis number									

Bit n	Meaning
0	Machine axis (n+1) is not braked
1	Machine axis (n+1) is braked

Further information

The block LSINU_DeprecatedTransferSelData, function 3, is available to read the braking interface from the PLC user program.

8.3.14 PLC interface: Axis pair-specific activation of the protection function

If the supplementary function "Activation/deactivation of function "Axis pair collision protection" via NC/PLC interface signal (Page 193)" is active, using the axis-specific interface signal, the protection function for the axis pair can be activated and deactivated from the PLC user program:

```
<Axis>.coupling.out.mcsCollisionProtection (enable collision protection)
```

Activation

The protection function is activated if the axis-specific interface signal for one of the two machine axes of the axis pair is set.

Deactivation

The protection function is deactivated if the axis-specific interface signal for both machine axes of the axis pair is reset.

PLC signals

Basic Program Plus	Basic Program	
<Axis>.coupling.out.mcsCollisionProtection	---	DB31,DBX24.3

8.4 Boundary conditions

8.4.1 Axes

Same axis types

Both machine axes of an axis pair must be of the **same** axis type:

- Linear axis:
 - MD30300 \$MA_IS_ROT_AX = 0
 - MD30310 \$MA_ROT_IS_MODULO = 0
- Rotary axis:
 - MD30300 \$MA_IS_ROT_AX = 1
 - MD30310 \$MA_ROT_IS_MODULO = 0

Modulo rotary axes

None of the machine axes of an axis pair may be a **modulo rotary axis**:

- MD30300 \$MA_IS_ROT_AX = 1 (rotary axis)
- MD30310 \$MA_ROT_IS_MODULO = 1 (error: modulo rotary axis !)

8.4.2 Interpolatory couplings

Assumption

1. A machine axis is part of an interpolatory coupling, e.g.:
 - Generic coupling (CP)
 - Coupled motion (TRAIL)
 - Master value coupling (LEAD)
 - Electronic gear (EG)
 - Synchronous spindle (COUP)
2. The machine axis is **not** traversed in the **first** channel of the NC.
3. The machine axis is monitored using the "axis pair protection" function.

Effect

If the machine axis is not traversed in the first channel of the NC, then the components, created from the interpolatory couplings for the position and velocity setpoint are only available after a **deadtime of one interpolator clock cycle** of the "axis pair collision protection" function. Therefore, the machine axes are monitored, offset by these components. The absolute value of the components is independent of the interpolator clock cycle and the actual velocity and acceleration of the machine axis.

8.4.3 Simultaneous programming of the protected axes

If the protected axes are programmed simultaneously in an NC block with different traversing distances, then the monitoring function can respond, although the programmed target positions do not cause a violation of the safety window.

8.5 Examples

8.5.1 Collision protection

The figure shows the arrangement of 3 machine axes and the offset and orientation of the machine coordinate systems (machine).

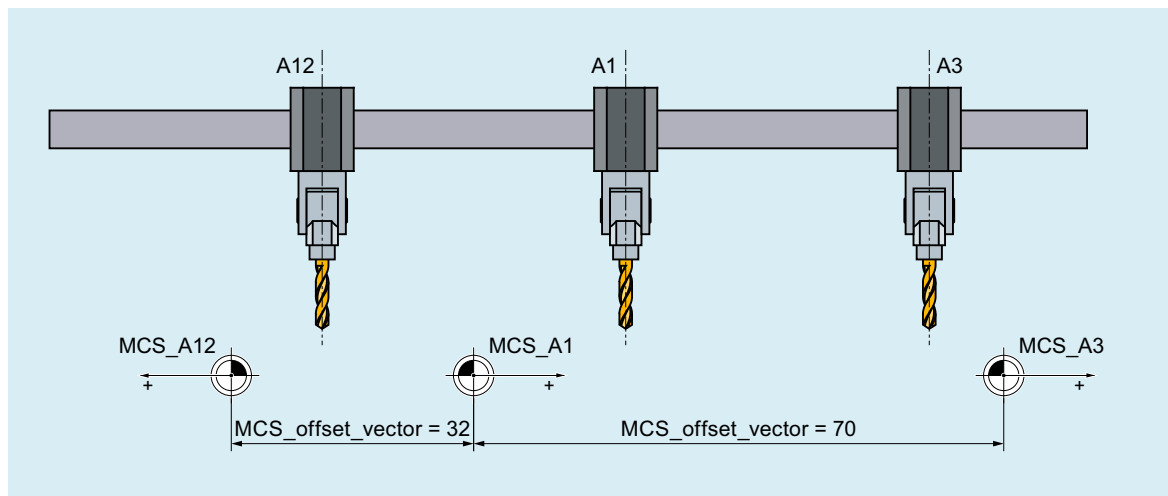


Figure 8-2 Collision protection for 2 axis pairs

Parameter assignment: Protection function 1

Axis pair: 1st machine axis A3, 2nd machine axis A1

- MD61516 \$MN_CC_PROTECT_PAIRS[0] = 01 03

Retraction direction: A1 in negative direction, A3 in positive direction

- MD61517 \$MN_CC_PROTECT_SAFE_DIR[0] = 00 01

Offset vector from machine coordinate system machine_A1 to machine_A3 with reference to machine_A3

- MD61518 \$MN_CC_PROTECT_OFFSET[0] = 70.0

Example protection window, 10.0 mm

- MD61519 \$MN_CC_PROTECT_WINDOW[0] = 10.0

Orientation of the machine coordinate systems to one another: same direction

- MD61532 \$MN_CC_PROTECT_DIR_IS_REVERSE[0] = 0

Protection window extension: none

- MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[0] = 0.0

Parameter assignment: Protection function 2

Axis pair: 1st machine axis A1, 2nd machine axis A12

- MD61516 \$MN_CC_PROTECT_PAIRS[1] = 12 01

Retraction direction: A12 in positive direction, A1 in positive direction

- MD61517 \$MN_CC_PROTECT_SAFE_DIR[1] = 01 01

Offset vector from machine coordinate system machine_A12 to machine_A1 with reference to machine_A1

- MD61518 \$MN_CC_PROTECT_OFFSET[1] = 32.0

Example protection window, 5.0 mm

- MD61519 \$MN_CC_PROTECT_WINDOW[1] = 5.0

Orientation of the machine coordinate systems to one another: opposite direction

- MD61532 \$MN_CC_PROTECT_DIR_IS_REVERSE[1] = 1

Protection window extension: by 5.0 mm to give a total of 10.0 mm

- MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[1] = 5.0

8.5.2 Collision protection and distance limiter

The figure shows the arrangement of two machine axes, the offset and orientation of the machine coordinate systems (machine), and the minimum and maximum distance vectors.

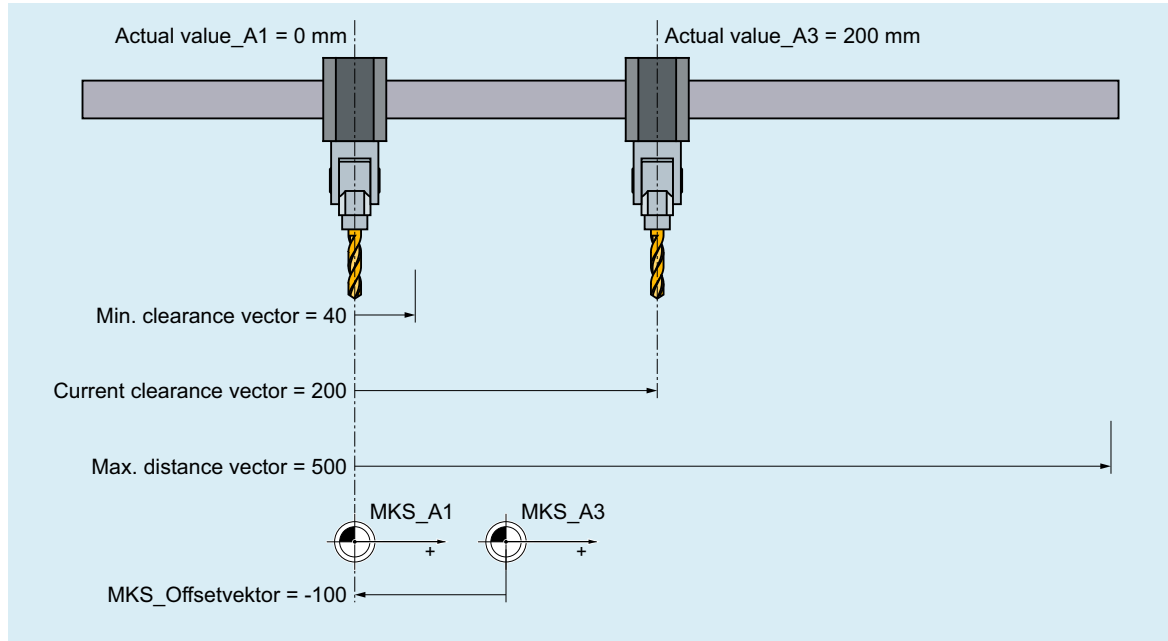


Figure 8-3 Collision protection and distance limiter for one axis pair

Parameter assignment: Protection function 1 - Collision protection

Axis pair: 1st machine axis A1, 2nd machine axis A3

- MD61516 \$MN_CC_PROTECT_PAIRS[0] = 03 01

Retraction direction: A1 in negative direction, A3 in positive direction

- MD61517 \$MN_CC_PROTECT_SAFE_DIR[0] = 01 00

Offset vector from machine coordinate system machine_A3 to machine_A1 with reference to machine_A1

- MD61518 \$MN_CC_PROTECT_OFFSET[0] = -100.0

Example protection window, 40.0 mm

- MD61519 \$MN_CC_PROTECT_WINDOW[0] = 40.0

Orientation of the machine coordinate systems to one another: same direction

- MD61532 \$MN_CC_PROTECT_DIR_IS_REVERSE[0] = 0

Protection window extension: none

- MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[0] = 0.0

Parameter assignment: Protection function 2 - Distance limiter

Axis pair: 1st machine axis A1, 2nd machine axis A3

- MD61516 \$MN_CC_PROTECT_PAIRS[1] = 03 01

Retraction direction: A1 in positive direction, A3 in negative direction

- MD61517 \$MN_CC_PROTECT_SAFE_DIR[1] = 00 01

Offset vector = "offset vector from machine coordinate system machine_A3 to machine_A1 with reference to machine_A1" - "maximum distance vector with reference to machine_A1"

Note**Maximum distance vector**

The maximum distance vector from the 1st machine axis to the 2nd machine axis is the vector from the origin of the machine coordinate system for the 1st machine axis to the maximum permissible position of the 2nd machine axis, with reference to the machine coordinate system for the 1st machine axis.

- MD61518 \$MN_CC_PROTECT_OFFSET[1] = -100.0 - 500.0 = 400.0

Example protection window, 20.0 mm

- MD61519 \$MN_CC_PROTECT_WINDOW[1] = 20.0

Orientation of the machine coordinate systems to one another: same direction

- MD61532 \$MN_CC_PROTECT_DIR_IS_REVERSE[1] = 0

Protection window extension: none

- MD61533 \$MN_CC_PROTECT_WINDOW_EXTENSION[1] = 0.0

If machine axis A1 has a value of 0, the settings above will limit the traversing range of machine axis A3 to between -60.0 and 380.0, with reference to machine_A3.

8.5 Examples

Punching and nibbling

9.1 Brief Description

Subfunctions

The functions specific to punching and nibbling operations comprise the following:

- Stroke control
- Automatic path segmentation
- Rotatable punch and die
- Clamp protection

They are activated and deactivated via language commands.

9.2 Stroke control

9.2.1 General information

Functionality

The stroke control is used in the actual machining of the workpiece. The punch is activated via an NC output signal when the position is reached. The punching unit acknowledges its punching motion with an input signal to the NC. No axis may move within this time period. Repositioning takes place after the punching operation.

High-speed signals

"High-speed signals" are used for direct communication between the NC and punching unit. Combined with the punch, they allow a large number of holes to be punched per minute since the punch positioning times are interpreted as machining delays.

PLC signals

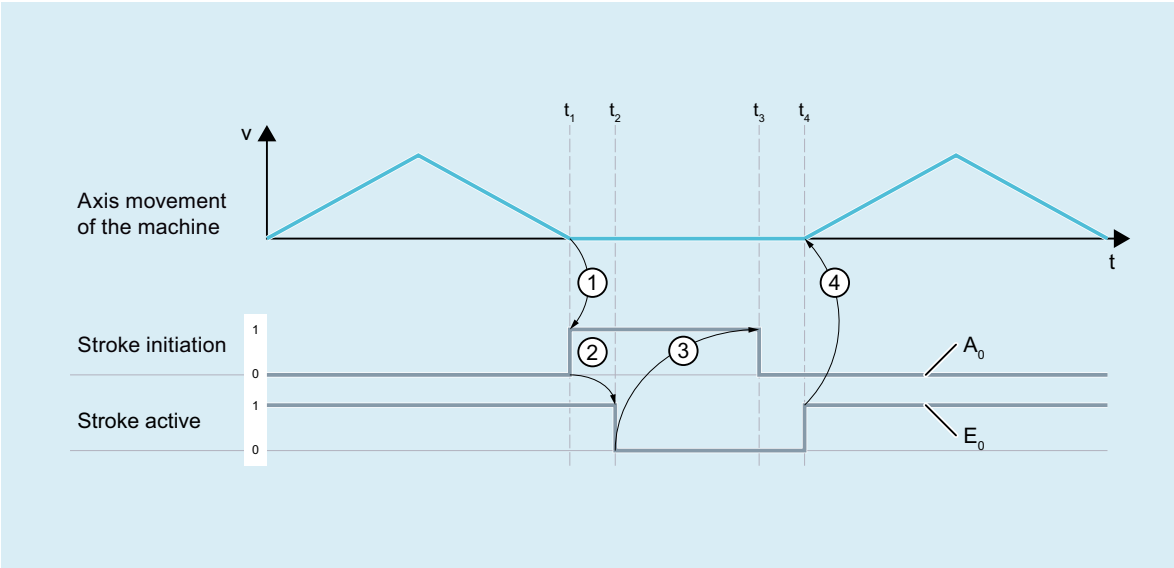
PLC interface signals are used for non-time-critical functions such as enabling and monitoring.

9.2.2 High-speed signals

Functionality

High-speed signals are used to synchronize the NC and punching unit. On the one hand, they are applied via a high-speed output to ensure that the punch stroke is not initiated until the metal sheet is stationary. On the other, they are applied via a high-speed input to ensure that the sheet remains stationary while the punch is active.

The high-speed digital inputs and outputs on the control are used to drive the punching unit. The following signal chart illustrates the signal sequence.



- (a) Axis motion of the machine as function $v(t)$
- (b) "Stroke initiation" signal
- (c) "Stroke active" signal

Figure 9-1 Signal chart

Note

The "Stroke active" signal is high-active for reasons relating to open-circuit monitoring.

The chronological sequence of events for punching and nibbling is controlled by the two signals A_0 and E_0 :

A_0	Set by the NC and identical to stroke initiation.
E_0	Defines the status of the punching unit and identical to the "Stroke active" signal.

The signal states characterize and define times t_1 to t_4 in the following way:

t_1	The motion of the workpiece (metal sheet) in relation to the punch is completed at instant t_1 . Depending on the criterion defined for stroke initiation (see Section "Criteria for stroke initiation (Page 209)"), the high-speed output O_0 is set for punch initiation ①.
t_2	The punching unit signals a punch movement via high-speed input E_0 at instant t_2 . This is triggered by signal A_0 ②. For safety reasons, signal E_0 is high-active (in the case of an open circuit, "Stroke active" is always set and the axes do not move). The "Stroke active" signal is not reset again until the tool has moved away from the metal sheet (t_4).
t_3	The NC reacts to the "Stroke active" signal at instant t_3 by canceling the "Stroke initiation" signal ③. From this point in time onwards, the NC is in a waiting state. It simply waits for cancellation of the "Stroke active" signal so it can initiate the next axis motion. The next stroke can be initiated only after signal A_0 has disappeared.
t_4	The punching operation is complete at instant t_4 , i.e. the punch has exited from the metal sheet again. The NC reacts to a signal transition in signal E_0 by starting an axis motion ④. The reaction of the NC to a signal edge change ④ is described in the section headed "Axis start after punching".

Note

The stroke time is determined by the period $\Delta t_h = t_4 - t_1$.

Reaction times at instant t_4 between the signal transition of E_0 and the start of the axis motion must also be added.

9.2.3 Criteria for stroke initiation

Initiate a stroke

The stroke initiation must be set, at the earliest, for the point in time at which it can be guaranteed that the axes have reached a standstill. This ensures that at the instant of punching, there is absolutely no relative movement between the punch and the metal sheet in the machining plane.

The following diagram shows the various criteria that can be applied to stroke initiation.

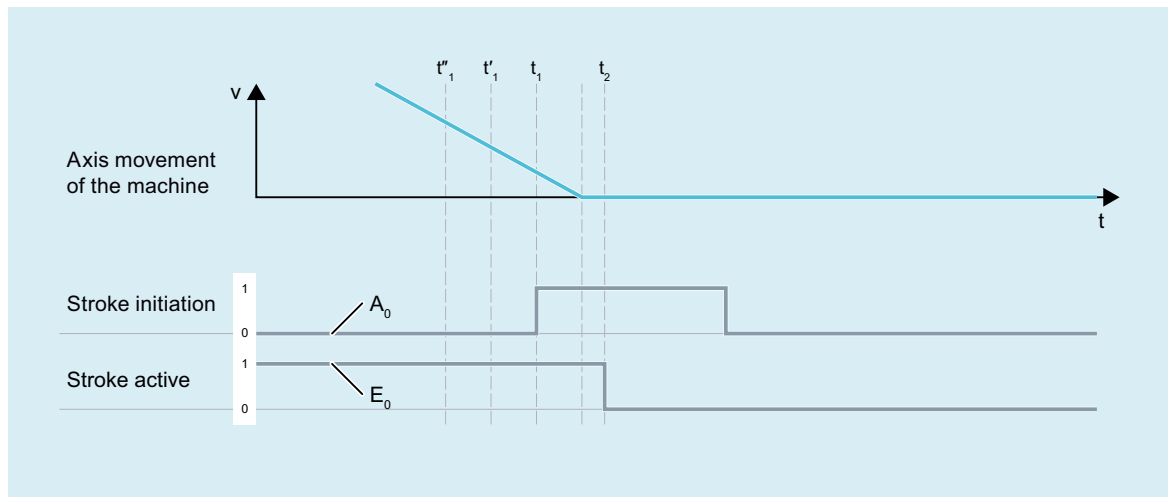


Figure 9-2 Signal chart: Criteria for stroke initiation

The time interval between t_1 and t_2 is determined by the reaction of the punching unit to setting of output A_0 . This cannot be altered, but can be utilized as a lead time for minimizing dead times.

The diagram above shows the default setting with which the output is set when the "Exact stop fine window" is reached (G601; default setting of G group 12). The punch initiation times t''_1 and t'_1 are programmed by means of G602 and G603 (see table below).

Programming	Activation	Description
G603	Stop interpolation	The interpolation reaches the block end. In this case, the axes continue to move until the overtravel has been traversed, i.e. the signal is output at an appreciable interval before the axes have reached zero speed (see t''_1).
G602	Reach the coarse in-position window	The signal is output once the axes have reached the coarse in-position window. If this criterion is selected for stroke initiation output, then the instant of stroke initiation can be varied through the size of interpolation window (see t'_1).
G601	Reach the fine in-position window	In this case, it can always be ensured that the machine will have reached a standstill at the instant of punching provided that the axis data are set appropriately. However, this variant also results in a maximum dead time (see t_1).

Note

The initial setting of the G group with G601, G602 and G603 (G group 12) is defined via machine data:

MD20150 \$MC_GCODE_RESET_VALUES[11]

The default setting is G601.

G603

Depending on velocity and machine dynamics, approximately 3 - 5 interpolation cycles are processed at the end of interpolation before the axes reach zero speed.

MD26018 \$MC_NIBBLE_PRE_START_TIME

In conjunction with the above machine data, it is possible to delay, and therefore optimize, the instant between reaching the end of interpolation and setting the high-speed output for "Stroke ON".

The following setting data is available in addition to MD26018:

SD42402 \$SC_NIBPUNCH_PRE_START_TIME

SD42402 can be changed from the part program and therefore adapted to the punching process depending on the progress of the part program run.

The following applies to the delay time:

MD26018 = 0 → SD42402 is operative

MD26018 ≠ 0 → MD26018 is operative

If the "Punching with dwell time, PDELAYON" is active, then the dwell time programmed in connection with this function is active. Neither MD26018 nor SD42402 is operative in this case.

9.2.4 Axis start after punching

Input signal "Stroke ON"

The start of an axis motion after stroke initiation is controlled via input signal "Stroke ON".

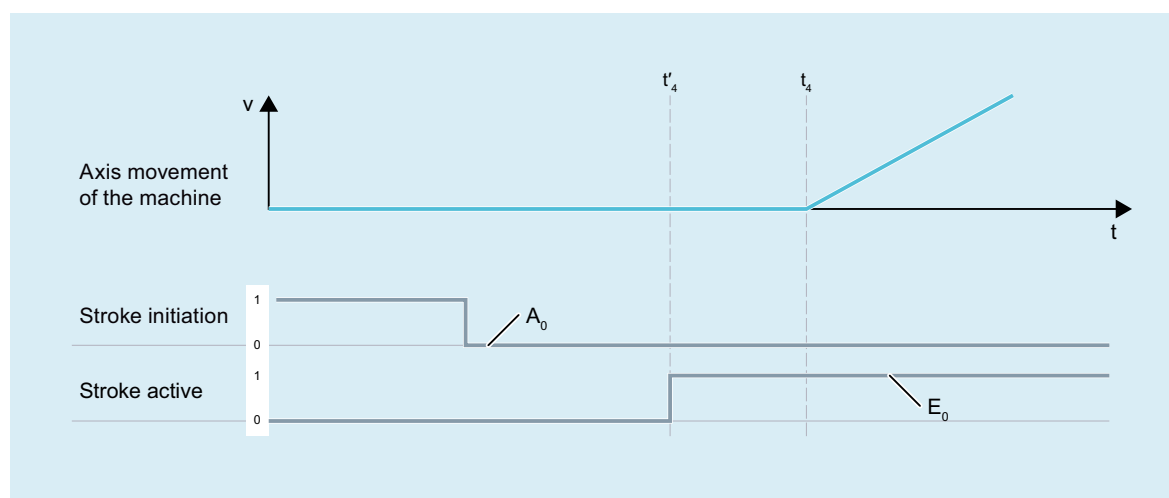


Figure 9-3 Signal chart: Axis start after punching

In this case, the time interval between t_4 and t'_4 acts as a switching-time-dependent reaction time. It is determined by the interpolation sampling time and the programmed punching/nibbling mode.

PON/SON

When the punching unit is controlled via `PON/SON`, the maximum delay time is calculated to be:

$$|t'_4 - t_4| = 3 \times \text{interpolator clock cycle}$$

PONS/SONS

If the punch is controlled by `PONS/SONS`, the delay time is determined by:

$$|t'_4 - t_4| \leq 3 \times \text{position controller cycles}$$

Prerequisites: Stroke time ($t_4 - t_2$) > 4 interpolator clock cycles

9.2.5 PLC signals specific to punching and nibbling

Function

In addition to the signals for direct stroke control, there are channel-specific NC/PLC interface signals for influencing the punching process and for status display.

Signals

Signal	Effect
<code><Chan>.nibbelingAndPunching.out.enableStroke</code> (no stroke enable)	Prevents the NC from initiating any punching operation. The NC waits until the enable signal is available before continuing the part program.
<code><Chan>.nibbelingAndPunching.out.suppressStroke</code> (stroke suppression)	Allows the part program to be processed without initiating a punching operation (dry run). With active path segmentation, the axes traverse in "stop and go" mode.
<code><Chan>.nibbelingAndPunching.out.noStroke</code> (delayed stroke)	Activates delayed stroke output if permitted by <code>PDELAYON</code> .
<code><Chan>.nibbelingAndPunching.out.manualStrokeInterface1</code> (manual stroke initiation)	Enables the operator to initiate a punching operation (controlled via the PLC) even without executing the part program. Manual stroke initiation is acknowledged with the signal: <code><Chan>.nibbelingAndPunching.in.strokeManualInitiationAck</code> (acknowledgement of manual stroke initiation)

PLC signals

Basic Program Plus	Basic Program	
<Chan>.nibbelingAndPunching.out.enableStroke	LBP_Chan*.A_StrokEnab	DB21,DBX3.0
<Chan>.nibbelingAndPunching.out.suppressStroke	LBP_Chan*.A_StrokSup	DB21,DBX3.2
<Chan>.nibbelingAndPunching.out.noStroke	LBP_Chan*.A_StrokStop	DB21,DBX3.4
<Chan>.nibbelingAndPunching.out.manualStrokeInterface1	LBP_Chan*.A_ManStrokEnab	DB21,DBX3.1
<Chan>.nibbelingAndPunching.in.strokeManualInitiationAck	LBP_Chan*.E_AcknManStrokEnab	DB21,DBX38.1

9.2.6 Punching and nibbling-specific reactions to standard PLC signals

Stroke control and "feed stop"

The control responds to the NC/PLC interface signal <Chan>.geoAxis1.out.disableFeed "feed stop" with regard to stroke control as follows:

Case 1: The interface signal is detected before the instant of time t_1 :

- Stroke initiation is suppressed.
- The next stroke is not initiated until the next start or until the "feed stop" has been cancelled.
- Machining is then continued as if there had been no interruption.

Case 2: The interface signal is detected at the instant of time t_1

- The current stroke is executed to completion.
- The control then dwells in the state characterized by t_4 .
- To enable this behavior without any problems, time monitoring of the "stroke active" and "stroke initiation" signals is omitted.

PLC signals

Basic Program Plus	Basic Program	
<Chan>.geoAxis1.out.disableFeed	LBP_Chan*.FDStop	DB21,DBX12.3

9.2.7 Signal monitoring

Oscillating signal

Owing to aging of the punch hydraulics, punch overshoot may cause the "stroke active" signal to oscillate at the end of a stroke.

In this case, depending on the subsequent machine data, alarm 22054 "improper punching signal" is displayed.

MD26020 \$MC_NIBBLE_SIGNAL_CHECK

Reset behavior

With an NC reset, the subsequent NC/PLC interface signal is reset **immediately** and does not wait for acknowledgement by the high-speed input.

`<Chan>.nibblingAndPunching.in.strokeInitiationActive` (stroke initiation active)

A currently activated stroke cannot be suppressed.

PLC signals

Basic Program Plus	Basic Program	
<code><Chan>.nibblingAndPunching.in.strokeInitiationActive</code>	<code>LBP_Chan*.E_StrokeEA</code>	<code>DB21,DBX38.0</code>

9.3 Activation and deactivation

9.3.1 Language commands

Punching and nibbling functions are activated and deactivated via configurable language commands. These replace the special M functions that were used in earlier systems.

For further information

Programming Manual Advanced

Groups

The language commands are subdivided into the following groups:

Group 35	
The actual punching and nibbling-specific functions are activated and deactivated by means of the following language commands:	
PON	= punching ON
SON	= nibbling ON
PONS	= punching ON, activated in the position controller
SONS	= nibbling ON, activated in the position controller
SPOF	= punching/nibbling OFF

Group 36	
This group includes the commands which have only a preparatory character and which determine the real nature of the punching function:	
PDELAYON	= punching with delay ON
PDELAYOF	= punching with delay OFF
Since the PLC normally needs to perform some preliminary tasks with respect to these preparatory functions, they are programmed before the activating commands.	

Group 38	
This group contains the commands for switching over to a second punch interface. It can be used, for example, for a second punching unit or set of hammer shears. A second I/O pair which can be used for punching functionality is defined via machine data.	
SPIF1	= first interface is active
SPIF2	= second interface is active

Note

Only one function at a time can be active within a G group (similar, for example, to the various interpolation modes G0, G1, G2, G3, etc. which are also mutually exclusive).

SPOF**Punching and nibbling OFF**

The **SPOF** function terminates all punching and nibbling functions. In this state, the NC responds neither to the "Stroke active" signal nor to the PLC signals specific to punching and nibbling functions.

If **SPOF** is programmed together with a travel command in one block (and in all further blocks if punching/nibbling is not activated with **SON** or **PON**), the machine approaches the programmed position without the initiation of a punching operation. **SPOF** deselects **SON**, **SONS**, **PON** and **PONS** and corresponds to the Reset condition.

Programming example:

Program code	Comment
:	
N20 G90 X100 SON	; Activate nibbling
N25 X50 SPOF	; Deactivate nibbling,
	; Positioning without stroke initiation
:	

SON**Nibbling ON**

SON activates the nibbling function and deselects the other functions in G group 35 (e.g. **PON**).

In contrast to punching, the first stroke is made at the start point of the block with the activating command, i.e. before the first machine motion.

SON has a modal action, i.e. it remains active until either SPOF or PON is programmed or until the program end is reached.

The stroke initiation is suppressed in blocks without traversing information relating to the axes designated as punching or nibbling axes (typically those in the active plane). If a stroke still needs to be initiated, then one of the punching/nibbling axes must be programmed with a 0 traversing path. If the first block with SON is a block without traversing information of the type mentioned, then only one stroke takes place in this block since the start and end points are identical.

Programming example:

Program code	Comment
:	
:	
N70 X50 SPOF	; Positioning without punch initiation
N80 X100 SON	; Activate nibbling, initiate a stroke before the
	; Motion (X=50) and at the end of the programmed
	; motion (X=100)
:	
:	

SONS

Nibbling ON (in position control cycle)

SONS behaves in the same way as SON. The function is activated in the position control cycle, thus allowing time-optimized stroke initiation and an increase in the punching rate per minute.

PON

Punching ON

PON activates the punching function and deactivates SON.

PON has a modal action like SON.

In contrast to SON, however, a stroke is not executed until the end of the block or, in the case of automatic path segmentation, at the end of a path segment. PON has an identical action to SON in the case of blocks which contain no traversing information.

Programming example:

Program code	Comment
:	
N100 Y30 SPOF	; Positioning without punch initiation
N110 Y100 PON	; Activate punching, punch initiation at the end of the
	; positioning operation (Y=100)
:	

PONS

Punching ON (in position control cycle)

PONS behaves in the same way as PON. For explanation, please refer to SONS.

PDELAYON

Punching with delay ON

PDELAYON is a preparatory function. This means that PDELAYON is generally programmed before PON. The punch stroke is output with a delay when the programmed end position is reached.

The delay time in seconds is programmed in setting data:

SD42400 \$SC_PUNCH_DWELLTIME

If the defined value cannot be divided as an integer into the interpolator clock cycle, then it is rounded to the next divisible integer value.

The function has a modal action.

PDELAYOF

Punching with delay OFF

PDELAYOF deactivates punching with delay function, i.e. the punching process continues normally. PDELAYON and PDELAYOF form a G group.

Programming example:

SPIF2 activates the second punch interface. This means that the stroke is controlled via the second pair of high-speed I/Os (see Section "Functional expansions (Page 218)", MD26004 and MD26006).

Program code	Comment
:	
N170 PDELAYON X100 SPOF	; Position without punch initiation, activate delayed punch initiation
:	
:	
N180 X800 PON	; Activate punching. The punch stroke is output with a delay when the end position is reached
:	
:	
N190 PDELAYOF X700	; deactivate punching with delay, normal punch initiation on. End of programmed motion
:	

SPIF1

Activation of first punch interface

`SPIF1` activates the first punching interface. This means that the stroke is controlled via the first pair of high-speed I/Os (see Section "Functional expansions (Page 218)", MD26004 and MD26006).

The first punch interface is always active after a reset or control system power up. If only one interface is used, then it need not be programmed.

SPIF2

Activation of second punch interface

`SPIF2` activates the second punch interface. This means that the stroke is controlled via the second pair of high-speed I/Os (see Section "Functional expansions (Page 218)", MD26004 and MD26006).

Programming example:

Program code	Comment
:	
N170 SPIF1 X100 PON	; At the end of the block, a stroke is initiated at the first fast output. The "Stroke active" signal is monitored at the first input.
:	
:	
N180 X800 SPIF2	; The second stroke is initiated at the second fast output. The "Stroke active" signal is monitored at the second input.
:	
:	
N190 SPIF1 X700	; All further strokes are controlled with the first interface.
:	

9.3.2 Functional expansions

Alternate interface

Machines that alternately use a second punching unit or a comparable medium can be switched over to a second I/O pair.

The second I/O pair can be defined via the following machine data:

MD26004 `$MC_NIBBLE_PUNCH_OUTMASK`

MD26006 `$MC_NIBBLE_PUNCH_INMASK`

The interface is switched by command `SPIF1` or `SPIF2`.

Full punching/nibbling functionality is available on both interfaces.

Example:

Hardware assignment for stroke control

Define the high-speed byte in each case on the CPU as a high-speed punch interface:

MD26000 \$MC_PUNCHNIB_ASSIGN_FASTIN = 'H00030001' → Byte 1

MD26002 \$MC_PUNCHNIB_ASSIGN_FASTOUT = 'H00000001'

Remark:

The first and second bits are inverted.

Screen form for high-speed input and output bits:

MD26004 \$MC_NIBBLE_PUNCH_OUTMASK[0]	= 1	First interface output bit → Bit 1 SPIF1
MD26004 \$MC_NIBBLE_PUNCH_OUTMASK[1]	= 2	Second interface output bit → Bit 2 SPIF2
MD26006 \$MC_NIBBLE_PUNCH_INMASK[0]	= 1	First interface input bit → Bit 1 SPIF1
MD26006 \$MC_NIBBLE_PUNCH_INMASK[1]	= 2	Second interface input bit → Bit 2 SPIF2

Automatically activated pre-initiation time

Dead times due to the reaction time of the punching unit can be minimized if the stroke can be initiated before the interpolation window of the axes is reached. The reference time for this is the interpolation end. The stroke is automatically initiated with G603 and delayed by the set value in relation to the time that the end of interpolation is reached.

The delay time for stroke initiation can be adjusted in machine data:

MD26018 \$MC_NIBBLE_PRE_START_TIME

Example:

With an IPO cycle of 5 ms, a stroke shall be released two cycles after reaching the interpolation end:

⇒ MD26018 \$MC_NIBBLE_PRE_START_TIME = 0.01 [s]

A pre-initiation time can also be programmed in setting data:

SD42402 \$SC_NIBPUNCH_PRE_START_TIME

This setting takes effective only if MD26018 = 0 has been set.

Monitoring of the input signal

If the "stroke active" signal is fluctuating between strokes due to plunger overshoots, for example, the message "undefined punching signal" can be also be output when interpolation is stopped.

The message output is dependent on the setting in machine data:

MD26020 \$MC_NIBBLE_SIGNAL_CHECK

MD26020 = 0	No alarm
MD26020 = 1	Alarm

Minimum period between two strokes

A minimum time interval between two consecutive strokes can be programmed in setting data:

SD42404 \$SC_MINTIME_BETWEEN_STROKES

Example:

There must be a minimum delay of at least 1.3 seconds between two stroke initiations irrespective of physical distance:

⇒ SD42404 \$SC_MINTIME_BETWEEN_STROKES = 1.3 [s]

If a punching dwell time (PDELAYON) is also programmed, then the two times are applied additively.

If a pre-initiation time at G603 is programmed, it will be effective only if the end of interpolation is reached before the time set in SD42404.

The programmed time becomes operative immediately. Depending on the size of the block buffer, strokes that have already been programmed can be executed with this minimum interval. The following programming measures (example) can be taken to prevent this:

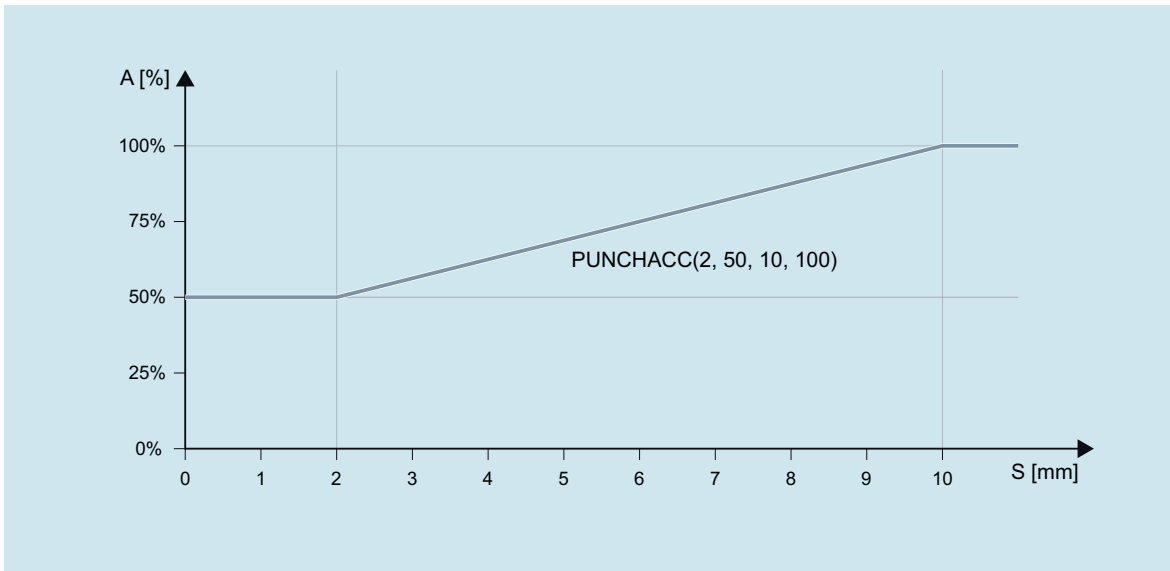
Program code
N...
N100 STOPRE
N110 \$SC_MINTIME_BETWEEN_STROKES = 1.3

The function is not active when SD42404 = 0.

Path-dependent acceleration

An acceleration characteristic can be defined using the PUNCHACC (Smin, Amin, Smax, Amax) language command. This command can be used to define different acceleration rates depending on the distance between holes.

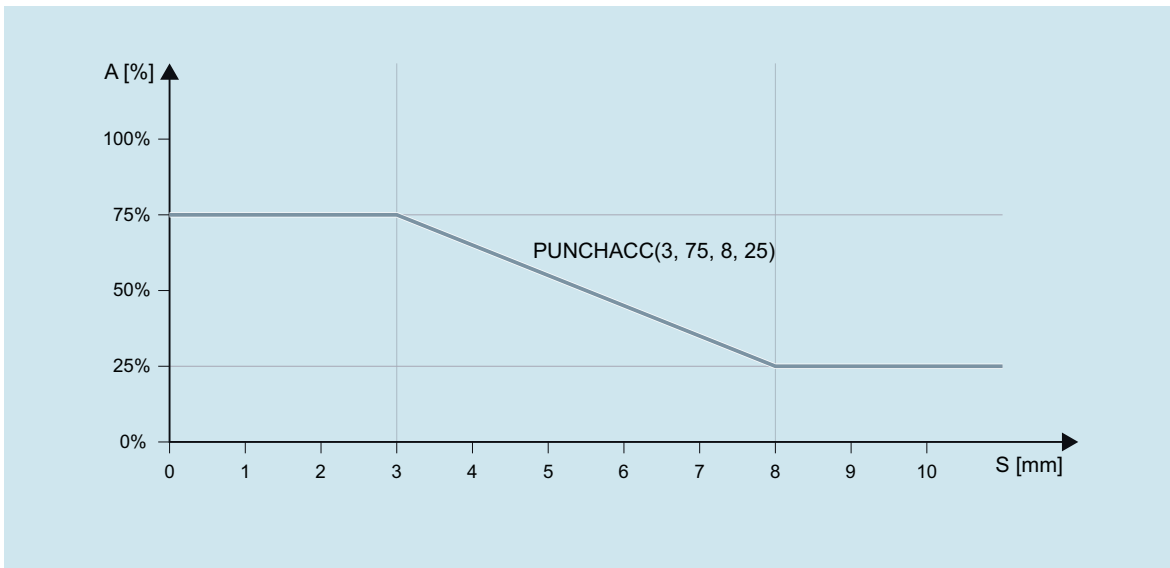
Example 1:



The characteristic defines the following acceleration rates:

Distance between holes	Acceleration
< 2 mm	The axis accelerates at a rate corresponding to 50% of maximum acceleration.
2 - 10 mm	Acceleration is increased to 100%, proportional to the spacing.
> 10 mm	The axis accelerates at the maximum rate (100%).

Example 2:



The characteristic defines the following acceleration rates:

Distance between holes	Acceleration
< 3 mm	The axis accelerates at a rate corresponding to 75% of maximum acceleration.
3 - 8 mm	Acceleration is reduced to 25%, proportional to the spacing.
> 10 mm	The axis accelerates at a rate of 25%.

If a reduced acceleration rate has already been programmed via `ACC`, then the acceleration limits defined with `PUNCHACC` refer to the reduced acceleration rate.

The function is deselected with:

$$S_{\min} = S_{\max} = 0$$

The acceleration rate programmed beforehand with `ACC` remains operative.

Block search

In the case of a search for a block containing a nibbling function, it is possible to program whether the punch stroke is executed at the block beginning or suppressed.

The setting is programmed in machine data:

MD11450 \$MN_SEARCH_RUN_MODE

Bit	Value	Meaning
5	0	Punch stroke at beginning of block is suppressed.
	1	Punch stroke at beginning of block is executed.

Further information:

Function Manual Basic Functions; Mode Group, Channel, Program Operation, Reset Response, Block Search

9.3.3 Compatibility with earlier systems

Use of M functions

As in earlier versions, macro technology allows special M functions to be used instead of language commands (compatibility).

The M functions and equivalent language commands as used in earlier systems are as follows:

M20, M23	≙	SPOF
M22	≙	SON
M25	≙	PON
M26	≙	PDELAYON

Note

M functions can be configured in machine data.

When M functions are assigned to language commands, it must be noted that M functions are organized in auxiliary function groups.

Examples

DEFINE M20 AS SPOF	Punching/nibbling OFF
or	
DEFINE M20 AS SPOF M=20	Punching with auxiliary function output
DEFINE M20 AS SPOF PDELAYOF	Punching/nibbling OFF and punching with delay OFF
DEFINE M22 AS SON	Nibbling ON
or	
DEFINE M22 AS SON M=22	Nibbling ON with auxiliary function output
DEFINE M25 AS PON	Punching ON
or	
DEFINE M25 AS PON M=25	Punching ON with auxiliary function output
DEFINE M26 AS PDELAYON	Punching with delay ON
or	
DEFINE M26 AS PDELAYON M=26	Punching and auxiliary function output

Programming example:

Program code	Comment
:	
:	
N100 X100 M20	; position without punch initiation
N110 X120 M22	; Activate nibbling, before and after motion
	; stroke initiation
:	
N120 X150 Y150 M25	; Activate punching, initiate stroke at end
	; of the motion
:	
:	

9.4 Automatic path segmentation

9.4.1 General information

Function

One of the following two methods can be applied to automatically segment a programmed traversing path:

- Path segmentation with maximum path segment programmed via language command `SPP`
- Path segmentation with a number of segments programmed via language command `SPN`

Both functions generate sub-blocks independently.

In earlier systems

- `SPP<number>` corresponds to `E<number>`
- `SPN<number>` corresponds to `H<number>`

Since addresses `E` and `H` now represent auxiliary functions, language commands `SPP` and `SPN` are used to avoid conflicts. The new procedure is therefore not compatible with those implemented in earlier systems. Both language commands (`SPP` and `SPN`) can be configured.

Note

The values programmed with `SPP` are either mm or inch settings depending on the initial setting (analogous to axes).

The automatic path segmentation function ensures that the path is divided into equidistant sections with linear and circle interpolation.

When the program is interrupted and automatic path segmentation is active (`SPP/SPN`), the contour can be reentered only at the beginning of the segmented block. The first punch stroke is executed at the end of this sub-block.

`SPP` and `SPN` can be activated only if geometry axes are configured.

SPP

The automatic path segmentation function `SPP` divides the programmed traversing path into sections of equal size according to the segment specification.

The following conditions apply:

- Path segmentation is active only when `SON` or `PON` is active.
(Exception: MD26014 `$MC_PUNCH_PATH_SPLITTING = 1`)
- `SPP` is modally active, i.e. the programmed segment remains valid until it is programmed again, but it can be suppressed on a block-by-block (non-modal) basis by means of `SPN`.
- The path segments are rounded off by the control system if required so that a total programmed distance can be divided into an integral number of path sections.
- The path segment unit is either mm/stroke or inch/stroke (depending on axis settings).

- If the programmed *SPP* value is greater than the traversing distance, then the axis is positioned on the programmed end position without path segmentation.
- *SPP* = 0, reset or program end delete the programmed *SPP* value. The *SPP* value is not deleted when punching/nibbling is deactivated.

SPN

The automatic path segmentation function *SPN* divides the traversing path into the programmed number of path segments.

The following conditions apply:

- Path segmentation is active only when *SON* or *PON* is active.
(Exception: MD26014 \$MC_PUNCH_PATH_SPLITTING = 1)
- *SPN* has a non-modal action.
- Any previously programmed *SPP* value is suppressed for the block containing *SPN*, but is re-activated again in the following blocks.

Supplementary conditions

- The path segmentation function is operative with linear and circular interpolation. The interpolation mode remains unchanged, i.e. circles are traversed when circular interpolation is selected.
- If a block contains both *SPN* (number of strokes) and *SPP* (stroke path), then the number of blocks is activated in the current block while the stroke path is activated in all blocks that follow.
- Path segmentation is active only in conjunction with punching or nibbling functions.
(Exception: MD26014 \$MC_PUNCH_PATH_SPLITTING = 1).
- Any programmed auxiliary functions are output before, during the first or after the last sub-block.
- In the case of blocks without traversing information, the rules which govern the programming of *SON* and *PON* also apply to *SPP* and *SPN*. In other words, a stroke is initiated only if an axis motion has been programmed.

9.4.2 Operating characteristics with path axes

MD26010

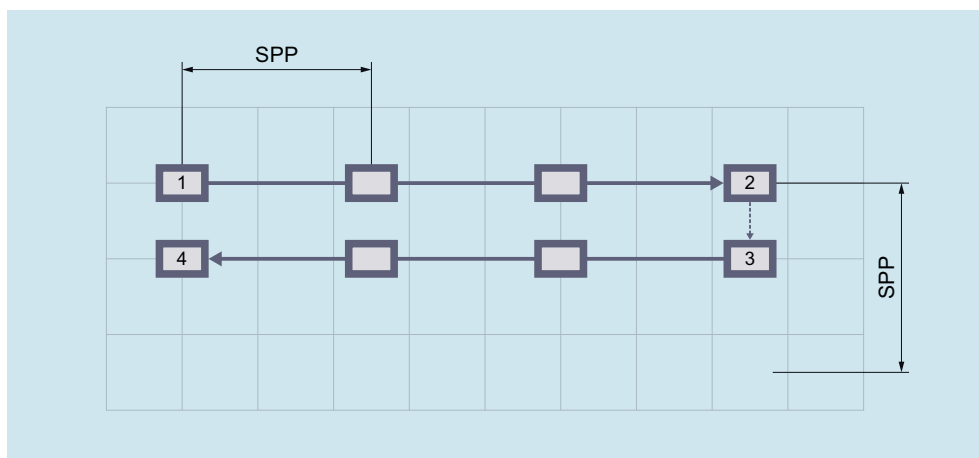
All axes defined and programmed via machine data:

MD26010 \$MC_PUNCHNIB_AXIS_MASK

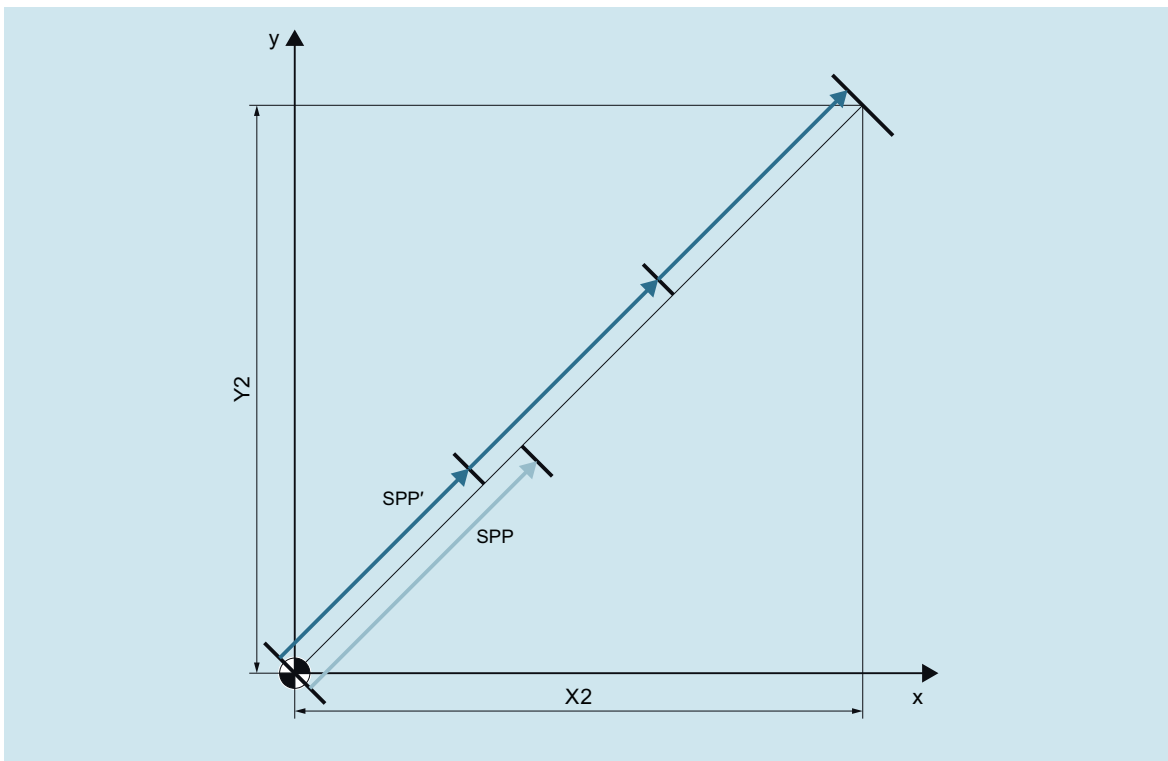
are traversed along path sections of identical size with *SPP* and *SPN* until the programmed end point is reached. This also applies to rotatable tool axes if programmed. The response can be adjusted for single axes.

Example of SPP

Program code	Comment
N1 G01 X0 Y0 SPOF	; positioning without punch initiation
N2 X75 SPP=25 SON	; nibbling with feedrate value 25 mm; initiate punch ; before the first movement and after each ; path segment.
:	
:	
N3 Y10	; positioning with reduced SPP value, because
:	; traversing distance < SPP value, and initiate punch ; after the movement.
:	
:	
N4 X0	Reposition with punch initiation
:	after each path section.
:	



If the programmed path segmentation is not an integral multiple of the total path, then the feed path is reduced.



X2/Y2: Programmed traversing distance
 SPP: Programmed SPP value
 SPP': Automatically rounded-off offset distance

Figure 9-4 Path segmentation

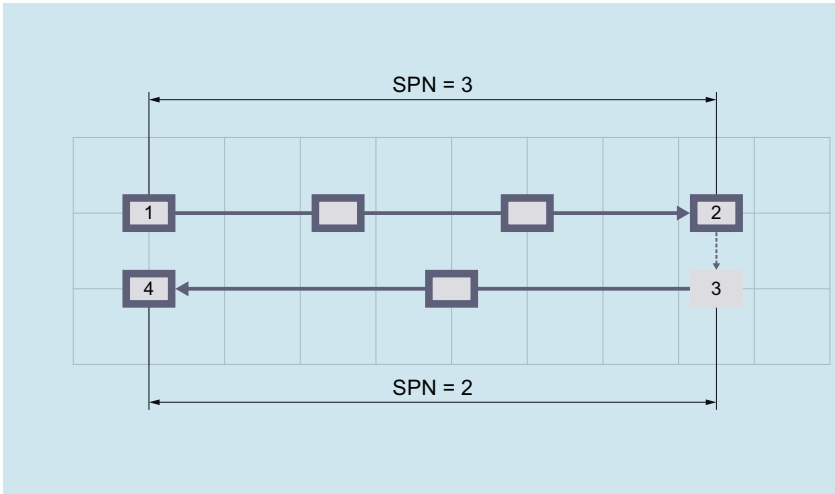
Example of SPN

The number of path segments per block is programmed via *SPN*.

A value programmed via *SPN* takes effect on a non-modal basis for both punching and nibbling applications. The only difference between the two modes is with respect to the first stroke. This is normally executed at the beginning of the first segment with nibbling operations and at its end with punching operations. This means that when *n* segments are programmed, *n* strokes are executed with punching operations but *n*+1 with nibbling. Furthermore, where no travel information is available, only a single stroke is executed, even if several are programmed. Should it be necessary to generate several strokes at one position, then the corresponding number of blocks without traversing information must be programmed.

Program code	Comment
N1 G01 X0 Y0 SPOF	; positioning without punch initiation
N2 X75 SPN=3 SON	; activate nibbling. The total path is
	; divided into 3 path segments. Before the first
	; movement and at the end of each path segment, a
	; stroke is initiated.

Program code	Comment
:	
:	
:	
:	
N3 Y10 SPOF	; positioning without punch initiation
N4 X0 SPN=2 PON	; activate punching. The total path is divided into 2
	; path segments. Because punching has been activated,
	; the first stroke is initiated at the end of the first
	path
	; segment.
:	
:	
:	
:	



Example

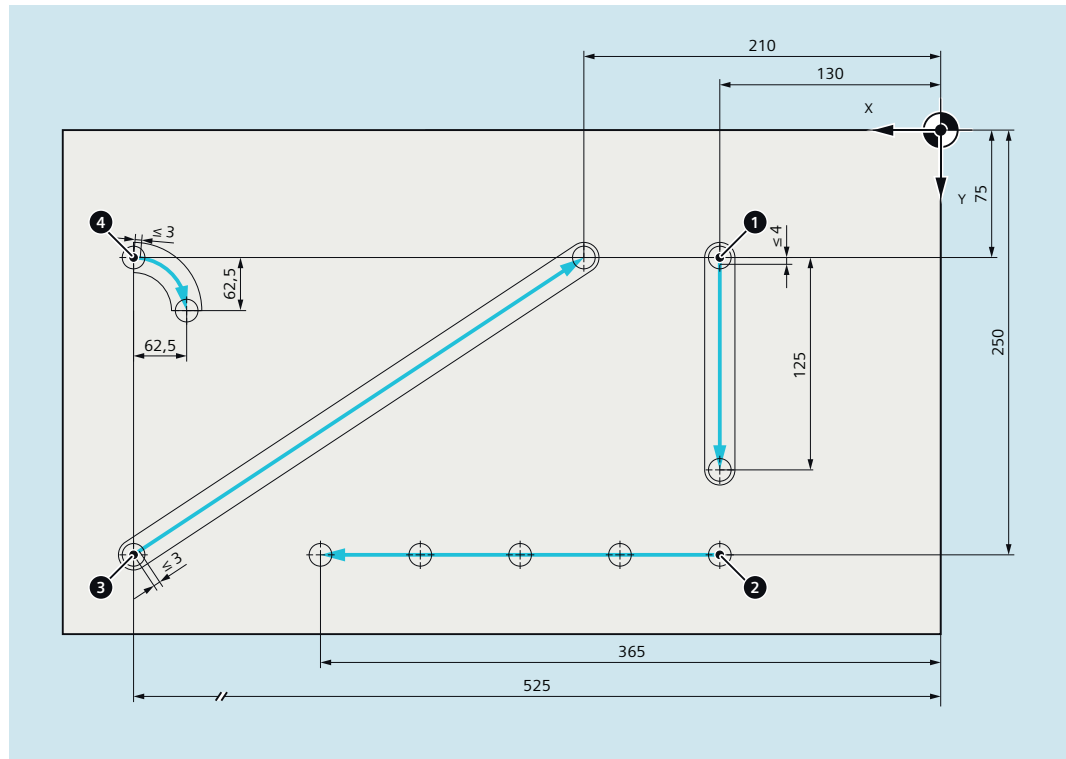


Figure 9-5 Workpiece

Extract from program

Program code	Comment
N100 G90 X130 Y75 F60 SPOF	; positioning to starting point (1) of the ; vertical nibbling path segments
N110 G91 Y125 SPP=4 SON	; end point coordinates (incremental) ; path segment: 4 mm, activate nibbling
N120 G90 Y250 SPOF	; absolute dimensioning, positioning at ; the start point (2) of the horizontal ; nibbling path segment
N130 X365 SPN=4 SON	; end point coordinates, 4 path segments, ; activate nibbling
N140 X525 SPOF	; positioning to starting point (3) of the ; inclined nibbling path
N150 X210 Y75 SPP=3 SON	; end point coordinates, path segment: 3 mm, ; activate nibbling
N160 X525 SPOF	; positioning to starting point (4) of the ; nibbling path on pitch circle path
N170 G02 G91 X-62.5 Y62.5 I0 J62.5 SON	; incremental circular interpolation with ; activate parameters,

Program code	Comment
N180 G00 G90 Y300 SPOF	; nibbling ; positioning

9.4.3 Response in connection with single axes

MD26016

The path of single axes programmed in addition to path axes is distributed evenly among the generated intermediate blocks as standard.

In the following example, the additional rotary axis C is defined as a synchronous axis.

If the axis is additionally defined as "punching-nibbling axis" (MD26010 \$MC_PUNCHNIB_AXIS_MASK = 1), a different behavior of the synchronized axis can be generated depending on the machine data MD26016 \$MC_PUNCH_PARTITION_TYPE.

Program example:

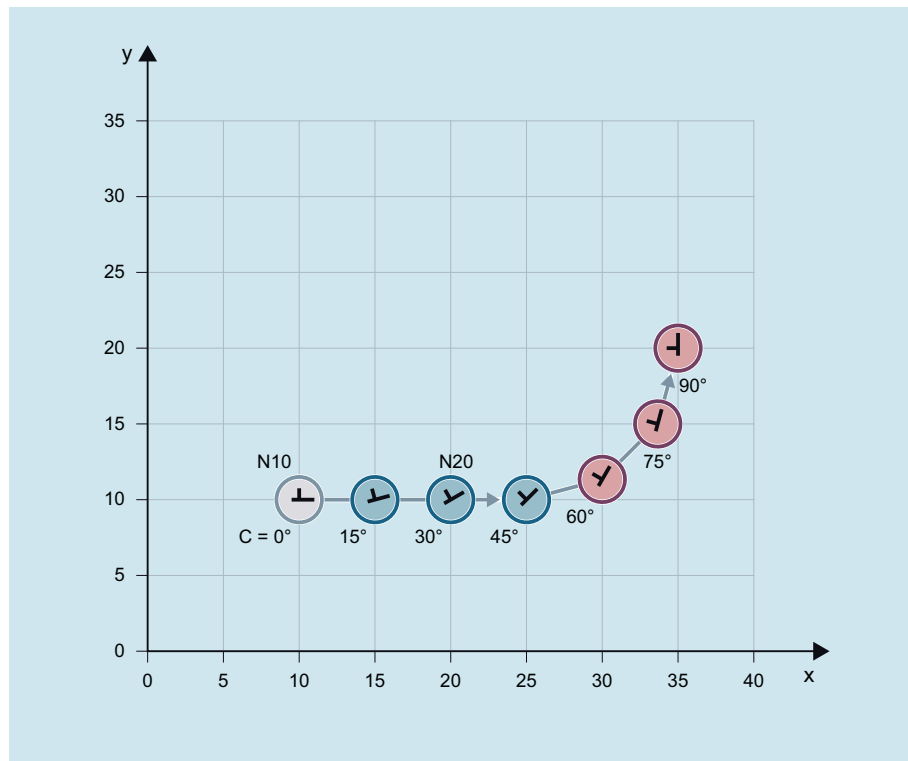
Program code

```
N10 G90 G1 PON X10 Y10 C0 F10000
N20 SPP=5 X25 C45
N30 G3 SPN=3 X35 Y20 I0 J10 C90
```

Case 1: MD26016 \$MC_PUNCH_PARTITION_TYPE = 0 (default setting)

With this setting, the axes behave as standard, i.e. the programmed special axis motions are distributed among the generated intermediate blocks of the active path segmentation function in all interpolation modes.

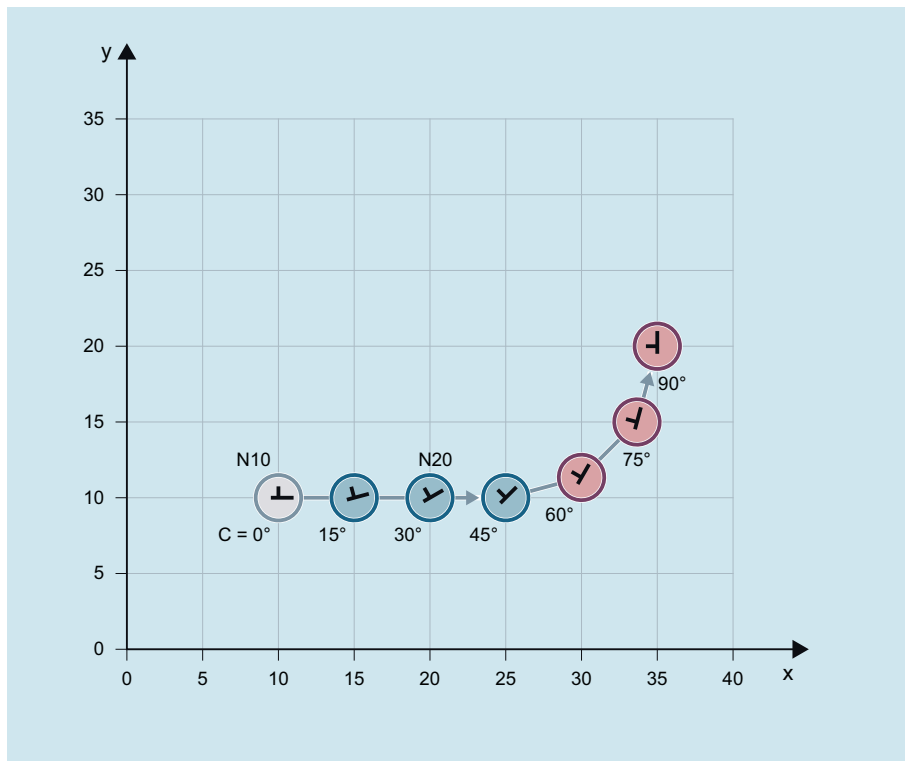
In block N20, the C axis is rotated through 15° in each of the three intermediate blocks. The axis response is the same in block N30, in the case of circular interpolation (three sub-blocks, each with 15° axis rotation).



Case 2: MD26016 \$MC_PUNCH_PARTITION_TYPE = 1

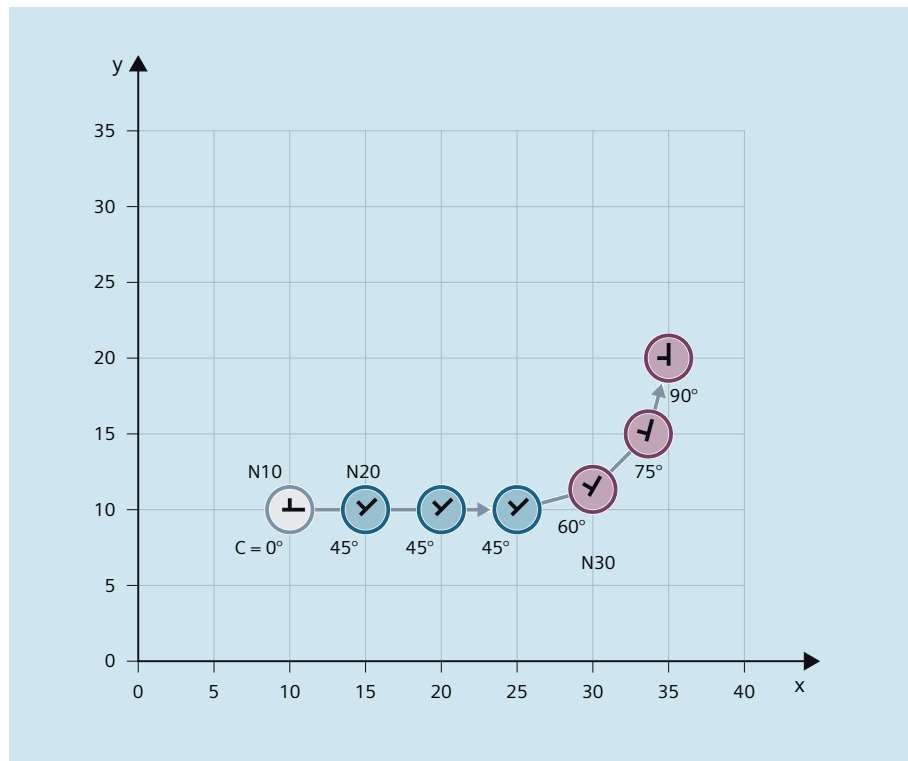
In contrast to the behavior described above, here the synchronous axis travels the entire programmed rotation path in the first sub-block of the selected path segmentation function.

Applied to the example, the C axis already reaches the programmed end position $C=45$ when it reaches X position $X=15$. It behaves in the same way in the circular interpolation block below.

**Case 3: MD26016 \$MC_PUNCH_PARTITION_TYPE = 2**

If only the behavior with active path segmentation for linear interpolation is to be as in case 2, but for circular interpolation it is to be the default behavior (see case 1), then set MD26016 = 2.

The axis behavior for the example is then as follows: In block N20, the C axis is rotated to $C=45^\circ$ in the first sub-block. The following circular interpolation block rotates the C axis through 15° in every sub-block.



This behavior may be especially desired for the axis of the rotatable tool if it is used to align the tool in a defined direction, for example tangential to the contour, and tangential control is **not** to be used for this purpose. However, it is not a substitute for the tangential control function since the start and end positions of the rotary axis must always be programmed.

Note

Additional offset motions of the special axes (here rotary axis C) are implemented via a work offsets.

Constraints

- If the C axis is not defined as a "Punch-nibble axis", then the C axis motion path is not segmented in block N30 in the above example nor is a stroke initiated at the block end.
- If the functionality described above is to be implemented in a variant not specific to nibbling applications, but with alignment of the special axis, then stroke initiation can be suppressed by the following PLC interface signal:
<Chan>.nibblingAndPunching.out.suppressStroke (stroke suppression)
(Application: e.g. alignment of electron beam during welding)
A similar behavior is made possible by the following machine data setting:
MD26014 \$MC_PUNCH_PATH_SPLITTING = 1
In this case, the path is segmented irrespective of punching or nibbling functions.

PLC signals

Basic Program Plus	Basic Program	
<Chan>.nibblingAndPunching.out.suppressStroke	LBP_Chan*.A_StrokSup	DB21,DBX3.2

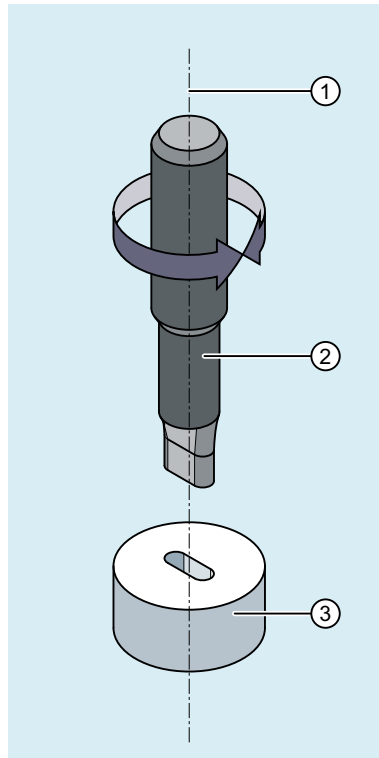
9.5 Rotatable tool

9.5.1 General information

Function overview

The following two functions are provided for nibbling/punching machines with rotatable punch and lower die:

- Coupled motion
for synchronous rotation of punch and die
- Tangential control
for normal alignment of rotary axes for punches in relation to workpiece



- 1 Tool axis
- 2 Stamp
- 3 Die

Figure 9-6 Representation of a rotatable tool axis

9.5.2 Coupled motion of punch and die

Function

Using the standard function "Coupled motion", it is possible to assign the axis of the die as a coupled motion axis to the rotary axis of the punch.

Activation

The "Coupled motion" function is activated or deactivated with language commands `TRAILON` and `TRAILOF` respectively.

Further information

Function Manual Axes and Spindles; Axis Couplings

Example

Example of a typical nibbling machine with rotatable punches where C is the punch axis and C1 the die axis:

Program code	Comment
:	
:	
TRAILON (C1, C, 1)	;Enabling the coupled-motion grouping
G01 X100 Y100 C0 PON	;Initiate stroke with C axis /
	;C1 axis position C=0=C1
X150 C45	;Initiate stroke with C axis /
	;C1 axis position C=45=C1
:	
:	
M30	

Basic position

No coupled-motion groupings are active after power up. Once the two tool axes have approached the reference point, the coupled-motion grouping is not generally separated again.

This can be achieved by:

- Program single activation of the coupled-motion grouping (see example above)
- Program MD setting:
MD20110 \$MC_RESET_MODE_MASK, bit 8=1

In this way, the coupled-motion grouping remains active after RESET/part program start or end.

9.5.3 Tangential control

Function

The "tangential control" function aligns the rotary axis used for positioning the punching/nibbling tool so that it is tangential to the programmed path. In connection with tangential control, this axis is called the tangential axis. The linear axes of the machining plane are the leading axes of the tangential coupling. The tangential axis is the following axis that is aligned to the path tangent of the lead axes. An additive offset angle for the tangent angle can also be programmed for the tangential axis.

If the punching/nibbling tool (die and die plate) is moved via two separate rotary axes, the second rotary axes can be coupled to the tangential axis via "coupled motion".

Note

Switch-on sequence

In the NC program, the tangential control must first be defined and then the coupled motion must be activated.

Further information

A detailed description of the tangential control can be found in the Axis and Spindles Function Manual.

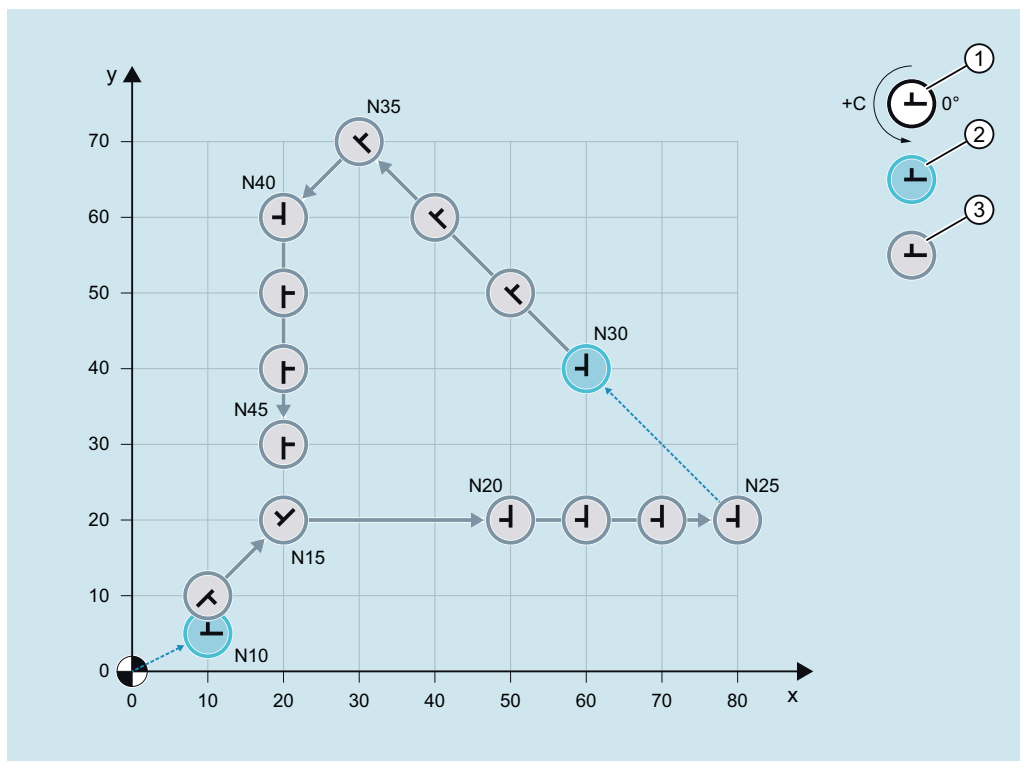
Example: Linear interpolation

Assumptions

- The punching/nibbling tool is subdivided into bottom and top tool
- The bottom tool and top tool are moved by different rotary axes (C/C1).
- The linear axes X and Y are the leading axes (machining plane G17)

Program code	Comment
N2 TANG (C, X, Y, 1, "B")	; definition tangential control
	; leading axes: X, Y
	; following axis: C
	; coupling factor: 1
	; basic coordinate system
N5 G0 X10 Y5	; starting position
N8 TRAILON (C1, C, 1)	; coupled motion ON
N10 Y10 C225 PON F60	; C/C1 axis → 225°
	; nibbling initiation ON, → stroke
N15 X20 Y20 C45	; C/C1 axis → 45° → stroke
N20 X50 Y20 C90 SPOF	; C/C1 axis → 90°
	; stroke initiation: OFF

Program code	Comment
N25 X80 Y20 SPP=10 SON	; path segmentation: ON ; path segment: 10 mm → 4 lifts ; punch initiation: ON
N30 X60 Y40 SPOF	; positioning ; stroke initiation: OFF
N32 TANGON (C, 180)	; tangential control: ON ; offset angle: 180°
N35 X30 Y70 SPN=3 PON	; path segmentation: 3 path segments → lift ; nibbling initiation: ON
N40 G91 C45 X-10 Y-10	; C/C1: 90° (-135°+180°+45°) → lift
N42 TANGON (C, 0)	; tangential control: ON ; offset angle: 0°
N45 G90 Y30 SPN=3 SON	; path segmentation → 3 lifts ; punch initiation: ON
N50 SPOF TANGOF	; stroke initiation: OFF ; tangential control: OFF
N55 TRAILOF (C1, C)	; coupled motion: OFF
N60 M2	

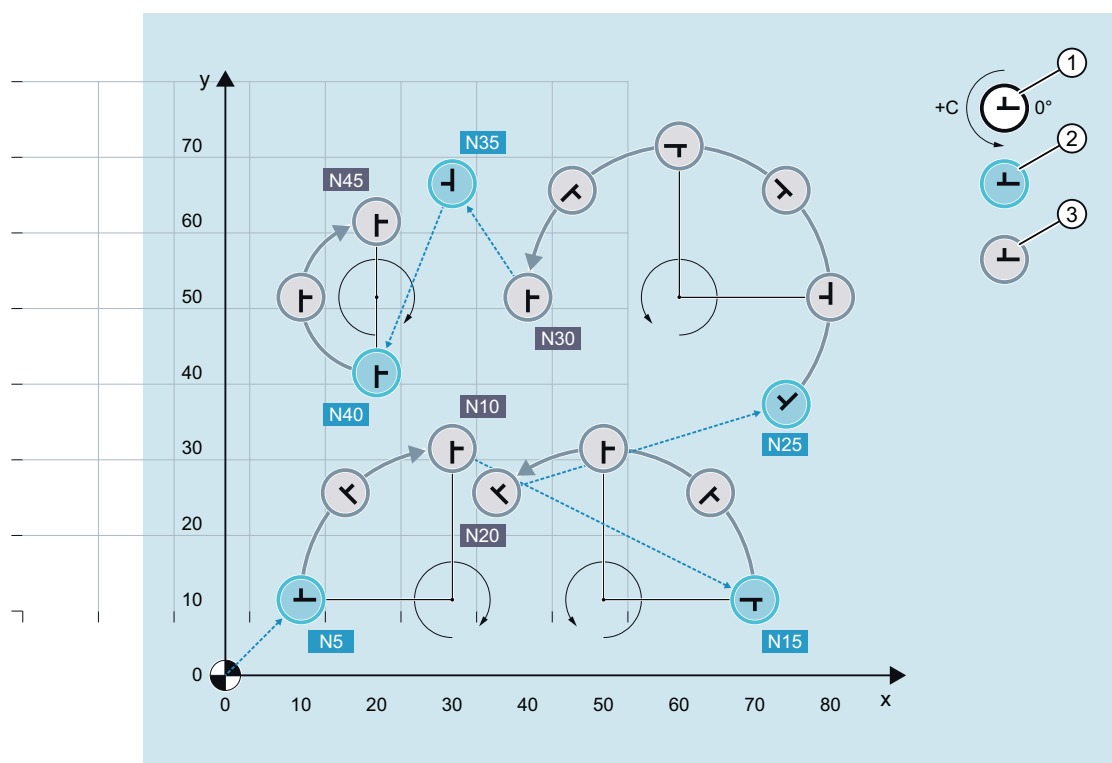


- ① Mounting position 0°
- ② Positioning
- ③ Stroke

Example: Circular interpolation

In circular interpolation mode, particularly when path segmentation is active, the tool axes rotate along a path tangentially aligned to the programmed path axes in each sub-block.

Program code	Comment
N2 TANG (C, X, Y, 1, "B")	; definition tangential control ; leading axes: X, Y ; following axis: C ; coupling factor: 1 ; basic coordinate system
N5 G0 F60 X10 Y10	; starting position
N8 TRAILON (C1, C, 1)	; coupled motion: ON
N9 TANGON (C, -90)	; tangential control: ON ; offset angle: -90° or 270°
N10 G02 X30 Y30 I20 J0 SPN=2 PON	; circular interpolation ; path segmentation → 2 lifts
N15 G0 X70 Y10 SPOF	; positioning ; stroke initiation: OFF
N17 TANGON (C, 90)	; tangential control: ON ; offset angle: 90°
N20 G03 X35,86 Y24,14 CR=20 SPP=16 SON	; circular interpolation ; path segmentation → 4 lifts ; punch initiation: ON
N25 G0 X74,14 Y35,86 C0 PON	; positioning ; C/C1 axis: 0° ; nibbling initiation: ON
N27 TANGON (C, 0)	; tangential control: ON ; offset angle: 0°
N30 G03 X40 Y50 I-14,14 J14,14 SPN=5 SON	; circular interpolation ; path segmentation → 5 lifts ; punch initiation: ON
N35 G0 X30 Y65 C90 SPOF	; positioning ; stroke initiation: OFF
N40 G91 X-10 Y-25 C180	; positioning, incremental ; C/C1 axis: +180° ; ⇒ 270° absolute
N43 TANGOF	; tangential control: OFF
N45 G90 G02 Y60 I0 J10 SPP=2 PON	; circular interpolation ; path segmentation → 2 lifts
N50 SPOF	; stroke initiation: OFF
N55 TRAILOF (C1, C)	; coupled motion: OFF
N60 M2	



- ① Mounting position 0°
- ② Positioning
- ③ Stroke

9.6 Protection zones

Clamping protection zone

The "clamping protection zone" function is contained as a subset in the "Protection zones" function. Its purpose is to simply monitor whether clamps and tool could represent a mutual risk.

Note

No by-pass strategies are implemented for cases where the clamp protection is violated.

Further information

Function Manual Monitoring and Compensating; Axis Monitoring, Protection Zones

9.7 Supplementary conditions

Availability of function "Punching and nibbling"

The function is an option ("Punching and nibbling functions"), which must be assigned to the hardware through the license management.

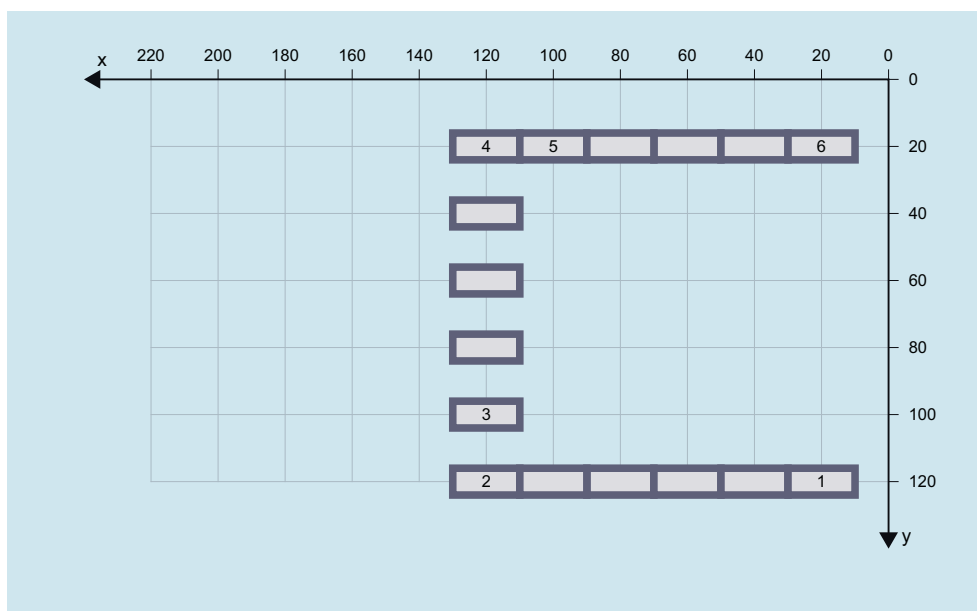
9.8 Examples

9.8.1 Examples of defined start of nibbling operation

Example 1

Example of defined start of nibbling operation

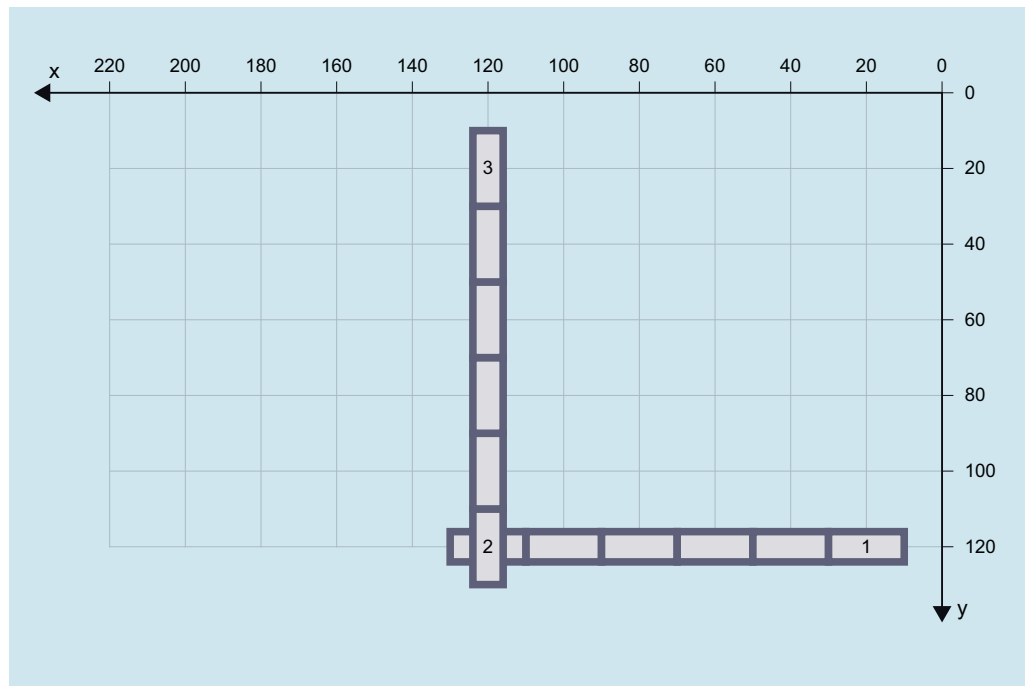
Program code	Comment
:	
:	
N10 G0 X20 Y120 SPP= 20	; Position 1 is approached
N20 X120 SON	; Defined start of nibbling, first stroke at ; "1", last stroke at "2"
N30 Y20	; Defined start of nibbling, first stroke at ; "3", last stroke at "4"
N40 X20	; Defined start of nibbling, first stroke at ; "5", last stroke at "6"
N50 SPOF	
N60 M2	



Example 2

This example utilizes the "Tangential control" function. Z has been selected as the name of the tangential axis.

Program code	Comment
:	
:	
N5 TANG (Z, X, Y, 1, "B")	; Define tangential axis
N8 TANGON (Z, 0)	; Select tangential control
N10 G0 X20 Y120	; Position 1 is approached
N20 X120 SPP=20 SON	; Defined start of nibbling, ; tangential control selected, ; first stroke at "1", last stroke at "2"
N30 SPOF TANGOF	; Deselect nibbling mode and deselect ; tangential control
N38 TANGON (Z, 90)	; Select tangential control
N40 Y20 SON	; Defined start of nibbling, ; tangential control selected, ; first stroke at "2" rotated 90 degrees to ; block N20, last stroke at "3"
N50 SPOF TANGOF	; Deselect nibbling mode and deselect ; tangential control
N60 M2	



Examples 3 and 4 for defined start of nibbling

Example 3: Programming of SPP

Program code	Comment
:	
:	
N5 G0 X10 Y10	; Positioning
N10 X90 SPP=20 SON	; Defined start of nibbling, ; 5 punches initiated
N20 X10 Y30 SPP=0	; One punch is initiated at the end of the path
N30 X90 SPP=20	; 4 punches initiated at intervals of 20 mm
N40 SPOF	
N50 M2	

Example 4 Programming of SPN

Program code	Comment
:	
:	
N5 G0 X10 Y10	; Positioning
N10 X90 SPN=4 SON	; Defined start of nibbling, 5 ; 5 punches initiated

Program code	Comment
N20 X10 Y30 PON	; One punch is initiated at the end of the path
N30 X90 SPN=4	; 4 punches initiated
N40 SPOF	
N50 M2	

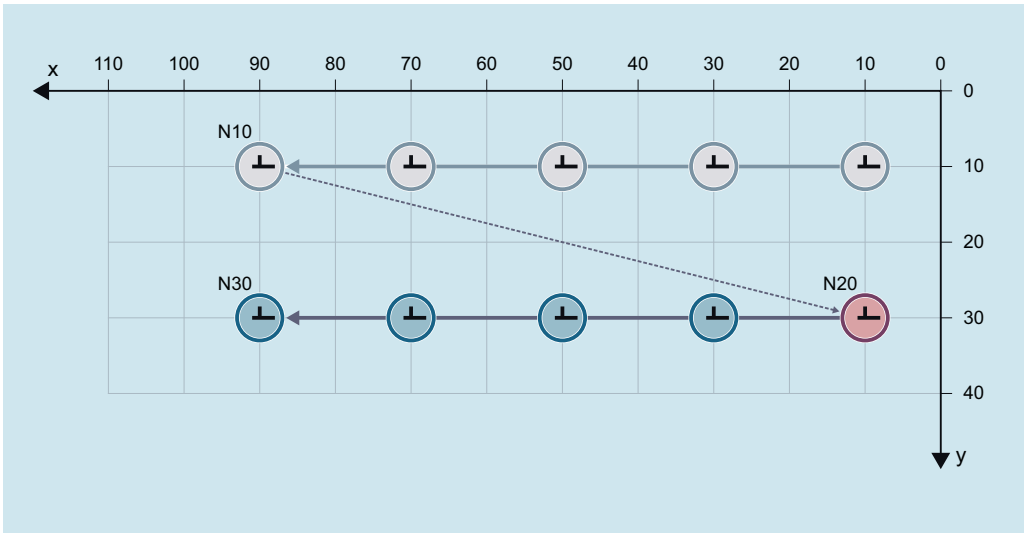


Figure 9-7 Examples 3 and 4 for defined start of nibbling

Examples 5 and 6 without defined start of nibbling

Example 5 Programming of SPP

Program code	Comment
:	
:	
N5 G0 X10 Y30	; Positioning
N10 X90 SPP=20 PON	; No defined start of nibbling, ; 4 punches initiated
N15 Y10	; One punch is initiated at the end of the path
N20 X10 SPP=20	; 4 punches initiated at intervals of 20 mm
N25 SPOF	
N30 M2	

Example 6 Programming of SPN

Program code	Comment
:	

Program code	Comment
:	
N5 G0 X10 Y30	; Positioning
N10 X90 SPN=4 PON	; No defined start of nibbling, ; 4 punches initiated
N15 Y10	; One punch is initiated at the end of the path
N20 X10 SPN=4	; 4 punches initiated
N25 SPOF	
N30 M2	

Example 7 Application example of SPP programming

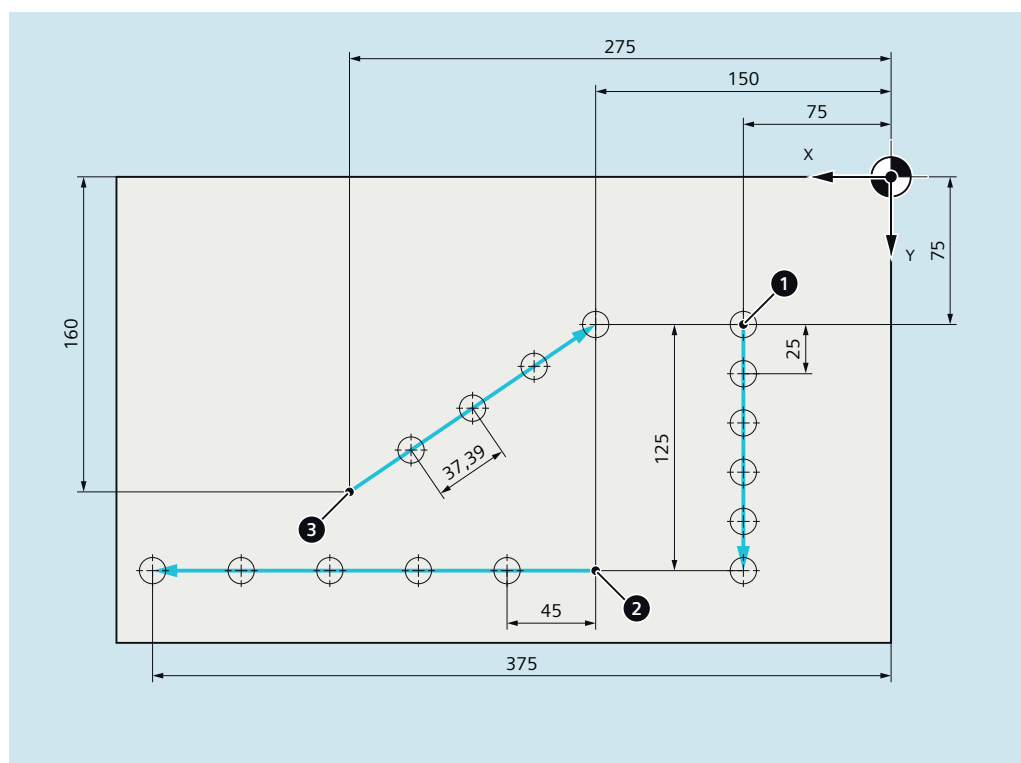


Figure 9-8 Workpiece

Extract from program:

Program code	Comment
N100 G90 X75 Y75 F60 PON	; Positioning to starting point (1) of the ; vertical line of holes, punch single hole
N110 G91 Y125 SPP=25 PON	; End point coordinates (incremental), ; path segment: 25 mm, activate punching
N120 G90 X150 SPOF	; Absolute dimensioning, position at ; the start point (2) of the horizontal line of ; holes

Program code	Comment
N130 X375 SPP=45 PON	; End point coordinates, path segment: 45 mm
N140 X275 Y160 SPOF	; Positioning to starting point (3) of the ; oblique line of holes
N150 X150 Y75 SPP=40 PON	; End point coordinates, programmed ; path segment: 40 mm, calculated ; path segment: 37.39 mm
N160 G00 Y300 SPOF	; Positioning

Appendix

A.1 List of abbreviations

A	
A	Output
AFIS	Automatic Filter Switch: Automatic filter switch
ASCII	American Standard Code for Information Interchange: American coding standard for the exchange of information
ASIC	Application Specific Integrated Circuit: User switching circuit
ASUP	Asynchronous subprogram
AUTO	Operating mode "Automatic"
AUXFU	Auxiliary Function: Auxiliary function
AWL	Statement list

B	
BAG	Mode group
BCD	Binary Coded Decimals: Decimal numbers encoded in binary code
BICO	Binector Connector
BIN	Binary Files: Binary files
BKS	Basic coordinate system
BM	Operating alarm
BO	Binector Output
BTSS	Operator panel interface

C	
CLC	Clearance control
CNC	Computerized Numerical Control: Computerunterstützte numerische Steuerung
COM	Communication
CP	Communication Processor
CPU	Central Processing Unit: Central processing unit
CST	Configured Stop: Configured stop

D	
DDS	Drive Data Set: Drive data set
DIR	Directory: Directory
DO	Drive Object
DRF	Differential Resolver Function: Differential resolver function (handwheel)

D	
DRY	Dry Run: Dry run feedrate
DWORD	Double word (currently 32 bits)

E	
E	Input
EES	Execution from External Storage
E/A	Input/Output
ESR	Extended stop and retract
ETC	ETC key ">"; Softkey bar extension in the same menu

F	
FDD	Feed Disable: Feed disable
FdStop	Feed Stop: Feed stop
FIFO	First In First Out: Memory that works without address specification and whose data is read in the same order in which they was stored
FM	Error message
FUP	Control system flowchart (PLC programming method)
FW	Firmware

G	
GEO	Geometry, e.g. geometry axis
GP	Basic program (PLC)
GUD	Global User Data: Global user data

H	
HEX	Abbreviation for hexadecimal number
HiFu	Auxiliary function
HMI	Human Machine Interface: SINUMERIK user interface
HSA	Main spindle drive
HT	Handheld Terminal
HW	Hardware

I	
IBN	Commissioning
INC	Increment: Increment
INI	Initializing Data: Initializing data
IPO	Interpolator

J	
JOG	JOGging: Setup mode

K	
KOP	Ladder diagram (PLC programming method)

L	
LED	Light Emitting Diode: Light-emitting diode
LMS	Position measuring system
LR	Position controller

M	
Main	Main program: Main program (OB1, PLC)
MCP	Machine Control Panel: Machine control panel
MD	Machine Data
MDA	Manual Data Automatic: Manual input
MDS	Motor Data Set: Motor data set
MELDW	Message word
MKS	Machine coordinate system
MM	Motor Module
MPF	Main Program File: Main program (NC)
MPI	Multi Point Interface
MSTT	Machine control panel

N	
NC	Numerical Control: Numerical control with block preparation, traversing range, etc.
NCU	Numerical Control Unit: NC hardware unit
NCK	Numerical Control Kernel
NCSD	NC Start Disable
NST	Interface signal
NV	Work offset
NX	Numerical Extension: Axis expansion module

O	
OB	Organization block in the PLC
OEM	Original Equipment Manufacturer
OP	Operation Panel: Operating equipment

A.1 List of abbreviations

P	
PCU	PC Unit: PC box (computer unit)
PG	Programming device
PI	Program instance
PLC	Programmable Logic Control: Controller
PN	PROFINET
PO	Power On
POS	Position/positioning
PPO	Parameter process data object: Cyclic data telegram for PROFIBUS DP transmission and "Variable speed drives" profile
PPU	Panel Processing Unit (central hardware for a panel-based CNC, e.g. SINUMERIK 828D)
PROFIBUS	Process Field Bus: Serial data bus
PRT	Program test
PTP	Point to Point: Point-to-point
PZD	Process data: Process data part of a PPO

R	
REF	REference point approach function
REPOS	REPOSition function
RESU	Retrace support
RID	Read In Disable
RP	R Parameter, arithmetic parameter, predefined user variable

S	
SA	Synchronized action
SBL	Single Block: Single block
SBT	Safe Brake Test
SCC	Safety Control Channel
SCL	Structured Control Language
SD	Settingdatum or setting data
SDI	Safe Direction
SERUPRO	Search-Run by Program Test: Block search via program test
SIC	Safety Info Channel
SKP	Skip: Function for skipping a part program block
SLP	Safe Limited Position
SLS	Safely Limited Speed
SMI	Sensor Module Integrated
SOS	Safe Operating Stop
SPF	Sub Program File: Subprogram (NC)
SS1	Safe Stop 1
SS2	Safe Stop 2
STO	Safe Torque Off

S	
STW	Control word
SUG	Grinding wheel peripheral speed
SW	Software

T	
TCU	Thin Client Unit
TIA	Totally Integrated Automation
TM	Terminal Module (SINAMICS)
TO	Tool Offset: Tool offset
TOA	Tool Offset Active: Identifier (file type) for tool offsets
TOFF	Online tool length offset
TRANSMIT	Transform Milling Into Turning: Coordination transformation for milling operations on a turning machine

U	
UP	Subprogram
USB	Universal Serial Bus

V	
VDI	Internal communication interface between NC and PLC

W	
WKS	Workpiece coordinate system
WPD	Work Piece Directory: Workpiece directory
WZ	Tool
WZV	Tool management

Z	
ZSW	Status word (of drive)

A.2 Available IPCs

IPCs recommended for SINUMERIK

Panel IPC	
IPC 477E 22" Win 7	6AV7241-3YA04-0FA0
IPC 477E 24" Win 7	6AV7241-5SB04-0FA0
IPC 477E 15" Win10	6AV7241-1WA07-0FA0
IPC 477E 19" Win10	6AV7241-3XB07-0FA0
IPC 477E 22" Win10	6AV7241-3YA07-0FA0
IPC 477E 24" Win10	6AV7241-5SB07-0FA0

Box IPC	
IPC 427E (Standard) Win7	6AG4141-1AA14-0FA0
IPC 427E (High) Win7	6AG4141-5AB14-0FA0
IPC 427E (Standard) Win10	6AG4141-1AA17-0FA0
IPC 427E (High) Win10	6AG4141-5AB17-0FA0

Index

A

- Acceleration
 - Channel-specific, 198
 - Function-specific, 198
- Acceleration characteristic, 220
- Alternate interface, 218
- ASUB enable, 156
- Automatically activated pre-initiation time, 219
- Axis pair collision protection
 - Maximum number of axis pairs, 191
- Axis replacement, 171

B

- Block search, 222

C

- CC_FASTOFF, 186
- CC_FASTON, 184
- CC_FASTON_CONT, 185
- CC_PREPRE, 145, 157
- CC_RESU.MPF, 159
- CC_RESU_ASUP.SPF, 163
- CC_RESU_BS_ASUP.SPF, 163
- CC_RESU_END.SPF, 162
- CC_RESU_INI.SPF, 160
- Clamping protection zone, 240
- CLC_RESU_LENGTH_BS_BUFFER, 168
- Clearance control
 - 3D control direction, 65
 - Collision monitoring, 39
 - Commissioning, 40
 - Constraints, 80
 - Programming, 56
 - Technological properties, 38
- Compile cycle
 - Interface version, 21
 - Software version, 29
- Compile cycles, 19
- Continue machining ASUB, 163

D

- Definition of an axis pair, 193

- Disable, 183
- Distance vector
 - Maximum, 205

E

- Enable, 193
- END program, 162

G

- G0 edge change, 176
- GUD variables, 168

I

- I/Os, 175, 182
- INI program, 160
- Integrated outputs, 175
- Interruption point, 146

L

- Language command
 - SPN, 225, 227
 - SPP, 224, 226
- Language commands, 214
- Laser cutting, 32
- Linear axis, 201

M

- Main program, 159
- Maximum switching frequency, 179
- MD10360, 182
- MD10362, 44
- MD10366, 44
- MD10620, 131
- MD10640, 131
- MD10712, 50
- MD11450, 222
- MD11602, 156
- MD11604, 156
- MD18351, 154
- MD19610, 193
- MD20050, 158, 183
- MD20110, 138, 236

- MD20140, 138
 - MD20150, 132, 210
 - MD21100, 131
 - MD21104, 132
 - MD24100, 92, 138, 140
 - MD24110, 92
 - MD24120, 92, 141, 158
 - MD24460, 138
 - MD26000, 219
 - MD26002, 219
 - MD26004, 218
 - MD26006, 218
 - MD26010, 225, 230
 - MD26014, 224, 225, 233
 - MD26016, 230
 - MD26018, 211, 219
 - MD26020, 214, 219
 - MD28090, 43, 152, 182
 - MD28100, 43, 152, 182
 - MD28105, 153
 - MD30132, 84
 - MD30300, 201
 - MD30310, 201
 - MD32000, 68
 - MD32200, 68
 - MD32230, 68
 - MD36000, 50
 - MD36010, 50
 - MD36020, 50
 - MD36040, 51
 - MD36060, 51
 - MD36100, 83
 - MD36110, 83
 - MD36120, 83
 - MD36130, 83
 - MD37100, 83
 - MD60900+i, 151
 - MD60940, 42
 - MD60948, 182, 184
 - MD60972, 193
 - MD61516, 193
 - MD61517, 194
 - MD61518, 194
 - MD61519, 195
 - MD61532, 196
 - MD61535, 193
 - MD62500, 50
 - MD62502, 50
 - MD62504, 39, 50
 - MD62528, 68
 - MD62560, 183
 - MD62571, 154
 - MD62572, 154
 - MD62574, 155
 - MD62575, 168
 - MD62580, 152
 - MD62600, 88, 106, 114, 121, 125, 141
 - MD62601, 104, 107, 114, 121
 - MD62602, 125, 127, 128, 129, 141
 - MD62603, 94, 107, 114, 121, 141
 - MD62604, 96, 107, 114, 121, 141
 - MD62605, 100, 106, 114, 121, 141
 - MD62606, 96, 114, 121, 141
 - MD62607, 95, 107, 115, 121, 141
 - MD62608, 96, 115, 121
 - MD62609, 96, 115, 121
 - MD62610, 96, 107, 120, 122
 - MD62611, 96, 107, 114, 120, 122
 - MD62612, 94, 107, 115, 121
 - MD62613, 94, 107, 115, 121
 - MD62614, 122, 141
 - MD62615, 141
 - MD62616, 97, 122, 141
 - MD62617, 103, 107, 114, 121, 141
 - MD62618, 103, 107, 114, 121, 141
 - MD62620, 101, 107, 114, 121, 141
 - MD62629, 105
 - MD62630, 105
 - MD62631, 105
 - MD62632, 106
 - MD62636, 135
 - MD62637, 136
 - MD63514, 198
 - MD65520, 68
 - Minimum interval between two consecutive strokes, 220
 - Minimum switching position clearance, 179
 - Modulo rotary axes, 201
 - Monitoring of the input signal, 219
- O**
- Offset vector, 194
 - Option, 193
 - Orientation of the machine axes, 196
- P**
- p1414, 38
 - Path segmentation, 224
 - Plane, 151
 - Program continuation point, 146
 - Protection window, 195

PUNCHACC, 220

R

RESET response, 158
RESU ASUB, 163
RESU main program, 159
RESU working plane, 151
Retraceable contour range, 146
Retraction direction, 194
Rotary axis, 201

S

SD42400, 217
SD42402, 211, 219
SD42404, 220
Series machine startup, 155
Switching criteria, 176
Switching instants, 179
Switching position, 176
Switching position offset, 180, 181

T

Tool offsets, 173
Transformation, 172

V

Velocity threshold value, 177

